

FOUNDATIONS OF KNOWLEDGE REPRESENTATION AND REASONING

João Pavão Martins

Departamento de Engenharia Informática
Instituto Superior Técnico
Universidade Técnica de Lisboa

Copyright ©1996, 1997, 1998, 1999, 2001, 2004, 2005, 2006 João Pavão Martins

All Rights Reserved

No part of this book may be reproduced in any form or by any means, electronic or mechanical, including photocopy, recording, or any information storage or retrieval system without permission in advance, in writing, from the author:

João Pavão Martins

Grupo de Inteligência Artificial

Departamento de Engenharia Informática

Instituto Superior Técnico

Av. Rovisco Pais

1096 Lisboa CODEX

Portugal

`jpm@gia.ist.utl.pt`

Contents

Glossary of notations	vii
Preface	xi
1 What is Knowledge Representation?	1
1.1 World, models, and representations	3
1.2 Cognitive agents	6
1.3 What is knowledge?	11
1.4 Underlying hypotheses	14
1.4.1 Physical symbol system hypothesis	14
1.4.2 Knowledge representation hypothesis	15
1.5 Levels of approach	16
1.5.1 The knowledge level	16
1.5.2 The symbol level	17
1.6 Declarative and procedural knowledge	18
1.7 Other aspects of knowledge representation	21
1.8 Bibliographic notes and historical remarks	22
2 Classical Logic	25
2.1 Propositions and arguments	26
2.2 Components of a logic	32
2.2.1 The deductive system	33
2.2.2 The semantics	34
2.2.3 Deductive system vs. semantics	34
2.3 Propositional logic	35
2.3.1 Deductive system	38
Theorems and derived rules of inference	49
New logical symbols	51
How do we construct proofs?	53

	Properties of the deductive system	56
2.3.2	Semantics	61
2.3.3	Soundness and completeness of propositional logic . .	64
2.4	First-order logic	65
2.4.1	Deductive system	73
2.4.2	Semantics	76
2.4.3	Soundness and completeness of first-order logic	84
2.5	Representation using logic	84
2.5.1	Kinship relations	85
2.5.2	World of airplanes	93
2.5.3	Blocks world	98
2.6	Summary	100
2.7	Bibliographic notes and historical remarks	101
2.8	Exercises	102
3	Non-Classical Logics	109
3.1	Modal logic	111
3.1.1	The language of propositional modal logic	112
3.1.2	Semantics for propositional modal logic	113
3.1.3	Properties of the accessibility relation	119
3.1.4	Axioms and readings of the modal operators	121
3.1.5	Deductive system for propositional modal logic	124
	Rules for normal modal logic	124
	Rules for other modal logics.	127
3.1.6	Soundness and completeness	128
3.1.7	First-order modal logic	128
	The language of first-order modal logic	128
	Remarks on first-order modal logic	129
3.2	Relevance logic	130
3.2.1	Deductive system	132
3.3	Summary	138
3.4	Bibliographic notes and historical remarks	138
3.5	Exercises	139
4	Non-Monotonic Formalisms	143
4.1	Basic principles	144
4.2	Default Logic	149
4.2.1	Deductive system	149
4.2.2	Semantics	159
4.3	Auto-epistemic Logic	169

4.4	Circumscription	172
4.5	Summary	174
4.6	Bibliographic notes and historical remarks	174
4.7	Exercises	177
5	Belief Revision	181
5.1	The need for belief revision	183
5.2	Justification-based systems	185
5.2.1	JTMS	188
5.2.2	Interaction between the JTMS and the problem solver	194
5.2.3	Removal of contradictions	196
5.3	Dependency networks	198
5.3.1	Computation of labels for nodes	199
5.4	Assumption-based systems	200
5.4.1	Computation of labels for nodes	201
5.4.2	Interaction between the ATMS and the problem solver	202
5.4.3	Removal of contradictions	203
5.5	Other approaches	203
5.5.1	Multiple Belief Reasoner	204
5.5.2	Logic-based TMS	206
5.6	Applications	207
5.7	Bibliographic notes and historical remarks	208
5.8	Exercises	210
6	Semantic Networks	213
6.1	Basic principles of semantic networks	218
6.2	SNePS: A case study	224
6.2.1	Arcs in SNePS	225
6.2.2	Nodes in SNePS	225
6.2.3	Knowledge representation in SNePS	229
6.2.4	Logical connectives in SNePS	238
6.2.5	Network representation of the connectives	248
6.2.6	Putting it all together	251
6.2.7	Inference in SNePS	254
6.2.8	Building and using SNePS networks	256
6.2.9	Examples	262
6.3	Other kinds of semantic networks	266
6.3.1	Conceptual graphs	266
6.3.2	Semantic primitives	268
6.4	Concluding remarks	271

6.5	Bibliographic notes and historical remarks	271
6.6	Exercises	273
7	Frames	277
7.1	Basic concepts	279
7.2	Reasoning with frames	284
7.2.1	Property inheritance	284
7.2.2	Procedural attachment	286
7.2.3	Rule-based reasoning	287
7.3	KEE: A case study	289
7.3.1	Basics	289
7.3.2	TELLANDASK	292
7.3.3	Reasoning with KEE	295
7.4	Formal semantic of frames	297
7.5	Scripts	301
7.6	Concluding remarks	302
7.7	Bibliographic notes and historical remarks	302
7.8	Exercises	304
8	Description Logics	307
8.1	Basic principles	309
8.1.1	TBox – The terminological component	310
8.1.2	ABox – The assertional component	315
8.2	Terminological formalism	316
8.2.1	Concept descriptions	318
8.2.2	Role descriptions	322
8.2.3	Knowledge bases — the TBox	324
8.2.4	Models and semantic structures	325
8.2.5	Terminological languages	325
8.3	Assertional formalism	326
8.3.1	Interpretations	327
8.4	Reasoning in description logics	328
8.4.1	Reasoning in the terminological component	328
8.4.2	Reasoning in the assertional component	333
8.4.3	Knowledge bases vs. Data bases	334
8.5	Implementations of description logics	335
8.5.1	The terminological language	335
8.5.2	The assertional language	338
8.6	Knowledge modeling in description logics	339
8.7	RACER: A case study	348

8.7.1	Building RACER knowledge bases	350
8.7.2	Evaluation functions	353
8.7.3	Queries	354
8.7.4	Retrieval	354
8.7.5	Example	356
8.8	Bibliographic notes and historical remarks	364
8.9	Exercises	365
9	Knowledge Sharing	371
9.1	KIF	372
9.1.1	KIF syntax	374
	Terms	375
	Sentences	375
	Definitions	376
9.1.2	KIF semantics	379
9.1.3	Knowledge bases in KIF	380
9.2	Knowledge Query Manipulation Language	381
9.3	Ontologies	382
9.3.1	Ontology languages	386
9.3.2	Vocabulary vs. taxonomy vs. ontology	387
9.4	Time ontology	388
9.4.1	Instants and intervals	390
9.4.2	Directionality of time	394
9.4.3	Interval relations	396
9.4.4	Convexity	401
9.4.5	Density	402
9.4.6	Concatenation of intervals	402
9.4.7	Aggregates of temporal entities	404
9.4.8	Temporal units	407
	Clock and calendar intervals	409
	Time standards	413
	Days	414
	Weeks	414
	Months and years	415
	Time stamps	417
9.4.9	Linking time and events	418
9.5	Bibliographic notes and historical remarks	420
	References	423

Glossary of notations

Notation	Description	Page
$\mapsto$	Function mapping	16
<i>tell</i>	Knowledge base addition	16, ??
<i>ask</i>	Knowledge base query	16, ??
$\{e_1, \dots, e_n\}$	Set with elements $e_1, \dots, e_n$	26
(Δ, α)	Argument	26
$\therefore$	Therefore	26
$\vdash$	Derivability	33, 43
$\models$	Logical consequence	34, 63
$\mathcal{L}_{PL}$	Language of propositional logic	36
$\neg$	Negation	36
$\wedge$	Conjunction	36
$\vee$	Disjunction	36
$\rightarrow$	Implication	36
P_i	Predicate symbol	36
$\cup$	Set union	37
Prem	Rule of premise	39
Rep	Rule of repetition	39
$\wedge I$	Rule of conjunction introduction	39
$\wedge E$	Rule of conjunction elimination	40
Hyp	Rule of hypothesis	41
Reit	Rule of reiteration	41
$\emptyset \vdash \alpha$	Theorem	43
$\vdash \alpha$	Theorem	43
$\emptyset$	Empty set	43
$\rightarrow I$	Rule of implication introduction	43
$\rightarrow E$	Rule of implication elimination	43
$\neg I$	Rule of negation introduction	45
$\neg E$	Rule of negation elimination	45

Notation	Description	Page
$\vee I$	Rule of disjunction introduction	46
$\vee E$	Rule of disjunction elimination	46
Theo	Derived rule of theorem	49
$\rightarrow E'$	Derived rule of implication elimination	50
MT	Modus tollens	51
$\neg\neg I$	Derived rule of double negation introduction	51
$\neg\vee I$	Derived rule of negated disjunction introduction	51
$\stackrel{\text{def}}{=}$	By definition	53
$\leftrightarrow$	Equivalence	52
$\leftrightarrow I$	Rule of equivalence introduction	53
$\leftrightarrow E$	Rule of equivalence elimination	53
■	Q.E.D. (end of proof)	57
$Th(\Delta)$	Theory generated from Δ	60
V	Valuation function	61
$\mathcal{L}_{FOL}$	Language of first-order logic	65
$\forall$	Universal quantification	68
$\exists$	Existential quantification	68
f_i^n	Function symbol (with n arguments)	68
P_i^n	Predicate symbol (with n arguments)	68
x_i	Individual variable	68
$P_i^n(t_1, t_2, \dots, t_n)$	Atomic <i>wff</i>	70
$\{x_1/t_1, \dots, x_n/t_n\}$	Substitution	72
$\cdot$	Application of substitution	72
$\forall I$	Rule of universal introduction	73
$\forall E$	Rule of universal elimination	74
$\exists I$	Rule of existential introduction	75
$\exists E$	Rule of existential elimination	75
(D, F, R)	Conceptualisation	78
D	Universe of discourse	78, 233
F	Set of functions	79
R	Set of relations	79
I	Interpretation function	80, 243
$\models_I \alpha$	α is true under the interpretation I	82
$\mathcal{L}_{PML}$	Language of propositional modal logic	28
$\square$	Necessary	29
$\diamond$	Possible	29
(W, R)	Kripke structure	30

Notation	Description	Page
(W, R, V)	Kripke model	31
$R(w_1, w_2)$	Accessibility relation between w_1 and w_2	30
$\models_M \alpha$	Model M satisfies $wff \alpha$	33
$M \models_w \alpha$	α is true in world w according to model M	33
$\mathcal{L}_{FOML}$	Language of first order modal logic	45
$\Rightarrow$	Entailment	48
$\mathcal{L}_{PRL}$	Language of propositional relevance logic	48
$\mathbb{N}$	Set of natural numbers	48
2^S	Set of sets of S	48
$\mathcal{L}_{PRL}^+$	Language of extended propositional relevance logic	48
$\frac{\alpha(x):\beta_1(x),\dots,\beta_m(x)}{\gamma(x)}$	General default rule	65
$\frac{\alpha:\beta_1,\dots,\beta_m}{\gamma}$	Closed default rule	66
$\frac{\alpha(x):\beta(x)}{\beta(x)}$	Normal default rule	66
(R, Δ)	Default theory	67
$\Gamma(\Omega)$	Extension generating operator	70
$\geq_r$	Preference relation introduced by default rule r	76
$\geq_R$	Preference relation introduced by set R	77
$Mod(\Delta)$	Set of models of Δ	78
T	Translation function in a TMS	105
$(n_1, \dots, n_m)$	Sequence	109
$\langle A_M \mid A_{Nm} \mid c \rangle$	General justification	114
ma	Set of monotonic antecedents	114
nma	Set of non-monotonic antecedents	114
$conc$	Conclusion of a justification	114
$\cap$	Set intersection	115
$\langle A_M \parallel c \rangle$	Monotonic justification	117
$\wedge \Rightarrow$	And-entailment	157
$\vee \Rightarrow$	Or-entailment	157
$\mathbb{X}_i^j$	And-or	157
$C_i^n(\alpha_1, \dots, \alpha_n)$	Ordered set of all the i -combinations of the elements of the set $\{\alpha_1, \dots, \alpha_n\}$	161

Notation	Description	Page
${}_k c_i^n(\alpha_1, \dots, \alpha_n)$	k -th element of $C_i^n(\alpha_1, \dots, \alpha_n)$	161
${}_k \bar{c}_i^n(\alpha_1, \dots, \alpha_n)$	$\{\alpha_1, \dots, \alpha_n\} \setminus {}_k c_i^n(\alpha_1, \dots, \alpha_n)$	161
$\setminus$	Set difference	161
$f\langle\{\alpha_1, \dots, \alpha_n\}\rangle$	The set $\{f(\alpha_1), \dots, f(\alpha_n)\}$	161
$\binom{n}{j}$	Number of combinations of n things j at time	161
E	Extension function	234
$[D, E]$	Semantic structure	234
$\top$	Universal concept	234
$\perp$	Inconsistent concept	234
$\sqcap$	Concept or role intersection	235, 238
$\sqcup$	Concept or role union	235, 238
$\forall R.C$	Value restriction	235
$\exists R.\top$	Limited existential quantification	236
$\exists R.C$	Full existential quantification	236
$\geq n R$	Number restriction	237
$\leq n R$	Number restriction	237
$\{i_1, \dots, i_n\}$	Concept enumeration	237
$\ S \ $	Cardinality of set S	237
U	Universal role	238
$R_1 \odot R_2$	Role composition	238
R^+	Transitive closure	239
R^-	Inverse role	239
$\equiv$	Concept or role equality	240
$\sqsubseteq$	Concept or role inclusion	240
$\mathcal{T}$	Terminology	241
$\models_{[D, E]} \mathcal{T}$	Model of a terminology	241
$\mathcal{AL}$	Attributive language	241
$\mathcal{ALUEN}$	Attributive language with union, full existential quantification, and number restrictions	242
O	Set of entities (individuals or objects)	243
C	Set of concepts	243
R	Set of roles	243
$[D, I]$	Description logic interpretation	243
$\models_{[D, I]} \Delta$	Model of a description	244
$\mathcal{T} \models_{[D, E]} C \sqsubseteq D$	D subsumes C in terminology $\mathcal{T}$	245

Preface

This book corresponds to a compilation of the notes that have been used in the course on knowledge representation offered by Instituto Superior Técnico, Technical University of Lisbon (Portugal). It should be considered as a rough draft of a forthcoming book on Knowledge Representation. Comments and suggestions about its contents will be most welcome.

Acknowledgments

I am grateful to Stuart C. Shapiro, Bill Rapaport, Maria R. Cravo, Ana Cardoso Cachopo, Helena Sofia Pinto, Pedro Oliveira, Pedro Lemos, Sérgio Costa, Paulo Filipe Andrade, Paulo Sebastião, Pedro Martins, Vanda Ribeiro, Adriano Andrade, Marcio Faustino, and Haythem Ismail for comments on earlier drafts of this book.

Chapter 1

What is Knowledge Representation?

Knowledge comes, but wisdom lingers.
Lord Alfred Tennyson, *Locksley Hall*

Computer programs are based on a model of the world they deal with. Models are ubiquitous not only in computer programs but also in human thinking and language. When we program, we write down an explicit description of a model, encoding, in an appropriate language, the rules, facts, and data thought to be relevant to the behaviour of the program. The program works by following these rules and manipulating these representations. In fact, computers differ from other machines because they run by manipulating representations, which are always formulated in terms of models. This can be summarised in the slogan *no computation without representation*.¹

This book is about models and their representation using relevant languages. However, our aim is far more ambitious than just writing computer programs; ultimately, we are aiming at being able to provide a computer with declarations about a given domain or state of affairs and let the computer use these declarations to reflect intelligent behaviour regarding the domain or state of affairs. We are well aware of the distance that we are from this ultimate goal, but the progress that has been made over the last half cen-

¹[Smith 85, p. 20].

ture leads us to believe that we are approaching this goal. In this book we address the problem of how to represent, inside a computer, what we know, believe, or assume, and how that computer can use whatever we have written to guide its behaviour. This is all part of the exciting field of Artificial Intelligence.

Artificial Intelligence (AI) is the branch of Computer Science that is engaged in the development of programs that behave intelligently — the kind of behaviour found in humans. Since this behaviour is largely dictated by the knowledge that the agent that exhibits it contains, it is just natural that an important area of AI addresses the problem of how to represent and to use knowledge. In this book, we are focused on the influence of knowledge in intelligent behaviour.

The actual development of AI is largely due to the concept of a computer system capable of using a portion of explicitly represented knowledge to reason about its goals, about its environment, about other agents, and even about itself.

There are two main reasons that justify the importance of knowledge representation in AI:

- When we consider the way humans perform tasks such as playing chess, planning activities, or solving mathematical problems, we usually bear in mind *what they know* in order to perform that task — knowledge is associated with intelligent behaviour;
- For several years, it has been recognised that in order to obtain intelligent behaviour in a program, it is more important *to have large amounts of knowledge* than to have sophisticated algorithms.

The idea behind knowledge representation is very simple. It consists of writing down, using a specific language (the knowledge representation language), sentences or figures that correspond to the description of a world or the state of a world. When a computer manipulates this representation, it reaches conclusions or predictions about the world or its state.

Knowledge representation has links to many other scientific areas besides artificial intelligence, psychology, neurology, linguistics, philosophy, logic, and computer science:

- with *psychology*, it shares an interest in the study of models of memory

and reasoning;

- with *neurology*, it shares an interest in physiological models of the human nervous system;
- with *linguistics*, it shares an interest in the syntax, semantics, and pragmatics of language;
- with *philosophy*, it shares an interest in the nature of concepts and propositions;
- with *logic*, it shares an interest in deductive reasoning; and
- with *computer science*, it shares an interest in information systems and programming languages.

When we consider the area of knowledge representation there are several questions that arise and whose answers we address in this first chapter: What is knowledge? How does knowledge influence intelligent behaviour? Are there assumptions that support the work on knowledge representation? At what level should we consider the analysis of knowledge? What are the main concerns of knowledge representation?

Before we address these questions, we take a deeper look into the notions of models and representations and discuss the general architecture of a program based on knowledge.

1.1 World, models, and representations

Most computer programs address the solution of a problem in the real-world (the world with which we interact in our daily life). The real world has an infinite richness and complexity of which we only perceive part through our senses (sight, hearing, smell, taste, and touch). The real world is far too complex and detailed to be fully perceived or understood by an agent.

In order to handle the richness of the world, humans have dealt with models for centuries. A *model* corresponds to an abstraction of the world that only captures its relevant aspects regarding a certain problem or a certain task. An abstraction introduces simplification that removes irrelevant aspects. This perspective is used by engineers with approximate models (such as points, masses, frictionless surfaces) that disregard unnecessary aspects of

the world. Engineers have dealt with models to predict the behaviour of real world objects: scale models of aeroplanes were used to study lift and air friction; scale models of bridges and buildings have been used to study the effects of forces, and so on. In the second half of the 20th century, physical models were gradually replaced by computer models and programs that mimic the behaviour of the real-world entity under consideration. In Mathematics, models are used, for example, to compute areas and volumes, disregarding all small physical imperfections that occur in real-world objects. Mathematical models are so widely used that it is hard to think of anything that they haven't been used for — from social and economic systems, to genetic structures, to the mass and charge of sub-atomic particles.²

What is important to keep in mind, is that every model deals with its subject matter at some particular level of abstraction, paying attention to certain details, throwing away others, grouping together similar aspects into common categories, and so on. Sometimes what is ignored is at a too low level, sometimes at a too high level, it all depends on the purposes for which the model is being used.

Models have to ignore things because they view the world at a level of abstraction, otherwise they would drown in the infinite richness of the embedding world. Every model does a certain amount of “violence” to its subject matter, in order to get at the underlying regularities that group things together. If we do not commit this act of violence — if we do not ignore some of the things that exist in the world — we would become so overcome with complexity that we would be unable to think or to act.

For this reason, models are inherently partial. All thinking, and all computation, are similarly partial. Furthermore thinking and computation *have to be* partial, otherwise they would not be able to work.

The partial agreement of models with the real world is balanced by the ease and efficiency of its use. Their usefulness corresponds to their ability to predict the behaviour of the world — by using models, we can find out what the state of the real world will be (with respect to those aspects existing in the model) after the execution of a certain number of actions, without the need to perform those actions in the real world.

When we design a computer system, we have first to develop a model of the world. But we do not, in general, ever get to hold the model in our hand —

²Mathematics is unique, with respect to models, because its subject matter is the world of models and abstract structures.

computer systems are based on models that are purely abstract. Rather, we develop two concrete things, structured in terms of the abstract underlying model:

- A *specification*, a formal description in some formal language, formulated in terms of the model, in which the desired behaviour is described.
- The *program*, a set of instructions and representations, also formulated in the terms of the model, which the computer uses as the basis for its actions.

The program has to say how the behaviour is to be achieved, typically in a step by step fashion. The specification is less constrained — all it has to do is to specify what the proper behaviour should be, independent of how it is accomplished. Specifications are essentially declarative — they are indicative sentences or claims. Programs, on the other hand, are procedural — they must contain instructions that lead to a certain sequence of actions.

The solutions for most of the problems that we face in our daily life depend, almost exclusively, on our knowledge about the application domain. For example, the success of medical diagnosis depends on the knowledge that the doctor has about different illnesses and their symptoms, the types of medication to use and their side effects, and even about the psychology of the patient. This knowledge is contained in a mental model that the doctor has about a variety of things, the patient, the drug, the absorption rate, the desired balance between therapy and toxicity, and so on. The model of the absorption rate, for example, will consider the patient's weight, age, and sex but will not pay attention to the patients' tastes in music. In order to reach our goal — to build programs that behave based on knowledge — we have to explicitly *represent* this mental model in a computer program. It is this representation that the computer will manipulate in order to reach its conclusions — the computer will neither manipulate the model nor the real-world.

Following this discussion, in this book, we will be concerned about three types of entities (Figure 1.1³):

³The images in this figure, from left to right, were obtained in the following internet pages, digitaldisplay.com/specs/, www.avenza.com/products.examples.html, and www.kestrel.edu/home/projects/vizo/.

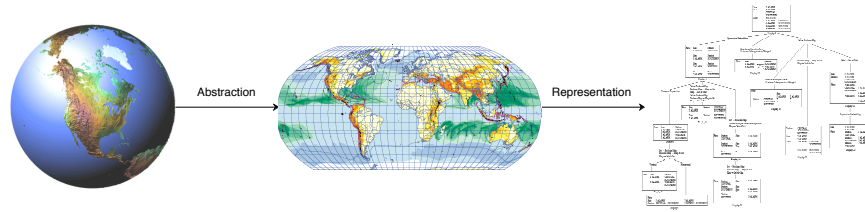


Figure 1.1: Relationship between the world, model, and representation.

- The *world*. Something that has an infinite richness and complexity of which we only perceive part.
- A *model* of the world. An abstraction of the world that only captures its relevant aspects regarding a certain problem or a certain task.
- The *representation* of the model of the word. A set of rules and statements that express the model using a certain language.

1.2 Cognitive agents

Our ultimate goal is building programs that take their actions based on a representation of knowledge about a given situation – these programs are known as *cognitive agents* or *knowledge-based agents*.

In the most general case, cognitive agents act upon the surrounding world, thus, they have to “sense” the world and “act” upon the surrounding world. The general architecture of a cognitive agent is shown in Figure 1.2:

- The agent has a set of *sensors* that enable the agent to perceive the word that surrounds it.
- The agent has a set of *actuators* that enable the agent to act upon this world.
- The results of the sensors are processed and provided to a general *processing machinery* that is the responsible for the agent’s actions in the outside world. We assume that the processing machinery works based on the *knowledge* that the agent has about the outside world

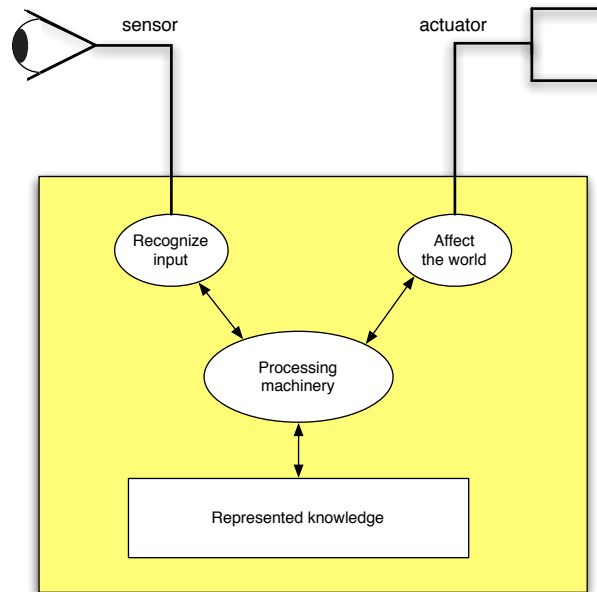


Figure 1.2: General architecture of a cognitive agent.

and about itself. This knowledge is represented in a component that is used by the processing machinery.

A more detailed view on the relation between the processing machinery and the represented knowledge considers that there are two main components:

1. The *knowledge base*, a component that contains a representation of a model of the real world.
2. The *inference engine*, a set of general-purpose procedures that manipulate the knowledge base and are independent of the kind of knowledge that is represented in the knowledge base. The inference engine queries the knowledge base for knowledge that is explicitly stored and uses inference to provide knowledge that was implicit in the knowledge base (Figure 1.3).

From this architecture, it should be clear that in a formalism for knowledge representation we have to consider two aspects, the notational aspect and the inferential aspect.

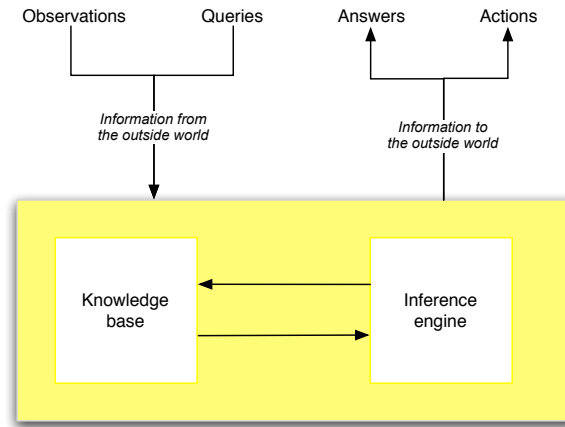


Figure 1.3: Inference engine and knowledge base.

1. The *notational aspect* is concerned with how knowledge is represented in the knowledge base. In particular, it is concerned with the development of a precise formal language that specifies what basic structures are used to represent knowledge and how these basic structures can be combined.

The choice of a formalism, according to the notational aspect, depends on the ease of representation of important aspects to be modelled by the language, on the expressive power of the language, and on the possibility of supplying an interpretation for the sentences of the language.

2. The *inferential aspect* is concerned with how to produce inferences from the static structures of the language (contained in the knowledge base), that is, how to make explicit the knowledge that is implicit. It is thus concerned with the choice of inference rules to be included in the representation and the choice of what kind of control to perform on the inference.

The behaviour of the system is dictated by the use of the knowledge in the knowledge base by the inference engine.

A *knowledge representation system* can be defined as a collection of data structures (the knowledge base) and procedures (the inference engine) that manipulate these data structures. When the procedures and the data struc-

tures are used in a proper way, the program that uses them reflects intelligent behaviour. We should note that data structures are knowledge in the same sense that an encyclopaedia is knowledge — it is a source of knowledge, but, without a reader, it is only ink on paper. Knowledge representation is closely related to reasoning, since it needs to make explicit things that are implicit.

In this sense, the knowledge base corresponds to an internal representation of a model of the outside world. This internal representation is updated with the results of perception, and is used (along with reasoning) to compute the action to be taken and to predict the effect that the action will have on the external world. In case the action is carried out in the world, the predicted results are compared with the actual results.

The following corresponds to a well accepted characterisation for the behaviour of a cognitive agent:⁴

1. The agent has knowledge about the surrounding environment and about the way this environment is affected by its actions.
2. The agent has a certain number of goals, which also correspond to knowledge.
3. The actions performed by the agent are based on its knowledge and are executed in order to achieve some of its goals.

Under this perspective, the activity of a cognitive agent mainly consists of: *perceiving*, through its sensing devices, the world that surrounds it; *reasoning* about that world; and *making decisions* about what actions to execute. Actions are dictated by the knowledge of the agent, by its goals, and by the current perception of the outside world.

A cognitive agent is, thus, involved in an infinite loop of perceiving, reasoning, and acting (Figure 1.4).

The acting cycle requires the agent to acquire an internal representation that corresponds to a model of the external world, which should be kept up to date with the current state of the world.⁵

⁴Note that this description applies both to computing systems and to human beings.

⁵Typically, an AI program receives the information corresponding to sensors through the keyboard and the actuators correspond to commands printed on the screen. The question of whether this way of interacting with the outside world has an influence on the intelligent capabilities of the system is discussed in [Kirsh 91].

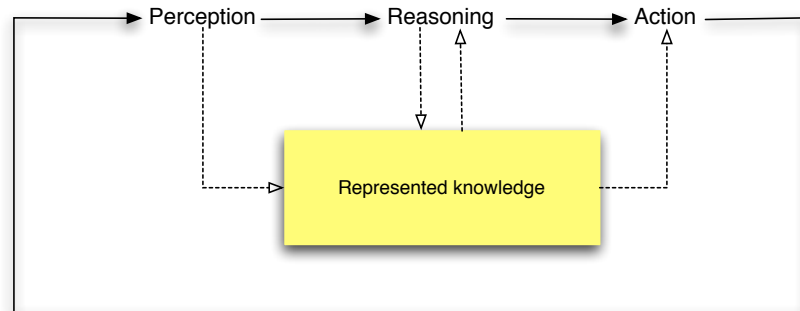


Figure 1.4: Acting cycle of a cognitive agent.

The knowledge represented in the program evolves through three types of operations:

- Operations that rely on a set of sensors that inspect the outside world (note that these sensors can only identify the characteristics of the real world that exist in the abstraction) and modify the knowledge in the system.
- Operations that correspond to reasoning procedures, through which the program reflects upon its knowledge and either modifies its knowledge or decides to act upon the world.
- Operations that rely on a set of actuators that, based on knowledge in the knowledge base, act upon the outside world (again, these actuators can only act upon the characteristics of the real world that exist in the abstraction).

We said that models are inherently partial. However, action is not partial. When we reach out our hand and move something, it is the real world that we are affecting, not our model of it. Models, thinking, and some aspects of computation are abstract, but action is not. This difference between action and models is very important. Even if our thought is formulated in terms of a model, to act is to take leave of the model and participate in the whole, rich, infinitely variegated world. For this reason, action plays a crucial role, in grounding the abstract process of modelling. One form that grounding can take, is to provide feedback on how well the modelling is going. The world doesn't care about the model. Feedback is a special case of a very

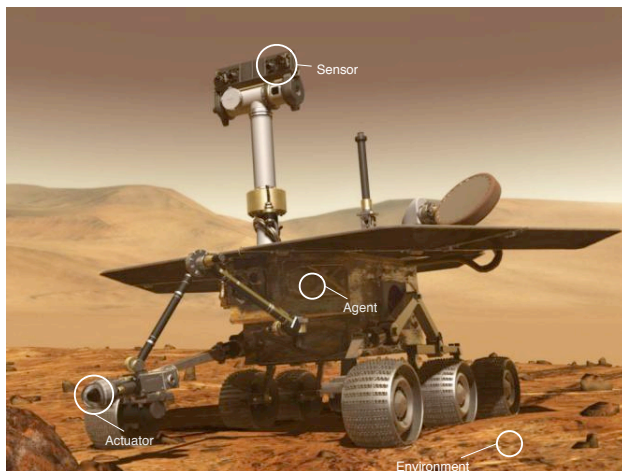


Figure 1.5: An agent and its environment.

general phenomenon: we often learn, when we act, just how good or bad our model was. We learn to assess the sensed experience, the inner resources to revise and re-conceptualise, and to recover from minor mistakes and failures.

Another, more realistic, view of a cognitive agent is shown in 1.5.⁶

1.3 What is knowledge?

The attempt to define what knowledge is is very old. Both Plato (427–347 B.C.) and Aristotle (384–322 B.C.) addressed this question, and there is a long philosophic tradition of discussions about knowledge-related questions: what is knowledge? what is its nature and origin? how can we classify it? and what connections does it have with the notion of truth?

When we try to define the meaning of “having knowledge”, we should analyse, among others, the meanings of *knowledge* and of the verb *to know*. If we look in a dictionary, we find:⁷

know *v.* *ℰ* *n.* –*v.* **1** *tr.* (often foll. by *that*, *how*, *what*, etc.) **a** have in

⁶This image was obtained from <http://photojournal.jpl.nasa.gov/catalog/PIA04240>.
Courtesy NASA/JPL-Caltech.

⁷[Hawkins and Allen 81].

the mind; have learn; be able to recall (*knows a lot about cars; knows what to do*). **b** (also *absol.*) be aware of (a fact) (*he knows I am waiting; I think he knows*). **c** have a good command of (a subject or language) (*knew German; knows his tables*). **2 tr.** be acquainted or friendly with (a person or thing). **3 tr.** **a** recognise; identify (*I knew him at once; knew him for an American*). **b** (foll. by *to* + infin.) be aware of (a person or thing) as being or doing what is specified (*knew them to be rogues*). **c** (foll. by *from*) be able to distinguish (one from another) (*did not know him from Adam*). **4 tr.** be subject to (*her joy knew no bounds*). **5 tr.** have personal experience of (fear, etc.). **6 tr.** (as **known** *adj.*) **a** publicly acknowledge (*a known thief; a known fact*). **b Math.** (of a quantity etc.) having a value that can be stated. **7 intr.** have understanding or knowledge. **8 tr.** *archaic* have sexual intercourse with. *-n.* (in phr. **in the know**) *colloq.* well-informed; having special knowledge.

knowledge *n.* **1 a** (usu. foll. by *of*) awareness or familiarity gained by experience (of a person, a fact, or thing) (*have no knowledge of that*). **b** a person's range of information (*is not within his knowledge*). **2 a** (usu. foll. by *of*) a theoretical or practical understanding of a subject, language, etc. (*has a good knowledge of Greek*). **b** the sum of what is known (*every branch of knowledge*). **3 Philos.** true, justified belief; certain understanding, as opp. to opinion. **4** = *carnal knowledge*.

As can be seen, *know* and *knowledge* may have many different meanings. In order to understand the differences between these meanings, let us consider the following sentences:⁸

1. I know how to go from Buffalo to Toronto.
2. I know how to type.
3. I know the expansion of π to four decimal places.
4. I know Bill.
5. I know Hamlet.
6. I know Denmark.

⁸Adapted from [Lehrer 80].

7. I know that the area of a rectangle is given by the product of its length by its height.

8. I know that you are telling the truth.

One of the meanings of “knowing” (illustrated by sentences 1, 2, and 3) corresponds to *having a certain kind of competence*. Under this meaning, we state that we know how to go from one place to another, we are conveying competence about being able to move ourselves or to explain to someone how to do it. When we state that we know how to use a device, we are stating our competence about its use. When we state that we know something (by heart), we are stating that we have the competence to recall and convey that information.

Another meaning of “knowing” (illustrated by sentences 4, 5, and 6) corresponds to *being acquainted with someone or with something*. The meaning of sentence 6 is harder to make precise, since it may mean that I have been to Denmark or that I am able to move around geographically or socially. This sentence may correspond to two different senses of the verb “to know”, competence and acquaintance.

A third meaning of “knowing” (represented by sentences 7 and 8) corresponds to *recognising something as information*. This is the meaning that is more interesting to us. Usually, knowing something according to one of the other meanings also entails having knowledge according to this meaning of the verb. In this book, we will mainly consider knowledge under this interpretation, since it is the most closely connected with cognition.

In this book, we will be using *knowledge* as a competence-like notion, being a potential for generating action. Our notion of knowledge is intimately linked with rationality.

In philosophy, there is a distinction between the words *knowledge* and *belief*, with knowledge meaning justified belief. In AI, we follow a more permissive approach, and we associate the word “knowledge” with both these senses. So, roughly we can say that, in AI, knowledge is a collection of facts and relations among them such that, when these are interpreted in an adequate way, they give rise to intelligent or rational behaviour. The amount of knowledge that an agent (either natural or artificial) possesses can be measured by the variety of situations in which the agent is able to achieve successful results.

1.4 Underlying hypotheses

Before we start addressing the representation of knowledge, we should look into its underlying assumptions. These assumptions correspond to the two hypotheses discussed in this section.

1.4.1 Physical symbol system hypothesis

A physical symbol system is composed of a set of entities, called *symbols*, that correspond to physical patterns. Symbols can be grouped into symbol structures or *expressions*. At any moment, the physical symbol system contains a collection of these symbol structures. Besides the symbol structures, the physical symbol system has a collection of procedures that operate on symbol structures, producing symbol structures (these procedures create, modify, reproduce, and destroy symbol structures).

A *physical symbol system* is a machine that, over time, produces a succession of these symbol structures. The system exists in a world containing other objects besides the symbol structures. This description is so general that it can be applied to any computer program. In fact, the description that was provided may correspond to any model of universal computation. It provides

- memory to contain symbol information;
- symbols to provide a pattern to match or index information;
- operations to manipulate symbols;
- composability (the operators may make any symbol structure).

The *physical symbol system hypothesis* states that a physical symbol system is a necessary and sufficient condition for intelligent behaviour. By *necessary* condition, we mean that any system that shows intelligent behaviour can be viewed as a physical symbol system; by *sufficient* condition, we mean that any physical symbol system of adequate size can be organised so that it exhibits intelligent behaviour; finally, by intelligent behaviour we mean the kind of behaviour found in humans.

The importance of this hypothesis to AI stems from the fact that it states that *it is possible* to write a computer program (nothing more than a physical

symbol system) that exhibits intelligent behaviour. This hypothesis makes no commitment about what the “knowledge” of this program is nor to the way the knowledge is represented.

The physical symbol system hypothesis means that we will be interested in:

1. finding symbol patterns that correspond to important aspects of the domain being considered — the knowledge representation part;
2. finding the operations that manipulate these symbol structures to produce potential solutions to our problems — the reasoning part.

1.4.2 Knowledge representation hypothesis

Over the years, there have been many disputes among researchers in knowledge representation. There is, however, some agreement about fundamental questions. One of the points of agreement is called the *knowledge representation hypothesis*:

1. Any mechanical process that exhibits intelligent behaviour is composed of structures that are assigned, by an external observer, to a propositional representation of the knowledge used by that process.
2. Independently of the meaning that external agents may assign to these propositional structures, they play a formal but causal and essential role in the behaviour that manifests that knowledge.

The fact that these structures are propositional in nature and could be graphs, trees, arrays, records, or any other data structure, means that this hypothesis spans a large number of knowledge representation formalisms (thus the reason for agreement!). These structures are stored in the *knowledge base* component. The second part of this hypothesis states that it is the represented knowledge, independently of how we interpret it, that generates the behaviour of the system.

The knowledge representation hypothesis complements the physical symbol system hypothesis. Whereas the physical symbol system hypothesis states that a physical symbol system is capable of generating intelligent behaviour, the knowledge representation hypothesis assigns that behaviour to the structures that represent the knowledge of the system.

In order to enable the existence of an explicit knowledge base, there should be a well-defined language that will be used to represent the knowledge of the system, the *knowledge representation language*. The system must also have a set of procedures, the *inference engine*, that enable the generation of information that is implicit in the knowledge base.

The knowledge about the application domain contains the necessary power to create intelligent behaviour. This body of knowledge is explicitly represented in the knowledge base, using the knowledge representation language, and it is what is responsible for intelligent behaviour.

1.5 Levels of approach

Knowledge representation can be approached at two levels, known as the knowledge level and the symbol level. This distinction is similar to the distinction between the definition of an abstract data type and the choice of a representation for the elements of that type.

1.5.1 The knowledge level

At the *knowledge level*, we look at the knowledge base as an abstract data type that can be accessed and manipulated through a small number of operations, sometimes known as the *services* that the knowledge-based system can provide. These operations include a constructor, called *tell*, and a selector, called *ask*.

- The operation *tell* adds knowledge to the knowledge base:

$$tell : knowledge \times statement \mapsto knowledge$$

- The operation *ask* takes a question (a sentence that may contain variables) and uses reasoning to check whether the information in the query is implicit in the knowledge base:

$$ask : knowledge \times question \mapsto answer$$

Clearly, these two operations are not enough for a complete approach at the knowledge level. Among others, we will need operations to create an empty

knowledge base, an operation to simulate forgetting, an operation to remove contradictions, and so on. However, *tell* and *ask* correspond to the main interaction between a program (or user) and a knowledge base.

At this level, we are not concerned with the structures nor with the mechanisms that will be used to maintain the knowledge base. The capability of the knowledge base is measured in terms of the functions that use it — the statements that it is able to accept and the answers that it is capable of producing. We look at knowledge from *outside* of the system and we are only concerned with what is true in the world according to this knowledge.

One of the main characteristics of knowledge is the notion of competence. This notion is applicable to an agent whose behaviour is dictated by the knowledge in the knowledge base and can be considered as an abstraction (or heuristic) for reasoning about the beliefs of that agent. The notion of *competence* can be formalised in the following way: if the agent imagines the world in such a way that a given statement, say α , is true, and if that statement implies another statement, say, β , then (disregarding whether or not the agent is aware of it), the agent imagines the world in such a way that β is also true. In this way, the knowledge level analysis corresponds to the study of what is implicit in the beliefs of the agent.

The analysis at the knowledge level enables to establish the theoretical foundations that enable an evaluation and comparison of the different knowledge representation formalisms. This is done because it concentrates on the services supplied by a knowledge representation system, independently of how those services are accomplished.

1.5.2 The symbol level

In computer science, a good solution depends, most of the time, on a good representation. It is therefore important to have an adequate representation for the knowledge and the reasoning components of a cognitive agent. Researchers in AI soon recognised that the primitives offered by a knowledge representation system, together with the mechanisms available to manipulate this knowledge, strongly affect the system's capabilities of perceiving, knowing, and acting.

The symbol level looks at the task of knowledge representation under the perspective of a specific representational formalism. When we approach representation at the symbol level, we are interested in defining symbol struc-

tures that enable the representation of knowledge and in defining procedures that perform inference based on those structures. We look at knowledge from *within* the system and we are concerned with how the system knows what it knows.

Most work in knowledge representation has been done at this level, and most of this book will be devoted to knowledge representation at the symbol level. In particular, we study four main formalisms for knowledge representation: semantic networks, frames, terminological logics, and representations based on logic.

1.6 Declarative and procedural knowledge

There are two distinct ways of representing knowledge: using declarations (the declarative representation) or using procedures (the procedural representation). For many years, researchers have argued which of these ways should be adopted to represent knowledge. This dispute was known as the “declarative vs. procedural controversy”. The dispute corresponds to the philosophic distinction between *knowing that* and *knowing how*.

The procedural position was that knowing is “knowing how” and, therefore, that knowledge should be represented as a set of procedures, each one representing a fragment of knowledge.

The declarative position was that knowing is “knowing that” and, therefore, that knowledge should be represented as a set of statements, describing a particular domain. These statements will, in turn, be manipulated by a general-purpose procedure that will be responsible for the inference of new statements.

From the formal point of view, there is no distinction between these two positions. We know that programs may be looked at as data, and, according to this perspective, everything is declarative. On the other hand, we may look at declarations as statements for a program; under this perspective, everything is procedural.

The controversy is not concerned with the discussion of whether a given piece of knowledge corresponds to a statement or to a program but rather in finding out what we gain if we look at it in one way or another. Clearly, each one of these types of representation has advantages and disadvantages. The advantages of one representation are disadvantages in the other, and

vice versa.

The first advantage that can be pointed out for the declarative position corresponds to the *flexibility* and *economy* of the representation. The procedural position requires that, when we specify knowledge, we have to state how it is going to be used. Usually, there is more than one way of using that knowledge, and it is not reasonable that every possible way of using that knowledge should be specified beforehand.

Consider, for example, the statement “All lawyers from Chicago are rich”.⁹ Using a procedural representation, this statement could be represented as:

```
(To-prove-that
  (Rich ?X)
  (Prove-both (Profession ?X Lawyer) (Livesin ?X Chicago)))
```

This representation should be read in the following way: if we want to find out whether *X* is rich, then we will have to simultaneously verify that *X* is a lawyer and that *X* lives in Chicago.

Using a declarative representation, for example, first-order logic, this piece of knowledge could be represented by the formula:

$$\forall(x)[(Profession(x, Lawyer) \wedge LivesIn(x, Chicago)) \rightarrow Rich(x)].$$

This statement could be used, for example, to prove that:¹⁰

1. David is rich, by showing that he is a lawyer and that he lives in Chicago;
2. Stuart does not live in Chicago, knowing that he is a lawyer and that he is not rich.

In a procedural representation, the same fragment of knowledge would have to be represented in different ways in order to enable each one of these uses. Under this perspective, the declarative representation is more flexible and economical than the procedural representation, since the same “representation” can be used in different ways.

The second advantage for the declarative representation concerns the *understanding* of the representation and the possibility of *learning*. In fact,

⁹This example is due to [Winograd 75].

¹⁰Depending on what inference procedure is available.

the declarative representation is preferable in respect to the possibility of understanding what is represented and to the possibility of modifying that knowledge.

Since the declarative representation means that knowledge is represented as a set of statements, the addition, replacement, or removal of knowledge is obtained through the addition, replacement or removal of statements. For this reason, learning is easier in a declarative representation. In a procedural representation, a piece of knowledge may be embedded in a complex procedure; therefore, the addition, replacement, or removal of knowledge is obtained through the modification of that procedure. Everyone that has ever programmed knows how the change of a procedure may have unforeseen consequences in other parts of the program.

The final advantage of the declarative position corresponds to *ease of access* and *ease of communication* of the representation. Most of the things that we know are expressed as declarations: natural language is mainly declarative and corresponds to the easiest way to convey information. This aspect has a direct consequence for the addition of knowledge to a program and to the communication of its contents to others.

The main advantage of procedural representation is related to *procedural modelling*. Many things that we know can be looked at as procedures, and it is difficult to describe them through declarations. Consider, for example, the addition of two integer numbers. Without a procedure to compute the addition, we would need a list of every pair of integers and their corresponding sum. In this case, we replace flexibility by an enormous gain in computation.

Another advantage of procedural representation is related to the use of *meta-knowledge*. Certain aspects of knowledge about knowledge are more easily expressed by procedures rather than by declarations. For example, the nearness relation is transitive, provided that it is not applied many times in the same deduction.

Finally, procedural representation has advantages for *computational efficiency*. Using specialised procedures gives rise to more efficient programs than the use of general-purpose procedures.

In the late 1970s, people reached the conclusion that both kinds of representation were important. Most knowledge used in current AI programs is declarative, and in most representations, there is the possibility of calling

specialised procedures, when the use of procedures is required. This possibility of calling procedures in a declarative representation is called *procedural attachment*.

1.7 Other aspects of knowledge representation

The underlying theme of knowledge representation is how to include knowledge about the world in a computing system so that, given the required reasoning capabilities, the system is able to use that knowledge to adapt itself to deal with that world. The main problems that knowledge representation faces are the following:

1. In what form should knowledge be represented? This problem deals with the study of structures to represent the different kinds of knowledge.
2. How can the reasoning mechanism use a limited amount of knowledge (explicit knowledge) to produce the remaining knowledge that is necessary for the task (implicit knowledge)?
3. How may knowledge (both implicit and explicit knowledge) be used to guide the behaviour of the system?
4. What can the system do in the face of incomplete or wrong information?
5. How can the system reason in the face of problems that are computationally difficult or undecidable?

Knowledge representation has focused on different ways to express (or represent) information, ranging from informal models of human memory to complex formal languages. Different systems may be more suited to solve certain kinds of problems. However, the main effort in knowledge representation has been focused on the development of general-purpose languages for knowledge representation.

Since the development of a representation is of no use without considering the reasoning that can be performed with that representation, knowledge representation is actually taken as synonymous with *knowledge representation and reasoning*. For this reason, knowledge representation is grounded

in the study of logics, in which the study of the language is done together with the study of inference rules and interpretations. This supplies a standard for measuring the performance of a computing system for knowledge representation.

The main areas that knowledge representation addresses are:

1. *Representation of knowledge.* This requires generation of theories about particular domains. Recently, work has been done towards the development of generic ontologies, axiomatizations that span several domains, development of re-usable knowledge bases, and aspects associated with knowledge elicitation and knowledge engineering.
2. *Foundational mathematics* that corresponds to the development of complex logics and methods for comparing them.
3. *Cognitive science*, the study of the structure of human thought and its parameters.
4. *Reasoning*, the study of the different forms of commonsense reasoning.
5. *Technology*, the development of computing systems to support the needs of the fields of AI.

1.8 Bibliographic notes and historical remarks

Important reference works in the field of knowledge representation include [Bobrow and Collins 75], [Brachman and Smith 80], [Cercone and McCalla 87], and [Davis and Morgenstern 04]. [Brachman and Levesque 85] presents a collection of the most influential papers published in knowledge representation prior to 1984, as well as an annotated reference list of papers in this area. [Brachman 90] presents a perspective of the main problems addressed in the field of knowledge representation as well as a discussion of the historical evolution of the field. [Winograd 75] presents an excellent discussion of the declarative vs. procedural controversy. The discussion presented in Section 1.6 was based on this paper.

The proceedings of the conference on the principles of knowledge representation and reasoning [Brachman, Levesque, and Reiter 89], [Allen, Fikes, and Sandewall 91], [Nebel, Rich, and Swartout 92], [Doyle, Sandewall, and Torasso 94], [Aiello, Doyle, and Shapiro 96], [Cohn, Schubert, and Shapiro

98], [Cohn, Giunchiglia, and Selman 00], [Fensel, Giunchiglia, McGuinness, and Williams 02], and [Dubois, Welty, and Williams 04] present an excellent perspective on the problems facing researchers of knowledge representation.

The characterization of the behaviour of a cognitive agents is mainly due to Allen Newell (1927–1992) [Newell 81]. Also due to Allen Newell, is the distinction between the knowledge level and the symbol level [Newell 81].

The idea of having a knowledge base separated from the inference engine is due to [McCarthy 68]. McCarthy proposed a system called ADVICE-TAKER that would decide how to act by using formal reasoning procedures being applied over a body of explicitly represented knowledge. The system would not need to be programmed for specific tasks but it would be told what it needed to know through information that was added to the knowledge base and was expected to infer non-explicitly added knowledge using the formal reasoning procedure.

The first physical symbol system hypothesis, was stated by [Newell and Simon 76] and is discussed in detail in [Newell 81].

The knowledge representation hypothesis was formulated by Brian Cantwell Smith [Smith 82].

Although most of the approaches to knowledge representation are based on the hypotheses presented in Section 1.4, there are approaches, namely Brook’s anti-representationalism [Brooks 86, 91], that work based on a architecture, the *the subsumption architecture* (also called “Brooksian architecture”) that is based on the synergy between sensation and actuation in lower animals such as insects. Brooks argues that instead of building complex agents in simple worlds, we should follow the evolutionary path and start building simple agents in the real, complex and unpredictable world. The *subsumption architecture* is built in layers. Each layer gives the system a set of pre-wired behaviours. The higher levels build upon the lower levels to create more complex behaviours. The behaviour of the system as a whole is the result of many interacting simple behaviours. The layers operate asynchronously. The layers of the subsumption architecture are composed of networks of *finite state machines* augmented with timers. The timers enable state changes after pre-programmed periods of time.

A distinguishing characteristic of the subsumption architectures is its *lack of a world-model*. Brooks sees this as an advantage. By not having a representation, the architecture avoids the time necessary to read and write

them, the time-cost of algorithms that utilise them, and the difficulty of maintaining the accuracy of world-models. These are all problems that are major issues in any system with a world model which also attempts to operate in complex and dynamic environments. However, this approach is a double-edged sword. While it is true that such systems are fast, they also depend entirely on the world to tell them what to do — they are purely reflexive.

Chapter 2

Classical Logic

Logic is invincible because in order to
combat logic it is necessary to use logic
Pierre Boutroux [Kline 72]

One of the characteristics of rational agents is the need to justify the things they believe and the need to understand the way other agents justify their beliefs. One of the main goals of logic, as a discipline, is to start from a situation described by a set of sentences, that are assumed to be true, and to find out what other sentences have to be true in that situation — logic addresses the study of the notions of deductive consequence and of consistency. Logic has been used in Computer Science in general, and in Artificial Intelligence in particular, to develop languages to model situations we encounter and to provide mechanisms to reason about these situations in such a way that we can guarantee that results produced are the right ones.

For the reasons above, the discipline of logic has a fundamental role in knowledge representation and reasoning. In this chapter, we look into some basic aspects of logic, presenting what is known as *classical logic*.

Although it is not easy to define in a few sentences what logic is, since it addresses a large number of problems and does not have well-defined borders — at one end of the spectrum, it addresses philosophical problems, and, at the other end, it addresses mathematical problems —, for the sake of this chapter, we can consider that logic addresses the analysis of arguments, or the analysis of the methods to distinguish valid from invalid arguments.

The logic presented in this chapter is just the starting point of our work. In the course of this book we consider other aspects of logic, whose languages correspond either to subsets or to supersets of the language presented here.

We start this chapter with a general presentation of logic-related concepts. These may strike you at first as not being related to knowledge representation, however this connection becomes quite clear when we reach Section 2.5.

2.1 Propositions and arguments

For the sake of this chapter, we consider that the role of logic is the analysis of sound reasoning, starting from a situation described by a certain number of declarative sentences, that are assumed to be true, and to find out what other sentences have to be true in that situation. By *declarative sentences* we mean sentences that make assertions of statements about something, such as “Socrates is a man”, “All birds have feathers”, or “The moon is a star”; we explicitly leave out sentences such as “What is knowledge representation?” and “Provide and answer to Exercise 4”.

The sentences that we start from are called the *premises*, and the new sentences that are generated from them are called the *conclusions*. The pair (*premises*, *conclusion*) is called an *argument*, and the process of generating conclusions from premises is called *inference*. In our daily life, we use some stretches of discourse intended as supporting a conclusion from premises; these stretches of discourse are used both to support the things we believe and to justify our actions.

An *argument* is a pair (Δ, α) , where Δ (the *premises*) is a finite set of propositions and α (the *conclusion*) is a single proposition.

In natural language, premises and conclusion are usually separated by words such as “then”, “therefore”, “thus”, and so on. In our arguments, we use the symbol “ $\therefore$ ” to separate the premises from the conclusion. Thus, the argument ($\{\text{all men are mortal, Socrates is a man}\}$, Socrates is mortal), is written as:

all men are mortal
Socrates is a man
 $\therefore$ Socrates is mortal

A *proposition* is taken as a declarative sentence (a statement or an assertion)

to which, in principle, it is possible to assign the value *true* or the value *false*. The truth value of a proposition is not absolute and timeless, but depends on the interpretative context. For example, “the grass is green” may be true in the context of a well-cared-for golf course and is likely to be false in the context of a house lawn in Western New York in the middle of a cold and icy winter.

An argument may either be valid or invalid. The argument (Δ, α) is said to be *valid* (equivalently, we can say that Δ logically implies α , or that α is a *logical consequence* of Δ) when it is logically impossible for all the premises to be true and the conclusion to be false. Otherwise, the argument is said to be *invalid*.

We should clearly distinguish the notions of validity and invalidity from the notions of truth and falsity. Validity and invalidity are attributes of arguments, whereas truth and falsity are attributes of propositions.

The validity (invalidity) of an argument is independent of the particular contents of the propositions it contains and only depends on the existence (or not) of a certain relation between the truth (either factual or hypothetical) of the premises and the conclusion. A valid argument may have false premises and a true conclusion; an invalid argument may have true premises and true conclusion. This is shown in Table 2.1.¹

The only relation between the validity or invalidity of an argument and the truth values of its components is the following: an argument whose premises are all true and whose conclusion is false is invalid.

The “almost” independence between the validity/invalidity of an argument and the truth/falsity of the sentences that are part of it is expressed by the *principle of truth-value irrelevancy*: except in the case where all the premises are true and the conclusion is false, the truth values of the propositions that are part of an argument do not determine whether the argument is valid or invalid.

This principle means that the validity (or invalidity) of an argument is not a function of the truth value of its propositions. We can then ask what is it a function of. The answer is simple (but it does not help us much at this moment): It is a function of whether the conclusion follows from the premises.

¹I am grateful to John Corcoran, Bill Rapaport, and Ana Cardoso Cachopo for providing some of these arguments.

<i>Truth values of premises and conclusion</i>	<i>Valid argument</i>	<i>Invalid argument</i>
$(True, True)$	all men are mortal Socrates is a man $\therefore$ Socrates is mortal	all even numbers are multiples of 2 $\therefore$ all multiples of 2 are even numbers
$(True, False)$	—	all dogs are animals $\therefore$ all animals are dogs
$(False, True)$	all birds are human all humans have feathers $\therefore$ all birds have feathers	all animals are dogs $\therefore$ all dogs are animals
$(False, False)$	all cats are canines all canines have feathers $\therefore$ all cats have feathers	all dogs are cats $\therefore$ all cats are dogs

Table 2.1: Illustration of truth-value irrelevancy.

Let us consider the following arguments:

A_1 : all men are mortal
Socrates is a man
 $\therefore$ Socrates is mortal

A_2 : every person is a human
 $\therefore$ every human is a person

A_3 : all stars are heavenly bodies
the Sun is a star
 $\therefore$ the Sun is a heavenly body

How do we tell whether they are valid or invalid? We know that logic aims at defining methods to distinguish valid from invalid arguments. This distinction should be made independently from the subject matter dealt with by the arguments; otherwise, there will be no independent subject called

logic. In order to reach this goal, in logic, we consider classes composed of similar arguments that differ only in the specific terms of the subject matter of the argument. For example, arguments A_1 and A_3 , above, are similar except that A_1 deals with zoology and A_3 deals with astronomy. In logic, it is said that arguments A_1 and A_3 have the same form.

In order to evaluate the validity or invalidity of an argument, we consider that each of its propositions contains logical terms (words such as “and”, “or”, “not”, “implies”, “all”, and so on) and terms that are specific to the domain of the subject matter of the argument. With this distinction, we define the *form* of an argument as being an argument where all the specific (that is non-logical) terms are replaced by a symbol that is associated with their grammatical category, for example, proper names, nouns, adjectives, and so on.

Under this definition, the form of arguments A_1 and A_3 is:

$$\begin{aligned} F_1: & \text{All } \boxed{A} \text{ are } \boxed{B} \\ & \boxed{C} \text{ is an } \boxed{A} \\ & \therefore \boxed{C} \text{ is a } \boxed{B} \end{aligned}$$

where $\boxed{A}$ and $\boxed{B}$ are nouns and $\boxed{C}$ is a proper name.

Argument A_2 has the form:

$$\begin{aligned} F_2: & \text{every } \boxed{A} \text{ is a } \boxed{B} \\ & \therefore \text{every } \boxed{B} \text{ is an } \boxed{A} \end{aligned}$$

where $\boxed{A}$ and $\boxed{B}$ are nouns.

The form of an argument may be studied independently of the subject matter of the propositions. In fact, an argument is valid or invalid because of its form and not because of its subject matter. This means that all the arguments with the same form are either valid or invalid. This is stated by the *principle of form* – if two arguments have the same form, then they are both valid or both invalid.

Under this perspective, we can see that argument A_2 is invalid, because the following argument (with the same form) has true premise and false conclusion:

$$\begin{aligned} A_4: & \text{Every dog is a mammal} \\ & \therefore \text{Every mammal is a dog} \end{aligned}$$

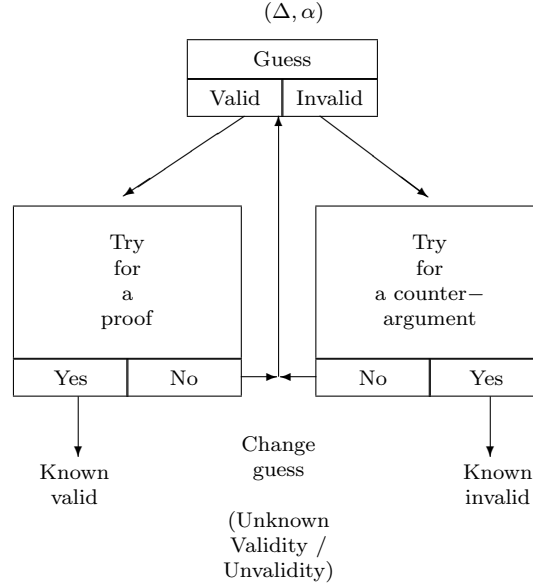


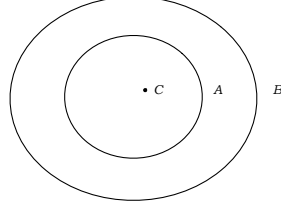
Figure 2.1: Logic methodology chart.

Thus, logic studies arguments in respect to their form (this is the reason why it is called a *formal* system).

In mathematics, there is a set of symbols that is used to represent the entities that mathematics deals with; in logic, we use symbols to represent the entities that are common in arguments, the logical terms. We usually represent “and” by “ $\wedge$ ”, “or” by “ $\vee$ ”, “not” by “ $\neg$ ”, “implies” by “ $\rightarrow$ ”, “all” by “ $\forall$ ”, and so on. Using this notation, an argument with form F_1 , for example, will be written as:

$$\begin{array}{l}
 F_1': \forall x[A(x) \rightarrow B(x)] \\
 A(C) \\
 \therefore B(C)
 \end{array}$$

In Figure 2.1, we present the main guidelines of the methodology for deciding whether a given argument is valid or invalid. According to this methodology, given an argument (Δ, α) , the first step to determine its validity or invalidity corresponds to producing a “guess” about the category of the argument. This guess is produced by analysing the premises and the conclusion and

Figure 2.2: Diagram to show the validity of F_1 .

trying to find out (intuitively) whether the conclusion is a consequence of the premises.

1. If our guess is that the argument is valid, we have to prove that it is impossible for all the premises to be true and the conclusion to be false. If we can find this proof, the argument becomes *known to be valid*. Thus, knowledge about the validity of an argument involves abstract reasoning.

Let us consider, as an example, argument A_1 , and let us try to prove that it is valid. We will consider its form, F_1 , and we will represent, using a Venn diagram,² all entities that satisfy property A and all entities that satisfy property B. Since the first proposition asserts that all As are Bs, the entity corresponding to A has to fall within the set corresponding to B (Figure 2.2). The second proposition of this argument states that C is an A. Using the diagram of Figure 2.2, we can then conclude that C must be placed inside the set A and therefore that it must belong to B. Therefore, the argument is known to be valid.

2. If our guess is that the argument is invalid, we have to find an argument in the same form as the original argument, whose premises are all true and whose conclusion is false. This is called a *counter-argument*. If we can come up with a counter-argument, the original argument becomes *known to be invalid*. Thus, knowledge about the invalidity of an argument involves factual knowledge.

For example, argument A_2 can be shown to be invalid, by producing argument A_4 , that has the same form with a true premise and a false

²Venn diagrams were invented by John Venn (1834–1923) in 1880 and were improved by Leonhard Euler (1700–1783) and by Clarence Irving Lewis (1883–1964).

conclusion.

If we cannot prove the validity or invalidity of an argument, we can change our guess and try to prove otherwise. If we fail even so, the validity of the argument is unknown.

Although any argument is either valid or invalid, the validity or invalidity of the argument may be unknown. Let us consider the following argument: ($\{\text{axioms of arithmetic}\}$, “for $n > 2$, there are no integer values x , y and z such that $z^n = x^n + y^n$ ”). This argument, known as *Fermat’s last theorem* remained with unknown validity or invalidity from the 17th century to 1993 (the year in which it was proved), despite the efforts of many mathematicians over those 300 years to prove it. This argument was valid, but it only became known to be valid 300 years after its statement.

In this chapter, we present the methodology for evaluating arguments with respect to their form and for deciding whether they are valid or invalid.

2.2 Components of a logic

Before we can talk about a logic, we have to specify the language to be used in the description of our arguments. This language establishes what things we can talk about, and with what level of detail.

The language of a logic is defined through a set of formation rules that specify which are the legal sentences in the logic, the so called *well-formed formulas* (*wffs*, for short). If we denote by $\mathcal{L}$ the language that corresponds to the well-formed formulas, an argument is a pair (Δ, α) , where $\Delta \subset \mathcal{L}$ and $\alpha \in \mathcal{L}$.

As with any language, over the sentences of $\mathcal{L}$, we can perform two types of operations:

- We can perform symbol-manipulation operations, looking at the language just as a set of symbols. Through these symbol-manipulation operations, we will carry out proofs, sequences of propositions starting with the premises that attempt to reach a given conclusion;
- We can assign a meaning — and, consequently, a truth value — to the propositions that make up the arguments, with the goal of accessing the validity or invalidity of the arguments.

These two types of operations are done in different components of a logic, the deductive system and the semantics.

2.2.1 The deductive system

The *deductive system* contains a set of symbol-manipulation rules, the *rules of inference*, that specify how to generate new formulas from formulas that already exist. The rules of inference just consider the symbols in the sentences of the language and have no concerns about the meaning of these symbols. Thus, rules of inference are statements that enable us to write certain formulas from the existence of other formulas.

The deductive system may also contain a number of *wffs* that are accepted without proof, the *axioms*.

Given the argument (Δ, α) , if there is a sequence of inference rules that, when applied to the *wffs* in Δ (and to the *wffs* that are generated from Δ) produces α , we say that α is *derivable* from Δ , and we write

$$\Delta \vdash \alpha.$$

The sequence of *wffs* generated from Δ in order to obtain α (including Δ and α) is called a proof of α from Δ .

A *proof* is thus a finite sequence of *wffs* (usually written as a column) such that each *wff* is either a premise or the result of the application of a rule of inference to earlier *wffs*.³

Using the notion of a proof, we say that the proposition α is *derivable* from the set of propositions Δ , if there is a finite sequence of rules of inference that when applied to Δ (and to the propositions derived from it) produces α .

We should keep in mind that the symbol “ $\vdash$ ” depends on the deductive system (since it depends on the rules of inference that are defined), and thus, to be precise, we should index it with an identification of the corresponding deductive system. However, to make the notation easier, when there is no danger of confusion, we will use just “ $\vdash$ ”, meaning derivability in the deductive system under consideration.

³As we will shortly see, the lines of a proof may also correspond to axioms or to theorems, but this is not important for now.

2.2.2 The semantics

The *semantics* specifies the conditions under which the propositions (*wffs*) are true or false. The semantics is based on the notion of interpretation, informally a way to assign meaning to the symbols that compose the language.⁴ An interpretation allows the evaluation of the truth value of sentences.

Given the argument (Δ, α) , if there is no interpretation that makes all propositions in Δ true and α false, then we say that Δ *logically implies* α , or that α is a *logical consequence* of Δ , and we write

$$\Delta \models \alpha.$$

If $\Delta \models \alpha$, then, according to our definition of validity, the argument (Δ, α) is valid.

Note that the sentence “*if there is no interpretation that makes all propositions in Δ true and α false*” is quite strongly stated; it does not mean that *we are unable* to come up with an interpretation that makes all propositions in Δ true and α false, but it means that it is *impossible* to do so.

2.2.3 Deductive system vs. semantics

In the deductive system nothing is said about validity (it only involves syntactic manipulation of *wffs*); on the other hand, in the semantics, nothing is stated about the criteria used to determine validity (it only states the conditions for an argument to be valid or invalid). These components provide two different accounts for the relationship between premises and conclusion.

Although the deductive system and the semantics are distinct aspects of a logic, they should be compatible in order for the logic to make sense. A logic is said to be *sound* if any argument that is provable with its deductive system is valid according to its semantics; a logic is said to be *complete* if any argument valid according to its semantics is provable in its deductive system (Figure 2.3).

Informally, a logic being sound means that we make no mistakes in the proofs or that only valid arguments are deducible; and a logic being complete means that we can prove all valid arguments.

⁴A rigorous account of the semantics is provided in Section 2.4.2.

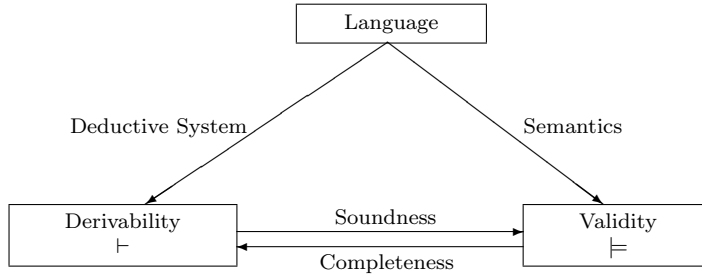


Figure 2.3: Relationship between the deductive system and the semantics.

The notions of soundness and completeness are not properties of the deductive systems and the semantics alone, but rather a relationship between the two components of a logic. In a logic that is sound and complete, the notions of provability and validity are extensionally equivalent in the sense that they apply exactly to the same arguments. Unfortunately, the notion of completeness only holds for simple logics, and most of the logics that are interesting from the AI point of view are not complete.

2.3 Propositional logic

An important aspect in the definition of a logic is the choice of the language to be used in the representation of propositions. In the definition of this language, a basic alphabet is established, and a set of rules is stated that determine how the basic components are combined in order to produce sentences or *well-formed formulas* (*wffs*, for short).

The definition of the language determines the things that can be expressed in the language and the degree of detail with which these things can be expressed. In classical logic, there are two main alternatives for the definition of the language, corresponding to propositional logic and to first-order logic.

In *propositional logic*, propositions have no “internal structure”; *first-order logic* is richer from the representation point of view in the sense that it is possible to represent the “parts” of a proposition. In order to understand this difference, let us consider the proposition “Socrates is a man”. This proposition may be represented in propositional logic by a symbol, say P , – called a predicate. This symbol does not give us any clue about the entities

“Socrates” and “man” nor to the relationship between them. It just stands by itself as a single symbol. First-order logic provides ways to represent these entities and their relationship.

From the standpoint of knowledge representation, propositional logic is almost useless. However, the advantage of presenting propositional logic is that it provides us with an easy understanding of some of the basic operations that are done in logic and leads the way for a better understanding of first-order logic, the logic we are really interested in.

In this section, we present the definition of the language of propositional logic, denoted by $\mathcal{L}_{PL}$.

Symbols of the language. In the definition of a language, the first thing that we have to define are the kinds of symbols that may be part of the sentences of the language.⁵ The language of propositional logic allows three kinds of symbols, punctuation symbols, logical symbols,⁶ and predicate symbols (or just predicates):

1. *Punctuation symbols:* ()
2. *Logical symbols:* $\neg \wedge \vee \rightarrow$
 - (a) the symbol $\neg$ is read *not* and corresponds to the negation operator;
 - (b) the symbol $\wedge$ is read *and* and corresponds to the conjunction operator;
 - (c) the symbol $\vee$ is read *or* and corresponds to the disjunction operator;
 - (d) the symbol $\rightarrow$ is read *implies* and corresponds to the implication operator.
3. *Predicate symbols,* for $i \geq 1$: P_i

A predicate corresponds to a symbol that stands for a proposition — and thus can be assigned a truth value. The reason for parameterizing predicates, instead of using individual letters to denote them, is that resorting to a parameter gives us an unlimited capability of defining predicates.

⁵In Computer Science, these are called *terminal symbols*.

⁶These are also called *logical connectives*.

The language of propositional logic, $\mathcal{L}_{PL}$, is composed of strings of symbols obtained from the set

$$\{ (,) \} \cup \{ \neg, \wedge, \vee, \rightarrow \} \cup \{ P_1, P_2, P_3, \dots \}.$$

When there is no danger of confusion, we will use upper-case Roman letters, P, Q, R , etc., without subscripts to denote predicates, therefore, the basic set of symbols of our language is represented as

$$\{ (,) \} \cup \{ \neg, \wedge, \vee, \rightarrow \} \cup \{ P, Q, R, \dots \}.$$

Sentences of the language. We know that not every combination of the terminal symbols corresponds to a valid sentence in a language. The legal sentences of the language of propositional logic are called *well-formed formulas* (*wffs*, for short) and are defined by the following formation rules:

1. Predicates are *wffs*;
2. If α is a *wff*, then $\neg\alpha$ is a *wff*;
3. If α and β are *wffs*, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, and $(\alpha \rightarrow \beta)$ are *wffs*;
4. Nothing else is a *wff*.

The *wff* $\neg\alpha$ is read “*it is not the case that α* ”; the *wff* $(\alpha \wedge \beta)$ is read “ *α and β* ”; the *wff* $(\alpha \vee \beta)$ is read “ *α or β* ”; and the *wff* $(\alpha \rightarrow \beta)$ is read “ *α implies β* ”, or, alternatively, “*if α then β* ”.

According to these definitions, if P, Q , and R are predicates, then the following are *wffs*:

$$\begin{aligned} & (P \wedge Q) \\ & ((P \wedge Q) \rightarrow R) \\ & ((P \vee ((P \wedge Q) \rightarrow R)) \wedge R) \end{aligned}$$

As an exercise, you should convince yourself that the following strings of symbols are not *wffs*:

$$\begin{aligned} & () \\ & (\rightarrow (\wedge P Q) R) \\ & (P \vee \rightarrow ((P \wedge Q) R)) \wedge R) (\end{aligned}$$

Whenever possible, redundant parenthesis are omitted, and the *wffs* above are written as:

$$\begin{aligned} &P \wedge Q \\ &(P \wedge Q) \rightarrow R \\ &(P \vee ((P \wedge Q) \rightarrow R)) \wedge R \end{aligned}$$

Before we go on, let us provide some examples of *wffs* in propositional logic. Let us assume that P stands for the sentence “It is raining”; that Q stands for the sentence “It is windy”; and that R stands for the proposition “I stay at home”. In this case, the *wff* $P \wedge Q$ stands for “It is raining *and* it is windy”; the *wff* $(P \wedge Q) \rightarrow R$ stands for “*If* it is raining *and* it is windy, *then* I stay at home”.

2.3.1 Deductive system

The deductive system specifies the rules of inference, rules that allow the introduction of new *wffs* from existing *wffs*. In our presentation of the deductive system, we use an approach called *natural deduction* that only contains rules of inference, and no axioms.⁷

The rules of inference *do not* belong to the language of the logic, $\mathcal{L}$, but *talk* about the entities of $\mathcal{L}$, that is, they belong to the *meta-language* of $\mathcal{L}$. They can be looked at as rewrite rules that state that from a certain number of *wffs* with a given form, we are allowed to write a new *wff* with a certain form. We use lower-case Greek letters (α, β, γ , etc.) as meta-variables whose domain is the set of *wffs*. For example, the *wff* $(P \wedge Q) \rightarrow R$, may be written as $\alpha \rightarrow \beta$, where α is a meta-variable whose value is the *wff* $(P \wedge Q)$ and β is a meta-variable whose value is the *wff* R ; but it can also be written as $(\alpha \wedge \beta) \rightarrow \gamma$, where α is a meta-variable whose value is the *wff* P , β is a meta-variable whose value is the *wff* Q , and γ is a meta-variable whose value is the *wff* R .

Typically, in natural deduction there are two rules of inference for each logical symbol, one that states how to introduce a *wff* that uses the logical symbol, the *introduction rule*, and another that states how to use a *wff* that contains the logical symbol, the *elimination rule*.

⁷An alternative approach, called *axiomatic approach* or *Hilbert-style approach*, besides a set of rules of inference, contains a distinguished set of *wffs*, the *axioms*, that are accepted without proof.

A proof is a sequence of numbered lines, each one either contains a premise or a *wff* that is introduced using a rule of inference. On the right hand side of each line there is a justification for how the line was introduced in the proof. The first lines of a proof contain all the premises used in the proof.

We start with a simple example, trying to prove that $\{P, Q\} \vdash P \wedge Q$. This result should be obvious – from the premises P and Q , we must be able to infer that $P \wedge Q$. We can start our proof by writing the premises:

1	P	Prem
2	Q	Prem

So far, we wrote two lines, line 1 contains *wff* P and is justified as a premise and line 2 contains *wff* Q and is also justified as a premise. In order to go on, we need to introduce rules of inference.

There is a rule of inference that enables to re-introduce in a proof a line that already exists in the proof. This rule, called *repetition*, written Rep, states that any *wff* may be repeated inside a proof. The statement of this rule is represented in the following way:

n	α	
$\vdots$	$\vdots$	
m	α	Rep, n

At this point, we should notice that, since the rules of inference belong to the meta-language, they talk about *wffs*. For this reason, rules of inference use variables that range over *wffs*.

The rule of repetition allows us, for example, to add a third line to our original proof:

1	P	Prem
2	Q	Prem
3	P	Rep, 1

Line 3 in the proof above states that the *wff* P is introduced by repetition of line 1.

Rules for conjunction. As for any logical symbol, there are two rules of inference to deal with conjunction. One of the rules tells us how to introduce (or how to build) a *wff* whose main logical symbol is a conjunction. This is called *conjunction introduction* rule and is written $\wedge I$. The conjunction

introduction rule states that in a proof with the *wffs* α and β , we can derive $\alpha \wedge \beta$:

$$\begin{array}{lll} n & \alpha & \\ n+1 & \beta & \\ n+2 & \alpha \wedge \beta & \wedge\text{I}, (n, n+1) \end{array}$$

The justification of line $n+2$, $\wedge\text{I}, (n, n+1)$, states that this line was obtained from lines n and $n+1$ using the conjunction introduction rule.

Using this rule, we can add a third line to our previous proof:

$$\begin{array}{lll} 1 & P & \text{Prem} \\ 2 & Q & \text{Prem} \\ 3 & P \wedge Q & \wedge\text{I}, (1, 2) \end{array}$$

And, thus, we can conclude that $\{P, Q\} \vdash P \wedge Q$, since $P \wedge Q$ was obtained in a proof with the premises P and Q .

As strange as it may sound, from lines 1 and 2 of the proof above we are not able to derive $Q \wedge P$ since this deduction does not follow the “pattern” specified in the $\wedge\text{I}$ rule – this rule enables us to introduce a conjunction, whose first element is the *wff* in line n and whose second element is the *wff* in line $n+1$. However, using the rule of repetition followed by conjunction introduction, we are able to obtain the desired result:

$$\begin{array}{lll} 1 & P & \text{Prem} \\ 2 & Q & \text{Prem} \\ 3 & P & \text{Rep, 1} \\ 4 & Q \wedge P & \wedge\text{I}, (2, 3) \end{array}$$

The second rule for conjunction tells us how we can use a *wff* whose main logical symbol is a conjunction. This rule is called *conjunction elimination*, written $\wedge\text{E}$, and is stated as follows: from the *wff* $\alpha \wedge \beta$, we can derive α , β , or both.

$$\begin{array}{lll} n & \alpha \wedge \beta & \\ n+1 & \alpha & \wedge\text{E}, n \end{array}$$

or

$$\begin{array}{lll} n & \alpha \wedge \beta & \\ n+1 & \beta & \wedge\text{E}, n \end{array}$$

It is important to notice that there are two forms of the conjunction elimination rule. The first one tells us that from a conjunction we may obtain the first element of the conjunction, and the second one tells us that from

a conjunction we may obtain the second element of the conjunction. Using the rule of conjunction introduction, we can produce the following proof for the argument $\{P \wedge Q, R\} \vdash P \wedge R$:

1	$P \wedge Q$	Prem
2	R	Prem
3	P	$\wedge E, 1$
4	R	Rep, 2
5	$P \wedge R$	$\wedge I, (3, 4)$

Rules for hypothetical proofs. Natural deduction uses the notion of *hypothetical proof*, a proof initiated with the introduction of a hypothesis. We can think of a hypothetical proof as an “environment”, where we consider the hypothesis that started the proof.

An hypothetical proof is initiated by the *rule of hypothesis*, written Hyp, which states that at any point, we may introduce any *wff* as a hypothesis, which starts a new hypothetical proof. This is represented as follows:

n	α	Hyp
$n + 1$	$\dots$	

Once we start a hypothetical proof, all the lines that we add belong to the hypothetical proof until we apply a rule of inference that ends the hypothetical proof (so far, we have not presented such a rule). Inside a hypothetical proof we may apply any rule of inference that we can apply in a regular proof. For example, the following sequence of lines correspond to part of a proof that contains a hypothetical proof:

1	$P \wedge Q$	Prem
2	P	$\wedge E, 1$
3	R	Hyp
4	R	
		Rep, 3

There is a special rule of inference, only applicable in hypothetical proofs, that enables us to repeat, inside the hypothetical proof, any *wff* that exists outside the hypothetical proof. This is called the rule of *re-iteration*, written Reit, that states that any *wff* in a proof may be re-iterated (repeated) in any

hypothetical proof inside the proof. This rule is represented in the following way:

$$\begin{array}{c|c}
 n & \alpha \\
 \vdots & \vdots \\
 m & \alpha \quad \text{Rei, } n
 \end{array}$$

With the rule of re-iteration, we may continue our previous proof in the following way:

$$\begin{array}{c|c}
 1 & P \wedge Q & \text{Prem} \\
 2 & P & \wedge\text{E, } 1 \\
 3 & \begin{array}{c|c} R & \text{Hyp} \end{array} \\
 4 & R & \text{Rep, } 3 \\
 5 & P & \text{Rei, } 2 \\
 6 & R \wedge P & \wedge\text{I, } (4, 5)
 \end{array}$$

At this point, we should notice that we have two kinds of proofs. The outermost proof that is not initiated by a hypothesis — called the *categorical proof* — and hypothetical proofs that are initiated by hypothesis. The *wffs* in a hypothetical proof are called *contingent*, since they may depend on the hypothesis that started the proof; the *wffs* in the outermost proof are called *categorical*.

So far, the need for hypothetical proofs is not clear. However this need is revealed when we consider the rules for implication.

Rules for implication. Before we present the rules for implication, let us look at the intuitive meaning of a *wff* containing an implication. Let P stand for the sentence “It rains” and let Q stand for the sentence “I stay at home”. The *wff* $P \rightarrow Q$ is read “If it rains, then I stay at home”. We should notice that this *wff* does not depend on whether or not it is raining, it just states that in the case that it rains, I should stay at home; if it does not rain, then I may or may not stay at home since the formula states nothing about this case. If we want to prove that $P \rightarrow Q$, then we should first get into some hypothetical reasoning “context” where we assume that it rains; if, within this “context”, we can show that I stay at home, then it is clear that the *wff* is proved. It should also be clear that this proof has nothing to do about the current weather conditions. This hypothetical reasoning “context” corresponds to a hypothetical proof.

The rule of *implication introduction*, written $\rightarrow I$, states that if, in a proof initiated by the hypothesis α , we are able to derive β , then we may derive $\alpha \rightarrow \beta$ in the proof that contains the hypothetical proof initiated by the hypothesis α :

$$\begin{array}{rcl}
 n & \left| \begin{array}{c} \alpha \\ \vdots \\ \beta \end{array} \right. & \text{Hyp} \\
 \vdots & & \\
 m & & \\
 m+1 & \alpha \rightarrow \beta & \rightarrow I, (n, m)
 \end{array}$$

The justification of the *wff* in line $m+1$, $\rightarrow I(n, m)$, states that this line was obtained in the proof initiated by the hypothesis in line n and by the *wff* in line m .

The rule of implication introduction is the first rule so far that allows us to end a hypothetical proof. As a first example, let us consider the proof

$$\begin{array}{rcl}
 1 & \left| \begin{array}{c} P \\ P \end{array} \right. & \text{Hyp} \\
 2 & & \text{Rep, 1} \\
 3 & P \rightarrow P & \rightarrow I, (1, 2)
 \end{array}$$

Besides being an example of the use of the implication introduction rule, the proof above is our first proof that does not use premises. Since the *wff* in line 3 of this proof does not depend on any premises, it stands by itself. This proof means that $\emptyset \vdash P \rightarrow P$. By definition, a *wff* that is obtained in a proof that does not contain premises is called a *theorem*. When $\emptyset \vdash \alpha$, that is, when α is a theorem, it is usual to write just $\vdash \alpha$.

In order to make use of a *wff* whose main logical connective is an implication, we need the rule of *implication elimination*, written $\rightarrow E$, also known as *modus ponens*. This rule states that in a proof that contains both α and $\alpha \rightarrow \beta$ we may derive β :

$$\begin{array}{rcl}
 n & \alpha & \\
 n+1 & \alpha \rightarrow \beta & \\
 n+2 & \beta & \rightarrow E, (n, n+1)
 \end{array}$$

As an example, we show the proof of the argument $\{P \rightarrow Q, P\} \vdash Q$:

1	$P \rightarrow Q$	Prem
2	P	Prem
3	$P \rightarrow Q$	Rep, 1
4	Q	$\rightarrow$ E, (2, 3)

As a more elaborated proof, we show that $(P \rightarrow (Q \rightarrow R)) \rightarrow ((P \wedge Q) \rightarrow R)$ is a theorem:

1	$P \rightarrow (Q \rightarrow R)$	Hyp
2	$P \wedge Q$	Hyp
3	P	$\wedge$ E, 2
4	$P \rightarrow (Q \rightarrow R)$	Rei, 1
5	$Q \rightarrow R$	$\rightarrow$ E, (3, 4)
6	Q	$\wedge$ E, 2
7	$Q \rightarrow R$	Rep, 5
8	R	$\rightarrow$ E, (6, 7)
9	$(P \wedge Q) \rightarrow R$	$\rightarrow$ I, (2, 8)
10	$(P \rightarrow (Q \rightarrow R)) \rightarrow ((P \wedge Q) \rightarrow R)$	$\rightarrow$ I, (1, 9)

The next proof shows that $(P \rightarrow (Q \rightarrow P))$ is a theorem. This result may strike us at first. Its English reading states that “*if P, then, for any proposition Q, if Q then P*”. We can re-read this statement as “any proposition implies a true proposition”. This result is known as one of the *paradoxes of implication* and generated reactions against classical logic.⁸

1	P	Hyp
2	Q	Hyp
3	P	Rei, 1
4	$Q \rightarrow P$	$\rightarrow$ I, (2, 3)
5	$P \rightarrow (Q \rightarrow P)$	$\rightarrow$ I, (1, 4)

⁸This aspect is further discussed in Chapter 2.

Rules for negation. The rule for the introduction of negation, *negation introduction*, written $\neg I$, uses the notion of proof by way of contradiction: if, in a proof initiated by the hypothesis α , we derive a contradiction – both β and $\neg\beta$ – then we can derive $\neg\alpha$ in the proof that contains the hypothetical proof initiated by the hypothesis α :

n	α	Hyp
$\vdots$	$\vdots$	
m	β	
$m + 1$	$\neg\beta$	
$m + 2$	$\neg\alpha$	$\neg I, (n, (m, m + 1))$

The justification of the *wff* in line $m + 2$, $\neg I(n, (m, m + 1))$, states that this line was obtained in the proof initiated by the hypothesis in line n and by the contradiction in lines m and $m + 1$.

The rule of negation introduction is another inference rule that ends a hypothetical proof.

Using the rule of negation introduction, together with our previous rules, we are able to prove the argument $\{P \rightarrow Q, \neg Q\} \vdash \neg P$:

1	$P \rightarrow Q$	Prem
2	$\neg Q$	Prem
3	P	Hyp
4	$P \rightarrow Q$	Rei, 1
5	Q	$\rightarrow E, (3, 4)$
6	$\neg Q$	Rei, 2
7	$\neg P$	$\neg I, (3, (5, 6))$

The previous proof presents a pattern of reasoning that is known as *modus tollens*: from $\alpha \rightarrow \beta$ and $\neg\beta$, we can derive $\neg\alpha$.

The rule for *negation elimination*, written $\neg E$, asserts that negating a proposition twice is the same as asserting the proposition. For example, if P states that it is raining, the *wff* $\neg\neg P$ states that it is not the case that it is not raining, which is the same as saying that it is raining. The rule of negation elimination states that if in a proof a *wff* is negated twice, we can conclude

that *wff*:

$$\begin{array}{lll} n & \neg\neg\alpha & \\ n+1 & \alpha & \neg\text{E}, n \end{array}$$

The next proof shows that a contradiction implies any proposition $((P \wedge \neg P) \rightarrow Q)$. This result is another of the *paradoxes of implication* and generated reactions against classical logic.⁹

1	$P \wedge \neg P$	Hyp
2	$\neg Q$	Hyp
3	$P \wedge \neg P$	Rei, 1
4	P	$\wedge\text{E}, 3$
5	$\neg P$	$\wedge\text{E}, 3$
6	$\neg\neg Q$	$\neg\text{I}, (2, (4, 5))$
7	Q	$\neg\text{E}, 6$
8	$(P \wedge \neg P) \rightarrow Q$	$\rightarrow\text{I}, (1, 7)$

Rules for disjunction. The intuitive meaning of a disjunction is that at least one of the elements of the disjunction holds (eventually both may hold). The rule of *disjunction introduction*, written $\vee\text{I}$, uses this principle and states that from the *wff* α , we can either derive $\alpha \vee \beta$, $\beta \vee \alpha$, or both, where β is any *wff*:

$$\begin{array}{lll} n & \alpha & \\ n+1 & \alpha \vee \beta & \vee\text{I}, n \end{array}$$

or

$$\begin{array}{lll} n & \alpha & \\ n+1 & \beta \vee \alpha & \vee\text{I}, n \end{array}$$

The rule of *disjunction elimination*, written $\vee\text{E}$, corresponds to reasoning by cases – if we have the *wff* $\alpha \vee \beta$ (intuitively meaning that at least one of the *wffs* α or β holds) and we are able to derive a third formula, γ , independently from each one of the *wffs* α and β , then surely γ holds.

The rule is stated as follows, if, in a proof we have (1) the *wff* $\alpha \vee \beta$; (2) a hypothetical proof from the hypothesis α containing the *wff* γ ; and (3) a

⁹This aspect is further discussed in Chapter 2.

hypothetical proof from the hypothesis β containing the *wff* γ , then we can derive the *wff* γ in that proof:

n	$\alpha \vee \beta$	
o	α	Hyp
$\vdots$	$\vdots$	
p	γ	
r	β	Hyp
$\vdots$	$\vdots$	
s	γ	
m	γ	$\vee E, (n, (o, p), (r, s))$

The justification of the *wff* in line m , $\vee E(n, (o, p), (r, s))$, states that this line was obtained from the *wff* in line n , together with two subproofs, one initiated by the hypothesis in line o and the *wff* in line p and the other initiated by the hypothesis in line r and the *wff* in line s .

As a first example, we prove the *disjunctive syllogism*, $((P \vee Q) \wedge \neg P) \rightarrow Q$:

1	$(P \vee Q) \wedge \neg P$	Hyp
2	$P \vee Q$	$\wedge E, 1$
3	P	Hyp
4	$\neg Q$	Hyp
5	P	Rep, 3
6	$\neg P$	$\wedge E, 1$
7	$\neg\neg Q$	$\neg I, (4, (5, 6))$
8	Q	$\neg E, 7$
9	Q	Hyp
10	Q	Rep, 9
11	Q	$\vee E, (2, (3, 8), (9, 10))$
12	$((P \vee Q) \wedge \neg P) \rightarrow Q$	$\rightarrow I, (1, 11)$

As another example, we show the proof of $(P \vee Q) \rightarrow \neg(\neg P \wedge \neg Q)$:

1	$P \vee Q$	Hyp
2	P	Hyp
3	$\neg P \wedge \neg Q$	Hyp
4	P	Rei, 2
5	$\neg P$	$\wedge E$, 3
6	$\neg(\neg P \wedge \neg Q)$	$\neg I$, (3, (4, 5))
7	Q	Hyp
8	$\neg P \wedge \neg Q$	Hyp
9	Q	Rei, 7
10	$\neg Q$	$\wedge E$, 8
11	$\neg(\neg P \wedge \neg Q)$	$\neg I$, (8, (9, 10))
12	$\neg(\neg P \wedge \neg Q)$	$\vee E$, (1, (2, 6), (7, 11))
13	$(P \vee Q) \rightarrow \neg(\neg P \wedge \neg Q)$	$\rightarrow I$, (1, 12)

Summary of inference rules. Tables 2.2, 2.3, and 2.4 show a summary of the rules of inference that we presented so far.

Name	Rule
Hypothesis	$\begin{array}{c c} n & \alpha \\ n+1 & \dots \end{array} \quad \text{Hyp}$
Repetition	$\begin{array}{c c} n & \alpha \\ \vdots & \vdots \\ m & \alpha \end{array} \quad \text{Rep, } n$
Re-iteration	$\begin{array}{c c} n & \alpha \\ \vdots & \vdots \\ m & \alpha \end{array} \quad \text{Rei, } n$

Table 2.2: Summary of rules for hypothesis and for repeating *wffs*.

	Introduction			Elimination		
$\rightarrow$	n	α	Hyp			
	$\vdots$	$\vdots$		n	α	
	m	β		m	$\alpha \rightarrow \beta$	
	$m+1$	$\alpha \rightarrow \beta$	$\rightarrow I, (n, m)$	$m+1$	β	$\rightarrow E, (n, m)$
$\wedge$	n	α		n	$\alpha \wedge \beta$	
	$n+1$	β		$n+1$	α	$\wedge E, n$
	$n+2$	$\alpha \wedge \beta$	$\wedge I, (n, n+1)$		and	
				n	$\alpha \wedge \beta$	
				$n+1$	β	$\wedge E, n$

Table 2.3: Summary of rules for the logical connectives (part 1).

Theorems and derived rules of inference

We have seen that a proof is a sequence of lines, each of them containing a *wff* and justified by a rule of inference.

There are two ways to shorten the proofs, the first one uses the notion of a theorem and the second one the notion of derived rule of inference.

Recall that a theorem is any *wff* that can be derived from an empty set of premises. Thus, if we need to use a formula that corresponds to a theorem, we may just introduce in our proof the lines that correspond to the proof of the theorem and obtain the corresponding *wff*. We are allowed to omit the lines that correspond to the proof of the theorem and just introduce the *wff* that corresponds to the theorem, justifying it as such. For example, we have shown in page 43 that $P \rightarrow P$ is a theorem, therefore, in a proof we are allowed to use the line

$$n \quad P \rightarrow P \quad \text{Theo}$$

Let us notice, furthermore that despite we have proved that $P \rightarrow P$, the *wff* P could have been replaced by any other *wff*, so instead of a single theorem, we have an infinite set of *wffs* of the form $\alpha \rightarrow \alpha$ as corresponding to theorems.

Another way to make our proofs simpler is to use derived rules of inference. A *derived rule of inference* is any pattern of reasoning that we can write down and is supported by other rules of inference. A derived rule of inference

	<i>Introduction</i>		<i>Elimination</i>	
$\vee$	$ \begin{array}{lcl} n & \alpha & \\ n+1 & \alpha \vee \beta & \vee I, n \\ & \text{and} & \\ n & \alpha & \\ n+1 & \beta \vee \alpha & \vee I, n \end{array} $		$ \begin{array}{lcl} n & \alpha \vee \beta & \\ o & \alpha & \text{Hyp} \\ & \hline \vdots & \vdots & \\ p & \gamma & \\ r & \beta & \text{Hyp} \\ & \hline \vdots & \vdots & \\ s & \gamma & \\ m & \gamma & \vee E, (n, (o, p), (r, s)) \end{array} $	
$\neg$	$ \begin{array}{lcl} n & \alpha & \text{Hyp} \\ & \hline \vdots & \vdots & \\ m & \beta & \\ m+1 & \neg \beta & \\ m+2 & \neg \alpha & \neg I, (n, (m, m+1)) \end{array} $		$ \begin{array}{lcl} n & \neg \neg \alpha & \\ n+1 & \alpha & \neg E, n \end{array} $	

Table 2.4: Summary of rules for the logical connectives (part 2).

corresponds to an abstraction, where we can pack several inference steps into a single step.

As a first example, let us consider the rule of implication elimination. Taking into account the rule of repetition, we can easily conclude that the following is a correct pattern of reasoning:

$$\begin{array}{lcl}
n & \alpha \rightarrow \beta & \\
n+1 & \alpha & \\
n+2 & \beta & \rightarrow E', (n, n+1)
\end{array}$$

Therefore, we will be allowed to use the rule of $\rightarrow E'$ in our proofs.

In page 45, we discovered a pattern of reasoning, called *modus tollens* — another example of a derived rule of inference —, that is formalized in the following way: in a proof that contains both $\neg \beta$ and $\alpha \rightarrow \beta$, we can derive

$\neg\alpha$.

n	$\neg\beta$	
$n + 1$	$\alpha \rightarrow \beta$	
$n + 2$	$\neg\alpha$	MT, $(n, n + 1)$

Another derived rule, *double negation introduction*, denoted by $\neg\neg$ I, is justified with the following proof:

1	P	Prem
2	$\neg P$	Hyp
3	P	Rei, 1
4	$\neg P$	Rep, 2
5	$\neg\neg P$	$\neg$ I, $(2, (3, 4))$

Therefore, in a proof that contains α we can derive $\neg\neg\alpha$:

n	α	
$n + 1$	$\neg\neg\alpha$	$\neg\neg$ I, n

As a last example, we present the rule of *negated disjunction introduction*, denoted by $\neg\vee$ I, that states that from $\neg\alpha$ and $\neg\beta$ we can derive $\neg(\alpha \vee \beta)$ — the proof in Figure 2.4 justifies this rule of inference.

n	$\neg\alpha$	
$n + 1$	$\neg\beta$	
$n + 2$	$\neg(\alpha \vee \beta)$	$\neg\vee$ I, $(n, n + 1)$

In summary, the *wffs* in a proof are either premises, theorems, or correspond to the application of inference rules to other *wffs* in the proof. The rules of inference either correspond to the rules that are directly associated with the connectives or may be derived rules of inference.

New logical symbols

The logical symbols that we presented so far correspond to the standard symbols in a logic. It can be shown that just two of the logical symbols that we used, $\neg$ and $\rightarrow$, are really necessary to express all the *wffs* in propositional logic. At this point, we should remember why we are discussing logic — our concern is defining languages to express knowledge and to reason

1	$\neg P$	Prem
2	$\neg Q$	Prem
3	$P \vee Q$	Hyp
4	P	Hyp
5	P	Rep, 4
6	Q	Hyp
7	$\neg P$	Hyp
8	$\neg Q$	Rei, 2
9	Q	Rep, 6
10	$\neg\neg P$	$\neg$ I, (7, (8, 9))
11	P	$\neg$ E, 10
12	P	$\vee$ E, (3, (4, 5), (6, 11))
13	$\neg P$	Rei, 1
14	$\neg(P \vee Q)$	$\neg$ I, (3, (12, 13))

Figure 2.4: Justification for derived rule of inference $\neg\vee$ I.

symbolically with the sentences of these languages. Under this perspective, it is just natural that we are interested to know how easily the languages enable us to write down expressions about the world we are modelling.

We are free to introduce new symbols defined by the combination of existing symbols. Formulas with these new logical symbols can be considered as “syntactic sugar” for more complex formulas using the standard logical symbols. The introduction of new logical symbols makes it easier to express statements in our language (a detailed case is discussed in Section 5.2.4). In this section, we illustrate this aspect by the introduction of a logical symbol called *equivalence* (also called *bi-conditional*) and represented by $\leftrightarrow$.

The introduction of equivalence leads us to revise the formation rules for *wffs* presented in page 37, with the rule:

3a If α and β are *wffs*, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, and $(\alpha \leftrightarrow \beta)$ are

wffs.

The *wff* $\alpha \leftrightarrow \beta$ is read “ α is equivalent to β ”, or, alternatively, “ α if and only if β ”.

We define a *wff* containing the logical symbol $\leftrightarrow$ as an abbreviation of the following *wff*:¹⁰

$$\alpha \leftrightarrow \beta \stackrel{\text{def}}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha) \quad (2.1)$$

Under this definition, we introduce the following obvious inference rules for equivalence.

The *equivalence introduction* rule, written $\leftrightarrow I$, states that in a proof with the *wffs* $\alpha \rightarrow \beta$ and $\beta \rightarrow \alpha$, we can derive $\alpha \leftrightarrow \beta$:

$$\begin{array}{lll} n & \alpha \rightarrow \beta & \\ n+1 & \beta \rightarrow \alpha & \\ n+2 & \alpha \leftrightarrow \beta & \leftrightarrow I, (n, n+1) \end{array}$$

The rule for *equivalence elimination*, written $\leftrightarrow E$, is stated as follows: from the *wff* $\alpha \leftrightarrow \beta$, we can derive $\alpha \rightarrow \beta$, $\beta \rightarrow \alpha$, or both.

$$\begin{array}{lll} n & \alpha \leftrightarrow \beta & \\ n+1 & \alpha \rightarrow \beta & \leftrightarrow E, n \end{array}$$

or

$$\begin{array}{lll} n & \alpha \leftrightarrow \beta & \\ n+1 & \beta \rightarrow \alpha & \leftrightarrow E, n \end{array}$$

How do we construct proofs?

One question that may be raised at this point is what is the strategy that one should follow in constructing a proof? To this end, we should first write down the premises (if they exist) and then attempt at obtaining a line with the *wff* that we want to prove.

Although we provide some basic guidelines on doing this job, these should be considered as hints and not as clear recipes. It is certainly possible to come up with cases, where simpler solutions can be found than those obtained by blindly following our suggestions.

The way to develop a proof depends much on the structure of the formula that we want to prove:

¹⁰The symbol $\stackrel{\text{def}}{=}$, is read “by definition”.

- If we are trying to prove a *wff* of the form $\alpha \rightarrow \beta$ then, usually the way to go is to start a hypothetical proof with the hypothesis α and inside this proof aiming at proving β . Notice that by doing this, we have now shifted our attention — from the initial problem of trying to prove $\alpha \rightarrow \beta$, we are now concerned with obtaining just the *wff* β .¹¹
- If we are trying to prove a *wff* of the form $\alpha \wedge \beta$ then the way to go is to try to prove separately α and β . Again, we shift our attention from the proof of $\alpha \wedge \beta$, to two separate proofs, of α and of β .
- If we are trying to prove a *wff* of the form $\alpha \vee \beta$ then the way to go is to try to prove one of the *wffs* α or β . Again, we shift our attention from the proof of $\alpha \vee \beta$, to another proof, either of α or of β .
- If we are trying to prove a *wff* of the form $\neg\alpha$ then, we may consider one of two alternatives:
 - either we have *wffs* that directly lead us to the derivation of $\neg\alpha$ or
 - we can try to introduce a proof with the hypothesis α and try to derive a contradiction within that hypothetical proof.
- If we are trying to prove a *wff* that corresponds to a single predicate, then we should first try to find applications of inference rules that lead us to the introduction of a *wff* corresponding to the predicate, if we fail, we can try to prove it by way of contradiction, starting a hypothetical proof with the negation of the predicate and trying to produce a contradiction within that hypothetical proof,¹² or, if there are disjunctions in our proof, trying to use reasoning by cases.

We now show some additional proofs, explaining our reasoning while constructing these proofs. We start by the proof of the theorem $P \rightarrow (Q \rightarrow (P \wedge Q))$. Since we are trying to prove an implication, we should start a hypothetical proof with the hypothesis P and to try to prove $Q \rightarrow (P \wedge Q)$

¹¹As an example of an exception to this rule, suppose that we were trying to prove the *wff* $Q \rightarrow R$ in a proof that contains both P and $P \rightarrow (Q \rightarrow R)$, in this case our hint above is not useful and we should be clever enough to apply the rule of $\rightarrow$ E.

¹²Notice that it is quite similar to the logic methodology chart shown in Figure 2.1.

within this proof.

1	P	Hyp
	$\hline$	
	$\text{Aim at filling these lines}$	
n	$Q \rightarrow (P \wedge Q)$	

Since our goal is now to prove an implication $Q \rightarrow (P \wedge Q)$, we should start another hypothetical proof with hypothesis Q and try to prove $P \wedge Q$ within this proof.

1	P	Hyp
	$\hline$	
2	Q	Hyp
	$\hline$	
	$\text{Aim at filling these lines}$	
m	$P \wedge Q$	
n	$Q \rightarrow (P \wedge Q)$	

In order to prove the conjunction, we should try to prove P and Q separately, which can easily be done, and from then on the remaining part of the proof is trivial:

1	P	Hyp
	$\hline$	
2	Q	Hyp
	$\hline$	
3	P	Rei, 1
4	Q	Rep, 2
5	$P \wedge Q$	$\wedge$ I, (3, 4)
6	$Q \rightarrow (P \wedge Q)$	$\rightarrow$ I, (2, 5)
7	$P \rightarrow (Q \rightarrow (P \wedge Q))$	$\rightarrow$ I, (1, 6)

As a second example, we now try to prove the theorem $P \vee \neg P$. This theorem is known as the *law of excluded middle*. Here the situation is a little trickier. Despite the fact that we are trying to prove a disjunction, we cannot find any way to prove any of the disjuncts (as a theorem) – in fact we cannot just prove any arbitrary predicate from nothing. Therefore, we will attempt to use a proof by contradiction from the hypothesis that corresponds to the

negation of the theorem we want to prove.

1	$\neg(P \vee \neg P)$	Hyp
	$\hline$	
	<i>Aim at filling these lines</i>	
n	α	
$n + 1$	$\neg\alpha$	

The main difficulty here is to “see” what contradiction should be derived. Since we have negated $P \vee \neg P$ and this *wff* holds, according to our intuitions, we will try to obtain $P \vee \neg P$. In order to obtain this formula, we will start by introducing the hypothesis P . From then on, the proof is obvious:

1	$\neg(P \vee \neg P)$	Hyp
	$\hline$	
2	P	Hyp
	$\hline$	
3	$P \vee \neg P$	$\vee I, 2$
	$\hline$	
4	$\neg(P \vee \neg P)$	Rei, 1
5	$\neg P$	$\neg I, (2, (3, 4))$
6	$P \vee \neg P$	$\vee I, 5$
7	$\neg(P \vee \neg P)$	Rep, 1
8	$\neg\neg(P \vee \neg P)$	$\neg I, (1, (6, 7))$
9	$P \vee \neg P$	$\neg E, 8$

Properties of the deductive system

The deductive system of propositional logic (as well as the deductive system of first-order logic, see Section 2.4.1) has a certain number of properties, some of which are presented in this section.

Theorem 1 (Deduction Theorem) *If Δ is a set of wffs, α and β are wffs, and if $(\Delta \cup \{\alpha\}) \vdash \beta$, then $\Delta \vdash (\alpha \rightarrow \beta)$.*

PROOF: Let us assume that $(\Delta \cup \{\alpha\}) \vdash \beta$. This means that there is a proof that starts with all the premises in Δ and with the premise α and whose last line is β . Let us assume that $\Delta = \{\delta_1 \dots \delta_n\}$. The first $n + 1$ lines of this

proof contain the premises $\delta_1 \dots \delta_n, \alpha$ and the last line of this proof, say m , contains the *wff* β :

1	δ_1	Prem
$\vdots$	$\vdots$	
n	δ_n	Prem
$n + 1$	α	Prem
$\vdots$	$\vdots$	
m	β	

Let us construct a new proof whose first n lines are the premises $\delta_1, \dots, \delta_n$. The $n + 1^{th}$ line of the new proof starts a hypothetical proof that contains the hypothesis α . The remaining lines of the new proof are constructed in the following way: if line k ($k > n + 1$) of the original proof corresponds to the application of the rule of repetition to a premise in line i ($1 \leq i \leq n$), then line k of the new proof has the same *wff* but it is obtained by the rule of re-iteration applied to the same premise; otherwise, the line of the new proof is equal to the line of the original proof:

1	δ_1	Prem
$\vdots$	$\vdots$	
n	δ_n	Prem
$n + 1$	α	Hyp
$\vdots$	$\vdots$	
m	β	

The m^{th} line of the new proof contains *wff* β and is contained in the hypothetical proof that was initiated by the hypothesis α . Using the rule of implication introduction, we can add line $m + 1$ to the new proof, containing the *wff* $\alpha \rightarrow \beta$ and justified by $\rightarrow I, (n + 1, m)$:

1	δ_1	Prem
$\vdots$	$\vdots$	
n	δ_n	Prem
$n + 1$	α	Hyp
$\vdots$	$\vdots$	
m	β	
$m + 1$	$\alpha \rightarrow \beta$	$\rightarrow I, (n + 1, m)$

The new proof shows that $\Delta \vdash (\alpha \rightarrow \beta)$. ■

Corollary 1 *If $\Delta = \{\alpha_1, \dots, \alpha_n\}$ is a set of wffs, if β is a wff, and if $\Delta \vdash \beta$, then $\vdash \alpha_1 \rightarrow (\dots (\alpha_n \rightarrow \beta))$.*

PROOF: The proof corresponds to n applications of Theorem 1. ■

Corollary 2 *If Δ is a set of wffs, α and β are wffs, and if $\Delta \vdash (\alpha \rightarrow \beta)$ then $(\Delta \cup \{\alpha\}) \vdash \beta$.*

PROOF: The proof uses a line of reasoning similar to the proof of Theorem 1. ■

Theorem 2 (Transitivity of $\vdash$) *If Δ is a set of wffs, $\alpha_1, \dots, \alpha_n$ are wffs, β is a wff, and if $\Delta \vdash \alpha_1, \dots, \Delta \vdash \alpha_n$, and $\{\alpha_1, \dots, \alpha_n\} \vdash \beta$, then $\Delta \vdash \beta$.*

PROOF: Let $\Delta = \{\delta_1 \dots \delta_m\}$. Let us also assume that for $1 \leq i \leq n$, $\Delta \vdash \alpha_i$. This means that there are n proofs, each one starting with the premises $\delta_1 \dots \delta_m$, and having the wff α_i in line k_i :

1	δ_1	Prem
$\vdots$	$\vdots$	
m	δ_m	Prem
$\vdots$	$\vdots$	
k_i	α_i	

Let us assume that $\{\alpha_1, \dots, \alpha_n\} \vdash \beta$. This means that there is a proof that starts with the premises $\alpha_1, \dots, \alpha_n$ and whose last line, say l , contains the wff β :

1	α_1	Prem
$\vdots$	$\vdots$	
m	α_n	Prem
$\vdots$	$\vdots$	
l	β	

Let us construct the following proof: The first n lines contain the premises $\delta_1 \dots \delta_m$; lines $n+1$ to $k_1 - n$ are identical to the lines $n+1$ to k_1 of the proof of $\Delta \vdash \alpha_1$; for $2 \leq j \leq n$, we have lines identical to lines $n+1$ to k_j of

the proof of $\Delta \vdash \alpha_j$, updating the appropriate line references.

1	δ_1	Prem
$\vdots$	$\vdots$	
m	δ_m	Prem
$\vdots$	$\vdots$	
k_1	α_1	
$\vdots$	$\vdots$	
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m)$	α_n	

We use repetition, to write down the lines with the *wffs* $\alpha_1 \dots \alpha_n$:

1	δ_1	Prem
$\vdots$	$\vdots$	
m	δ_m	Prem
$\vdots$	$\vdots$	
k_1	α_1	
$\vdots$	$\vdots$	
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m)$	α_n	
$m + \sum_{i=1}^n (k_i - m) + 1$	α_1	Rep
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m) + n$	α_n	Rep

We now write down, lines $n+1$ to l of the proof of $\{\alpha_1, \dots, \alpha_n\} \vdash \beta$, updating the corresponding line references:

1	δ_1	Prem
$\vdots$	$\vdots$	
m	δ_m	Prem
$\vdots$	$\vdots$	
k_1	α_1	
$\vdots$	$\vdots$	
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m)$	α_n	
$m + \sum_{i=1}^n (k_i - m) + 1$	α_1	Rep
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m) + n$	α_n	Rep
$\vdots$	$\vdots$	
$m + \sum_{i=1}^n (k_i - m) + n + l$	β	

The new proof shows that $\Delta \vdash \beta$. ■

Theorem 3 (Monotonicity) *If Δ_1 and Δ_2 are sets of wffs, α is a wff, and $\Delta_1 \vdash \alpha$, then $(\Delta_1 \cup \Delta_2) \vdash \alpha$.*

PROOF: Let $\Delta_1 = \{\delta_{11} \dots \delta_{1n}\}$ and $\Delta_2 = \{\delta_{21} \dots \delta_{2m}\}$. Since $\Delta_1 \vdash \alpha$, there is a proof starting with premises $\delta_{11} \dots \delta_{1n}$, and ending with wff α :

1	δ_{11}	Prem
$\vdots$	$\vdots$	
n	δ_{1n}	Prem
$\vdots$	$\vdots$	
k	α	

Let us construct a proof that starts with the premises $\delta_{11} \dots \delta_{1n}, \delta_{21} \dots \delta_{2m}$ and whose lines after line $n + m$ are identical to lines $n + 1$ to line k of the proof of $\Delta_1 \vdash \alpha$, with the update of the appropriate line references:

1	δ_{11}	Prem
$\vdots$	$\vdots$	
n	δ_{1n}	Prem
$n + 1$	δ_{21}	Prem
$\vdots$	$\vdots$	
$n + m$	δ_{2m}	Prem
$\vdots$	$\vdots$	
$m + k$	α	

The new proof shows that $(\Delta_1 \cup \Delta_2) \vdash \alpha$. ■

Given a set of wffs $\Delta \subset \mathcal{L}$, the set of all wffs derivable from Δ is called the *theory generated from Δ* , and is written $Th(\Delta)$. Formally,

$$Th(\Delta) = \{\alpha : \Delta \vdash \alpha\}.$$

Theorem 4 *For any Δ , the set $Th(\Delta)$ is infinite.*

PROOF: It can be easily seen that for any Δ , $Th(\Delta) \neq \emptyset$. In fact, even if $\Delta = \emptyset$, $Th(\emptyset)$ contains all the theorems. Given any wff in $Th(\Delta)$, say α , just by the rule of $\vee I$, we can generate infinitely many wffs, $\alpha \vee \alpha$, $\alpha \vee \alpha \vee \alpha$, ..., $\alpha \vee \dots \vee \alpha$. ■

Corollary 3 (Monotonicity) *If Δ_1 and Δ_2 are sets of wffs and $\Delta_1 \subset \Delta_2$, then $Th(\Delta_1) \subset Th(\Delta_2)$.*

PROOF: We will prove that for any α such that $\alpha \in Th(\Delta_1)$ we have that $\alpha \in Th(\Delta_2)$. Let $\Delta_2 = \Delta_1 \cup \Delta_0$ (this is a direct consequence of $\Delta_1 \subset \Delta_2$).

Let α be any *wff* such that $\alpha \in Th(\Delta_1)$. This means that $\Delta_1 \vdash \alpha$. Theorem 3 guarantees that $(\Delta_1 \cup \Delta_0) \vdash \alpha$. Therefore, $\alpha \in Th(\Delta_2)$. ■

Theorem 5 (Fixed point) *If Δ is a set of wffs, then $Th(\Delta) = Th(Th(\Delta))$.*

PROOF: We will show that $Th(\Delta) \subset Th(Th(\Delta))$ and $Th(Th(\Delta)) \subset Th(\Delta)$.

1. $Th(\Delta) \subset Th(Th(\Delta))$.

Let $\alpha \in Th(\Delta)$. Since $\{\alpha\} \vdash \alpha$, we can conclude that $\alpha \in Th(Th(\Delta))$.

2. $Th(Th(\Delta)) \subset Th(\Delta)$.

Let us assume by way of contradiction that $Th(Th(\Delta)) \not\subset Th(\Delta)$. This means that there exists an α such that $\alpha \in Th(Th(\Delta))$ and $\alpha \notin Th(\Delta)$.

Since $\alpha \in Th(Th(\Delta))$ and $\alpha \notin Th(\Delta)$, the *wff* α is produced by a proof with the premises $\beta_1, \dots, \beta_n$ such that $\{\beta_1, \dots, \beta_n\} \subset Th(\Delta)$. By Theorem 2, this means that $\alpha \in Th(\Delta)$, which is a contradiction. Therefore $Th(Th(\Delta)) \subset Th(\Delta)$.

Therefore $Th(\Delta) = Th(Th(\Delta))$ ■

2.3.2 Semantics

In the semantics we look at the *wffs* and logical connectives under the perspective of their meaning. In the semantics of propositional logic, we define a *valuation function*, a function, V , from the sentences in our language to the truth values, *true* (T) and *false* (F):

$$V : \mathcal{L}_{PL} \mapsto \{T, F\}$$

Given a *wff* α , and a valuation function V , we say that V satisfies the *wff* α if and only if $V(\alpha) = T$; otherwise the valuation function does not satisfy the *wff* α .

One of the interesting aspects of the semantics is that it is *compositional*, that is, the truth value of a formula only depends on the truth values of the sub-formulas that compose it and on the logical symbol that relates those sub-formulas:

1. In what concerns atomic predicates, the *valuation function* is an association between predicates and the truth values T and F .

Using one of our previous examples, let R correspond to the sentence “It rains” and let U correspond to the sentence “I use an umbrella”. A valuation function, V , corresponds to the assignment of a truth value to each of these propositions. For example,

$$V(R) = T$$

$$V(U) = F$$

For each predicate, we can only have two valuation functions, one that assigns the value T to the predicate and another that assigns the value F .

2. In order to assign meaning to *wffs* that contain logical symbols, we have to define the meaning of the logical symbols themselves. We define this meaning as a function that maps truth values into truth values. These meanings are specified resorting to tables that are known as *truth tables*.

- (a) Given a *wff* α , and a valuation function V , we say that V satisfies the *wff* $\neg\alpha$ if and only if V does not satisfy α :

α	$\neg\alpha$
T	F
F	T

- (b) Given two *wffs* α and β and a valuation function V , we say that V satisfies the *wff* $\alpha \wedge \beta$ if and only if V satisfies both *wffs* α and β :

α	β	$\alpha \wedge \beta$
T	T	T
T	F	F
F	T	F
F	F	F

- (c) Given two *wffs* α and β and a valuation function V , we say that V satisfies the *wff* $\alpha \vee \beta$ if and only if V satisfies at least one of the *wffs* α or β :

α	β	$\alpha \vee \beta$
T	T	T
T	F	T
F	T	T
F	F	F

- (d) Given two *wffs* α and β and a valuation function V , we say that V satisfies the *wff* $\alpha \rightarrow \beta$ if and only if V does not satisfy α or V satisfies β :

α	β	$\alpha \rightarrow \beta$
T	T	T
T	F	F
F	T	T
F	F	T

The truth values of *wffs* containing more than one logical symbol are obtained by composing the truth values of the *wffs* they contain. For example, given the predicates P , Q , and R , if we want to know under which conditions the *wff* $(P \wedge Q) \rightarrow R$ is true, we construct the following truth table:

P	Q	$P \wedge Q$	R	$(P \wedge Q) \rightarrow R$
T	T	T	T	T
T	T	T	F	F
T	F	F	T	T
T	F	F	F	T
F	T	F	T	T
F	T	F	F	T
F	F	F	T	T
F	F	F	F	T

Using the semantics, we can now evaluate if a given argument is valid. By definition, the argument (Δ, α) is valid if there is no interpretation — valuation function — that makes all the propositions in Δ *true* and the conclusion(α) *false*. In this case, we write $\Delta \models \alpha$.

For example, given the argument $(\{P \wedge Q, R\}, P \wedge R)$, the following truth table shows its validity — there is no line in this table, where the premises are both T and the conclusion is F .

P	Q	R	$P \wedge Q$	$P \wedge R$
T	T	T	T	T
T	T	F	T	F
T	F	T	F	T
T	F	F	F	F
F	T	T	F	F
F	T	F	F	F
F	F	T	F	F
F	F	F	F	F

We say that the wff α is a *tautology* if and only if it is true under all possible valuation functions (interpretations). If α is a tautology we write $\emptyset \models \alpha$ or just $\models \alpha$.

To verify whether α is a tautology, we create a truth table with all the predicates that compose α and we verify if α is always true, no matter the combination of the truth values that compose it. For example, the following truth table shows that the wff $P \vee \neg P$ is a tautology.

P	$\neg P$	$P \vee \neg P$
T	F	T
F	T	T

Given a wff α that contains n predicates, $P_1, P_2, \dots, P_n$, we can, in principle, construct a truth table for α . The main problem is that the truth table has 2^n lines, each line corresponding to a possible combination of the truth values for $P_1, P_2, \dots, P_n$. For large values of n this task may be impossible to complete.

2.3.3 Soundness and completeness of propositional logic

Theorem 6 (Soundness) *Propositional logic is sound: for any wffs, $\alpha_1 \dots \alpha_n, \beta$, if $\{\alpha_1 \dots \alpha_n\} \vdash \beta$ then $\{\alpha_1 \dots \alpha_n\} \models \beta$.*

PROOF: ■

Theorem 7 (Completeness) *Propositional logic is complete: for any wffs, $\alpha_1 \dots \alpha_n, \beta$, if $\{\alpha_1 \dots \alpha_n\} \models \beta$ then $\{\alpha_1 \dots \alpha_n\} \vdash \beta$.*

PROOF: ■

2.4 First-order logic

Propositional logic does not allow predicates to have a structure. Recall from page 35 that the representation of the proposition “Socrates is a man” did not allow for the representation of Socrates, man, nor for the relationship between these entities. When we want to use structure in the predicates we obtain a logic called *first-order logic*. In first-order logic, *wffs* have an “internal structure”.

First-order logic requires a new definition of *wffs*, introduces new logical symbols and new formation rules. In this section, we define the language of first-order logic, denoted by $\mathcal{L}_{FOL}$.

Symbols of the language. First-order logic is much richer than propositional logic. Besides containing all the symbols of propositional logic, first-order logic contains two additional logical symbols, the quantifiers, and allows for the use of functions and variables. Furthermore, predicates have a structure, in the sense that they correspond to relations with arguments.

Before we provide the list of the symbols of first-order logic language, we elaborate a little on the notions of function, variable, and relation.

The word “function” is used broadly to mean any determinate correspondence (or mapping) between two classes of entities. From the standpoint of set theory, a function is a set (possibly infinite) containing the elements (ordered pairs) that correspond to the mapping. As examples, we can think of functions corresponding to the capital of a country, to the year of birth of a person, and to the successor of a natural number. The following sets partly list these functions:¹³

$$\{(Portugal, Lisbon), (France, Paris), (Spain, Madrid), \dots\}$$

$$\{(Augustus_De_Morgan, 1806), (Alonzo_Church, 1903), \dots\}$$

$$\{(1, 2), (2, 3), (3, 4) \dots\}$$

In a more precise way, a *function* is a set of ordered pairs that does not contain two pairs with the same first component. The set of the first components of the pairs is called the *domain* of the function, and the set of the second components of the pairs is called the *range* of the function.

¹³Notice that the first set is finite; the second set may be very large — if we consider all persons — but is also finite; the third set is infinite.

To define a function, it is not common to list every pair that belongs to the function (in many cases, this is not even possible; for example, if the domain is infinite). In general, to define a function, we supply the domain of the function and some expression that takes an element of the domain (called the *argument* of the function) and computes the corresponding element of its range (called the *value* of the function). Under this perspective, the functions above may be defined by the following expressions (assuming that their domain was previously stated):

$$c(x) = \text{the capital of } x$$

$$b(x) = \text{the year of birth of } x$$

$$s(x) = x + 1$$

To compute the value of the function for a given value of its domain, all that has to be done is to replace the variable in the expression that defines the function by its value. For example, to compute the value of the function s for the argument 1 (that is, to compute the second component of the pair whose first component is 1), all we need to do is to replace x by 1 in $x + 1$ to obtain 2. Taking into account that functions are mappings, they can be used to describe entities in the language. For example, using the function s , $s(3)$ corresponds to the entity 4.

Our discussion so far, only addressed functions with one argument, but we can easily extend the concept for functions with n arguments ($n \geq 0$): a function of n arguments is a set of $(n + 1)$ -ordered-tuples that does not contain two tuples with the same first n elements. Under this definition, functions with zero arguments correspond to singleton sets and are considered to be constants.

Variables are unnamed symbols that correspond to “place holders” for entities of the language. Variables “*per se*” do not stand for entities, but they may be replaced by appropriate entities of the language.

The word “relation” is used to mean any relationship between classes of objects. We start by discussing relations with two arguments and, as we did for the case of functions, we generalize it afterwards.

From the standpoint of set theory, a *relation* is any set of ordered pairs. A relation places no constraints on the kinds of pairs that belong to the set. As an example, we can think of the relation corresponding to the countries that share a common border. The following set lists some of the countries

involved in this relation:

$$\{(Portugal, Spain), (Spain, Portugal), (Spain, France), \dots\}$$

The pairs $(Spain, Portugal)$ and $(Spain, France)$ are part of the relation — since *Spain* has borders both with *Portugal* and with *France* —, whereas the pairs $(Portugal, Denmark)$ and $(Mexico, Canada)$ are not. The fact that there are at least two pairs with the same first component — $(Spain, Portugal)$ and $(Spain, France)$ — is enough to forbid this set to be classified as a function.

As with functions, relations have a domain and a range. A relation is usually defined supplying an indication of the sets to which the first and the second element of the pairs belong, together with an expression that takes elements from these sets and makes a statement about their relationship. Under this perspective, the relation above may be defined as:

$$B(x, y) = x \text{ has a border with } y$$

The notion of relation is easily extended to any number of arguments. Relations with one argument are usually called *classes* or *properties*.

From the mathematical point of view, any function can be seen as a relation — they are both sets of ordered pairs —, but functions place an additional constraint on the elements of the sets. However, the use of functions and the use of relations is quite different. Let us consider, for example, the relation S that relates a natural number with its successor and the function s that provides the successor of a natural number:

$$S(x, y) = y \text{ is the successor of } x$$

$$s(x) = x + 1$$

The relation S contains the pairs $\{(1, 2), (2, 3), (3, 4), \dots\}$. These are exactly the same pairs that belong to the function s . However, if we use the relation, we cannot talk about the successor of a given number, as it is the case when we use the function; with the relation, all that we can say is that the pair $(3, 4)$ is part of the relation S . A deeper discussion of when to use functions or relations is provided in Section 2.5.

Ending this short discussion on functions, variables, and relations, we now turn to the main topic of this section, the definition of the alphabet of first-order logic. The alphabet of first-order logic, besides punctuation symbols and logical symbols, contains an infinite supply of function symbols, relation symbols, and individual variables. It is defined as follows:

1. *Punctuation symbols*: $, () []$

2. *Logical symbols*: $\neg \wedge \vee \rightarrow \forall \exists$

Besides the symbols already used in propositional logic, the symbol $\forall$ is read *for all* and corresponds to the universal quantification operator; the symbol $\exists$ is read *exists* and corresponds to the existential quantification operator.

3. *Functions symbols* with n arguments, for $n \geq 0$ and $i \geq 1$: f_i^n .

The reason for parameterizing the functions, instead of using individual letters to denote them, stems from the fact that resorting to a parameter gives an unlimited capability of defining function symbols.

When there is no danger of confusion, functions will be denoted by lower-case Roman strings,¹⁴ $f, g, h, capital, father$, etc.

4. *Predicate symbols* with n arguments, for $n \geq 0$ and $i \geq 1$: P_i^n .

A predicate of n arguments represents a n -ary relation over the elements of the language.

When there is no danger of confusion, we will use strings beginning with an upper-case Roman letter, $P, Q, R, Man, Human$, etc., to denote predicates.

5. *Individual variables*, for $i \geq 1$: x_i

Individual variables have as domain objects of our language.

When there is no danger of confusion, we will use lower-case Roman letters, x, y, z , etc., to denote individual variables.

Terms. Terms denote entities in our language. Intuitively, terms correspond to noun phrases in natural language. We have different kinds of terms: constants — that correspond to functions of zero arguments; variables; and function applications, with the appropriate number of arguments.

Terms are recursively defined as follows:

1. Each function with zero arguments (a constant) is a term;
2. Each variable is a term;

¹⁴Sequences of one or more letters.

3. If $t_1, t_2, \dots, t_n$ are terms, then $f_i^n(t_1, t_2, \dots, t_n)$ is a term;
4. Nothing else is a term.

As an example, let us consider that *Portugal* and *Augustus_De_Morgan* are constants;¹⁵ that *capital* is a function of one argument whose value is the capital city of the country that corresponds to its argument, *father* is a function of one argument whose value is the father of the person that corresponds to its argument; and that x is a variable. Then, according to the definition above, the following are terms:

Portugal
Augustus_De_Morgan
capital(Portugal)
father(Augustus_De_Morgan)
father(father(father(Augustus_De_Morgan)))
 x
capital(x)
father(x)

In the terms above, we are using an approach that is quite common in programming. Rather than using single-letter designations for terms, we use a string that provides some hint about the intended meaning of the term. We should note, however, that there is nothing of formal meaning associated with this notation used for terms.

A term that contains no variables is called a *ground term*. In our example above, the first five terms are ground terms and the last three are not ground terms.

¹⁵Here we are violating the convention that terms start with a lower-case Roman letter. However, since in common language, proper names are written starting with an upper-case, we will adopt this convention for proper names.

Well-formed formulas. Just as in the propositional logic case, *wffs* are defined on the basis of predicates — the application of a predicate to the appropriate number of arguments — and on the combination of *wffs* using the logical symbols:

1. If $t_1, t_2, \dots, t_n$ are terms, then $P_i^n(t_1, t_2, \dots, t_n)$ is a *wff* (this is called an *atomic wff*);
2. If α is a *wff*, then $\neg\alpha$ is a *wff*;
3. If α and β are *wffs*, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, and $(\alpha \rightarrow \beta)$ are *wffs*;
4. If α is a *wff* containing zero or more occurrences of the individual variable x , then $\forall x[\alpha]$ and $\exists x[\alpha]$ are *wffs*;
5. Nothing else is a *wff*.

The *wff* $\forall x[\alpha]$ is read “for all x , α ”; the *wff* $\exists x[\alpha]$ is read “there exists an x such that α ”.

As an example, let us suppose that *Lives_in* is a predicate of one argument, *Share_border* is a predicate of two arguments; *Fought* is a predicate of three arguments; *capital* is a function of one argument; *Portugal* and *Spain* are constants (functions with zero arguments); and x , y , and w are variables. Then, the following expressions are *wffs*:

$$\text{Share_border}(\text{Portugal}, \text{Spain})$$

$$\text{Share_border}(x, y)$$

$$\forall x [\forall y [\text{Share_border}(x, y) \rightarrow \exists w [\text{Fought}(w, x, y)]]]$$

$$\text{Lives_in}(x, \text{capital}(\text{Portugal}))$$

A *wff* that contains no variables is called a *ground formula*. In our example above, the first *wff* is a ground formula.

As we did in the case of propositional logic, whenever possible, redundant parenthesis will be omitted in *wffs*. We also abbreviate a sequence of nested quantifiers of the same kind, for example $\forall x [\forall y [\dots]]$, by a single occurrence of the quantifier followed by the list of corresponding variables, for example, $\forall x, y [\dots]$. This is illustrated in the following *wff*:

$$\forall x, y [\text{Share_border}(x, y) \rightarrow \exists w [\text{Fought}(w, x, y)]]$$

Free and bound variables. In the wffs $\forall x[\alpha]$ and $\exists x[\alpha]$, the wff α is called the *scope* of the quantifier ($\forall$ or $\exists$), and the quantifier is said to *bind* the variable x .

For example, in the wff

$$\forall x, y [Share_border(x, y) \rightarrow \exists w [Fought(w, x, y)]],$$

the scope of the universal quantifier is the wff

$$Share_border(x, y) \rightarrow \exists w [Fought(w, x, y)]$$

and the scope of the existential quantifier is the wff

$$Fought(w, x, y).$$

The universal quantifier binds the variables x and y and the existential quantifier binds the variable w .

Note that α does not necessarily have to contain the variable x . For example, as in the wff $\forall x [Share_border(Portugal, Spain)]$. In this case, $\forall x[\alpha]$ and $\exists x[\alpha]$ mean the same as just α .

An occurrence of the variable x is said to be *bound* in the wff α if it belongs to the scope of the quantifier that introduces it; otherwise it is said to be *free*.

For example:

- $P(x)$ contains a free occurrence of the variable x (also said that $P(x)$ contains the free variable x);
- $\forall x[P(x)]$ contains the bound variable x ;
- $P(x) \rightarrow \exists x[Q(x)]$ contains a free occurrence of x (in $P(x)$) and a bound occurrence of x (in $Q(x)$).

In summary, an occurrence of the variable x in the wff α is bound in the wffs $\forall x[\alpha]$ and $\exists x[\alpha]$. A wff with no free variables is said to be *closed*.

Substitutions. We know that variables are place holders for terms. When dealing with wffs with variables, we often need to replace instances of variables by terms.

We define a *substitution* as a finite set of ordered pairs $\{x_1/t_1, \dots, x_n/t_n\}$, where each x_i ($1 \leq i \leq n$) is an individual variable, and each t_i ($1 \leq i \leq n$) is a term. In a substitution, all individual variables are different (that, is, $\forall i, j, 1 \leq i \leq n, 1 \leq j \leq n$ ($i \neq j \rightarrow x_i \neq x_j$)) and none of the variables x_i is the same as the corresponding term t_i ($1 \leq i \leq n$). There are two special cases of substitutions, the *empty substitution* (that corresponds to the empty set) and *ground substitutions* in which no term contains variables.

If a and b are constants; x , y , and z are variables; and f , g , and h are functions with one argument, the following are examples of substitutions

$$\{x/f(x), y/z\}$$

$$\{x/a, y/g(y), z/f(g(h(b)))\}$$

the following are *not* examples of substitutions¹⁶

$$\{x/x, y/z\}$$

$$\{x/a, y/g(y), x/b, c/f(g(h(b)))\}$$

The idea behind a substitution is that it will be used a guideline in the replacement of certain variables in a formula: each individual variable in the substitution will be replaced by the associated term. This justifies the constraints that we have placed on substitutions, namely that all its individual variables are different — otherwise we would not know what term to use in the replacement of the variable — and that no variable is the same as the corresponding term — otherwise the replacement will be useless.

The *application of the substitution* $s = \{x_1/t_1, \dots, x_n/t_n\}$ to the wff α (represented by $\alpha \cdot s$) is the wff obtained from α by replacing all occurrences of the free variable x_i by t_i ($1 \leq i \leq n$).

For example, given the wff $P(x, f(a, y))$, which contains the free variables x and y , and the ground substitution $s = \{x/a, y/f(a, b)\}$, the application of the substitution s to $P(x, f(a, y))$ yields

$$P(x, f(a, y)) \cdot \{x/a, y/f(a, b)\} = P(a, f(a, f(a, b))).$$

The application of the substitution $s_1 = \{x/a, y/x\}$ to the previous formula produces

$$P(x, f(a, y)) \cdot \{x/a, y/x\} = P(a, f(a, x)).$$

¹⁶As an exercise, you should explain why.

Given the same substitution, and the *wff* $\forall x[P(x, f(a, y))]$, we have

$$\forall x[P(x, f(a, y))] \cdot \{x/a, y/f(a, b)\} = \forall x[P(x, f(a, f(a, b)))].$$

Let us now consider a final example, that brings some problems. We consider again the *wff* $\forall x[P(x, f(a, y))]$ and the substitution $\{x/a, y/x\}$. The application of this substitution to this formula yields

$$\forall x[P(x, f(a, y))] \cdot \{x/a, y/x\} = \forall x[P(x, f(a, x))].$$

The application of the substitution brought an undesirable side effect. The variable x that is bound by the universal quantifier is now introduced in the second argument of the predicate P . This changes the meaning of the *wff* $\forall x[P(x, f(a, y))]$, that was not intended by the application of the substitution. To avoid these cases, a new notion, that of a term is free for a variable in a formula is introduced.

Given a term t , a variable x and a *wff* α , we say that t is free for x in α if no free instance of the variable x in α occurs in the scope of a quantifier binding any variable occurring in t .

2.4.1 Deductive system

Proofs in first-order logic are similar to those of propositional logic, except that they have new rules of inference to handle the introduction and the elimination of quantifiers. This means that any proof rule presented in Section 2.3.1 is also applicable in first-order logic.

Rules for universal quantification. Before we start considering the rules for universal quantification, let us consider the intuitive meaning of the *wff* $\forall x[\alpha(x)]$. This formula simply says that no matter what term, t_0 we substitute for x , the formula $\alpha(t_0)$ is true. For this reason, the rule of *universal introduction*, written $\forall I$, has to take this into account – we have to prove that $\alpha(t_0)$ for an arbitrary individual t_0 . In order to guarantee that there are no restrictions about the individual t_0 that has the property $\alpha(t_0)$, we use a technique similar to what was used in the implication introduction: we create a new “context” in which there is a “fresh” individual – a variable that has never appeared on the proof before – and try to prove that this individual has the property α . This is accomplished by the start of a

new hypothetical proof, that introduces a new variable, x_0 that was never mentioned in the proof. The new hypothetical proof stipulates the scope of a variable rather the scope of an hypothesis as in the rule of $\rightarrow$ I.

The rule of universal introduction can be stated as follows: if, in a proof initiated by the introduction of a variable x_0 that has never used in the proof before, we are able to derive the *wff* $\alpha(x_0)$,¹⁷ then, we can conclude that $\forall x[\alpha(x)]$ in the proof that contains the hypothetical proof initiated by the variable x_0 :

$$\begin{array}{ccc}
 n & x_0 & | \\
 \vdots & & \vdots \\
 m & & \alpha(x) \cdot \{x/x_0\} \\
 m+1 & \forall x[\alpha(x)] & \quad \forall I, (n, m)
 \end{array}$$

In the representation of this inference rule, x_0 to the left of the start of the hypothetical proof denotes that x_0 is a new variable that had never been used before in the proof. We have also introduced a new type of hypothetical proof that starts with the assumption of a new variable and ends with the introduction of a universally quantified formula.

Before we present some examples, we will also state the rule for *universal elimination*, written $\forall E$, that exploits the meaning of the universal quantifier. From $\forall x[\alpha(x)]$ we may infer $\alpha(x) \cdot \{x/t\}$, where “ t ” is any term (given the obvious condition that t be free for x in α):

$$\begin{array}{ccc}
 n & \forall x[\alpha(x)] & \\
 n+1 & \alpha(x) \cdot \{x/t\} & \quad \forall E, n
 \end{array}$$

As an example, we present the proof of the argument $(\{\forall x[P(x) \rightarrow Q(x)], \forall x[Q(x) \rightarrow R(x)]\}, \forall x[P(x) \rightarrow R(x)])$.

¹⁷Notice that $\alpha(x_0) = \alpha(x) \cdot \{x/x_0\}$.

1	$\forall x[P(x) \rightarrow Q(x)]$	Prem
2	$\forall x[Q(x) \rightarrow R(x)]$	Prem
3	$x_0 \mid P(x_0) \rightarrow Q(x_0)$	$\forall E, 1$
4	$Q(x_0) \rightarrow R(x_0)$	$\forall E, 2$
5	$\mid \overline{P(x_0)}$	Hyp
6	$\mid P(x_0) \rightarrow Q(x_0)$	Rei, 3
7	$\mid Q(x_0)$	$\rightarrow E, (5, 6)$
8	$\mid Q(x_0) \rightarrow R(x_0)$	Rei, 4
9	$\mid R(x_0)$	$\rightarrow E, (7, 8)$
10	$\mid P(x_0) \rightarrow R(x_0)$	$\rightarrow I, (5, 9)$
11	$\forall x[P(x) \rightarrow R(x)]$	$\forall I, (3, 10)$

Rules for existential quantification. The rule of *existential introduction*, written $\exists I$, is quite simple. From $\alpha(t)$, where t is any term, we may infer $\exists x[\alpha(x)]$, where x is free in $\alpha(x)$:

$$\begin{array}{lll} n & \alpha(t) & \\ n+1 & \exists x[\alpha(x)] & \exists I, n \end{array}$$

The rule for *existential elimination*, written $\exists E$, is a bit more complicated. The *wff* $\exists x[\alpha(x)]$ states that there is some individual that satisfies the property α . Although we know that there is such an individual, we do not know *which* is the individual that has this property. Therefore, we assume that x_0 is *the* individual such that $\alpha(x_0)$. In order to be fair, we could have made no statements about this individual before, and thus we create a “context” where this is a fresh new individual that was never mentioned before. If, within this context we are able to derive a statement β that does not mention x_0 , then β must hold, independently of x_0 .

The rule of existential elimination uses a notion of subproof that combines the subproofs initiated by a hypothesis and initiated by the introduction of a variable. It states that from $\exists x[\alpha(x)]$ and a proof initiated by the introduction of a variable x_0 (that has never used in the proof before) together with

the assumption that $\alpha(x_0)$, we are able to derive the *wff* β that does not use the variable x_0 , then, we can conclude that β in the proof that contains the hypothetical proof initiated by the variable x_0 and the assumption $\alpha(x_0)$:

n	$\exists x[\alpha(x)]$	
m	x_0	$\alpha(x) \cdot \{x/x_0\}$
$\vdots$		$\vdots$
k		β
$k+1$	β	$\exists E, (n, (m, k))$

Using the rules for quantifiers, we now prove a theorem, that is part of the *De Morgan laws* for quantifiers,¹⁸ $\exists x[P(x)] \rightarrow \neg \forall x[\neg P(x)]$:

1	$\exists x[P(x)]$	Hyp
2	x_0	$P(x_0)$
3		$\forall x[\neg P(x)]$
4		$P(x_0)$
5		$\neg P(x_0)$
6		$\neg \forall x[\neg P(x)]$
7	$\neg \forall x[\neg P(x)]$	$\exists E, (1, (2, 6))$
8	$\exists x[P(x)] \rightarrow \neg \forall x[\neg P(x)]$	$\rightarrow I, (1, 7)$

Summary of inference rules. Table 2.5 shows a summary of the rules of inference for quantifiers.

2.4.2 Semantics

Semantics specifies under what conditions the *wffs* of our language are true or false. Recall that a *wff* is a sequence of symbols combined using well-defined syntactic rules. In order to evaluate the truth or falsity of a *wff* it is necessary to “interpret” each of the symbols in the *wff*, by assigning

¹⁸Named after the mathematician Augustus De Morgan (1806–1871).

	Introduction	Elimination
$\forall$	$ \begin{array}{c c} n & x_0 \\ \vdots & \vdots \\ m & \alpha(x) \cdot \{x/x_0\} \\ m+1 & \forall x[\alpha(x)] \end{array} \quad \forall I, (n, m) $	$ \begin{array}{c} n \quad \forall x[\alpha(x)] \\ n+1 \quad \alpha(x) \cdot \{x/t\} \quad \forall E, n \end{array} $
$\exists$	$ \begin{array}{c} n \quad \alpha(t) \\ n+1 \quad \exists x[\alpha(x)] \quad \exists I, n \end{array} $	$ \begin{array}{c c} n & \exists x[\alpha(x)] \\ m & x_0 \mid \alpha(x) \cdot \{x_0/x\} \quad \text{Hyp} \\ \vdots & \vdots \\ k & \beta \\ k+1 & \beta \quad \exists E, (n, (m, k)) \end{array} $

Table 2.5: Summary of rules for the quantifiers.

a meaning to them. Afterwards, we have to assess whether the meaning assigned to the *wff* agrees with the world that we are considering. If it agrees, we say that the *wff* is true; otherwise, we say that it is false.

As an example, let us suppose that we want to describe the situation represented in Figure 2.5, using the following set of *wffs*:

$$Tree(t_1) \quad (2.2)$$

$$Tree(t_2) \quad (2.3)$$

$$House(h_1) \quad (2.4)$$

$$House(h_2) \quad (2.5)$$

$$Adjacent(h_2, t_1) \quad (2.6)$$

$$Adjacent(t_1, t_2) \quad (2.7)$$

$$Adjacent(t_2, h_1) \quad (2.8)$$

$$Adjacent(h_2, left(t_2)) \quad (2.9)$$

Together with *wffs* that relate some of the predicates involved in the formulas:

$$\forall(x, y)[Adjacent(x, y) \rightarrow Adjacent(y, x)] \quad (2.10)$$

$$\forall(x, y, z)[(Adjacent(x, y) \wedge Adjacent(y, z) \rightarrow InBetween(x, y, z)] \quad (2.11)$$

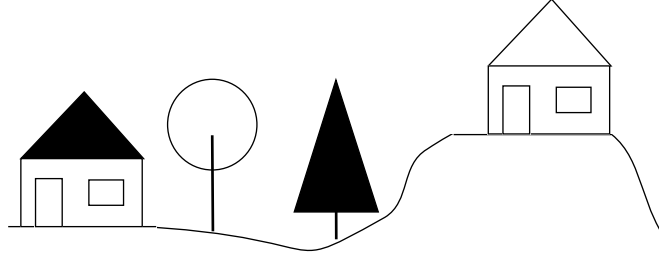


Figure 2.5: Situation in the world.

The *wff* $Adjacent(t_1, t_2)$ will be meaningless until we provide a meaning for each of the symbols $Adjacent$, t_1 and t_2 . Let us suppose that we provide meanings for each of these symbols in such a way that the resulting meaning for the *wff* $Adjacent(t_1, t_2)$ is “the black tree is next to the white tree”. According to this interpretation, the *wff* $Adjacent(t_1, t_2)$ is true.

Notice that what we have done so far is quite imprecise. First of all, the meaning of a *wff* was expressed through a sentence in natural language, with all the ambiguities that are inherent to it (for example, being “next to” is ambiguous; does “next to” mean being immediately next to an object or is it possible to have other objects in between?). Second, the description of the world was done by resorting to a figure, which cannot be considered as a formal description of the world.

In this section, we describe how the process of evaluating the truth value of *wffs* under a given interpretation is done.

In order to describe how to formalise a world or a situation, we define the notion of a conceptualisation. A conceptualization corresponds to a model of the world, in the sense introduced in Section 1.1. The reason we call it conceptualization, rather than model, stems from the fact that the word “model” has a different meaning in logic, as presented in this section.

A *conceptualisation* is a triple (D, F, R) , where:

1. D , the *universe of discourse*, is the set of entities that correspond to the world or situation about which we will be talking. The universe of discourse specifies all entities that we may consider. Note that these are a subset of all entities in the world, containing only those that we are interested in.

These entities may either be real objects (such as the planet Venus, the city of Lisbon, Augustus De Morgan, or a set of countries), abstract concepts (for example, loyalty, beauty, or the set of natural numbers), or fictional concepts (for example, Sherlock Holmes, Santa Claus, or Cinderella).

2. F is the set of functions that can be applied to the entities of the universe of discourse. Like in the specification of the universe of discourse, where we only considered a subset of the entities existing in the real world, here, we only consider a subset of all functions that can be applied to the entities of the universe of discourse, the functions that we are interested in. For example, if our universe of discourse includes natural numbers, one of the possible functions in D will be the successor function s ; if our universe of discourse includes the set of countries and cities, one of the possible functions in D will be *capital*, which, given a country returns the capital city of that country.
3. R is the set of all relations or properties that can be applied to the entities of the universe of discourse. Again, among all possible relations over the entities of the universe of discourse, we are only interested in a subset, the relations that are relevant for our intended application. For example, if the universe of discourse includes natural numbers, a possible relation would be *Smaller*; if our universe of discourse includes the set of countries, a possible relation would be *Share_border*.

A conceptualisation formally describes a world or a situation. To be more precise, and using the notions introduced in Chapter 1, a conceptualisation formally describes a *model of the world* — we are abstracting from the real-world all aspects that are irrelevant to the case at hand. In the example provided at the outset of this section, we completely ignored that there is a hill, where one of the houses is located. It is also important to stress that, given a world or a situation, there is not just one conceptualisation that describes it. The choice of the conceptualisation depends on the aspects that are relevant for the intended application.

Let us consider, again, the situation represented in Figure 2.5. We can create the following conceptualisation:

1. *Universe of discourse.*

$$D = \{\odot, \blacktriangle, \text{house icon}, \text{house icon}\}$$

2. *Set of functions.* We will just consider the two following functions:

$$f_1 = \{(\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\hat{\triangle}, \blacktriangle)\}$$

$$f_2 = \{(\hat{\triangle}, \circlearrowleft), (\circlearrowleft, \blacktriangle), (\blacktriangle, \hat{\triangle})\}$$

Thus:

$$F = \{ \{(\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\hat{\triangle}, \blacktriangle)\}, \{(\hat{\triangle}, \circlearrowleft), (\circlearrowleft, \blacktriangle), (\blacktriangle, \hat{\triangle})\} \}$$

3. *Set of relations.* We will consider four relations:

$$R_1 = \{(\circlearrowleft), (\blacktriangle)\}$$

$$R_2 = \{(\hat{\triangle}), (\hat{\triangle})\}$$

$$R_3 = \{(\hat{\triangle}, \circlearrowleft), (\circlearrowleft, \hat{\triangle}), (\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\blacktriangle, \hat{\triangle}), (\hat{\triangle}, \blacktriangle)\}$$

$$R_4 = \{(\hat{\triangle}, \circlearrowleft, \blacktriangle), (\circlearrowleft, \blacktriangle, \hat{\triangle})\}$$

Thus:

$$R = \{ \{(\circlearrowleft), (\blacktriangle)\}, \{(\hat{\triangle}), (\hat{\triangle})\}, \\ \{(\hat{\triangle}, \circlearrowleft), (\circlearrowleft, \hat{\triangle}), (\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\blacktriangle, \hat{\triangle}), (\hat{\triangle}, \blacktriangle)\}, \\ \{(\hat{\triangle}, \circlearrowleft, \blacktriangle), (\circlearrowleft, \blacktriangle, \hat{\triangle})\} \}$$

From the notion of conceptualisation, we can define the notion of interpretation. An *interpretation* is a total function (I) whose domain are the entities in our language and whose range is the entities in our conceptualisation.

The fact that an interpretation is a *function* whose domain is the entities in our language means, among other things, that each constant in our language is associated with only one entity in our conceptualisation, but different constants in our language may be associated with the same entity in our conceptualisation. If α is an entity in our language, we write $I(\alpha)$ to denote the result of the application of the interpretation function I to the entity α . In this chapter, we are only concerned with the interpretation of closed *wffs*.

Given the conceptualisation (D, F, R) , an interpretation, I , should satisfy the following conditions:

1. Each individual constant f_i^0 is associated with an entity of the universe of discourse D ;

2. Each function f_i^n ($n > 0$) is associated with a function in F . If f_i^n is a function with n arguments, corresponding to the function $I(f_i^n)$ of our conceptualisation, and if $t_1, \dots, t_n$ are terms, then $f_i^n(t_1, \dots, t_n)$ corresponds to the entity $I(f_i^n)(I(t_1), \dots, I(t_n))$ of our conceptualisation;
3. Each predicate P_i^n is associated with a relation of R .

For example, given our running example, we can consider the following interpretation (where the symbol $\mapsto$ represents the association between the symbols of the language and the objects of the conceptualisation):

$$\begin{aligned}
I(t_1) &\mapsto \circlearrowleft \\
I(t_2) &\mapsto \blacktriangle \\
I(h_1) &\mapsto \boxed{\circlearrowleft} \\
I(h_2) &\mapsto \boxed{\blacktriangle} \\
I(left) &\mapsto \{(\circlearrowleft, \boxed{\blacktriangle}), (\blacktriangle, \circlearrowleft), (\boxed{\circlearrowleft}, \blacktriangle)\} \\
I(right) &\mapsto \{(\boxed{\circlearrowleft}, \circlearrowleft), (\circlearrowleft, \blacktriangle), (\blacktriangle, \boxed{\circlearrowleft})\} \\
I(Tree) &\mapsto \{(\circlearrowleft), (\blacktriangle)\} \\
I(House) &\mapsto \{(\boxed{\circlearrowleft}), (\boxed{\blacktriangle})\} \\
I(Adjacent) &\mapsto \{(\boxed{\blacktriangle}, \circlearrowleft), (\circlearrowleft, \boxed{\blacktriangle}), (\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\blacktriangle, \boxed{\circlearrowleft}), (\boxed{\circlearrowleft}, \blacktriangle)\} \\
I(InBetween) &\mapsto \{(\boxed{\circlearrowleft}, \circlearrowleft, \blacktriangle), (\circlearrowleft, \blacktriangle, \boxed{\circlearrowleft})\}
\end{aligned}$$

Given the *wff* α , and an interpretation I , we say that I *satisfies* α , under the following conditions:

1. If α is an atomic *wff*, a *wff* of the form $P_i^n(t_1, \dots, t_n)$, the interpretation I satisfies α if and only if the n -tuple $(I(t_1), \dots, I(t_n))$ is an element of the relation $I(P_i^n)$.
2. The interpretation I satisfies the *wff* $\neg\alpha$ if and only if I does not satisfy α .
3. The interpretation I satisfies the *wff* $\alpha \wedge \beta$ if and only if I satisfies both *wffs* α and β .

4. The interpretation I satisfies the *wff* $\alpha \vee \beta$ if and only if I satisfies at least one of the *wffs* α or β .
5. The interpretation I satisfies the *wff* $\alpha \rightarrow \beta$ if and only if I does not satisfy α or I satisfies β .
6. The interpretation I satisfies the *wff* $\forall x[\alpha]$ if and only if, for all substitutions $\{x/a\}$, where “ a ” is any individual constant, I satisfies $\alpha \cdot \{x/a\}$.
7. The interpretation I satisfies the *wff* $\exists x[\alpha]$ if and only if there is a substitution $\{x/a\}$, where “ a ” is an individual constant, such that I satisfies $\alpha \cdot \{x/a\}$.

In the example that we have been considering, the interpretation I satisfies the *wffs* $Tree(t_1)$ and $Adjacent(h_2, left(t_2))$ for the following reasons:

1. $Tree(t_1)$.

The tuple $(I(t_1)) = (\circlearrowleft)$ belongs to the relation $I(Tree) = \{(\circlearrowleft), (\blacktriangle)\}$;

2. $Adjacent(h_2, left(t_2))$.

The tuple $(I(h_2, left(t_2))) = (I(h_2), I(left(t_2))) = (\blacksquare, I(left(I(t_2)))) = (\blacksquare, I(left)(\blacktriangle)) = (\blacksquare, \circlearrowleft)$ belongs to the relation $I(Adjacent) = \{(\blacksquare, \circlearrowleft), (\circlearrowleft, \blacksquare), (\circlearrowleft, \blacktriangle), (\blacktriangle, \circlearrowleft), (\blacktriangle, \blacksquare), (\blacksquare, \blacktriangle)\}$.

We say that the *wff* α is *true under the interpretation* I if and only if I satisfies α ; otherwise, we say that α is *false under the interpretation* I . If α is true under the interpretation I , we write $\models_I \alpha$.

An interpretation that satisfies all the *wffs* in a set of *wffs* is said to be a *model* of that set of *wffs*. If there is a model for a set of *wffs*, the set is said to be *satisfiable*.

Under this perspective, we say that the argument (Δ, α) is valid if and only if all the models of the premises are also models of the conclusion.

Going back to our previous example, the interpretation I satisfies all the *wffs* in the following set of *wffs* (expressions 2.2 to 2.11):

$Tree(t_1)$

$Tree(t_2)$

$House(h_1)$

$House(h_2)$

$Adjacent(h_2, t_1)$

$Adjacent(t_1, t_2)$

$Adjacent(t_2, h_1)$

$Adjacent(h_2, left(t_2))$

$\forall(x, y)[Adjacent(x, y) \rightarrow Adjacent(y, x)]$

$\forall(x, y, z)[(Adjacent(x, y) \wedge Adjacent(y, z) \rightarrow InBetween(x, y, z)]$

Thus, this interpretation is a model of these *wffs*.

However, if we consider the interpretation J , defined in the following way:

$J(t_1) \mapsto \blacktriangle$

$J(t_2) \mapsto \circ$

$J(h_1) \mapsto \hat{\square}$

$J(h_2) \mapsto \hat{\square}$

$J(left) \mapsto \{(\circ, \hat{\square}), (\blacktriangle, \circ), (\hat{\square}, \blacktriangle)\}$

$J(right) \mapsto \{(\hat{\square}, \circ), (\circ, \blacktriangle), (\blacktriangle, \hat{\square})\}$

$J(Tree) \mapsto \{(\circ), (\blacktriangle)\}$

$J(House) \mapsto \{(\hat{\square}), (\hat{\square})\}$

$J(Adjacent) \mapsto \{(\hat{\square}, \circ), (\circ, \hat{\square}), (\circ, \blacktriangle), (\blacktriangle, \circ), (\blacktriangle, \hat{\square}), (\hat{\square}, \blacktriangle)\}$

$J(InBetween) \mapsto \{(\hat{\square}, \circ, \blacktriangle), (\circ, \blacktriangle, \hat{\square})\}$

The *wffs* $Adjacent(h_2, t_1)$, and $Adjacent(t_2, h_1)$, for example, have the value false; thus, J is not a model of the *wffs* above.

2.4.3 Soundness and completeness of first-order logic

The following properties can be shown for first-order logic. Their proof is beyond the aim of this book.

Theorem 8 (Soundness) *First-order logic is sound.*

Theorem 9 (Completeness) *First-order logic is complete.*

2.5 Representation using logic

In this section, we turn into the main objective of this book, knowledge representation and reasoning, and start discussing how we can use logic in the task of representing knowledge.

As with any representation formalism, the first step that we have to take when using logic for knowledge representation is to “invent” the model of whatever we need to represent. This invention corresponds to the conceptualisation, where we define the objects that we want to talk about (the universe of discourse), the functions, and the relations that we want to use.

Before we start representing knowledge, we have to define the constants, functions, and relations that are needed. The constants correspond to the named individuals of the domain that we want to talk about. Among the relations, we have to define some *classes* or *properties* — one-argument relations — that circumscribe the type of basic entities or describe properties that these have. In doing this, we are defining the basic *vocabulary* that is used in the *wffs* of our representation. In reference to the formation rules presented in page 67, we are complementing the basic symbols of the language (punctuation and logical symbols) with domain-specific function and predicate symbols.

It is a good practice to write down some indication about the conceptualisation (although not the conceptualisation itself) before starting any representation task.

Afterwards, the representation task consists in defining *wffs* — that will be considered as premises — that relate the several constants, relations, and functions of the domain. It is common to say that these *wffs* correspond to



Figure 2.6: The Simpson's family tree.

proper axioms — propositions that hold for the domain and are accepted without proof.

2.5.1 Kinship relations

As a first example, we discuss concepts regarding family relations. To illustrate our example, we use a well-known family (Figure 2.6).¹⁹

Vocabulary. In the vocabulary, we define the constants that we want to use, together with the relations and functions. Alongside with each relation and function, we provide an intended meaning for it. We use the expression

¹⁹This image was obtained from <http://duffzone.co.uk/desktops/tree/>. Reprinted with kind permission from www.duffzone.co.uk.

intended meaning because the actual meaning is denoted by the tuples that belong to the set that corresponds to the relation or to the function (as described in Section 2.4.2).

We use the following constants to denote each of the members of the family, *Hugo* (Hugo Simpson), *Bart* (Bart J. Simpson), *Lisa* (Lisa Simpson), *Maggie* (Maggie Simpson), *Herb* (Herb Powers), *Homer* (Homer J. Simpson), *Marge* (Marge B. Simpson), *Selma* (Selma Bouvier), *Paty* (Paty Bouvier), *Mrs.Y* (the person without name, shown in the lower left-hand corner of the tree), *Abe* (Abraham Simpson), *Penelope* (Penelope Olsen), *Jackie* (Jackie Bouvier), and *Mr.Bouvier* (Mr. Bouvier).

We use two classes (relations with one argument), *Male* and *Female*, with the following intended meanings:

$$Male(x) = x \text{ is a male}$$

$$Female(x) = x \text{ is a female}$$

The main relation we use to introduce ground *uffs* is that of being a parent. The two argument relation, *Parent* has the following intended meaning:

$$Parent(x, y) = x \text{ is a parent of } y$$

We also use the relations, *Father*, *Mother*, *Grandfather*, *Grandmother*, *Grandparent*, *Ancestor*, *Son*, *Daughter*, *Brother*, *Sister*, and *Sibling* with the following intended meanings:

$$Grandparent(x, y) = x \text{ is a grandparent of } y$$

$$Ancestor(x, y) = x \text{ is an ancestor of } y$$

$$Father(x, y) = x \text{ is the father of } y$$

$$Mother(x, y) = x \text{ is the mother of } y$$

$$Grandfather(x, y) = x \text{ is a grandfather of } y$$

$$Grandmother(x, y) = x \text{ is a grandmother of } y$$

$$Child(x, y) = x \text{ is a child of } y$$

$$Son(x, y) = x \text{ is a son of } y$$

$$Daughter(x, y) = x \text{ is a daughter of } y$$

$$Brother(x, y) = x \text{ is a brother of } y$$

$$Sister(x, y) = x \text{ is a sister of } y$$

$$Sibling(x, y) = x \text{ and } y \text{ are siblings}$$

Formulas. With these relations, based on the information in Figure 2.6, we can write down the following ground formulas:

$$\text{Male}(\text{Mr.Bouvier}) \quad (2.12)$$

$$\text{Male}(\text{Abe}) \quad (2.13)$$

$$\text{Male}(\text{Homer}) \quad (2.14)$$

$$\text{Male}(\text{Herb}) \quad (2.15)$$

$$\text{Male}(\text{Hugo}) \quad (2.16)$$

$$\text{Male}(\text{Bart}) \quad (2.17)$$

$$\text{Female}(\text{Jackie}) \quad (2.18)$$

$$\text{Female}(\text{Penelope}) \quad (2.19)$$

$$\text{Female}(\text{Mrs.Y}) \quad (2.20)$$

$$\text{Female}(\text{Marge}) \quad (2.21)$$

$$\text{Female}(\text{Selma}) \quad (2.22)$$

$$\text{Female}(\text{Patty}) \quad (2.23)$$

$$\text{Female}(\text{Lisa}) \quad (2.24)$$

$$\text{Female}(\text{Maggie}) \quad (2.25)$$

$$\text{Parent}(\text{Mr.Bouvier}, \text{Marge}) \quad (2.26)$$

$$\text{Parent}(\text{Jackie}, \text{Marge}) \quad (2.27)$$

$$\text{Parent}(\text{Mr.Bouvier}, \text{Selma}) \quad (2.28)$$

$$\text{Parent}(\text{Jackie}, \text{Selma}) \quad (2.29)$$

$$\text{Parent}(\text{Mr.Bouvier}, \text{Patty}) \quad (2.30)$$

$$\text{Parent}(\text{Jackie}, \text{Patty}) \quad (2.31)$$

$$\text{Parent}(\text{Abe}, \text{Homer}) \quad (2.32)$$

$$\text{Parent}(\text{Pamela}, \text{Homer}) \quad (2.33)$$

$$\text{Parent}(\text{Abe}, \text{Herb}) \quad (2.34)$$

$$\text{Parent}(\text{Mrs.Y}, \text{Herb}) \quad (2.35)$$

$$\text{Parent}(\text{Homer}, \text{Hugo}) \quad (2.36)$$

$$Parent(Marge, Hugo) \quad (2.37)$$

$$Parent(Homer, Bart) \quad (2.38)$$

$$Parent(Marge, Bart) \quad (2.39)$$

$$Parent(Homer, Lisa) \quad (2.40)$$

$$Parent(Marge, Lisa) \quad (2.41)$$

$$Parent(Homer, Maggie) \quad (2.42)$$

$$Parent(Marge, Maggie) \quad (2.43)$$

Using the relations, we can write down the following *wffs* that correspond to the proper axioms of our domain.

$$\forall x, y, z [(Parent(x, y) \wedge Parent(y, z)) \rightarrow Grandparent(x, z)] \quad (2.44)$$

$$\forall x, y [Parent(x, y) \rightarrow Ancestor(x, y)] \quad (2.45)$$

$$\forall x, y, z [(Ancestor(x, y) \wedge Parent(y, z)) \rightarrow Ancestor(x, z)] \quad (2.46)$$

$$\forall x, y [(Parent(x, y) \wedge Male(x)) \leftrightarrow Father(x, y)] \quad (2.47)$$

$$\forall x, y [(Parent(x, y) \wedge Female(x)) \leftrightarrow Mother(x, y)] \quad (2.48)$$

$$\forall x, y [(Grandparent(x, y) \wedge Male(x)) \leftrightarrow Grandfather(x, y)] \quad (2.49)$$

$$\forall x, y [(Grandparent(x, y) \wedge Female(x)) \leftrightarrow Grandmother(x, y)] \quad (2.50)$$

$$\forall x, y [Parent(x, y) \leftrightarrow Child(y, x)] \quad (2.51)$$

$$\forall x, y [(Child(x, y) \wedge Male(x)) \leftrightarrow Son(x, y)] \quad (2.52)$$

$$\forall x, y [(Child(x, y) \wedge Female(x)) \leftrightarrow Daughter(x, y)] \quad (2.53)$$

$$\forall x, y [(Brother(x, y) \vee Sister(x, y)) \rightarrow Sibling(x, y)] \quad (2.54)$$

You should notice that some of these *wffs* use the equivalence connective. The use of this connective means that the *wff* contains the necessary and sufficient conditions for the definition of a predicate — for example, with *wff* 2.47, *Father* is completely defined by the combination of *Parent* and *Male*. *Wffs* of this kind are usually called *full definitions*.

Another kind of *wff* that is very common to use, called *disjointness*, state that some combinations of relation are distinct. For example, we know that the relation *Ancestor* is not reflexive:

$$\forall x, y [Ancestor(x, y) \rightarrow \neg Ancestor(y, x)] \quad (2.55)$$

The set of all individuals, functions, relations, and proper axioms are usually called the *ontology* of the domain. This is further discussed in Chapter 9.

Proofs. We can now use *wffs* 2.12 to 2.54 as premises in order to derive information about the family. For example, the following proof (that only uses the relevant premises — 2.13, 2.14, 2.17, 2.32, 2.38, 2.44, and 2.49) shows that *Abe* is a grandfather of *Bart*:

1	$Male(Abe)$	Prem
2	$Male(Homer)$	Prem
3	$Male(Bart)$	Prem
4	$Parent(Abe, Homer)$	Prem
5	$Parent(Homer, Bart)$	Prem
6	$\forall x, y, z [(Parent(x, y) \wedge Parent(y, z))$ $\rightarrow Grandparent(x, z)]$	Prem
7	$\forall x, y [(Grandparent(x, y) \wedge Male(x))$ $\leftrightarrow Grandfather(x, y)]$	Prem
8	$(Parent(Abe, Homer) \wedge$ $Parent(Homer, Bart)) \rightarrow$ $Grandparent(Abe, Bart)$	$\forall E, 6$
9	$Parent(Abe, Homer) \wedge Parent(Homer, Bart)$	$\wedge I, (4, 5)$
10	$Grandparent(Abe, Bart)$	$\rightarrow E, (8, 9)$
11	$(Grandparent(Abe, Bart) \wedge Male(Abe)) \leftrightarrow$ $Grandfather(Abe, Bart)$	$\forall E, 6$
12	$(Grandparent(Abe, Bart) \wedge Male(Abe)) \rightarrow$ $Grandfather(Abe, Bart)$	$\leftrightarrow E, 11$
13	$Grandparent(Abe, Bart) \wedge Male(Abe)$	$\wedge I, (10, 1)$
14	$Grandfather(Abe, Bart)$	$\rightarrow E, (12, 13)$

Let us now consider the following puzzle:²⁰ *When Homer, Marge, Bart, and Lisa gather together for breakfast, each of them eats either doughnut or cereal. (1) If Homer eats cereal, then Marge eats doughnut; (2) Either Homer or Bart eats cereal, but not both; (3) Marge and Bart do not both eat doughnut. What does Homer eat?* In order to solve this puzzle, we have first to translate its statement into first-order logic. We introduce the constants *doughnut* and *cereal* and the relation *Eats*, with the following meaning

$$Eats(x, y) = x \text{ eats } y$$

The first statement that we represent is that *each of them eats either doughnut or cereal*. This is represented by *wffs* 2.56 and 2.57; *If Homer eats cereal, then Marge eats doughnut* is represented by the *wff* 2.58, *Either Homer or*

²⁰Adapted from [Summers 72, p. 3].

Bart eats cereal, but not both is represented by the wffs 2.59 and 2.60, and *Marge and Bart do not both eat doughnut* is represented by the wff 2.61:

$$\forall x [Eats(x, doughnut) \vee Eats(x, cereal)] \quad (2.56)$$

$$\forall x [Eats(x, doughnut) \rightarrow \neg Eats(x, cereal)] \quad (2.57)$$

$$Eats(Homer, cereal) \rightarrow Eats(Marge, doughnut) \quad (2.58)$$

$$Eats(Homer, cereal) \vee Eats(Bart, cereal) \quad (2.59)$$

$$\neg(Eats(Homer, cereal) \wedge Eats(Bart, cereal)) \quad (2.60)$$

$$\neg(Eats(Marge, doughnut) \wedge Eats(Bart, doughnut)) \quad (2.61)$$

A characteristic of this puzzle is that there is no straightforward path from the propositions in the puzzle's statement to the puzzle's solution. In other words, in solving this puzzle one has to raise hypotheses, reason from them and if a contradiction is detected replace some of those hypotheses.

In order to make the reasoning simpler, we use the derived rule on inference $\neg\vee I$ (presented in page 51) and also introduce a derived rule of inference, called R_1 , that states that from $\alpha \vee \beta$, $\alpha \rightarrow \neg\beta$ and $\neg\beta$ we can derive α (as an exercise, you are asked to justify this derived rule of inference — see page 104):

$$\begin{array}{ll} n & \alpha \vee \beta \\ n+1 & \alpha \rightarrow \neg\beta \\ n+2 & \neg\beta \\ n+3 & \alpha \end{array} \quad R_1, (n, n+1, n+2)$$

The proof in Figure 2.7 shows that Homer eats doughnut (as you might be expecting).

1	$\forall x [Eats(x, doughnut) \vee Eats(x, cereal)]$	Prem
2	$\forall x [Eats(x, doughnut) \rightarrow \neg Eats(x, cereal)]$	Prem
3	$Eats(Homer, cereal) \rightarrow Eats(Marge, doughnut)$	Prem
4	$Eats(Homer, cereal) \vee Eats(Bart, cereal)$	Prem
5	$\neg(Eats(Homer, cereal) \wedge Eats(Bart, cereal))$	Prem
6	$\neg(Eats(Marge, doughnut) \wedge Eats(Bart, doughnut))$	Prem
7	$Eats(Homer, cereal)$	Hyp
8	$Eats(Homer, cereal) \rightarrow Eats(Marge, doughnut)$	Rei, 3
9	$Eats(Marge, doughnut)$	$\rightarrow E$, (7, 8)
10	$Eats(Bart, doughnut)$	Hyp
11	$Eats(Marge, doughnut) \wedge$ $Eats(Bart, doughnut)$	$\wedge I$, (9, 10)
12	$\neg(Eats(Marge, doughnut) \wedge$ $Eats(Bart, doughnut))$	Rei, 6
13	$\neg Eats(Bart, doughnut)$	$\neg I$, (10, (11, 12))
14	$Eats(Bart, cereal)$	Hyp
15	$Eats(Homer, cereal) \wedge Eats(Bart, cereal)$	$\wedge I$, (7, 14)
16	$\neg(Eats(Homer, cereal) \wedge Eats(Bart, cereal))$	Rei, 5
17	$\neg Eats(Bart, cereal)$	$\neg I$, (14, (15, 16))
18	$Eats(Bart, doughnut) \vee Eats(Bart, cereal)$	$\vee E$, 1
19	$\neg(Eats(Bart, doughnut) \vee Eats(Bart, cereal))$	$\neg \vee I$, (13, 17)
20	$\neg Eats(Homer, cereal)$	$\neg I$, (7, (17, 19))
21	$Eats(Homer, doughnut) \vee Eats(Homer, cereal)$	$\vee E$, 1
22	$Eats(Homer, doughnut) \rightarrow \neg Eats(Homer, cereal)$	$\vee E$, 2
23	$Eats(Homer, doughnut)$	R_1 , (21, 22, 20)

Figure 2.7: Homer eats doughnut.

Discussion. At this point, we should think a bit about the representation that we have used so far. We know that a person has only one father, although nothing in our representation imposes this. If we want to impose this extra restriction, we could write the following *wff*:

$$\forall x, y, z [(Father(x, z) \wedge Father(y, z)) \rightarrow Eq(x, y)]$$

where *Eq* is the relation

$$Eq(x, y) = x \text{ is equal to } y$$

The relation *Eq* is so much used in logic, that often the expression *Eq*(*x*, *y*) abbreviated as $x = y$. Deciding whether two terms are equal may be far from being a trivial matter (this is further discussed in Chapter 5). For the purposes of this section, we consider that this relation satisfies the following theorems:

$$\forall x [x = x]$$

$$\forall x, y [(x = y) \rightarrow (y = x)]$$

$$\forall x, y, z [(x = y) \wedge (y = z) \rightarrow (x = z)]$$

Besides having introduced the relation *Father*, we could have chosen to use the function *father*, as well. This function is defined as

$$father(x) = \text{the father of } x$$

In this case, we could use the following *wffs*

$$\forall x [Male(father(x))]$$

$$\forall x [(Parent(father(x), x) \wedge Male(father(x)) \leftrightarrow Father(father(x), x)]$$

Wff 2.54 could have, alternatively, been written as two separated *wffs*,

$$\forall x, y [Brother(x, y) \rightarrow Sibling(x, y)] \quad (2.62)$$

$$\forall x, y [Sister(x, y) \rightarrow Sibling(x, y)] \quad (2.63)$$

The decision to use either *wff* 2.54 or *wffs* 2.62 and 2.63 is mainly a matter of ease of use.



Figure 2.8: Parts of an airplane.

2.5.2 World of airplanes

In this section, we present an example of a domain that enables to introduce two important aspects related with knowledge representation, dealing with properties of objects and dealing with hierarchies. Furthermore, we introduce the notion of *structured objects* (objects composed of parts) and *abstract individuals*.

To introduce some basic concepts, we use the representation in Figure 2.8²¹ that shows the main parts of an airplane. This figure shows that a “typical” airplane is composed of several components, wings, fuselage, fin, landing gear and so on. Each one of these components has, in turn, other components — for example, a wing has an aileron and a flap. Most of airplanes also have a component, not explicitly shown in Figure 2.8, an engine.

If we use the predicates *Plane* and *Has_part*, with the intended meanings

$$Plane(x) = x \text{ is an airplane}$$

$$Has_part(x, y) = x \text{ has } y \text{ as one of its parts}$$

²¹Courtesy Cislunar Aerospace, Inc.

we could write the *wff*

$$\forall x[Plane(x) \rightarrow Has_part(x, left_wing)] \quad (2.64)$$

However, in this *wff* there is nothing that tells us what a *left_wing* is, other than being a term denoted by a particular symbol. If we want to talk about the left wing of an airplane we should provide a definition for it. We introduce two predicates, *Wing* and *Left_wing* with the following intended meanings

Wing(*x*) = *x* is a wing

Left_wing(*x*) = *x* is a left wing

These predicates are related by the expression

$$\forall x[Left_wing(x) \rightarrow Wing(x)] \quad (2.65)$$

We now rewrite *wff* 2.64 as follows

$$\forall x[Plane(x) \rightarrow \exists y [Left_wing(y) \wedge Has_part(x, y)]] \quad (2.66)$$

We know that planes may have tens of thousands of parts and, if we go on, with this representation, we will end up in a big mess. We also know from programming that abstraction is a key issue in dealing with complex situations or objects. We use the abstraction concept to describe our airplane as a certain number of abstraction levels.

Each abstraction level corresponds to a *structured object*, an object composed by a certain number of components. The components of structured objects will be related by an assertion that states that if an object has a part that corresponds to a certain component, which, in turn, is composed by other parts, then the parts of the component are also part of the original object:

$$\forall x, y, z [(Has_part(x, y) \wedge Has_part(y, z)) \rightarrow Has_part(x, z)] \quad (2.67)$$

Using this approach, we can then introduce the highest abstraction level that defines the main components of an airplane:

$$\begin{aligned} \forall x[Plane(x) \rightarrow & (\exists y [Fuselage(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Left_wing(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Right_wing(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Fin(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Stabilizer(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Engine(y) \wedge Has_part(x, y)] \wedge \\ & \exists y [Landing_gear(y) \wedge Has_part(x, y)])] \end{aligned} \quad (2.68)$$

1	$\forall x, y [(Has_part(x, y) \wedge Has_part(y, z))$	
	$\rightarrow Has_part(x, z)]$	Prem
3	$Plane(CS-TJF)$	Prem
4	$\exists y [Left_wing(y) \wedge Has_part(CS-TJF, y)]$	
5	$y_0 \mid Left_wing(y_0) \wedge Has_part(CS-TJF, y_0)$	Hyp
6	$Left_wing(y_0)$	
7	$Wing(y_0)$	
8	$\exists y [Flap(y) \wedge Has_part(y_0, y)]$	
9	$y_1 \mid Flap(y_1) \wedge Has_part(y_0, y_1)$	
10	$Has_part(CS-TJF, y_0)$	$\wedge E, 5$
11	$Has_part(y_0, y_1)$	$\wedge E, 9$
12	$Has_part(CS-TJF, y_0) \wedge Has_part(y_0, y_1)$	$\wedge I, 10, 11$
15	$Has_part(CS-TJF, y_1)$	$\rightarrow E, 1, 12$

Figure 2.9: Reasoning about parts of airplanes.

We should note that sentence 2.68 is not according to our definition of *wff*. In fact, the logical connective $\wedge$ only takes two arguments, two *wffs*, and not an arbitrary number of arguments as we used above. However, since it can be shown (see the exercise in page 103) that $\wedge$ is associative

$$(\alpha \wedge (\beta \wedge \gamma)) \leftrightarrow ((\alpha \wedge \beta) \wedge \gamma)$$

we use $(\alpha \wedge \beta \wedge \gamma)$ as an abbreviation of $(\alpha \wedge (\beta \wedge \gamma))$.

We can go on with a description of each one of the parts that compose an airplane, for example,

$$\begin{aligned} \forall x [Wing(x) \rightarrow (\exists y [Flap(y) \wedge Has_part(x, y)] \wedge \\ \exists y [Aileron(y) \wedge Has_part(x, y)])] \end{aligned} \quad (2.69)$$

At this point, we can use *wffs* 2.68 and 2.69 to reason about a particular airplane, say *CS-TJF*. In the proof in Figure 2.9 we have omitted several

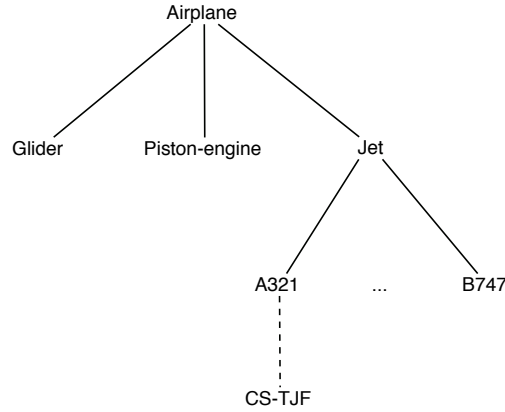


Figure 2.10: Classification of airplanes regarding propulsion power.

steps to make the outline of the reasoning simpler (the *wffs* that are originated from missing steps in the proof have no justifications). In this proof, within the “context” of assuming that y_0 and y_1 are the terms that denote the left wing of $CS-TJF$ and the flap of this left wing, respectively, we were able to conclude that y_1 is a part of $CS-TJF$.

Another aspect that we may want to do when talking about airplanes is to classify them according to certain criteria. In this classification task, it is common to group the entities of the domain in a *hierarchy*, where each element of the hierarchy either corresponds to a *class* (a set of entities that satisfy a certain one-argument relation) or corresponds to an *instance* (a named individual of the domain).

For example, if we use a classification regarding propulsion power, airplanes can be divided into *Gliders* (airplanes that do not have any propulsion), *Piston-engine* powered, and *Jet* engine powered. Among jet airplanes, we have, among others, the *A321* (Airbus 321) and the *B747* (Boeing 747).

The corresponding hierarchy, in an intuitive form, is shown in Figure 2.10. In this figure, solid lines represent class/subclass relations. For example, a *Jet* is an *Airplane*, and an *A321* is a *Jet*. Dashed lines represent the relation between a class and its members. For example, *CS-TJF* is a particular instance of an *A321*.

In order to represent the hierarchy in first-order logic, we start by writing

wffs that denote the relations between the classes in the hierarchy:²²

$$\forall x[Gl\ddot{u}der(x) \rightarrow Plane(x)] \quad (2.70)$$

$$\forall x[Piston_engine(x) \rightarrow Plane(x)] \quad (2.71)$$

$$\forall x[Jet(x) \rightarrow Plane(x)] \quad (2.72)$$

$$\forall x[A321(x) \rightarrow Jet(x)] \quad (2.73)$$

$$\forall x[B747(x) \rightarrow Jet(x)] \quad (2.74)$$

Wffs such as 2.70 to 2.74 that state the specialisation of relations — one relation is subsumed by another relation — are usually called *subtypes*.

We may also state the converse of the subtype relation, where some relations completely account for the super-type such as in

$$\forall x[Plane(x) \rightarrow (Gl\ddot{u}der(x) \vee Pist\ddot{o}n_engine(x) \vee Jet(x))] \quad (2.75)$$

$$\forall x[Jet(x) \rightarrow (A321(x) \vee \dots \vee B747(x))] \quad (2.76)$$

Wffs of this form express *exhaustiveness*.

Another kind of *wffs* that is common to use when we are expressing classifications correspond to *disjointness* relations — Gliders, Piston engines, and Jets are mutually disjoint:

$$\forall x[Gl\ddot{u}der(x) \rightarrow \neg Pist\ddot{o}n_engine(x)] \quad (2.77)$$

$$\forall x[Gl\ddot{u}der(x) \rightarrow \neg Jet(x)] \quad (2.78)$$

$$\forall x[Piston_engine(x) \rightarrow \neg Gl\ddot{u}der(x)] \quad (2.79)$$

$$\forall x[Piston_engine(x) \rightarrow \neg Jet(x)] \quad (2.80)$$

$$\forall x[Jet(x) \rightarrow \neg Gl\ddot{u}der(x)] \quad (2.81)$$

$$\forall x[Jet(x) \rightarrow \neg Pist\ddot{o}n_engine(x)] \quad (2.82)$$

As an exercise, you should convince yourself that *wffs* 2.77, 2.78, and 2.82 would be enough to express the disjointness relations in this case.

Finally, we may also want to state that a particular airplane, say *CS-TJF*, is an Airbus 321, whose name is “Luiz_Vaz_de_Camões”.²³

$$A321(CS-TJF) \quad (2.83)$$

$$Name_of(CS-TJF, Luiz_Vaz_de_Cam\ddot{o}es) \quad (2.84)$$

²²We are assuming that the meaning of the individual predicates is clear.

²³This is an airplane that belongs to the fleet of TAP Portugal.

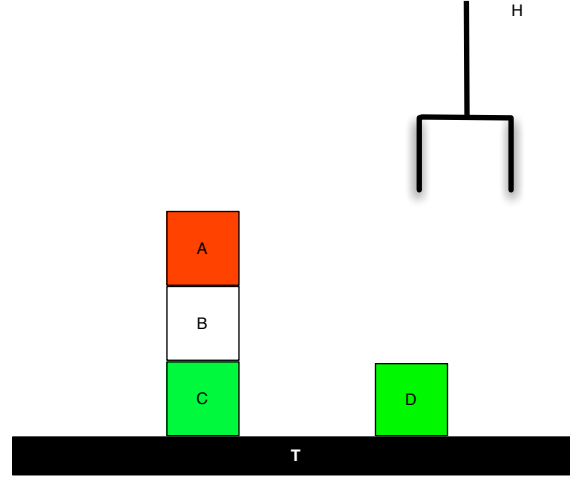


Figure 2.11: Example of the blocks world.

2.5.3 Blocks world

As a final example, we describe a very simple domain, known as the *blocks world*. The blocks world is composed of a table, on top of which there may be several blocks, and a robot's hand that is able to manipulate the position of the blocks in the table. This domain is commonly used to represent and to reason about actions and change (see Chapter ??). In Figure 2.11, we show a simple world of blocks, composed of four blocks, associated with the symbols A, B, C, and D. The table is associated with symbol the T and the robot's hand with the symbol H.

In the representation of the blocks world we use the predicates *Block*, *Hand*, *Table*, and *Colour* with the following intended meanings:

$Block(x) = x$ is a block

$Hand(x) = x$ is the robot's hand

$Table(x) = x$ is the table

$Colour(x, y) =$ The colour of x is y

The following *wffs* represent information about our world:

$$Hand(H) \tag{2.85}$$

$$Table(T) \quad (2.86)$$

$$Block(A) \quad (2.87)$$

$$Block(B) \quad (2.88)$$

$$Block(C) \quad (2.89)$$

$$Block(D) \quad (2.90)$$

$$Colour(A, red) \quad (2.91)$$

$$Colour(B, red) \vee Colour(B, green) \quad (2.92)$$

$$Colour(C, green) \quad (2.93)$$

$$Colour(D, green) \quad (2.94)$$

We also define the predicates *Clear_top*, *On_table*, *On_top*, and *Over* with the following intended meanings:

$$Clear_top(x) = \text{There is no block on top of } x$$

$$On_table(x) = x \text{ is directly on the table}$$

$$On_top(x, y) = x \text{ is directly on top of } y$$

$$Over(x, y) = x \text{ is stacked over } y$$

These predicates are related by the following *wffs*

$$\forall x, y [Clear_top(x) \rightarrow \neg On_top(y, x)] \quad (2.95)$$

$$\forall x, y [On_top(x, y) \rightarrow \neg Clear_top(y)] \quad (2.96)$$

$$\forall x, y [On_top(x, y) \rightarrow Over(x, y)] \quad (2.97)$$

$$\forall x, y, z [(Over(x, y) \wedge Over(y, z)) \rightarrow Over(x, z)] \quad (2.98)$$

The following *wffs* make statements about the blocks in Figure 2.11:

$$Clear_top(A) \quad (2.99)$$

$$Clear_top(D) \quad (2.100)$$

$$On_top(A, B) \quad (2.101)$$

$$On_top(B, C) \quad (2.102)$$

$$On_table(C) \quad (2.103)$$

$$On_table(D) \quad (2.104)$$

If we define the predicate *Over_colour* as with the intended meaning

$Over_colour(c_1, c_2) =$ There is a block of colour c_1 over a block of colour c_2

we can write the *wff*

$$\forall x, y, c_1, c_2 [(On_top(x, y) \wedge Colour(x, c_1) \wedge Colour(y, c_2)) \rightarrow Over_colour(c_1, c_2)] \quad (2.105)$$

The following proof shows that there is a *green* block *On_top* of a *red* block:

1	$On_top(A, B)$	Prem
2	$On_top(B, C)$	Prem
3	$Colour(A, red)$	Prem
4	$Colour(C, green)$	Prem
5	$Colour(B, red) \vee Colour(B, green)$	Prem
6	$Colour(B, red)$	Hyp
7	$Colour(C, green)$	Rei, 4
8	$On_top(B, C)$	Rei, 2
9	$Over_colour(red, green)$	
10	$Colour(B, green)$	Hyp
11	$Colour(A, red)$	Rei, 3
12	$On_top(A, B)$	Rei, 1
13	$Over_colour(red, green)$	
14	$Over_colour(red, green)$	$\vee I, (5, (6, 9), (10, 13))$

2.6 Summary

In this chapter, we presented an introduction to classical logic, described the motivations for the development of a logic and the components of a logic, the deductive system, and the semantics. We discussed the two approaches for the development of the deductive system, natural deduction and axiomatic systems. We described how the semantics of a logic is evaluated and stated some of the properties that relate the deductive system with the semantics.

We described the role of logic in knowledge representation, by presenting some examples of application domains.

2.7 Bibliographic notes and historical remarks

Logic has been studied for more than 2000 years. An historical account of the development of logic can be found in [Kneale and Kneale 88].

The methodology for deciding whether a given argument is valid or invalid was first discovered by Aristotle (about 350 B.C.) and was re-articulated in modern form by Alfred Tarski (1901–1983). A good discussion of form of an argument can be found in [Haack 78, pp. 22–27]. The truth-value semantics of propositional logic was invented in 1854 by George Boole (1815–1864) [Boole 54]. Gottlob Fregue (1848–1925) introduced in 1879 the concept of formal language (the original paper is reprinted in [van Heijenoort 67]). The methodology we presented to compute the truth value of *wffs* was introduced by Alfred Tarski in 1936. A reprint of the original paper by Tarski can be found in [Corcoran 83]. The completeness of first-order logic was first proved by Kurt Gödel (1906–1978) [Gödel 30].

Natural deduction systems have their origins in the work of Gerhard Gentzen (1909–1945) in 1934 (reprinted in [Gentzen 69]) and Stanislaw Jaskowski (1906–1965) [Jaskowski 34]. A good presentation of an alternative to natural deduction — axiomatic systems — can be found in [Mendelson 87].

Books on logic usually focus on three broad categories, *philosophical logic* [Haack 78], *mathematical logic* [Church 56], [Fitch 52], [Kleene 52], [Lemon 78], [Tarski 65], [Whitehead and Russell 10], and *computer science* [Ben-Ari 03], [Huth and Ryan 04]. The encyclopedia articles by [Rapaport 92a, 92b] present a rigorous account on logic and on the role of logic in Artificial Intelligence.

Logic has been proposed as a knowledge-representation formalism since the very beginnings of the work Artificial Intelligence. However, the first proposal to use logic as a knowledge-representation formalism precedes the birth of Artificial Intelligence, and was done in the 17th century by Gottfried Leibnitz (1646–1716) — see [Brachman and Levesque 85, p. xiv].

The first paper on knowledge representation in Artificial Intelligence, written by John McCarthy in 1958 (reprinted years later, [McCarthy 68]), proposed that the knowledge of an intelligent program should be expressed by declar-

ative sentences, in a way that was independent of the way knowledge was going to be used; it also proposed that the reasoning performed by such a program should use logical operations over those declarative sentences. McCarthy's approach originated what is known as the *logician approach to AI*, which has a large number of followers — see [Hayes 77], [Moore 82, 84b], [Israel 87], [McCarthy 88], and [Nilsson 91]. The compilation by [Brachman and Smith 80] presents a perspective of the positions of several researchers who work on knowledge representation. Many of the questions that are discussed directly concern the use of logic as a knowledge representation formalism. [Israel 93] provides a description of the different roles of logic in Artificial Intelligence.

In 1991 the journal *Artificial Intelligence* devoted a special issue to the foundations of AI. In this issue there are two important papers, one by Nils Nilsson [Nilsson 91], defending the logicist point of view, and an answer by Lawrence Birnbaum [Birnbaum 91] presenting the point of view of the people who do not believe in the logicist approach. The study of these papers helps understanding the arguments in favour and against logic-based knowledge representation systems.

2.8 Exercises

1. Tell whether the following arguments are valid or invalid:

- (a) Stu is human
Bill is human
∴ Stu is Bill
- (b) Homer Dodgson is Lewis Carroll
Mark Twain is not Lewis Carroll
∴ Mark Twain is not Homer Dodgson
- (c) Mark Twain is human
Mark Twain is not Lewis Carroll
∴ Lewis Carroll is not human
- (d) Homer Dodgson is Lewis Carroll
Homer Dodgson is a mathematician
Mathematicians are humans
∴ Lewis Carroll is human

- (e) Grass is green
 $\therefore$ Grass is green
2. Using the definition of *wff*, tell which of the following expressions are *wffs*:
- (a) $(P \vee (Q \wedge R)) \rightarrow ((P \vee Q) \wedge (P \vee R))$
 - (b) $((P \vee Q) \rightarrow R) \rightarrow (\neg P \vee R)$
 - (c) $((P_1 \vee P_2) \wedge (\neg P_3))$
 - (d) $(P \neg \vee Q) \wedge$
 - (e) $((P \vee Q) \wedge (\neg R)) \rightarrow D$
 - (f) $(P \vee Q) \wedge \rightarrow R$
 - (g) $P \rightarrow Q \rightarrow \rightarrow (P \vee Q)$
3. Using natural deduction, provide a proof for the following arguments:
- (a) $(\{P \rightarrow \neg P\}, \neg P)$
 - (b) $(\{P, Q\}, (P \wedge Q))$
4. Using natural deduction, provide the following proofs:
- (a) $\{\neg P \vee Q\} \vdash P \rightarrow Q$
 - (b) $\{\neg P \vee Q\} \vdash \neg (P \wedge \neg Q)$
 - (c) $\{\neg(P \wedge \neg Q)\} \vdash P \rightarrow Q$
 - (d) $\{(P \rightarrow (Q \wedge R))\} \vdash ((P \rightarrow Q) \wedge (P \rightarrow R))$
5. Using natural deduction, prove the following theorems, that are known as De Morgan's laws:
- (a) $\neg(\neg P \wedge \neg Q) \rightarrow (P \vee Q)$
 - (b) $\neg(P \wedge Q) \leftrightarrow (\neg P \vee \neg Q)$
 - (c) $(\neg P \vee \neg Q) \rightarrow \neg(P \wedge Q)$
 - (d) $(\neg P \wedge \neg Q) \rightarrow \neg(P \vee Q)$
6. Using natural deduction, prove the following theorems:
- (a) $(P \wedge Q) \leftrightarrow (Q \wedge P)$
 - (b) $(P \wedge (Q \wedge R)) \leftrightarrow ((P \wedge Q) \wedge R)$

- (c) $(P \vee Q) \leftrightarrow (Q \vee P)$
- (d) $(P \vee (Q \vee R)) \leftrightarrow ((P \vee Q) \vee R)$
- (e) $(P \rightarrow Q) \leftrightarrow (\neg P \vee Q)$

7. Using natural deduction, prove the following theorems:

- (a) $((P \rightarrow (P \rightarrow Q)) \wedge P) \rightarrow Q$
- (b) $(\neg P \rightarrow \neg Q) \rightarrow (Q \rightarrow P)$
- (c) $((P \rightarrow Q) \wedge (Q \rightarrow R)) \rightarrow (P \rightarrow R)$
- (d) $((P \vee Q) \rightarrow R) \rightarrow (P \rightarrow R)$
- (e) $(P \rightarrow (P \rightarrow Q)) \rightarrow (P \rightarrow Q)$
- (f) $(P \rightarrow (Q \rightarrow R)) \rightarrow ((P \rightarrow Q) \rightarrow (P \rightarrow R))$
- (g) $(P \wedge Q) \rightarrow \neg(P \rightarrow \neg Q)$
- (h) $(P \rightarrow \neg P) \rightarrow \neg P$
- (i) $(P \vee Q) \rightarrow (Q \vee P)$
- (j) $((P \vee Q) \vee R) \rightarrow (P \vee (Q \vee R))$
- (k) $(P \wedge (Q \vee R)) \rightarrow ((P \wedge Q) \vee (P \wedge R))$
- (l) $(P \vee \neg Q) \rightarrow \neg(\neg P \wedge Q)$
- (m) $((P \wedge Q) \vee P) \rightarrow P$

8. Present an argument that justifies the following derived rule of inference:

$$\begin{array}{ll}
 n & \alpha \vee \beta \\
 n+1 & \alpha \rightarrow \neg\beta \\
 n+2 & \neg\beta \\
 n+3 & \alpha
 \end{array}
 \quad R_1, (n, n+1, n+2)$$

9. Prove, using natural deduction:

- (a) $(F(a) \wedge \forall x [F(x) \rightarrow G(x)]) \rightarrow G(a)$
- (b) $(\forall x [F(x) \rightarrow G(x)] \wedge \forall y [G(y) \rightarrow H(y)]) \rightarrow \forall z [F(z) \rightarrow H(z)]$
- (c) $(\forall x [F(x) \rightarrow H(x)] \wedge \exists y [F(y)]) \rightarrow \exists z [H(z)]$

10. Prove the following theorems that correspond to two of the De Morgan laws for quantification:

- (a) $\neg\forall x[\neg P(x)] \rightarrow \exists x[P(x)]$
- (b) $\neg\exists x[\neg P(x)] \rightarrow \forall x[P(x)]$

11. Consider the following conceptualisation:

- *Universe of discourse.*
 $D = \{1, 2\}$
- *Set of functions.*
 $\{(1, 2), (2, 1)\}$
- *Set of relations.*
 $\{(1, 1), (1, 2)\}$

Suppose that the following *wffs* refer to a situation regarding this conceptualisation:

$$P(a, f(a)) \wedge P(b, f(b))$$

$$P(a, a)$$

$$\forall x [\exists y [P(x, y)]]$$

$$\forall x, y [P(x, y) \rightarrow P(f(x), f(y))]$$

Indicate which of the *wffs* above are satisfied by the following interpretation

$$I(a) \mapsto 1$$

$$I(b) \mapsto 2$$

$$I(f) \mapsto \{(1, 2), (2, 1)\}$$

$$I(P) \mapsto \{(1, 1), (1, 2)\}$$

12. Suppose that $P(x)$ and $Q(x)$ represent, respectively, that “ x is an integer number” and that “ x is a real number”. Using first-order logic, represent the following statements:²⁴

- (a) Every integer number is a real number.
- (b) Some real numbers are integer numbers.
- (c) Not every real number is an integer number.

13. Suppose that $P(x)$ stands for “ x is a politician” and that $H(x)$ stands for “ x is honest”. Express the following *wffs* in common language:²⁵

$$(a) \exists x [P(x)]$$

$$(b) \exists x [H(x)]$$

$$(c) \forall x [P(x) \rightarrow \neg H(x)]$$

²⁴Adapted from [Chang and Lee 73, p. 41].

²⁵*Ibid.*

- (d) $\exists x [P(x) \wedge H(x)]$
 - (e) $\exists x [P(x) \rightarrow H(x)]$
14. An Abelian group is a set A , together with a binary operator $+$, that satisfies certain properties. Suppose that the predicates, $A(x)$ represent that x is an Abelian group, $S(x, y, z)$ represents that “ $x + y = z$ ”, that the predicate $P(x, y)$ represents that “ x belongs to the set y ”, and that the predicate $E(x, y)$ represents that “ $x = y$ ”. Using first-order logic, express the following properties of an Abelian group:²⁶
- (a) For all x and y in A , if $x + y = z$ and $x + y = w$ then $z = w$.
 - (b) For all x, y and z in A , $(x + y) + z = x + (y + z)$.
 - (c) For all x and y in A , $x + y = y + x$.
 - (d) For all x and y in A , there exists z in A such that $x + y = z$.
15. Suppose that the predicates $P(x)$, $L(x)$, $B(x, y)$, $C(x, y, z)$ and $E(x, y)$ represent, respectively, that “ x is a point”, “ x is a straight line”, “point x belongs to the straight line y ”, “straight line x contains the points y and z ”, and “ x is equal to y ”. Using first-order logic, express the following propositions:
- (a) Given any straight line, there is at least a point that does not belong to the line.
 - (b) Given two points, they belong exactly to one straight line.
 - (c) There are at least two straight lines that go through the same point.
 - (d) Given any two distinct points there is one, and only one, straight line that goes through those points.
 - (e) Define the predicate $S(x, y)$ that corresponds to the fact that straight lines x and y are parallel. Two straight lines are parallel if they have no points in common.
16. Represent the following sentences, using first-order logic. Note that the representation of some of them may be harder than it seems at first. Actually, some of them are not representable in first-order logic. When this is the case explain why they are not representable. Note that a sentence may require more than one *wff*.

²⁶Adapted from [Chang and Lee 73, p. 42].

- (a) Only American citizens can be employed by the FBI.
 - (b) Every farmer who owns a donkey beats it.
 - (c) If a relation is reflexive, then for any x and y , if x and y satisfy the relation, then y and x also satisfy the relation.
 - (d) Merlin is either a cat or a dog.
 - (e) Any persistent person may learn logic.
 - (f) No politician is honest.
 - (g) Not every bird flies.
 - (h) If there is anything that can be done by someone, then Anna can do it.
 - (i) If anyone can sing, then so can Mary.
 - (j) John hates everyone that does not hate himself.
 - (k) Jill's father is married to her mother.
 - (l) Jack thinks that he remembers Jill's phone number.
 - (m) The number of pages of the book "The pearl" is even.
17. Based on the relations defined in Section 2.5.1, provide a definition for *Cousin*.
18. Represent in first-order logic the statements of the following puzzle (from [Summers 72]):
- Freeman knows five women: Ada, Bea, Cyd, Deb and Eve.*
- (a) *The women are in two age brackets: three women are under 30 and two women are over 30.*
 - (b) *Two women are teachers and the other three women are secretaries.*
 - (c) *Ada and Cyd are in the same age bracket.*
 - (d) *Deb and Eve are in different age brackets.*
 - (e) *Bea and Eve have the same occupation.*
 - (f) *Cyd and Deb have different occupations.*
 - (g) *Of the five women, Freeman will marry the teacher over 30.*

Using natural deduction, try to answer the following question: *Who will Freeman marry?*

Chapter 3

Non-Classical Logics

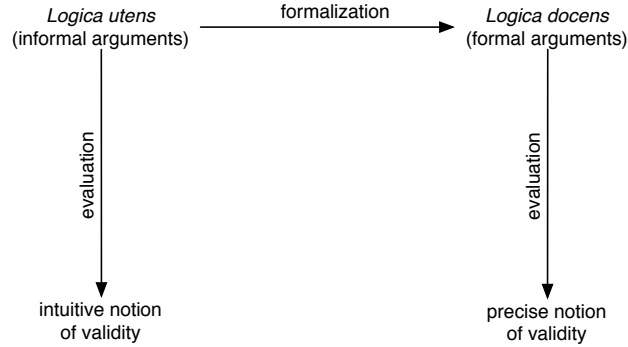
Ils n'avaient pas la même logique, l'un et l'autre. Ou peut-être ne voulait-elle pas avoir la même logique que le commissaire?

(They did not have the same logic. Or, perhaps didn't she want to have same logic as the deputy?)

Georges Simenon, *Les scrupules de Maigret*

The concepts of derivability and validity presented in Chapter 2 are defined for and applicable to formal arguments. In our daily life, we deal with informal arguments and we are able to classify them as valid or invalid (sometimes as good or bad arguments). In fact, we do much more than this, we consider arguments as “acceptable” or as “non-acceptable”, where an acceptable argument, besides being intuitively valid presents a coherent connection between premises and conclusion.

The question that we address at the outset of this chapter concerns the relationship between our intuitive notion of validity and the formal notion of validity introduced in a logic. Since logic formalizes arguments, representing them in a precise and generalizable form, in order for a logic system to be acceptable, it should present the following property: when an informal argument is translated into a corresponding formal argument, the formal argument must be valid if and only if the corresponding informal argument is intuitively valid.

Figure 3.1: *Logica utens* and *logica docens*.

We can start the development of a formal logic, based on the consideration of our intuitive concepts regarding the validity of informal arguments. We represent informal arguments using a formal language and we develop rules of inference and a method to evaluate the truth value of propositions. We do this task with the goal that informal arguments that are considered as valid, originate formal arguments that are valid.

Using the rules of inference and the semantics, some of the formal arguments that are classified as valid by the logic may correspond to informal arguments that are intuitively considered as invalid. In face of this situation, we can either revise the rules of the system in such a way that the arguments become invalid, or, if our intuitions about the invalidity of the argument are not particularly strong, we may revise our opinion about the validity of the informal argument. Once a logic becomes well established, it is likely that it will become the standard that we use to guide our intuitions about the validity and invalidity of informal arguments.

The intuitive logic that we use in our daily life to evaluate the validity or invalidity of arguments is called *logica utens*; the formal logic that is used for the evaluation of arguments is called *logica docens*. Figure 2.1 shows the relationship between *logica utens* and *logica docens* and the connections that these have with the notions of validity.

Since informal arguments contain ambiguities, the translation from informal arguments to the corresponding formal arguments entails the loss of something. The first step towards formalizing a logic corresponds to the choice of the set of logical connectives and of a logical language. This choice will de-

termine what formal arguments are representable in our logic and also what is lost when we go from an informal argument to its formal counterpart. Based on this choice, the formal system is developed. The results produced by this system are compared with our intuitive notion of validity. Once the system is “tuned”, it becomes the deciding factor about the validity or invalidity of arguments.

Different logics have been constructed with the goal of capturing different aspects of informal arguments. In this chapter we consider some logics, that are known as *non-classical logics*, because their set of valid arguments does not coincide with the set of valid arguments of classical logic.

Non-classical logics can be divided into two main groups:

1. *Logics that extend classical logic.* This group contains logics that accept all arguments of classical logic, plus some additional arguments.

In this case, the vocabulary of classical logic is extended in order for new notions to be representable. All axioms, rules of inference, and theorems of classical logic still belong to these logics. Examples are modal logic, epistemic logic, and deontic logic.

2. *Logics that challenge classical logic.* This group contains logics that do not accept as valid some of the arguments that are considered valid by classical logic.

The vocabulary of these logics may be the same as the vocabulary of classical logic, but either the axioms or the rules of inference are different. Examples are relevance logic, multiple-valued logic, and intuitionistic logic.

In this chapter, we look into logics that were developed following each one of these approaches.

3.1 Modal logic

In classical logic, formulas are either true or false in any model. It is often desirable to distinguish between several “kinds” (or “modes”) of truth, such as *necessarily true*, *believed to be true*, *known to be true*, *true in the future*, and so on.

Modal logic corresponds to a logic that extends classical logic by adding mechanisms to deal with propositions that include the notions of necessity and possibility:¹

- A proposition is necessarily true if it cannot be false under any circumstances. For example, *any person that has green eyes has eyes*.
- A proposition may be necessarily true, but may not be known to be true, for example, *adding a number to itself produces an even number*, is necessarily true but may not be known to be true by some people (children, for example).
- A proposition is possible if it is not necessarily false. For example, *the European Community is composed of 25 countries*, is a proposition that is true, but is not necessarily true, it has been false in the past and it is likely to be false in the future.

Although modal logic was introduced to deal with the notions of necessity and possibility, the same principles can be applied to other notions, thus the term “modal logic” is used more broadly for a family of related logical systems. These include logics for knowledge and belief (epistemic and doxastic logics), for tense and other temporal expressions (temporal logics), for moral expressions (deontic logics), and many others.

In this section, we look into modal logic. After the definition of the language, we define its semantics and present the deductive system. We start with propositional modal logic and afterwards we consider first-order modal logic.

3.1.1 The language of propositional modal logic

Since modal logic extends classical logic, its language uses some logical symbols that are not available in classical logic.

The language of propositional modal logic, denoted by $\mathcal{L}_{PML}$, is defined as follows:

Symbols of the language

1. *Punctuation symbols*: ()

¹These are called *modal notions*.

2. *Logical symbols:* $\neg \wedge \vee \rightarrow \Box \Diamond$

The symbols $\neg$, $\wedge$, $\vee$, and $\rightarrow$ have the same readings as in classical logic.

The symbol $\Box$ is read *it is necessary* and corresponds to the necessitation modal operator; the symbol $\Diamond$ is read *it is possible* and corresponds to the modal operator “possible”.

3. *Predicate symbols,* for $i > 0$, P_i

A predicate corresponds to a symbol that stands for a proposition. As before, when there is no danger of confusion, we will use upper-case roman letters, P , Q , R , etc., in place of predicate symbols.

Well formed formulas

1. Predicate symbols are *wffs*;
2. If α is a *wff*, then $\neg\alpha$, $\Box\alpha$, and $\Diamond\alpha$ are *wffs*;
3. If α and β are *wffs*, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, and $(\alpha \rightarrow \beta)$ are *wffs*;
4. Nothing else is a *wff*.

According to these definitions, if P , Q , and R are predicates, then the following are *wffs* of $\mathcal{L}_{PML}$:

$$\begin{aligned}
 &P \vee \Diamond Q \\
 &\Box((P \wedge Q) \rightarrow R) \\
 &\Diamond P \rightarrow \neg\Box\neg P \\
 &\Box\Box\Diamond\Box\neg P
 \end{aligned}$$

3.1.2 Semantics for propositional modal logic

As opposed to the approach we used in classical logic, in modal logic we start with the discussion of the semantics. This discussion enables a better understanding of the meaning of the modal operators.

The semantics of modal logic is based on the notion of possible worlds together with an accessibility relation between these worlds. Intuitively, a *possible world* is a hypothetical situation of the way the world could have

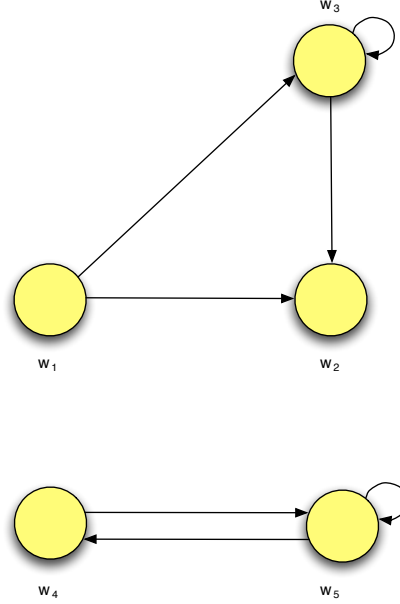


Figure 3.2: Example of a Kripke structure.

been.² Each possible world can be considered as a state of affairs in which we can evaluate *wffs* as we do in propositional logic. The accessibility relation defines if it is possible to go from the state of affairs described by one world to the state of affairs described by another world.

Informally, a *wff* of the form $\Box\alpha$ is true in a world, if α is true in every world that is accessible from that world; a *wff* of the form $\Diamond\alpha$ is true in a world, if α is true in at least one world accessible from that world.

We define a *Kripke structure* (usually referred to just as a *structure*) as a pair (W, R) , where W is a set of worlds, the *possible worlds* and R is a relation between worlds, the *accessibility relation*. If the accessibility relation holds between world w_1 and world w_2 , that is, if the world w_2 is accessible from world w_1 , we write $R(w_1, w_2)$.

As an example, let us consider a Kripke structure with five possible worlds, w_1, w_2, w_3, w_4 , and w_5 , and with the the following accessibility relation:

$$R = \{(w_1, w_2), (w_1, w_3), (w_3, w_2), (w_3, w_3), (w_4, w_5), (w_5, w_4), (w_5, w_5)\}.$$

²We should understand “world” as “universe”.

This Kripke structure is shown in Figure 2.2, where worlds are represented by circles and the accessibility relation between worlds is represented by arrows. In this structure, for example, both worlds w_2 and w_3 are accessible from w_1 ; world w_4 is not accessible from w_1 ; and no world is accessible from w_2 .

Alongside with a Kripke structure, the semantics of modal logic uses a *valuation function*, V , that is similar to the valuation function presented in Section 2.3.2, but takes as arguments a world and a propositional logic predicate and assigns a truth value to the predicate in the world.

The semantics of modal logic is defined based on a *Kripke model*, which is a triple $M = (W, R, V)$, where W , R , and V have the following meanings:³

1. W is a set of possible worlds;
2. $R \subset W \times W$ is the accessibility relation;
3. $V : W \times \mathcal{P}_{PL} \mapsto \{T, F\}$ is the valuation function.

A Kripke model $M = (W, R, V)$ determines, for each of the worlds in W , which *wffs* are true in that world. We write $M \models_w \alpha$ to mean that the *wff* α is true in the world w , in the Kripke model M .

Given a *wff* α and a Kripke model $M = (W, R, V)$, the truth value of α in the world $w \in W$ is defined by induction on the structure of α as follows:

1. If α is a predicate, $M \models_w \alpha$ if and only if, $V(w, \alpha) = T$;
2. $M \models_w \neg\alpha$ if and only if, $M \not\models_w \alpha$;
3. $M \models_w (\alpha \wedge \beta)$ if and only if, $M \models_w \alpha$ and $M \models_w \beta$;
4. $M \models_w (\alpha \vee \beta)$ if and only if, $M \models_w \alpha$ or $M \models_w \beta$;
5. $M \models_w (\alpha \rightarrow \beta)$ if and only if, $M \models_w \beta$ when $M \models_w \alpha$;
6. $M \models_w \Box\alpha$, if and only if, for every $x \in W$, such that $R(w, x)$, we have $M \models_x \alpha$;
7. $M \models_w \Diamond\alpha$, if and only if, there is an $x \in W$, such that $R(w, x)$, and $M \models_x \alpha$.

³Where $\mathcal{P}_{PL}$ is the set of predicates of propositional logic.

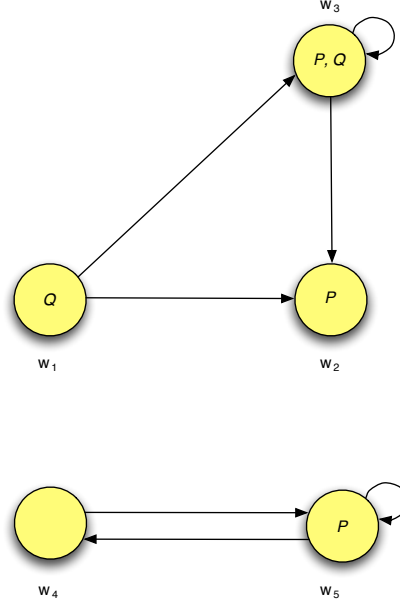


Figure 3.3: Example of a Kripke model.

The last two conditions show some similarities between the modal operators $\Box$ and $\Diamond$ and the quantifiers $\forall$ and $\exists$ of first order logic, except that they do not take variables as arguments.

As in classical logic, in modal logic, the logical symbol $\leftrightarrow$ is considered as an abbreviation for the conjunction of two implications:

$$\alpha \leftrightarrow \beta \stackrel{\text{def}}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$$

Before we move on, let us consider an example. Suppose that based on the Kripke structure of Figure 2.2, for each world, the valuation function has the value T for the *wffs* shown in Figure 2.3. This figure indicates, among other things, that $V(w_1, Q) = T$. In this case, we have:⁴

$$M \models_{w_1} Q$$

$$M \models_{w_1} \neg P$$

$$M \models_{w_1} \Diamond Q$$

⁴As an exercise, you should convince yourself about these relations.

$$\begin{aligned}
M &\models_{w_1} \Diamond \neg Q \\
M &\models_{w_1} \Box P \\
M &\models_{w_1} \Diamond P \\
M &\models_{w_3} P \rightarrow Q \\
M &\models_{w_3} \Box P \\
M &\models_{w_4} \neg P \\
M &\models_{w_4} \Diamond P \\
M &\models_{w_5} \Diamond P
\end{aligned}$$

There are two relations that are not so obvious and that may strike you at first:

$$M \models_{w_2} \Box P \quad (3.1)$$

$$M \models_{w_2} \neg \Diamond P \quad (3.2)$$

We should notice that world w_2 has no world accessible from it. According to our definition, $M \models_{w_2} \Box P$ if and only if for every $x \in W$, such that $R(w_2, x)$, we have $M \models_x P$. Since there is no x that is accessible from w_2 , the *wff* P is true in every world that is accessible from w_2 and, thus, formula 2.1. With a similar reasoning we can conclude formula 2.2.

The reasoning just outlined leads us to conclude that in a Kripke model, for every world, w , that has no world accessible to it, and for any *wff* α , we have:

$$M \models_w \Box \alpha \quad (3.3)$$

$$M \models_w \neg \Diamond \alpha \quad (3.4)$$

A model M is said to *satisfy* a *wff* α , if α is true in every world of the model. In this case, we write $\models_M \alpha$.

We say that a set of *wffs*, Δ , semantically entails a *wff*, α , written $\Delta \models \alpha$, if, in any world, w of any model $M = (W, R, V)$, we have $M \models_w \alpha$, whenever $M \models_w \delta$, for any $\delta \in \Delta$.

A *wff* α is said to be true, sometimes also said to be *valid*,⁵ if $\emptyset \models \alpha$.

⁵You should not confuse this use of the word “valid” with the concept of validity presented in Chapter 2.

Based on these definitions, for any *wffs* α and β , the following *wffs* can be proved to be true:

$$\Diamond\alpha \leftrightarrow \neg\Box\neg\alpha \quad (3.5)$$

$$\neg\Box\alpha \leftrightarrow \Diamond\neg\alpha \quad (3.6)$$

$$\Box(\alpha \wedge \beta) \leftrightarrow \Box\alpha \wedge \Box\beta \quad (3.7)$$

$$\Diamond(\alpha \vee \beta) \leftrightarrow \Diamond\alpha \vee \Diamond\beta \quad (3.8)$$

Formulas 2.5 and 2.6 correspond to our intuitions that α is possible if $\neg\alpha$ is not necessary and that α is necessary if $\neg\alpha$ is not possible; formulas 2.7 and 2.8 correspond to De Morgan laws for the modal operators.

For example, the following is a proof that formula 2.6 is true.⁶ Let M be a Kripke model,

1. Let us assume that $\models_M \neg\Box\alpha$. By definition, for every $w \in W$ $M \models_w \neg\Box\alpha$. Therefore, for any $w \in W$ there is a $x \in W$ such that $R(w, x)$ and $M \not\models_x \alpha$. This means that $M \models_x \neg\alpha$, which, in turn, means that $M \models_w \Diamond\neg\alpha$. Since w is an arbitrary world, we have $\models_M \Diamond\neg\alpha$. Therefore $\neg\Box\alpha \rightarrow \Diamond\neg\alpha$.
2. Let us assume that $\models_M \Diamond\neg\alpha$. By definition, for every $w \in W$ there is a $x \in W$ such that $R(w, x)$ and $M \models_x \neg\alpha$, this means that $M \models_x \neg\alpha$. Since w is an arbitrary world, we have $\models_M \neg\Box\alpha$. Therefore $\Diamond\neg\alpha \rightarrow \neg\Box\alpha$. ■

Another important formula that can be shown to be true, known as *Axiom K*,⁷ is:

$$\Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta) \quad (3.9)$$

To prove Axiom K , let M be a Kripke model and let us suppose that $\models_M \Box(\alpha \rightarrow \beta)$. By definition, this means that for any $w \in W$ and for every $x \in W$ such that $R(w, x)$, $M \models_x (\alpha \rightarrow \beta)$. Let us now assume that $M \models_x \alpha$, since the rules of classical logic apply to modal logic, we can conclude that $M \models_x \beta$. Therefore, $\models_M \Box\alpha \rightarrow \Box\beta$. ■

So far, we discussed the semantics of modal logic without imposing any restrictions on the nature of the accessibility relation. The modal logic that does not impose any properties on the accessibility relation is known as

⁶The proofs of the other formulas are left as an exercise.

⁷In honour to the logician Saul A. Kripke, who introduced the possible world semantics.

normal modal logic. There are no other interesting true formulas for normal modal logic.

Depending on the restrictions that we impose on the accessibility relation, other interesting formulas surface, as described in the next section. The restrictions placed on the accessibility relation clearly depend on what we intend to model.

3.1.3 Properties of the accessibility relation

In this section, we discuss the consequences that the properties of the accessibility relation have on the formulas that are true in modal logic. Before we start discussing these consequences, let us list some of the basic properties that the accessibility relation, as a binary relation, may have:

- A relation R is *reflexive* if $\forall w \in W, R(w, w)$;
- A relation R is *symmetric* if $\forall w_1, w_2 \in W, R(w_1, w_2) \rightarrow R(w_2, w_1)$;
- A relation R is *transitive* if $\forall w_1, w_2, w_3 \in W, (R(w_1, w_2) \wedge R(w_2, w_3)) \rightarrow R(w_1, w_3)$;
- A relation R is *linear* if $\forall w_1 \in W, \exists w_2 \in W$ such that $R(w_1, w_2)$;
- A relation R is *Euclidean* if $\forall w_1, w_2, w_3 \in W, (R(w_1, w_2) \wedge R(w_1, w_3)) \rightarrow R(w_2, w_3)$.

It turns out that, depending on the properties of the accessibility relation, additional formulas may be shown to be true in modal logic.

Let us assume, for example, that the accessibility relation is reflexive. In this case, the following *wff*, known as *Axiom T*, can be shown to be true:

$$\Box\alpha \rightarrow \alpha \quad (3.10)$$

To prove Axiom *T*, let us suppose that M is a Kripke model such that $\models_M \Box\alpha$ and that the accessibility relation is reflexive. By definition, this means that for any $w \in W$ and for every $x \in W$ such that $R(w, x)$, we have $M \models_x \alpha$. Since R is reflexive, we have $\forall w, R(w, w)$, which means that for any $w \in W$, we have $M \models_w \alpha$ and thus $\models_M \alpha$. ■

The following additional results can be shown:⁸

⁸Their proofs are left as an exercise.

Name	Axiom	Property of the accessibility relation
K	$\Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta)$	<i>none</i>
T	$\Box\alpha \rightarrow \alpha$	Reflexive
D	$\Box\alpha \rightarrow \Diamond\alpha$	Linear
4	$\Box\alpha \rightarrow \Box\Box\alpha$	Transitive
5	$\Diamond\alpha \rightarrow \Box\Diamond\alpha$	Euclidean
B	$\alpha \rightarrow \Box\Diamond\alpha$	Symmetric

Table 3.1: Modal logic axioms and properties of accessibility relations.

1. $\Box\alpha \rightarrow \Diamond\alpha$ is true in every Kripke structure whose accessibility relation is linear. This is known as *Axiom D*.
2. $\Box\alpha \rightarrow \Box\Box\alpha$ is true in every Kripke structure whose accessibility relation is transitive. This is known as *Axiom 4*.
3. $\Diamond\alpha \rightarrow \Box\Diamond\alpha$ is true in every Kripke structure whose accessibility relation is Euclidean. This is known as *Axiom 5*.
4. $\alpha \rightarrow \Box\Diamond\alpha$ is true in every Kripke structure whose accessibility relation is symmetric. This is known as *Axiom B*.

These results are summarized in Table 2.1, that also lists the historical names of the corresponding axioms.

Different modal logics are, therefore, originated when we consider different properties of the accessibility relation. These modal logics are identified by the names of the axioms that they contemplate.⁹ For example, modal logic $KT45$ is the modal system that contains axioms K , T , 4 , and 5 .¹⁰

There are relationships between the main systems of modal logic, as shown in Figure 2.4, where we show the name of the logic, the main axiom that it introduces, and the corresponding properties of the accessibility relation. In this figure, an arrow going from system S_1 to system S_2 means that all the truths of S_1 are truths of S_2 , but there are truths in S_2 that are not truths in S_1 . In this sense, system $KT45$ is the system with the larger number of truths.

⁹This notation is due to [Chellas 80].

¹⁰This logic is also known as modal logic $S5$.

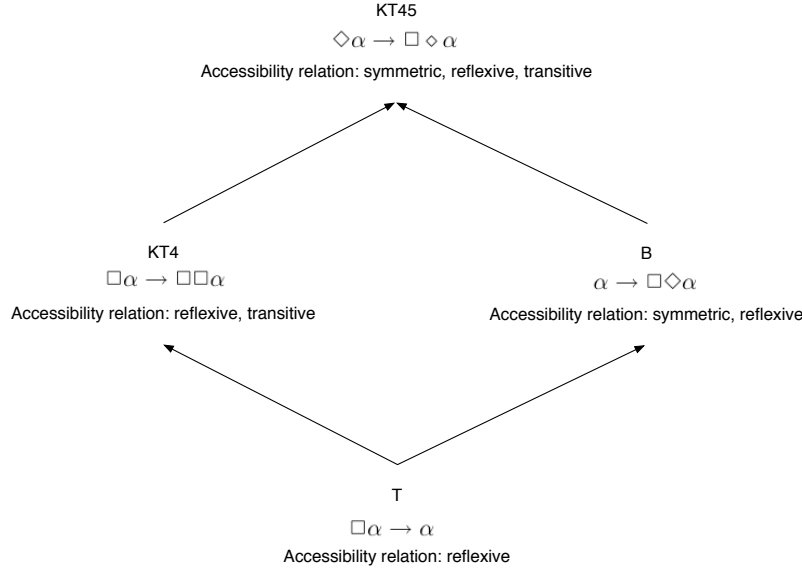


Figure 3.4: Relationship among some modal logic systems.

3.1.4 Axioms and readings of the modal operators

Different modal logics are designed to express different notions of possibility, necessity, belief, knowledge, temporal progression, and other modalities. Thus the term “modal logic” refers, not to a single logic, but rather to a family of related logical systems, depending on the readings of the modal operators.

These readings enable to represent sentences such as:

- It will rain tonight;
- John believes that Mary loves him;
- Mary knows that Shakespeare wrote Hamlet;
- When driving, we must fasten the seat-belt.

When we decide to use a certain modal logic, the choice of the axioms depends on the meaning we want to assign to the modal operators. In this section, we discuss some of the choices for different readings of the modal

Logic	$\Box\alpha$	$\Diamond\alpha$
Normal modal	α is necessary	α is possible
Epistemic	Agent knows that α	For all agent knows, α
Doxastic	Agent believes that α	α is consistent with the agent's beliefs
Deontic	It is obligatory that α	It is permitted that α
Temporal	It will always be the case that α	It will sometimes be the case that α

Table 3.2: Reading of modal operators for different logics.

operators, discuss the axioms that should be considered, and the meaning of the accessibility relation.

Table 2.2 shows some of the possible readings for the modal operator $\Box$, the associated reading for the modal operator $\Diamond$, and the name of the logic that they originate. The reading of the modal operator $\Diamond$ is obtained by considering that $\Diamond = \neg\Box\neg$.

- *Epistemic logic.* Epistemic logic deals with the notion of knowledge (of a given agent).

In epistemic logic, $\Box\alpha$ is read, agent knows that α , and $\Diamond\alpha$ ($\neg\Box\neg\alpha$) means that it is not the case that agent knows that α is false, which means that for as much as the agent knows, α is true or, alternatively, that α is possible.

Since one cannot know something that is false, if the agent knows α , then α must be true. In this case, Axiom *T*, $\Box\alpha \rightarrow \alpha$, should be considered.

We should also accept that the agent knows what it knows, which means that we should also consider Axiom *4*, $\Box\alpha \rightarrow \Box\Box\alpha$. Under this reading of $\Box$, Axiom *4* is known as *positive introspection*.

Furthermore, in the line of positive introspection, we should also accept Axiom *5*, $\Diamond\alpha \rightarrow \Box\Diamond\alpha$.

Axiom *K*, $\Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta)$ is accepted in all modal logics. Under this reading of $\Box$, Axiom *K* is known as *logical omniscience*,

the agent's knowledge is closed under logical consequence – the agent knows all the consequences of anything it knows.

Considering the axioms that we want to include in epistemic logic, we conclude that it corresponds to modal logic $KT45$, which requires the accessibility relation to be reflexive, transitive, and Euclidean. The accessibility relation $R(w_1, w_2)$ can be taken as meaning that w_2 could be the actual world, given the agent's knowledge at world w_1 .

- *Doxastic logic.* Doxastic logic deals with the notion of belief (of a given agent).

In doxastic logic, $\Box\alpha$ is read, agent believes that α (is true). In this case, $\Diamond\alpha$ ($\neg\Box\neg\alpha$) means that it is not the case that agent believes that α is false, which means that for as much as the agent believes, α is true or, alternatively, that α is consistent with the agent's beliefs.

It should be clear that in this case, Axiom T should not be used, since it would mean “if the agent believes in α , then α is true”, which contradicts the intuitive notion of belief – one can have false beliefs.

However, we should accept that the agent believes what it believes, thus we should accept Axiom 4 , $\Box\alpha \rightarrow \Box\Box\alpha$, another form of *positive introspection*, and, in the line of positive introspection, Axiom 5 , $\Diamond\alpha \rightarrow \Box\Diamond\alpha$.

Axiom K , $\Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta)$ is accepted in all modal logics. Under this reading of $\Box$, Axiom K is the belief's equivalent to logical omniscience.

Considering the axioms that we want to include in doxastic logic, we conclude that it corresponds to modal logic $K45$, which requires the accessibility relation to be transitive and Euclidean. The accessibility relation $R(w_1, w_2)$ can be taken as meaning that w_2 could be the actual world, given the agent's beliefs at world w_1 .

- *Deontic logic.* Deontic logic deals with the notion of moral codes.

In deontic logic, $\Box\alpha$ is read, it is obligatory that α (is true). In this case, $\Diamond\alpha$ ($\neg\Box\neg\alpha$) means that it is not obligatory that α is false, which means that it is permitted that α .

With this reading of $\Diamond$, we should accept Axiom D , $\Box\alpha \rightarrow \Diamond\alpha$, anything that is ought to be, is permitted to be.

Deontic logic corresponds to modal logic D , which requires the accessibility relation to be linear. The accessibility relation $R(w_1, w_2)$ can

be taken as meaning that w_2 is an acceptable world according to the information in world w_1 .

- *Temporal logic.* Temporal logic deals with different aspects related with time.

Different temporal logics assign different meanings to $\Box$. Suppose that $\Box\alpha$ is read, it will always be the case that α (is true), in other words, $\Box\alpha$ means that α will be true and will remain true. In this case, $\Diamond\alpha$ means that it is false that it will always be the case that α is false, which means that it will be the case that α is true.

When we consider temporal logic, the accessibility relation has the meaning of “later than”, which prevents it to be reflexive. This means that Axiom T ($\Box\alpha \rightarrow \alpha$) cannot be considered.

3.1.5 Deductive system for propositional modal logic

We discuss the deductive system based on the natural deduction approach. Recall that natural deduction contains two inference rules for each logical connective, the introduction and the elimination rule, besides other rules such as repetition and reiteration. Natural deduction for propositional logic also uses the notion of hypothetical proof, a proof initiated with the introduction of a hypothesis. Since modal logic extends classical logic, all rules of inference of classical logic are applicable in the deductive system of modal logic.

Rules for normal modal logic

In the deductive system of modal logic, we introduce a new type of proof, the *possible world* proof (*pw*-proof, for short) that corresponds to reasoning in an arbitrary accessible world. A *pw*-proof is drawn with two parallel vertical lines with the symbol “*pw*” in between.

Pw-proofs may be initiated at any moment and can use any of the inference rules of classical logic. In particular, the re-iteration rule is applicable to *pw*-proofs:

$$\begin{array}{c}
 n \quad \alpha \\
 \vdots \\
 m \quad \left| \begin{array}{c} pw \\ \hline \alpha \end{array} \right| \vdots
 \end{array}
 \quad \text{Rei, } n$$

In natural deduction there are no explicit rules for the logical symbol $\Diamond$, which must be written as $\neg\Box\neg$ in proofs.

Rules for necessitation. The rules for necessitation are associated with *pw*-proofs.

The rule of *necessitation introduction*, written $\Box I$, states that whenever a *wff* α occurs in a *pw*-proof, we can write $\Box\alpha$ in the proof that immediately contains the *pw*-proof:

$$\begin{array}{ccc} \vdots & \left| pw \right| & \vdots \\ n & & \alpha \\ m & \Box\alpha & \Box I, n \end{array}$$

The intuitive meaning of this rule is that, if we obtain a *wff* α in an arbitrary accessible world, then $\Box\alpha$ holds in the world that can access to that arbitrary world.

The rule for *necessitation elimination*, written $\Box E$, states that from the *wff* $\Box\alpha$, we can derive α in any *pw*-proof contained in the proof where $\Box\alpha$ occurs:

$$\begin{array}{ccc} n & \Box\alpha & \\ \vdots & \left| pw \right| & \vdots \\ m & & \alpha \quad \Box E, n \end{array}$$

The intuitive meaning of this rule is that, if the *wff* $\Box\alpha$ holds in a given world, then the *wff* α also holds in any world accessible from that world.

As an example, the following is a proof of $(\Box P \wedge \Box Q) \rightarrow \Box(P \wedge Q)$:

1	$\Box P \wedge \Box Q$	Hyp
2	$\Box P$	$\wedge E, 1$
3	$\Box Q$	$\wedge E, 1$
4	$\quad pw \quad P$	$\Box E, 2$
5	$\quad \quad Q$	$\Box E, 3$
6	$\quad \quad P \wedge Q$	$\wedge I, (4, 5)$
7	$\Box(P \wedge Q)$	$\Box I, 6$
8	$(\Box P \wedge \Box Q) \rightarrow \Box(P \wedge Q)$	$\rightarrow I, (1, 7)$

As another example, we prove that $\Diamond(P \vee Q) \rightarrow \Diamond P \vee \Diamond Q$, this corresponds to the proof that $\neg\Box\neg(P \vee Q) \rightarrow \neg\Box\neg P \vee \neg\Box\neg Q$. To shorten the proof, we use the De Morgan rules, $(\neg\alpha \wedge \neg\beta) \rightarrow \neg(\alpha \vee \beta)$ and $\neg(\alpha \wedge \beta) \rightarrow (\neg\alpha \vee \neg\beta)$, as derived rules of inference.

1	$\neg\Box\neg(P \vee Q)$	Hyp
2	$\quad \Box\neg P \wedge \Box\neg Q$	Hyp
3	$\quad \Box\neg P$	$\wedge E, 1$
4	$\quad \Box\neg Q$	$\wedge E, 1$
5	$\quad \quad pw \quad \neg P$	$\Box E, 3$
6	$\quad \quad \neg Q$	$\Box E, 4$
7	$\quad \quad \neg P \wedge \neg Q$	$\wedge I, (5, 6)$
8	$\quad \quad \neg(P \vee Q)$	De Morgan Rule (v1), 7
9	$\quad \Box\neg(P \vee Q)$	$\Box I, 8$
10	$\quad \neg\Box\neg(P \vee Q)$	Rei, 1
11	$\neg(\Box\neg P \wedge \Box\neg Q)$	$\neg I, (2, (9, 10))$
12	$\neg\Box\neg P \vee \neg\Box\neg Q$	De Morgan Rule (v2), 11
13	$\neg\Box\neg(P \vee Q) \rightarrow \neg\Box\neg P \vee \neg\Box\neg Q$	$\rightarrow I, (1, 12)$

Derived rule of inference. Since Axiom K ($\Box(\alpha \rightarrow \beta) \rightarrow (\Box\alpha \rightarrow \Box\beta)$) holds for any modal logic, we introduce the following rule, as a derived rule of inference:

$$\begin{array}{lll} n & \Box(\alpha \rightarrow \beta) & \\ n+1 & \Box\alpha \rightarrow \Box\beta & \text{Ax K, } n \end{array}$$

Rules for other modal logics.

We already know that depending on the modal axioms used, we will have different kinds of modal logics. The names of these axioms are usually used to identify the names of the logics that they generate.

Depending on which modal logic we are working on, we will have the additional rules of inference. These rules correspond to derived rules of inference originated by the direct application of the axioms discussed in Section 2.1.3.

Logics considering Axiom T :

$$\begin{array}{lll} n & \Box\alpha & \\ n+1 & \alpha & \text{Ax T, } n \end{array}$$

Logics considering Axiom 4 :

$$\begin{array}{lll} n & \Box\alpha & \\ n+1 & \Box\Box\alpha & \text{Ax 4, } n \end{array}$$

Logics considering Axiom 5 :

$$\begin{array}{lll} n & \neg\Box\neg\alpha & \\ n+1 & \Box\neg\Box\neg\alpha & \text{Ax 5, } n \end{array}$$

Similarly to what happened with classical logic, the *uffs* that are derivable in a logic depend on the deductive system that is used (see page 33). In classical logic, we do not index the derivability symbol, $\vdash$, with an indication of the deductive system because all classical logic systems are equivalent. In modal logic, however, the deductive system makes a difference in respect to derivability, and we associate the symbol $\vdash$ with an identification of the deductive system. For example, $\Delta \vdash_{KD5} \alpha$ means that “ α is derivable from Δ , in system $KD5$ ”.

As an example, we present a proof of $\vdash_{KT45} \Box \Diamond \Box P \rightarrow \Box P$. Before we start the proof, we should transform the *wff* $\Box \Diamond \Box P \rightarrow \Box P$ into a *wff* only containing the $\Box$ connective, obtaining, $\Box \neg \Box \neg \Box P \rightarrow \Box P$:

1	$\Box \neg \Box \neg \Box P$	Hyp
2	$\begin{array}{ l} pw \\ \hline \neg \Box \neg \Box P \end{array}$	$\Box E, 1$
3	$\begin{array}{ l} \neg \Box P \\ \hline \end{array}$	Hyp
4	$\begin{array}{ l} \Box \neg \Box P \\ \hline \end{array}$	Ax 5, 3
5	$\neg \Box \neg \Box P$	Rei, 2
6	$\neg \neg \Box P$	$\neg I, (3, (4, 5))$
7	$\Box P$	$\neg E, 6$
8	P	Ax T, 7
9	$\Box P$	$\Box I, 8$
10	$\Box \neg \Box \neg \Box P \rightarrow \Box P$	$\rightarrow I, (1, 9)$

3.1.6 Soundness and completeness

For different modal systems, soundness and completeness results can be shown. For example, for the system $KT4$, the soundness proof corresponds to proving that if $\emptyset \vdash_{KT4} \alpha$, that is, if α is derivable in this system, then α is true in every Kripke structure that is reflexive and transitive. The completeness proof corresponds to the converse proof. These proofs are outside the scope of this book.

3.1.7 First-order modal logic

The language of first-order modal logic

The modal logic discussed so far does not contemplate the existence of variables. Similarly to classical logic, the move from propositional modal logic into first-order modal logic requires a revision of the rules for the formation of *wffs* (taking into account more symbols) and the introduction of new rules of inference.

The language of first-order modal logic ($\mathcal{L}_{FOML}$), is defined on the basis of the same components as $\mathcal{L}_{FOL}$, with the addition of the unary logical symbols $\Box$ and $\Diamond$. Terms have the same definition as in $\mathcal{L}_{FOL}$ and *wffs* are defined as:

1. If $t_1, t_2, \dots, t_n$ are terms, then $P_i^n(t_1, t_2, \dots, t_n)$ is a *wff*;
2. If α is a *wff*, then $\neg\alpha$, $\Box\alpha$, and $\Diamond\alpha$ are *wffs*;
3. If α and β are *wffs*, then $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, and $(\alpha \rightarrow \beta)$ are *wffs*;
4. If α is a *wff* containing zero or more occurrences of the individual variable x , then $\forall x[\alpha]$ and $\exists x[\alpha]$ are *wffs*;
5. Nothing more is a *wff*.

Remarks on first-order modal logic

It would seem to be a simple matter to increase modal logic with quantifiers. However, this creates several problems, one of them concerns the correspondence between entities existing in the different possible worlds, as well as correspondences between the functions applied to these entities.

Some of the alternatives (not discussed in this book) are the possibility to create or to destroy entities in different worlds; the possibility of existence of functions (the rigid designators) that remain unchanged in different worlds and their identification.

The simplest alternative, the fixed-domain approach, assumes a single domain of quantification that contains all the possible entities. On the other hand, the world-relative approach, assumes that the domain of quantification changes from world to world, and contains only the entities that actually exist in a given world.

The *fixed-domain approach* requires no major adjustments to the classical machinery for quantifiers. Modal logics that are adequate for fixed domain semantics can be constructed by adding principles of a propositional modal logic to classical quantifier rules together with the *Barcan Formulas*:

$$\Box\forall x[P(x)] \rightarrow \forall x[\Box P(x)] \quad (3.11)$$

$$\forall x[\Box P(x)] \rightarrow \Box\forall x[P(x)] \quad (3.12)$$

$$\Box \exists x[P(x)] \rightarrow \exists x[\Box P(x)] \quad (3.13)$$

$$\exists x[\Box P(x)] \rightarrow \Box \exists x[P(x)] \quad (3.14)$$

In order to get an intuitive idea about what is stated in these formulas, let us look into the meaning of the individual formulas that compose them. Under the perspective of possible worlds, we obtain the following “interpretations”:

- $\Box \forall x[P(x)]$ states that in every world that is accessible from the current world, the property P holds for every entity;
- $\forall x[\Box P(x)]$ states that every entity in the current world satisfies the property P in every world that is accessible from the current world;
- $\Box \exists x[P(x)]$ states that in every world that is accessible from the current world, property P holds for some entity;
- $\exists x[\Box P(x)]$ states that there is an entity in the current world that satisfies the property P in every world that is accessible from the current world.

With these readings, *uff* 2.11, for example, states that if, in every world accessible from the current world, the property P holds for every object, then all the objects in the current world satisfy property P in all the worlds that are accessible from it.

The fixed-domain interpretation has advantages of simplicity and familiarity, but it does not provide a direct account of the semantics of certain quantifier expressions of natural language.

In the *world-relative approach*, each possible world has its own domain of quantification (the set of objects that actually exist in that world), and the domains vary from one world to the next.

3.2 Relevance logic

As an example of a logic that challenges classical logic, we present the relevance logic.¹¹ Classical logic does not consider relevance issues when it evaluates the validity of arguments. The irrelevancy between premises and

¹¹Also called *relevant logic* in UK and Australia.

conclusions is not well accepted in common reasoning and is considered a flaw in one's argument.

Reacting against the lack of account of relevance in classical logic, *relevance logic* challenges classical logic in the following two respects:

1. *Regarding the classical concept of validity:* Relevance logicians argue that if a proposition implies another, then there must be an element of causality that relevantly connects them, and, for that reason, they do not recognize some of the arguments classified as valid by classical logic. In particular, they explicitly deny the so-called paradoxes of implication: $\alpha \rightarrow (\beta \rightarrow \alpha)$, anything implies a true proposition; and $(\alpha \wedge \neg\alpha) \rightarrow \beta$, a contradiction implies anything.

To their (semantic) notion of implication corresponds a (syntactic) notion of derivability according to which β is derivable from α only if the derivation of β genuinely uses, and does not simply take a detour via, α .

2. *Regarding the applicability of the disjunctive syllogism:* Relevance logic distinguishes two kinds of disjunction, the truth functional and the intensional.
 - (a) In the *truth functional sense*, the truth value of the wff $\alpha \vee \beta$ is defined *only* in terms of the truth values of α and β . For this sense of “or”, the classical rule of $\vee$ -introduction¹² is applicable.
 - (b) The *intensional sense*, assumes that the disjuncts are relevant to each other, corresponds to the way “or” is used in everyday language, meaning something like “if it isn't one, then it is the other”. This case clearly involves an implication (or better, two implications), and disjunctive syllogism¹³ can be explained in terms of *modus ponens*.

Relevance logicians argue that classical logic confuses the two senses of “or”, using the first in the rule of $\vee$ -introduction and the second in the rule of $\vee$ -elimination. For this reason, they consider disjunctive syllogism a “fallacy of relevance” and explicitly deny that it is a valid rule of inference.

¹²From α we can derive either $\alpha \vee \beta$ or $\beta \vee \alpha$, regardless of the truth value of β or its connection with α (page 46).

¹³From $\alpha \vee \beta$ and $\neg\alpha$, infer β (page 47).

In this book we only discuss the deductive system of propositional relevance logic. Further readings are indicated in Section 2.4.

3.2.1 Deductive system

The language of propositional relevance logic is quite similar to the language of classical propositional logic, with the logical symbol $\rightarrow$ replaced by the symbol $\Rightarrow$. This is used to stress the fact that relevance logic uses a different kind of implication. The symbol $\Rightarrow$ is read *entails* and corresponds to the relevant implication operator (also called *entailment*). The language of propositional relevance logic is denoted by $\mathcal{L}_{PRL}$.

To define derivability, relevance logic modifies the classical rules of natural deduction. First, it associates each well formed formula (*wff*) in the line of a derivation with a set, the *identifier set*, that references all hypotheses actually used in its derivation. Second it restates the classical rules of inference to take these sets into account, blocking the irrelevant applications which are allowed in classical logic.

We discuss the rules of inference of a system for propositional relevance logic.¹⁴ The deductive system deals with extended *wffs* that belong to the language $\mathcal{L}_{PRL}^+$, which is defined as:

$$\mathcal{L}_{PRL}^+ = \mathcal{L}_{PRL} \times \{, \} \times 2^{\mathbb{N}}$$

Each extended *wff* of $\mathcal{L}_{PRL}^+$ associates a *wff* with a set of identifiers (its *identifier set*); if $\alpha \in \mathcal{L}_{PRL}$ and $a \subset \mathbb{N}$, an extended *wff* is written as α, a .

The basic idea behind extended *wffs* is to record in the identifier set a unique “stamp” (integer number) associated with each of the hypotheses used in the derivation of its *wff* and to make sure that each derived extended *wff* contains stamps of all hypotheses underlying its derivation and only those stamps.

Rule of hypothesis. This rule enables the introduction of any *wff* as a hypothesis, in any point of a proof. This hypothesis is associated with an identifier ($k \in \mathbb{N}$) that never appeared before in the proof and initiates a

¹⁴We will not discuss the semantics of relevance logic (this can be found in [Read 88]).

new hypothetical proof:

$$\begin{array}{c|c} n & \alpha, \{k\} \\ n+1 & \dots \end{array} \quad \text{Hyp}$$

Rule of repetition. Any *wff* may be repeated in a proof, keeping its identifier set:

$$\begin{array}{c|c} n & \alpha, a \\ \vdots & \vdots \\ m & \alpha, a \end{array} \quad \text{Rep, } n$$

Rule of re-iteration. Any *wff* may be re-iterated (repeated) in a sub-proof contained in the proof where the *wff* exists, keeping its identifier set:

$$\begin{array}{c|c} n & \alpha, a \\ \vdots & \vdots \\ m & \alpha, a \end{array} \quad \text{Rei, } n$$

Rules for entailment. As in classical logic, there are two rules for entailment, one to introduce a *wff* with the entailment and the other to use a *wff* with the entailment. Since entailment is a connective that differs from the implication connective of classical logic, it is just natural that there are differences between the entailment and the implication introduction rules.

The *entailment introduction* rule, written $\Rightarrow I$, presents similarities with the $\rightarrow I$ rule of classical logic, in the sense that it is based on a hypothetical proof, but it makes sure that the *wff* used for the introduction of the entailment was *in fact* derived from the hypothesis that initiated the hypothetical proof. This is done by placing restrictions on the identifier set.

If, in a proof initiated with the hypothesis $\alpha, \{k\}$, we can derive $\beta, a \cup \{k\}$ – notice that the identifier “stamp” of the hypothesis α belongs to the identifier set of β , we can infer $\alpha \Rightarrow \beta, a$ in the proof that contains the proof initiated

by the hypothesis $\alpha, \{k\}$:

n	$\alpha, \{k\}$	Hyp
$\vdots$	$\vdots$	
m	$\beta, a \cup \{k\}$	
$m+1$	$\alpha \Rightarrow \beta, a$	$\Rightarrow I, (n, m)$

We should further notice that since $\alpha \Rightarrow \beta$ does not depend on the hypothesis α , its identifier “stamp” is removed from the identifier set of the resulting extended *wff*.

Implication elimination, also known as *modus ponens*, and written $\Rightarrow E$. If, in a proof, we have both α, a and $\alpha \Rightarrow \beta, b$, we can infer $\beta, a \cup b$:

n	α, a	
$n+1$	$\alpha \Rightarrow \beta, b$	
$n+2$	$\beta, a \cup b$	$\Rightarrow E, (n, n+1)$

In this case β depends *both* on all hypotheses underlying α and all hypotheses underlying $\alpha \Rightarrow \beta$, thus its identifier set is the union of the identifier sets of the original extended *wffs*.

Rules for negation. In relevance logic there are four rules associated with negation. Two of them handle the negation introduction and elimination; the other two handle double negation and simply state that the *wff* $\neg\neg\alpha$ is the same as the *wff* α .

Negation introduction, written $\neg I$. If in a proof we have the extended *wff* $\alpha \Rightarrow \neg\alpha, a$, we can derive $\neg\alpha, a$:

n	$\alpha \Rightarrow \neg\alpha, a$	
$n+1$	$\neg\alpha, a$	$\neg I, n$

Negation elimination, also called *modus tollens*, written $\neg E$. If in a proof, we have the extended *wffs* $\neg\beta, a$ and $\alpha \Rightarrow \beta, b$, we can infer $\neg\alpha, a \cup b$:

n	$\neg\beta, a$	
$n+1$	$\alpha \Rightarrow \beta, b$	
$n+2$	$\neg\alpha, a \cup b$	$\neg E, (n, n+1)$

Double negation introduction, written $\neg\neg I$. If, in a proof, we have α, a , then we can infer $\neg\neg\alpha, a$:

$$\begin{array}{lll} n & \alpha, a & \\ n+1 & \neg\neg\alpha, a & \neg\neg I, n \end{array}$$

Double negation elimination, written $\neg\neg E$. If, in a proof, we have $\neg\neg\alpha, a$, then we can infer α, a :

$$\begin{array}{lll} n & \neg\neg\alpha, a & \\ n+1 & \alpha, a & \neg\neg E, n \end{array}$$

Rules for conjunction. *Conjunction introduction*, written $\wedge I$. If, in a proof, we have both α, a and β, a , then we can infer $\alpha \wedge \beta, a$:

$$\begin{array}{lll} n & \alpha, a & \\ n+1 & \beta, a & \\ n+2 & \alpha \wedge \beta, a & \wedge I, (n, n+1) \end{array}$$

The rule of conjunction introduction may seem too strongly stated, when it requires that both α and β have the same identifier set, but it is so to restrict the gratuitous introduction of irrelevancies.

Suppose (as one might expect) that $\wedge I$ allowed the conjunction of *wffs* with different identifier sets, resulting in a *wff* whose identifier was the union of the identifier sets of the parent extended *wffs*:

$$\begin{array}{lll} n & \alpha, a & \\ n+1 & \beta, b & \\ n+2 & \alpha \wedge \beta, a \cup b & \text{Wrong } \wedge I, (n, n+1) \end{array}$$

The following “proof” shows how we could then introduce irrelevancies in the system, using this version of the rule. The application of $\wedge E$ to the *wff* in line 5, which resulted from such use of the wrong $\wedge I$ rule, allows the identifier of hypothesis of line 2 to be “smuggled into” the identifier of P (line 6), thereby allowing the “proof” of $P \Rightarrow (Q \Rightarrow P)$, one of the paradoxes

of implication.

1	$P, \{1\}$	Hyp
2	$Q, \{2\}$	Hyp
3	$P, \{1\}$	Rei, 1
4	$Q, \{2\}$	Rep, 2
5	$P \wedge Q, \{1, 2\}$	Wrong $\wedge$ I, (3, 4)
6	$P, \{1, 2\}$	$\wedge$ E, 5
7	$Q \Rightarrow P, \{1\}$	$\Rightarrow$ I, (2, 6)
8	$P \Rightarrow (Q \Rightarrow P), \{ \}$	$\Rightarrow$ I, (1, 7)

Conjunction elimination, written $\wedge$ E. If, in a proof we have $\alpha \wedge \beta, a$, then we can infer either α, a , or β, a , or both:

$$\begin{array}{lll} n & \alpha \wedge \beta, a & \\ n+1 & \alpha, a & \wedge\text{E}, n \end{array}$$

$$\begin{array}{lll} n & \alpha \wedge \beta, a & \\ n+1 & \beta, a & \wedge\text{E}, n \end{array}$$

Rules for disjunction. *Disjunction introduction*, written $\vee$ I. If in a proof we have α, a , then we can infer $\alpha \vee \beta, a$, or $\beta \vee \alpha, a$, or both, where β is any *wff*:

$$\begin{array}{lll} n & \alpha, a & \\ n+1 & \alpha \vee \beta, a & \vee\text{I}, n \end{array}$$

$$\begin{array}{lll} n & \alpha, a & \\ n+1 & \beta \vee \alpha, a & \vee\text{I}, n \end{array}$$

Disjunction elimination, written $\vee$ E. If, in a proof, we have (a) the extended *wff* $\alpha \vee \beta, a$; (b) a proof initiated by the hypothesis $\alpha, \{k\}$, having $\gamma, b \cup \{k\}$ as one of its extended *wffs*; and (c) a proof initiated by the hypothesis $\beta, \{l\}$,

having $\gamma, b \cup \{l\}$ as one of its extended *wffs*, then, we can derive $\gamma, a \cup b$ in that proof:

n	$\alpha \vee \beta, a$	
o	$\alpha, \{k\}$	Hyp
$\vdots$	$\vdots$	
p	$\gamma, b \cup \{k\}$	
r	$\beta, \{l\}$	Hyp
$\vdots$	$\vdots$	
s	$\gamma, b \cup \{l\}$	
m	$\gamma, a \cup b$	$\vee E, (n, (o, p), (r, s))$

The following is a proof of $(P \Rightarrow (Q \Rightarrow R)) \Rightarrow ((P \wedge Q) \Rightarrow R)$:

1	$P \Rightarrow (Q \Rightarrow R), \{1\}$	Hyp
2	$P \wedge Q, \{2\}$	Hyp
3	$P, \{2\}$	$\wedge E, 2$
4	$P \Rightarrow (Q \Rightarrow R), \{1\}$	Rei, 1
5	$Q \Rightarrow R, \{1, 2\}$	$\Rightarrow E, (3, 4)$
6	$Q, \{2\}$	$\wedge E, 2$
7	$Q \Rightarrow R, \{1, 2\}$	Rep, 5
8	$R, \{1, 2\}$	$\Rightarrow E, (6, 7)$
9	$(P \wedge Q) \Rightarrow R, \{1\}$	$\Rightarrow I, (2, 8)$
10	$(P \Rightarrow (Q \Rightarrow R)) \Rightarrow ((P \wedge Q) \Rightarrow R), \{\}$	$\Rightarrow I, (1, 9)$

Although we have not proved it, in relevance logic, a *wff* of the form $(\alpha \wedge \neg \alpha) \Rightarrow \beta$ is not a theorem. Logics with this property are called *paraconsistent logics*.

3.3 Summary

In this chapter we discussed non-classical logics, presenting the motivations for their development and discussing two logics that have been used in AI, modal logic and relevance logic.

3.4 Bibliographic notes and historical remarks

The distinction between *logica utens* and *logica docens* was introduced by the american logician Charles Sanders Peirce (1839–1914), who, in turn, adopted it from medieval logicians. A good discussion about this distinction can be found in [Haack 78] that also presents an outstanding introduction to the issues associated with philosophical logic and with non-classical logics.

A good perspective on the work on non-classical logics can be found in [Rautenberg 87]. The books by [Ramsay 88] and [Turner 84] discuss non-classical logics used in AI; the first, addresses modal logic, temporal logic, and non-monotonic logics; and the second, addresses modal logic, dynamic logic, intuitionistic logic, temporal logic, fuzzy logic, and non-monotonic logics.

Modal logic was originally introduced by Clarence Irving Lewis (1883–1964) [Lewis 18] in an attempt to avoid the paradoxes of implication. The idea was to distinguish two sorts of truth: necessary truth and mere contingent truth. A good overview on modal logic can be found in [Garson 03a]. Classical books on modal logic include [Hughes and Cresswell 68] and [Chellas 80]. The possible worlds semantics was introduced by Gottfried Wilhelm Leibniz (1646–1716) and was formalized by Saul Aaron Kripke (1940–) [Kripke 63]. An excellent discussion of the notion of possible worlds is given in [Bradley and Swartz 79].

Modal logic has been used in AI to deal with knowledge and belief (for example auto-epistemic logic, discussed in Section 3.3, and the logic of [Konolige 86]) and in logics that deal with a changing world (for example, TWEAK of [Chapman 87]). A discussion on temporal logic can be found in [Garson 03b].

Relevance logic was introduced by Alan Ross Anderson (1924–1973) and Nuel Belnap (1931–) [Anderson and Belnap 62a, 62b], [Anderson and Belnap 75], [Anderson, Belnap, and Dunn 92]. A nice presentation can be found

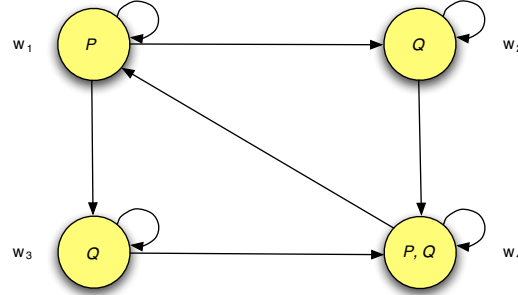


Figure 3.5: Kripke model.

in [Dunn and Restall 02].

Relevance logic can be used to keep track of dependencies among propositions (for example, in the belief revision system of [Martins and Shapiro 88]) or in systems that are not affected by contradictions [Joy, Murty and Krishna 90].

Discussions on paraconsistent logics can be found in [Jeffreys 42], [Tennant 84], [Priest, Routley and Norman 89], and [Priest 02].

3.5 Exercises

1. Consider the Kripke model shown in Figure 2.5.
 - (a) Write down the valuation function.
 - (b) Are there any important properties satisfied by the accessibility relation?
 - (c) Which of the following are true? Justify your answer.
 - i. $M \models_{w_1} \Box P$
 - ii. $M \models_{w_1} \Box Q$
 - iii. $M \models_{w_4} \Box P$
 - iv. $M \models_{w_4} P \rightarrow Q$

2. Consider the following Kripke model:

$$W = \{w_1, w_2, w_3\}$$

$$R = \{(w_1, w_2), (w_1, w_3), (w_2, w_1), (w_2, w_3), (w_3, w_1), (w_3, w_2)\}$$

$$V = \{(w_1, P, T), (w_1, Q, F), (w_2, P, T), (w_2, Q, T), (w_3, P, T), (w_3, Q, F)\}$$

- (a) Write down a graphical representation of this Kripke model.
 - (b) Are there any important properties satisfied by the accessibility relation?
 - (c) Which of the following are true? Justify your answer.
 - i. $M \models_{w_1} \Box P$
 - ii. $M \models_{w_2} \Diamond Q$
 - iii. $M \models_{w_3} \Box P$
3. Using the possible world semantics, prove the following equivalences:
 - (a) $\Diamond \alpha \leftrightarrow \neg \Box \neg \alpha$
 - (b) $\neg \Diamond \alpha \leftrightarrow \Box \neg \alpha$
 - (c) $\Box(\alpha \wedge \beta) \leftrightarrow \Box \alpha \wedge \Box \beta$
 - (d) $\Diamond(\alpha \vee \beta) \leftrightarrow \Diamond \alpha \vee \Diamond \beta$
 4. Prove that $\Box \alpha \rightarrow \Diamond \alpha$ is true in every Kripke structure whose accessibility relation is linear.
 5. Prove that $\Box \alpha \rightarrow \Box \Box \alpha$ is true in every Kripke structure whose accessibility relation is transitive.
 6. Prove that $\Diamond \alpha \rightarrow \Box \Diamond \alpha$ is true in every Kripke structure whose accessibility relation is Euclidean.
 7. Prove that $\alpha \rightarrow \Box \Diamond \alpha$ is true in every Kripke structure whose accessibility relation is symmetric.
 8. Prove that $\Box \alpha \leftrightarrow \Diamond \alpha$ is true in every Kripke structure whose accessibility relation is functional, that is, the accessibility relation satisfies the following property $\forall w \in W [\exists w_1 \in W, R(w, w_1) \wedge \forall w_1, w_2 \in W [(R(w, w_1) \wedge R(w, w_2)) \rightarrow w_1 = w_2]]$ (for every world, there is a unique world that is accessible from it).
 9. Prove that $\Box(\alpha \wedge \Box \alpha \rightarrow \beta) \vee \Box(\beta \vee \Box \beta \rightarrow \alpha)$ is true in every Kripke structure whose accessibility relation is linear, that is, the accessibility relation satisfies the following property $\forall w_1, w_2, w_3 \in W [((R(w_1, w_2) \wedge R(w_1, w_3)) \rightarrow R(w_2, w_3)) \vee w_2 = w_3 \vee R(w_3, w_2)]$.

10. Providing appropriate readings for the modal operators, represent the following propositions:
 - (a) It will rain tonight;
 - (b) John believes that Mary loves him;
 - (c) John believes that Mary loves him, but she doesn't;
 - (d) Mary knows that Shakespeare wrote Hamlet;
 - (e) When driving, we must fasten the seat-belt.
11. Considering that the reading of $\Box\alpha$ is "agent believes that α ", explain the meaning of the following formulas:
 - (a) $\Box\alpha \rightarrow \Diamond\alpha$
 - (b) $\Box\alpha \vee \neg\Box\alpha$
12. Using the deductive system, prove the following *wffs* in system K :
 - (a) $\Diamond\alpha \leftrightarrow \neg\Box\neg\alpha$
 - (b) $\neg\Diamond\alpha \leftrightarrow \Box\neg\alpha$
 - (c) $\Box(\alpha \wedge \beta) \leftrightarrow \Box\alpha \wedge \Box\beta$
 - (d) $\Diamond(\alpha \vee \beta) \leftrightarrow \Diamond\alpha \vee \Diamond\beta$
 - (e) $\Box(\alpha \rightarrow \beta) \rightarrow (\Diamond\alpha \rightarrow \Box\beta)$
13. Provide an intuitive reading for formula 2.12.
14. Provide an intuitive reading for formula 2.13.
15. Provide an intuitive reading for formula 2.14.
16. Prove the following theorems, using relevance logic:
 - (a) $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$
 - (b) $(A \Rightarrow (A \Rightarrow B)) \Rightarrow (A \Rightarrow B)$
 - (c) $A \Rightarrow ((A \Rightarrow B) \Rightarrow B)$

Chapter 4

Non-Monotonic Formalisms

“*Não há regra sem exceção.*”
(There is no rule without an exception)
Portuguese saying

A characteristic of human reasoning is the ability to deal with incomplete information. In our daily life, we are constantly faced with situations in which only part of the information is known. This fact does not prevent us from producing conclusions, even when the available information is not enough to guarantee the correctness of these conclusions. In this case, we may have to withdraw some of our conclusions in face of additional information. The kind of reasoning in which some of the conclusions may have to be revised in face of additional information is called *non-monotonic reasoning* and the formalisms that support this kind of reasoning are called *non-monotonic formalisms*. Some of these formalisms are presented in the form of a special kind of non-classical logics, called *non-monotonic logics*.

Whereas the basic notion underlying reasoning with complete information (the kind of reasoning corresponding to traditional logics) is the notion of truth, non-monotonic reasoning is based on the notion of rationality. We cannot expect all of our conclusions to be true; nevertheless, we do not want to produce just any conclusion whatsoever. To minimize the risk of producing wrong conclusions, we use our knowledge and experience to decide whether a conclusion should or should not be produced. Although some of our conclusions may turn out to be wrong, we require them to be rational.

The existence of a rational justification for a conclusion is a requirement for a conclusion produced by humans.

Rationality is a vague concept whose exact definition may be impossible to produce. As opposed to the notion of truth, rationality has two particular characteristics: (1) it depends on the agent that is performing the reasoning – two rational agents may not agree with what is rationally produced in a given situation; (2) it depends on how the conclusion is to be used.

A generally accepted definition of *rational conclusion*¹ states that an agent considers α as a rational conclusion, if the agent is prepared to use α as if it were true.

In the process of generating rational conclusions, human reasoning is based both on the presence of information and also on its absence. A rational conclusion is frequently connected to inference patterns of the form: “from α , lacking information about β , we can conclude γ ”. For example, knowing that a given person is employed, and lacking information about the fact that the person is not available to work, we may conclude that the person is available to work. The intuition behind this pattern of reasoning says that α supports the conclusion γ , while the lack of information about β guarantees its rationality.

In this reasoning pattern, the lack of information is usually identified with the absence of contradictory information. In this case, the pattern is written as “from α , in the lack of contradictory information, we may conclude γ ”. The most apparent property of this kind of reasoning is that it generates reliable conclusions, yet not irrevocable ones.

In this chapter we mainly look into two types of questions, “how are reasoning patterns represented?” and “what are the conditions for a given proposition to be believed?”.

4.1 Basic principles

One of the properties of classical logic is monotonicity, the conclusions that can be drawn from a set of premises are never invalidated if the set of premises increases, or, alternatively, the more information we have, the more

¹Provided by [Perlis 87].

conclusions we can draw.² This is stated by Theorem 3 (page 60). There are, however, many situations in which we want to produce conclusions that may have to be discarded in face of future information. This aspect of our reasoning is very desirable, because if we could only produce conclusions that were true, we would not be able to act upon the world. In fact, most of the time, we do not have complete information about what the state of the world is or what the consequences of our actions are.

In this chapter, we address the study of reasoning whose conclusions are more than just the logical consequences (in the classical sense) of a set of premises. Non-monotonic formalisms are an attempt to formalize this kind of reasoning, which is usually associated with sentences of the form:

In general, α ;
Typically, α ;
In the absence of the contrary, assume α ;
If α , I would have known it.

Non-monotonic reasoning mainly exploits reasoning with “everything that everyone knows” – also called *commonsense reasoning*. For example, given the sentence “In general, an employed person is available to work”, when we hear about an employed person, say Peter, we may be led to the conclusion that Peter is available to work, although there are a large number of exceptions to this rule: Peter may be on holidays; Peter may be sick; Peter may be on jury duty; and so on. We should notice that our conclusion about Peter being available to work is based not only on the information that, in general, employed persons are available to work and that Peter is an employed person, but also on the *assumption* that Peter is a normal employed person with respect to work availability. This assumption is based on the *lack* of information about the abnormality of work availability of Peter. If later on we learn that, for some reason, Peter is abnormal with respect to being available to work, we must withdraw the conclusion that Peter is available to work.

We should notice the difference between the sentence “In general, an employed person is available to work” and a sentence with a universal quantifier, for example, “Every employed person has a boss”. In the latter case, we can conclude that Peter has a boss, and this conclusion is not revisable. No

²This second statement may be misleading, since the number of conclusions that we can draw from a given set of premises is infinite (see Theorem 4). However, it translates nicely the underlying monotonic concept.

matter what we learn, we will never abandon the conclusion that Peter has a boss.³

We could try to use classical logic to represent the sentence that in general, an employed person is available to work. One attempt corresponds to the following *wff*:⁴

$$\forall x[(Employed(x) \wedge \neg Ab_w(x)) \rightarrow Available(x)] \quad (4.1)$$

which states that all employed persons that are not abnormal (with respect to being available to work) are available to work. We must now define what we mean by being abnormal with respect to being available to work, which we do with the following *wff*:⁵

$$\forall x[(OnHolidays(x) \vee Sick(x) \vee JuryDuty(x) \vee \dots) \rightarrow Ab_w(x)]. \quad (4.2)$$

The “...” in *wff* 3.2 indicates our inability to exhaustively enumerate all the possible conditions that may lead us to conclude the abnormality of an employed person. However, even if we could come up with all these conditions, from this *wff*, we could not conclude anything from the fact that Peter is an employed person, since there is not enough information to decide its normality or abnormality.

In face of the difficulty above, the goal of non-monotonic reasoning is to develop mechanisms that enable to “jump” to rational conclusions from incomplete information.

Using natural deduction, we may try to formalize this kind of reasoning with the following rule of inference (Abnormality Elimination), where t_1 is a term:

$$\begin{array}{ll} k & (Employed(t_1) \wedge \neg Ab_w(t_1)) \rightarrow Available(t_1) \\ l & Employed(t_1) \\ m & \text{Cannot prove that } Ab_w(t_1) \\ m+1 & Available(t_1) \end{array} \quad \text{Ab E, } (k, l, m)$$

There are two aspects worth noticing regarding our attempt to write this “rule of inference”.

- The first one is that we used specific predicates, *Employed*, *Ab_w*, and *Available* instead of variables that range over *wffs* (α , β , and so on) as

³As long as Peter is an employed person, but this is rather a quite different story.

⁴We use predicates with the obvious meaning.

⁵*Ibid.*

we have done up to now. In fact, we are talking about reasoning using a specific kind of knowledge (the knowledge that we have about what is expected from an employed person), rather than general knowledge.

- The second aspect is much deeper. The third line of the “inference rule” above mentions, not a *wff*, but rather the *lack of possibility* to derive a *wff*. This is a major deviation from the statement of the previous rules of inference. We are allowing that a rule of inference mentions, not a *wff*, but rather, the possible consequences of all *wffs* under the application of all rules of inference, including the rule of inference in question.

When we develop non-monotonic formalisms, we are opening the possibility of producing conclusions that are not true (in other words, we are accepting arguments that are not valid). However, we want to be able to infer propositions that are consistent with the premises, that is, propositions that are true in at least one of the models of the premises. For example, consider the set of premises:

{Peter is an employed person,
typically employed persons are available to work}.

The proposition “Peter is available to work” is consistent with this set, since it is true in at least one model of the premises (it belongs to a state of the world that we can conceive, based on these premises). On the other hand, “Peter is not available to work” is also consistent with this set of premises. However, “Peter is available to work” and “Peter is not available to work” cannot be inferred simultaneously.

Non-monotonic formalisms enable us to infer propositions that are consistent with the set of premises and that are mutually consistent. In general, the propositions that are inferred depend on the propositions that have been inferred. In our example, if we infer that “Peter is available to work”, we cannot further infer that “Peter is not available to work”, and vice-versa.

Monotonic inference, the kind of inference associated with traditional logics, can be seen as the mechanical application of all the inference rules in all possible ways to the premises and to the propositions generated from them. Once a proposition is generated, it is never removed. This process enables us to enumerate all the consequences of a set of premises. Whenever there

is a mechanical procedure to generate all the elements of a set, the set is said to be *recursively enumerable*.

On the other hand, non-monotonic inference, the kind of inference associated with non-monotonic formalisms, does not guarantee that a given proposition, once derived, will remain in all future steps, because a proposition inferred in a further step may invalidate it. This means that the set of consequences of a non-monotonic formalism is not recursively enumerable.

Given a set of premises, we are interested in computing the so-called extensions in a non-monotonic formalism. Intuitively, given a set of premises Δ and a non-monotonic formalism, an *extension*, Ω , of Δ in that formalism is a set of propositions that contains all the consequences of Δ in the classical sense and is closed under certain conditions: Extensions are fixed points with respect to the application of rules of inference. A *fixed point* with respect to the application of rules of inference is a set of propositions from which no further propositions can be generated by the application of rules of inference (see Theorem 5, page 61).

Given that a set of premises may have several extensions, we may wonder what theorems are generated by a set of premises. There are two different ways to define what a theorem is:

1. Theorems are the propositions that belong to every extension. This is called the *skeptical* approach, and is typically used in formalisms where many extensions are expected, without any intuitive criteria to select among them.
2. Theorems are the propositions that belong, to at least, one extension. This is called the *liberal* approach, and is used in formalisms where there are few extensions or when there is a criterion to select one extension as preferable.

Non-monotonic formalisms may be obtained either by creating new logics (as described in Sections 3.2 and 3.3) or by introducing additional mechanisms in classical logic (as described in Section 3.4).

4.2 Default Logic

Default logic extends classical logic in the sense described in Chapter 2. It uses the language of classical logic and all the rules of inference of classical logic. In addition, default logic also uses rules of inference, called default rules, that allow to “jump” to rational conclusions.

4.2.1 Deductive system

The deductive system of default logic, besides all the rules of inference of classical logic, contains an additional set of rules of inference, called *default rules*, that are applicable to a named set of predicate symbols. Default rules can be looked at as suggestions with respect to what we should believe in addition to what is dictated by classical logic.

Given the predicate symbols, $\alpha, \beta_1, \dots, \beta_m, \gamma$ ($m \geq 1$), a *default rule*, written⁶

$$\frac{\alpha(x) : \beta_1(x), \dots, \beta_m(x)}{\gamma(x)} \quad (4.3)$$

states that from $\alpha(t_1)$,⁷ where t_1 is a term, if it is consistent to assume $\beta_1(t_1), \dots, \beta_m(t_1)$, then we can infer $\gamma(t_1)$.

In a default rule,

- $\alpha(x)$ is called the *precondition* (also called the *prerequisite*) of the rule;
- $\beta_1(x), \dots, \beta_m(x)$ are called the *justifications* of the rule;
- $\gamma(x)$ is called the *consequent* of the rule.

As examples, if P, Q, R , and S are propositional logic predicates, *Employed* and *Available* are first-order predicates with one argument, the following are default rules:

$$\frac{P : Q, R}{S} \quad (4.4)$$

$$\frac{P : Q}{Q} \quad (4.5)$$

⁶In plain text we use the notation $\alpha(x) : \beta_1(x), \dots, \beta_m(x) / \gamma(x)$.

⁷Since predicates may have more than one argument, in the general case, x can be a vector, a sequence of one or more variables.

$$\frac{P : Q \wedge \neg R}{Q} \quad (4.6)$$

$$\frac{: Q}{R} \quad (4.7)$$

$$\frac{Employed(x) : Available(x)}{Available(x)} \quad (4.8)$$

There are a few particular cases of default rules:

1. A *closed* default rule is a rule where α , β_1 , ..., β_m , and γ have no variables. Default rules 3.4, 3.5, 3.6, and 3.7 are closed default rules.
2. A *normal* default rule is of the form

$$\frac{\alpha(x) : \beta(x)}{\beta(x)}.$$

Default rules 3.5 and 3.8 are normal default rules.

3. A *semi-normal* default rule is of the form

$$\frac{\alpha(x) : \beta(x) \wedge \neg \gamma(x)}{\beta(x)}.$$

Default rule 3.6 is a semi-normal default rule.

4. A *closed-world* default⁸ is a default rule with an empty precondition

$$\frac{: \beta_1(x), \dots, \beta_m(x)}{\gamma(x)}.$$

At first, it may seem that closed-world default rules do not fit within our definition of default rules. However, these are simply default rules whose precondition is always true (for example $A \vee \neg A$), and thus we simplify it without writing the precondition.

Default rule 3.7 is a closed-world default and is read “if it is possible to assume Q , we may conclude R ”.

⁸From the *closed world assumption* – the assumption that an agent knows all the atomic statements about a particular domain.

We should keep in mind that a default rule is a *rule of inference*. Using natural deduction, the use (elimination) of default rule 3.3 may be formalized as follows (Default Elimination), where t_1 is a term:

$$\begin{array}{ll}
 k & \alpha(t_1) \\
 l & \text{It is consistent to assume } \beta_1(t_1) \\
 \vdots & \vdots \\
 p & \text{It is consistent to assume } \beta_m(t_1) \\
 p+1 & \gamma(t_1)
 \end{array}
 \quad \text{Def E, } (k, l, \dots, p)$$

We should note the different nature of default rules as compared with classical inference rules. The applicability conditions of the default rule

$$\frac{\alpha(x) : \beta_1(x), \dots, \beta_m(x)}{\gamma(x)}$$

require that $\alpha(t_1)$ holds (which is similar to what happens in classical logic), but they also require that $\neg\beta_1(t_1), \dots, \neg\beta_m(t_1)$ are not derivable, using *all* rules of inference, which include the rule under consideration — this is what is meant by the *consistency* of the justifications. In other words, in order to find whether a given default rule is applicable, it is necessary to take into account the results produced by the application of all rules of inference, including the default rule itself. We can no longer just consider the *wffs* that exist in the proof up to the point of the application of the inference rule but we have to introduce additional mechanisms. Before we discuss these mechanisms, we need some further definitions.

A *default theory* is a pair (R, Δ) , composed of a set of default rules, R , and by a set of closed *wffs*, Δ ($\Delta \subset \mathcal{L}_{FOL}$). The *wffs* in Δ represent the basic knowledge and are treated as premises. Both R and Δ can be infinite sets.

The following are examples of default theories:

$$(\{\frac{P : Q}{Q}\}, \{P\}) \quad (4.9)$$

$$(\{\frac{Employed(x) : Available(x)}{Available(x)}\}, \{Employed(Peter), Sick(Peter)\}) \quad (4.10)$$

There are several particular cases of default theories:

1. A default theory that only contains closed default rules is called *closed*. Default theory 3.9 is closed.

2. A default theory that only contains normal default rules is called *normal*. Default theories 3.9 and 3.10 are normal.
3. A default theory that only contains semi-normal default rules is called *semi-normal*.

Going back to the applicability of default rules, given a default theory (R, Δ) , we want to compute the sets of *wffs* derivable from Δ using the rules of inference of classical logic and the default rules in R . These sets correspond, in classical logic, to the theorems of Δ . However, in default logic, there may be more than one of these sets or even none. Each one of these sets is called an *extension* of the default theory (R, Δ) . Each extension may be interpreted as a reasonable set of beliefs⁹ generated from Δ , using the default rules in R .

Before we go on, let us look into a few examples. Let us consider the statement that, in general, adults who are not students are employed. This is represented by the default rule

$$\frac{Adult(x) : \neg Student(x)}{Employed(x)}$$

Suppose, furthermore, that we know that Mary is an adult ($Adult(Mary)$). Given the default theory,

$$(\{\frac{Adult(x) : \neg Student(x)}{Employed(x)}\}, \{Adult(Mary)\})$$

It is reasonable to expect that its extension contains the *wffs* $Adult(Mary)$ and $Employed(Mary)$.

Going back to the example presented at the outset of this chapter that, typically, employed persons are available to work, we can represent this statement by the default rule

$$\frac{Employed(x) : Available(x)}{Available(x)}.$$

Exceptions to this default rule can be expressed by *wffs*, for example,

$$\forall x[Sick(x) \rightarrow \neg Available(x)].$$

⁹We use the word *belief* to stress that we are not dealing with propositions that are supposed to be true, but rather with propositions that it makes sense to consider, given the premises and the default rules.

Now, suppose that we know that Peter is an employed person and that Peter is sick. What do we expect to conclude from this information? The answer should be given by the extension of the default theory

$$(R, \Delta)$$

where

$$R = \left\{ \frac{Employed(x) : Available(x)}{Available(x)} \right\}$$

and

$$\Delta = \{ \forall x [Sick(x) \rightarrow \neg Available(x)], Employed(Peter), Sick(Peter) \}.$$

It is reasonable to expect the extension of default theory (R, Δ) to contain the *wffs* $\forall x [Sick(x) \rightarrow \neg Available(x)]$, $Employed(Peter)$, $Sick(Peter)$, and $\neg Available(Peter)$.

There are three properties that we expect that an extension of the default theory (R, Δ) should have:

1. It should contain Δ . Since Δ corresponds to the basic knowledge, the *wffs* in Δ must be part of any extension.
2. It should be closed with respect to derivability in the classical sense (using only the rules of inference of classical logic). This aspect guarantees that an extension is as complete as possible with respect to the classical notion of consequence.
3. It should be closed with respect to the applications of default rules in R . This aspect guarantees that all default rules that can be applied in the extension are applied.

Notice that these conditions say nothing about what should *not* exist in an extension. For example, the set of all *wffs* satisfies the three conditions above. In order to avoid the introduction, in an extension, of *wffs* without a proper justification, we should also require an extension to be a minimal set — there are no *wffs* that are added to the extension without a proper justification.

With this additional constraint, we avoid the case of an extension containing all *wffs*, but we still allow the existence of non-justified *wffs* in an extension, as illustrated by the following example. Let us consider the default theory

$$\left(\left\{ \frac{P : Q}{Q} \right\}, \{P\} \right).$$

There are two minimal sets that satisfy the three conditions above:

$$\Omega_1 = Th(\{P, Q\})$$

and

$$\Omega_2 = Th(\{P, \neg Q\}).$$

From our intuition, only Ω_1 should be considered as an extension of the default theory, since there is no reason to justify, in Ω_2 , the presence of $\neg Q$.

The source of the difficulty is the fact that the criteria for the application of a default rule take into account both the *wffs* that have already been derived and the *wffs* that have not been derived (but can be derived). This fact enables blocking the application of a default rule by the introduction of the negation of its justification. If this negated *wff* is not justified, it should not belong to the extension.

In order to come up with a proper definition of extension, let us suppose that both Ω and $\Gamma(\Omega)$ represent the same extension of the default theory (R, Δ) , and let us try to define $\Gamma(\Omega)$ in terms of Ω . In other words, let us suppose that we already know the extension Ω , and, based on this fact, we re-construct that extension, giving rise to $\Gamma(\Omega)$. Let us consider the following conditions:

1. $\Delta \subset \Gamma(\Omega)$.
2. $Th(\Gamma(\Omega)) = \Gamma(\Omega)$.
 $\Gamma(\Omega)$ is closed under derivability.
3. If $\alpha(x) : \beta_1(x), \dots, \beta_m(x) / \gamma(x) \in R$, $\alpha(t_1) \in \Gamma(\Omega)$, and $\neg\beta_1(t_1), \dots, \neg\beta_m(t_1) \notin \Omega$, then $\gamma(t_1) \in \Gamma(\Omega)$.

There is an important aspect to consider in this condition. Whereas we make sure that $\alpha(t_1) \in \Gamma(\Omega)$ — $\alpha(t_1)$ belongs to the extension we are constructing —, we test whether $\neg\beta_1(t_1), \dots, \neg\beta_m(t_1) \notin \Omega$ — the negation of $\beta_1(t_1), \dots, \beta_m(t_1)$ are not in our “guess” of the extension.

Although these conditions may resemble those stated before, there is a fundamental difference between them. Given Ω and $\Gamma(\Omega)$, we are able to formally distinguish what should be in the extension and what should not. This enables us to formally define an extension.

Let (R, Δ) be a default theory and let Ω be a set of *wffs* ($\Omega \subset \mathcal{L}_{FOL}$). Let $\Gamma(\Omega)$ be the smallest set of *wffs* of $\mathcal{L}_{FOL}$ that satisfies the following conditions:

1. $\Delta \subset \Gamma(\Omega)$.
2. $Th(\Gamma(\Omega)) = \Gamma(\Omega)$.
3. If $\alpha(x) : \beta_1(x), \dots, \beta_m(x) / \gamma(x) \in R$, $\alpha(t_1) \in \Gamma(\Omega)$, and $\neg\beta_1(t_1), \dots, \neg\beta_m(t_1) \notin \Omega$, then $\gamma(t_1) \in \Gamma(\Omega)$.

The set Ω is an *extension* of the default theory (R, Δ) if and only if $\Gamma(\Omega) = \Omega$, in other words, if Ω is a fixed point of the operator Γ .

Note that the operator Γ used to define the extensions of a theory is defined in a non-constructive way; that is, it is clearly insufficient to mechanically compute the extensions of a default theory. This operator relies on the existence of a “guess” of an extension.

Using the new definition of an extension, we consider again the default theory

$$(\{\frac{P : Q}{Q}\}, \{P\})$$

and the sets:

$$\Omega_1 = Th(\{P, Q\})$$

and

$$\Omega_2 = Th(\{P, \neg Q\}).$$

Since,

$$\Gamma(\Omega_1) = Th(\{P, Q\}) = \Omega_1$$

and

$$\Gamma(\Omega_2) = Th(P) \neq \Omega_2$$

only Ω_1 is an extension.

As a second example, the default theory

$$(\{\frac{: P}{\neg Q}, \frac{: Q}{\neg P}\}, \emptyset)$$

has two extensions: $\Omega_1 = Th(\{\neg P\})$ and $\Omega_2 = Th(\{\neg Q\})$.¹⁰

There is another, more intuitive, way to characterize an extension of a default theory, that results from the following theorem:

¹⁰ As an exercise, you should show these results.

Theorem 10 *Let Ω be a set of closed wffs ($\Omega \subset \mathcal{L}_{FOL}$), and let (R, Δ) be a closed default theory. Let*

$$\Omega_0 = \Delta$$

for $i \geq 0$, let¹¹

$$\Omega_{i+1} = Th(\Omega_i) \cup \left\{ \gamma : \frac{\alpha : \beta_1, \dots, \beta_m}{\gamma} \in R, \alpha \in \Omega_i, \neg\beta_1, \dots, \neg\beta_m \notin \Omega_i \right\}$$

Then, Ω is an extension of (R, Δ) if and only if

$$\Omega = \bigcup_{i=0}^{\infty} \Omega_i.$$

PROOF: See [Reiter 80, p. 89]. ■

As an example, let us consider the default theory

$$\left(\left\{ \frac{Q : P}{P}, \frac{R : \neg P}{\neg P} \right\}, \{Q, R\} \right).$$

It can be shown that both $\Omega = Th(\{Q, R, P\})$, and $\Sigma = Th(\{Q, R, \neg P\})$ are extensions of this default theory. For example, for Ω , we have:

$$\Omega_0 = \{Q, R\}$$

$$\Omega_1 = Th(\{Q, R\}) \cup \{P\}$$

$$\Omega_2 = Th(\{Q, R, P\}) \cup \{\}$$

Thus,

$$\Omega = \bigcup_{i=0}^{\infty} \Omega_i,$$

is an extension of the default theory.

As a last example, the following default theory has no extension¹²

$$\left(\left\{ \frac{P}{\neg P} \right\}, \emptyset \right).$$

¹¹For simplicity, we do not write down variables.

¹²The proof of this statement is left as an exercise.

At this point, we should note the significant difference between the default rules

$$\frac{\alpha : \beta_1, \beta_2}{\gamma}$$

and

$$\frac{\alpha : \beta_1 \wedge \beta_2}{\gamma}.$$

At first, these default rules may seem to be equivalent. To illustrate the difference between them, let us consider the default theory,

$$(\{\frac{\neg P, \neg Q}{R}\}, \{P \vee Q\}).$$

It can be shown that the extension of this default theory is $\Omega = Th(\{P \vee Q, R\})$.

However, if we consider the default theory

$$(\{\frac{\neg P \wedge \neg Q}{R}\}, \{P \vee Q\}),$$

its only extension is $\Omega = Th(\{P \vee Q\})$.

As a summary, default theories may have several extensions, eventually none. One problem with default theories, from the computational point of view, is that the computation of their extensions was introduced as a non-constructive way and thus is difficult to express with an algorithm.

Default theories were introduced as a way to formalize non-monotonic reasoning and their non-monotonic nature is expressed by the following theorem:

Theorem 11 (Non-monotonicity) *If (R, Δ) is a default theory with extension Ω , R' is a set of default rules, and Δ' is a set of wffs, then $(R \cup R', \Delta \cup \Delta')$ may have no extension Ω' such that $\Omega \subset \Omega'$.*

PROOF: See [Reiter 80, p. 91]. ■

It turns out that normal default theories are the most useful for expressing commonsense reasoning rules. Normal default theories have three important desirable properties:

1. *Guarantee of extensions.* Every closed normal default theory has an extension (Theorem 3).

2. *Semi-monotonicity.* If the set of default rules of a normal default theory increases, then, for every extension of the original default theory, there is an extension of the new default theory that contains it (Theorem 4).
3. Existence of a *mechanical decision procedure for formulas in an extension.* Given the closed normal default theory (R, Δ) and a wff $\gamma \in \mathcal{L}_{FOL}$, it is possible to write an algorithm to decide whether there is an extension Ω of (R, Δ) such that $\gamma \in \Omega$.

Theorem 12 (Existence of an extension) *Every closed normal default theory has an extension.*

PROOF: See [Reiter 80, pp. 95–96]. ■

Theorem 13 (Semi-monotonicity) *If R and R' are set of closed normal default rules, $R' \subseteq R$. Let Ω' be an extension of the default theory (R', Δ) . Then the default theory (R, Δ) has an extension Ω such that $\Omega' \subseteq \Omega$.*

PROOF: See [Reiter 80, p. 96]. ■

Although normal default rules yield default theories that correspond to many typical situations and that are easy to formalize, normal default theories may generate certain undesirable conclusions, namely due to the possible interaction between default rules, as is shown in the following example.

Let us consider the normal default theory $T = (R, \Delta)$ in which R has two default rules:

$$r_1 = \frac{Student(x) : Adult(x)}{Adult(x)}$$

and

$$r_2 = \frac{Adult(x) : Employed(x)}{Employed(x)}$$

and Δ has just one wff, $\Delta = \{Student(John)\}$. Default rule r_1 states that “Typically, students are adults”, and default rule r_2 states that “Typically, adults are employed”. These two default rules, taken together, enable the inference that “Typically, students are employed”, which, in general, is false.

In order to avoid transitivity in the application of default rules, we may increase R with the default rule:

$$r_3 = \frac{Student(x) : \neg Employed(x)}{\neg Employed(x)}.$$

The default theory $T' = (\{r_1, r_2, r_3\}, \{Student(John)\})$ has two extensions:

$$\Omega_1 = Th(\{Student(John), Adult(John), \neg Employed(John)\})$$

and

$$\Omega_2 = Th(\{Student(John), Adult(John), Employed(John)\}).$$

Although only extension Ω_1 is reasonable, nothing in the logic makes it preferable to Ω_2 . In order to avoid extension Ω_2 , we can modify default rule r_2 in the following way:

$$r'_2 = \frac{Adult(x) : Employed(x) \wedge \neg Student(x)}{Employed(x)}.$$

Default theory $T'' = (\{r_1, r'_2\}, \{Student(John)\})$ only has the required extension (Ω_1).

Default rule r'_2 is a *semi-normal* default rule. Semi-normal default theories do not have a guarantee of extension and do not have the semi-monotonic property.

4.2.2 Semantics

The semantics of default logic deals with sets of models, in the classical sense. The basic idea underlying the computation of the models of the extensions of the default theory (R, Δ) is to start with the set of all the models of Δ and to use the default rules in R to generate smaller sets of models. The smallest sets of models correspond, with some additional conditions, to the models of the extensions.

Recall, from Section 2.4.2, that an interpretation that satisfies all the *wffs* in a set of *wffs* is said to be a *model* of that set of *wffs*. We should notice that a set of *wffs* may have many models.

For example, let us assume that $\Delta = \{A, \neg B\}$. The interpretation with the valuation function

$$V_1(A) = T$$

$$V_1(B) = F$$

is a model of Δ . However, the interpretation with the valuation function

$$V_2(A) = T$$

$$V_2(B) = F$$

$$V_2(C) = F$$

is also a model of Δ .

We should note that the smaller a set of models is, the larger the number of satisfied *wffs* will be. In particular, the empty set of models satisfies all *wffs*.

The semantics of default logic is based on the introduction of a partial order among the sets of models of a default theory. Each default rule may be seen as expanding the description of the world by the addition of rule's consequent, and thus restricting the set of models to those that satisfy the consequent of the default rule.

Let $\mathcal{M}$ be a set of models and let $\mathcal{M}_1$ and $\mathcal{M}_2$ be two subsets of that set ($\mathcal{M}_1, \mathcal{M}_2 \in 2^{\mathcal{M}}$). Let

$$r = \frac{\alpha : \beta_1, \dots, \beta_m}{\gamma}$$

be a default rule. This default rule introduces a partial order $\geq_r$ on $2^{\mathcal{M}}$.

We say that the default rule r *prefers* the set of models $\mathcal{M}_1$ to the set of models $\mathcal{M}_2$, written as $\mathcal{M}_1 \geq_r \mathcal{M}_2$, if and only if:

$$\forall M \in \mathcal{M}_2 [M \models \alpha] \wedge \exists N_1, \dots, N_m \in \mathcal{M}_2 [N_i \models \beta_i, 1 \leq i \leq m] \wedge$$

$$\mathcal{M}_1 = \mathcal{M}_2 - \{M : M \models \neg\gamma\}$$

As an example, let us consider the default theory

$$(\{\frac{:Q}{Q}\}, \{P\})$$

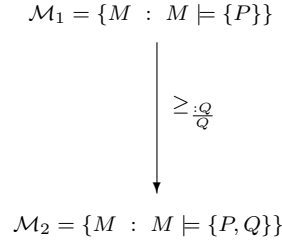
and the sets of models

$$\mathcal{M}_1 = \{M : M \models \{P\}\}$$

$$\mathcal{M}_2 = \{M : M \models \{P, Q\}\}.$$

Default rule : Q/Q introduces the following preference (this preference is represented in Figure 3.1)

$$\{M : M \models \{P, Q\}\} \geq_{\frac{Q}{Q}} \{M : M \models \{P\}\}$$

Figure 4.1: Partial order introduced by default rule : Q/Q .

Intuitively, the partial order $\geq_r$ captures the preference by r of more specialized descriptions of the word, in which the consequent of the rule is true, over other descriptions in which the preconditions of the rule are true, its justifications are consistent, but that do not satisfy the consequent.

The idea underlying the models preferred by a default rule can be extended to a set of default rules. Let R be a set of default rules, and let $\mathcal{M}$ be a set of models. Let $\mathcal{M}_1$ and $\mathcal{M}_2$ be two subsets of the set of models ($\mathcal{M}_1, \mathcal{M}_2 \in 2^{\mathcal{M}}$). The partial order, $\geq_R$ introduced by R on the set $2^{\mathcal{M}}$ is defined as the union of the partial orders introduced by the default rules in R .

We say that the set of default rules R prefers the set of models $\mathcal{M}_1$, to the set of models $\mathcal{M}_2$, written as $\mathcal{M}_1 \geq_R \mathcal{M}_2$, if and only if there exists a default rule in R that prefers the set of models $\mathcal{M}_1$ to the set of models $\mathcal{M}_2$, or there is a set of models $\mathcal{M}'$ such that the set of default rules R prefers the set of models $\mathcal{M}'$, to the set of models $\mathcal{M}_2$ and the set of default rules R prefers the set of models $\mathcal{M}_1$, to the set of models $\mathcal{M}'$:

$$(\exists r \in R[\mathcal{M}_1 \geq_r \mathcal{M}_2]) \vee (\exists \mathcal{M}' \in 2^{\mathcal{M}}[\mathcal{M}_1 \geq_R \mathcal{M}' \geq_R \mathcal{M}_2]).$$

As an example, let us consider the default theory

$$(\{\frac{P : \neg Q}{R}, \frac{P : \neg R}{S}\}, \{P\})$$

and the set of models

$$\begin{aligned}
\mathcal{M}_1 &= \{M : M \models \{P\}\} \\
\mathcal{M}_2 &= \{M : M \models \{P, R\}\}
\end{aligned}$$

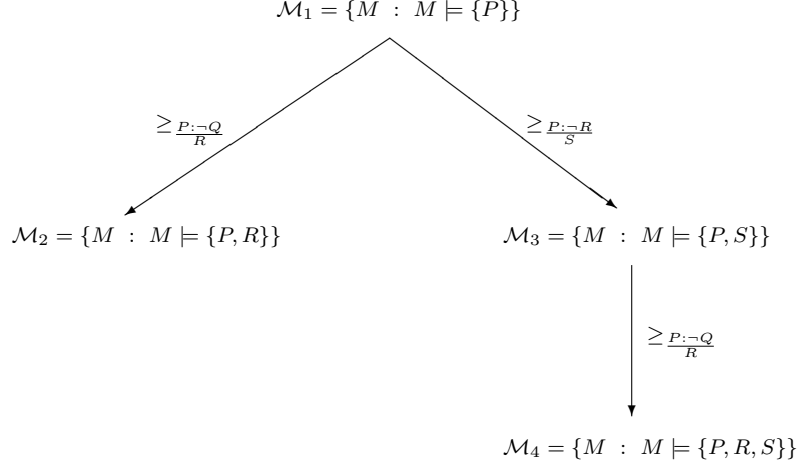


Figure 4.2: Partial order introduced by default rules $\frac{P:\neg R}{S}$ and $\frac{P:\neg Q}{R}$.

$$\mathcal{M}_3 = \{M : M \models \{P, S\}\}$$

$$\mathcal{M}_4 = \{M : M \models \{P, R, S\}\}.$$

The set of default rules $\{P : \neg Q/R, P : \neg R/S\}$ introduces the following preferences (Figure 3.2):

$$\{M : M \models \{P, R\}\} \geq_{\{\frac{P:\neg Q}{R}, \frac{P:\neg R}{S}\}} \{M : M \models \{P\}\} \quad (4.11)$$

$$\{M : M \models \{P, R, S\}\} \geq_{\{\frac{P:\neg Q}{R}, \frac{P:\neg R}{S}\}} \{M : M \models \{P\}\}. \quad (4.12)$$

The relation corresponding to expression 3.11 is justified by default rule $P : \neg Q/R$; the relation corresponding to expression 3.12 is justified by the existence of the set of models $\mathcal{M}_3$ and by the fact that

$$\mathcal{M}_4 \geq_{\frac{P:\neg Q}{R}} \mathcal{M}_3 \geq_{\frac{P:\neg R}{S}} \mathcal{M}_1.$$

For normal default theories (R, Δ) , it is enough to consider the maximal models with respect to $\geq_R$ containing elements of $2^{Mod(\Delta)}$.¹³ Each one of these maximal models corresponds to the model of an extension of the default theory (R, Δ) .

Non-normal default theories, since they do not satisfy the semi-monotonicity property, require a more complex approach. This approach is based on the

¹³ $Mod(\Delta)$, is the set models of Δ , that is, $Mod(\Delta) = \{M : M \models \Delta\}$.

notion of *stability*, which guarantees that the maximal models satisfy all the preference conditions of the default rules used to generate them.

Let us consider, again, the preference relation shown in Figure 3.2. The set of models

$$\mathcal{M}_4 = \{M : M \models \{P, R, S\}\}$$

was obtained from the set of default rules

$$\left\{ \frac{P : \neg Q}{R}, \frac{P : \neg R}{S} \right\}.$$

However, the application of the second default rule ($P : \neg R/S$) is in conflict with the application of the first default rule ($P : \neg Q/R$) that produced the set of models $\mathcal{M}_3$, where it was assumed that it was consistent to consider $\neg R$. To avoid this situation, the following notion of stability is introduced.

Let (R, Δ) be a default theory, and let $\mathcal{M} \in 2^{Mod(\Delta)}$. We say that $\mathcal{M}$ is *stable* in (R, Δ) if and only if there is $R' \subseteq R$ such that $\mathcal{M} \geq_{R'} Mod(\Delta)$ and for each default rule

$$\frac{\alpha : \beta_1, \dots, \beta_m}{\gamma} \in R' \exists N_1 \dots N_m \in \mathcal{M} [N_i \models \beta_i, 1 \leq i \leq m].$$

In other words, a set of models is stable in the default theory (R, Δ) if it is a specialization of the set of models of Δ and does not invalidate the justifications of any default rule used in the specialization.

The set of models $\mathcal{M}_4$ shown in Figure 3.2 is not stable, because given the default rule $(P : \neg R/S)$, there is no $M \in \mathcal{M}_4$ such that $M \models \neg R$.

Theorem 14 (Soundness) *If Ω is an extension of the default theory (R, Δ) , then $\{M : M \models \Omega\}$ is stable and maximal for (R, Δ) .*

PROOF: See [Etherington 88, pp. 174–175]. ■

This theorem guarantees that the extensions obtained in the deductive system correspond to extensions in the semantic system.

Theorem 15 (Completeness) *If $\mathcal{M}$ is a stable and maximal set of models of (R, Δ) , then $\mathcal{M}$ is the set of models for some extension of (R, Δ) . In other words, the set $\{\alpha : \forall M \in \mathcal{M}, M \models \alpha\}$ is an extension of (R, Δ) .*

PROOF: See [Etherington 88, pp. 175–176]. ■

This theorem guarantees that every extension obtained in the semantic system has a corresponding extension in the deductive system.

As a first example, let us consider the normal default theory $T_1 = (R, \Delta)$, where R has the following default rules:

$$\begin{aligned} r_1 &= \frac{E \vee F : A \wedge F}{A \wedge F} \\ r_2 &= \frac{A : B}{B} \\ r_3 &= \frac{A \wedge E : C}{C} \\ r_4 &= \frac{: \neg E}{\neg E} \end{aligned}$$

and Δ contains the following *wffs*:

$$\Delta = \{C \rightarrow D, (A \wedge B) \rightarrow E, E \vee D, D \rightarrow F\}$$

Table 3.1 shows all models of Δ , that is, all combinations of truth values that make all the *wffs* in Δ true. If we apply the definition of preferential models introduced by the default rules, we obtain the partial order presented in Figure 3.3, in which the maximal models are represented inside a rectangle. From the completeness theorem, we can conclude that T_1 has two extensions, which are defined by the sets of models $\mathcal{M}_3 = \{M_{14}, M_{17}\}$ and $\mathcal{M}_6 = \{M_1\}$.

Figure 3.3 makes use of an explicit representation of the models in each of the preferential sets of models. An alternative to represent the sets of models uses an implicit representation and is shown in Figure 3.4. In this figure:

1. The set of models $\mathcal{M}_1$ contains all the models that satisfy the *wffs* in the set

$$\Delta = \{C \rightarrow D, (A \wedge B) \rightarrow E, E \vee D, D \rightarrow F\}.$$

2. The set of models $\mathcal{M}_2$ contains all the models that satisfy

$$\Delta = \{C \rightarrow D, (A \wedge B) \rightarrow E, E \vee D, D \rightarrow F\}$$

<i>Model</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
M_1	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_2	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_3	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_4	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_5	<i>T</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_6	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_7	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_8	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>
M_9	<i>T</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>
M_{10}	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>
M_{11}	<i>T</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>
M_{12}	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>
M_{13}	<i>F</i>	<i>T</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{14}	<i>T</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{15}	<i>F</i>	<i>F</i>	<i>T</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{16}	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{17}	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{18}	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>	<i>T</i>
M_{19}	<i>T</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>
M_{20}	<i>F</i>	<i>T</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>
M_{21}	<i>T</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>
M_{22}	<i>F</i>	<i>F</i>	<i>F</i>	<i>F</i>	<i>T</i>	<i>F</i>

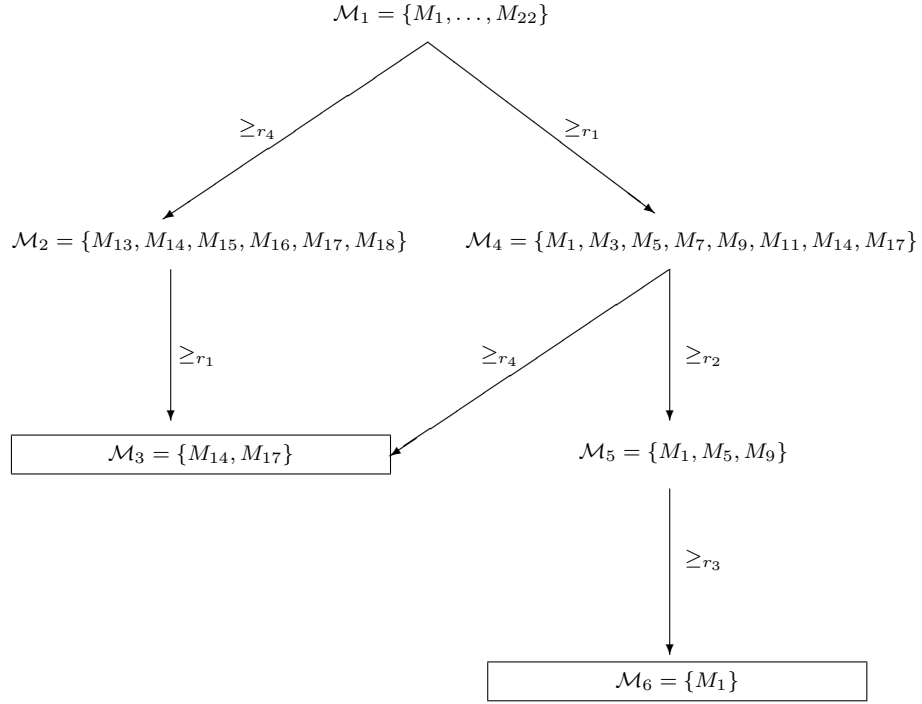
Table 4.1: Models of $\{C \rightarrow D, (A \wedge B) \rightarrow E, E \vee D, D \rightarrow F\}$.

and the consequent of the default rule r_4 , that is, the set of models that satisfies

$$\{C \rightarrow D, (A \wedge B) \rightarrow E, E \vee D, D \rightarrow F, \neg E\}$$

We should note that this set can be simplified in the following way: Since both $E \vee D$ and $\neg E$ have to be true, we may conclude that D has to be true; since D has to be true, $C \rightarrow D$ enables any truth value for C ; since both D and $D \rightarrow F$ have to be true, we may conclude that F has to be true; since $(A \wedge B) \rightarrow E$, and $\neg E$ have to be true, we may conclude that $\neg(A \wedge B)$ has to be true. Thus, $\mathcal{M}_2$ represents the set of models of

$$\{\neg(A \wedge B), D, \neg E, F\}.$$

Figure 4.3: Partial order between the models of the theory T_1 .

3. The set of models $\mathcal{M}_3$ contains all the models that satisfy

$$\{\neg(A \wedge B), D, \neg E, F\}$$

and the consequent of default rule r_1 , that is,

$$\{\neg(A \wedge B), D, \neg E, F, E \vee F, A \wedge F\}.$$

In a similar way, we may conclude that $\mathcal{M}_3$ represents the set of models

$$\{A, \neg B, D, \neg E, F\}.$$

4. The same line of reasoning is applied to the sets of models $\mathcal{M}_4$, $\mathcal{M}_5$, and $\mathcal{M}_6$.

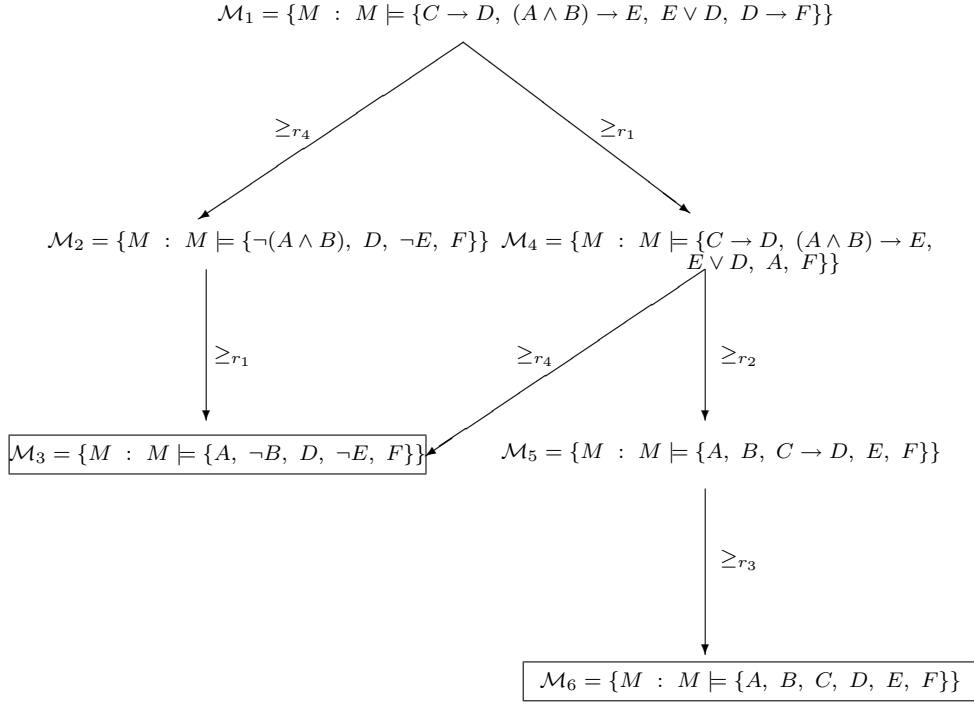


Figure 4.4: Partial order between the models of the theory T_1 (using an implicit representation of models).

Let us now consider the non-normal default theory

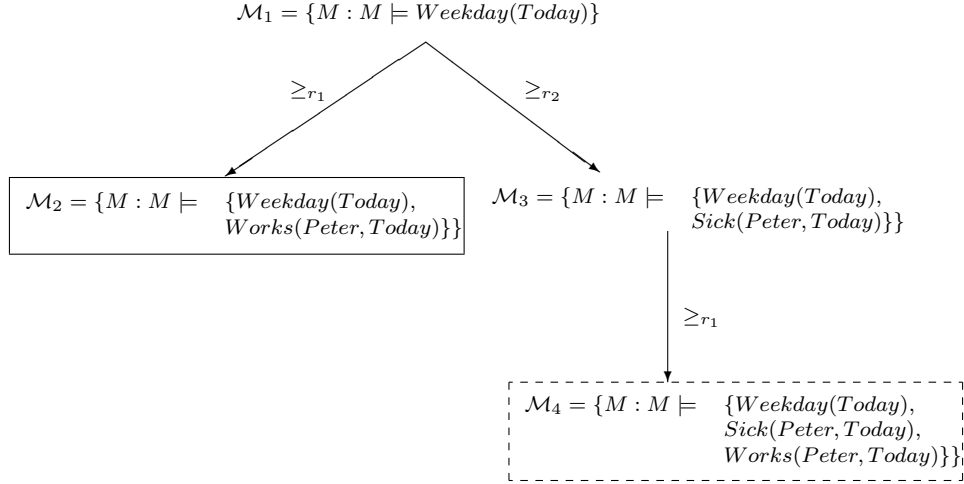
$$T_2 = (\{r_1, r_2\}, \{WeekDay(Today)\}),$$

where r_1 and r_2 are the following default rules:

$$r_1 = \frac{Weekday(x) : \neg HasExcuse(Peter, x)}{Works(Peter, x)}$$

$$r_2 = \frac{Weekday(x) : \neg Works(Peter, x)}{Sick(Peter, x)}$$

In other words, in an weekday, if it is consistent to assume that Peter has no excuse, then Peter works (r_1); in a weekday, if it is consistent to assume that Peter does not work, then Peter is sick (r_2). Today is a weekday.

Figure 4.5: Partial order between the models of the default theory T_2 .

In order to compute what can be concluded from this theory, we will determine the models of its extensions. Figure 3.5 shows the partial order introduced by the default rules of the theory T_2 . In fact,

$$\mathcal{M}_2 \geq_R \mathcal{M}_1$$

$$\mathcal{M}_4 \geq_R \mathcal{M}_3 \geq_R \mathcal{M}_1.$$

In this partial order, there are two maximal sets of models $\mathcal{M}_2$ and $\mathcal{M}_4$. The set of models $\mathcal{M}_2$ results from the application of default rule r_1 to the premisses; it states that Peter works, because it is consistent to assume that he has no excuse (nothing in the premisses lead to the conclusion of this fact). The set of models $\mathcal{M}_4$ results from the application of default rule r_2 , followed by the application of default rule r_1 ; it states that Peter works today and Peter is sick today.

Out of these two sets of models, only $\mathcal{M}_2$ is stable. In order for $\mathcal{M}_4$ to be stable, there should be $R' \subseteq R$ such that $\mathcal{M}_4 \geq_{R'} \mathcal{M}_1$, and for each default

rule¹⁴

$$\frac{\alpha : \beta}{\gamma} \in R' \exists M \in \mathcal{M}_4 [M \models \beta].$$

This condition is not satisfied, because $\forall M \in \mathcal{M}_4 : M \models \text{Works}(\text{Peter}, \text{Today})$. Which goes against the justification of default rule r_2 . In fact, the application of default rule r_1 goes against the justification of default rule r_2 ($\neg \text{Works}(\text{Peter}, \text{Today})$), and for this reason model $\mathcal{M}_4$ is not stable. In other words, T_2 has only one extension, defined by the set of models $\mathcal{M}_2$.

This situation never happens when only normal rules are used because the justification of a normal rule is equal to its conclusion.

4.3 Auto-epistemic Logic

Auto-epistemic logic uses the modal operator B , which is read “believes”. The word “auto-epistemic” stems from the fact that this logic corresponds to the introspection of knowledge by the agent that performs the reasoning, and thus it is adequate to model agents that reflect on their own beliefs. In auto-epistemic logic, it is possible to represent propositions such as “If I don’t believe that P then Q is true” and “If P is true, then I believe that P ”. Since the goal of auto-epistemic logic is to model the beliefs of an agent that reflects upon its own beliefs, we will be interested in sets of auto-epistemic formulas that are interpreted as the beliefs of such agents.

The language of auto-epistemic logic extends the language of first-order logic with the introduction of the modal operator B . The intuitive interpretation of $B\alpha$ is “I believe that α ”. For example, the *wff*

$$\forall x[(\text{Employed}(x) \wedge B\text{Available}(x)) \rightarrow \text{Available}(x)],$$

may be read as “If x is an employed person and I believe that x is available to work, then x is available to work”, in other words, I will believe that all employed persons are available to work, except for those that I explicitly know not being available to work. Like other non-monotonic logic formalisms, in auto-epistemic logic, it is not possible to define, in a constructive way, the set of *wffs* that may be deduced from a set of premises.

Auto-epistemic logic defines, in syntactic terms, the conditions that should be satisfied by the set of beliefs of an ideal rational agent. By *rational*, is

¹⁴Note that this is the application of the stability conditions to this particular theory.

meant an agent whose reasoning process only produces valid arguments; by *ideal*, is meant an agent that believes in all the consequences of its beliefs. Intuitively, an auto-epistemic theory generated from a set of premises corresponds to a maximal consistent set of propositions that may be deduced from that set of premises. This set should contain everything that can be derived from the premises, using classical logic, plus everything that can be deduced by reflection over the *wffs* that were deduced. An *auto-epistemic theory* is a set of *wffs*, Ω . An auto-epistemic theory is said to be *stable* if it satisfies the following conditions:

1. If $\{\alpha_1, \dots, \alpha_m\} \in \Omega$ and $\{\alpha_1, \dots, \alpha_m\} \vdash \beta$, then $\beta \in \Omega$.
2. If $\alpha \in \Omega$, then $B\alpha \in \Omega$.
3. If $\alpha \notin \Omega$, then $\neg B\alpha \in \Omega$.

The stability stems from the fact that if conditions 1, 2, and 3 are satisfied, no additional proposition may be inferred by a rational agent. These conditions were defined in [Stalnaker 80] and are known as *Stalnaker conditions*.

Considering an auto-epistemic theory as the set of beliefs to be held by a rational agent, Stalnaker conditions state that a rational agent knows all the consequences of its beliefs (it is logically omniscient), it believes in its beliefs and does not believe in what does not belong to its beliefs.

An auto-epistemic theory, Ω that is stable and consistent, besides Stalnaker conditions, satisfies two additional conditions:

1. If $B\alpha \in \Omega$, then $\alpha \in \Omega$.
2. If $\neg B\alpha \in \Omega$, then $\alpha \notin \Omega$.

In fact, assuming that $B\alpha \in \Omega$ and $\alpha \notin \Omega$, we could conclude, using the third Stalnaker condition, that $\neg B\alpha \in \Omega$, which contradicts the fact that Ω is consistent; assuming that $\neg B\alpha \in \Omega$ and $\alpha \in \Omega$ we could conclude, using the second Stalnaker condition, that Ω was not consistent because it would contain both $\neg B\alpha$ and $B\alpha$.

When dealing with auto-epistemic theories, we will be interested in computing the theories generated from a set of premises Δ . When we consider the stability conditions we can realize that they will not be sufficient for this job, once nothing prevents Ω from having *wffs* that are not derived from Δ

(neither in the classical sense nor by reflection on the beliefs of the agent). It is then necessary to add additional conditions that guarantee that the only *wffs* in an auto-epistemic theory are the premises and the *wffs* derivable from them. This is obtained through the definition of a theory grounded on a set of premises.

An auto-epistemic theory is *grounded* on a set of premises Δ if and only if all the *wffs* in Ω were derived (using classical logic) from the set

$$\Delta \cup \{B\alpha : \alpha \in \Omega\} \cup \{\neg B\alpha : \alpha \notin \Omega\}.$$

Defining the operator AE_Δ as:

$$AE_\Delta(\Omega) = Th(\Delta \cup \{B\alpha : \alpha \in \Omega\} \cup \{\neg B\alpha : \alpha \notin \Omega\}),$$

an auto-epistemic theory Ω is grounded on the set of premises Δ if and only if:

$$AE_\Delta(\Omega) = \Omega,$$

in other words, if Ω is a fixed point of the operator AE_Δ .

Consider, for example, the set of premises

$$\Delta = \{\forall x[\neg B\text{Available}(x) \rightarrow \neg \text{Available}(x)], \text{Available}(\text{Peter})\}.$$

The first *wff* in this set states that “If I don’t believe that someone is available to work, then that person is not available to work”; the second *wff* states that “Peter is available to work”. A grounded auto-epistemic theory generated from this set contains, among others, the following *wffs*

$$\{\neg B\text{Available}(\text{John}), \neg \text{Available}(\text{John})\}.$$

In fact, the first *wff* is generated from the non-derivability of $\text{Available}(\text{John})$:

$$\Delta \not\vdash \text{Available}(\text{John}).$$

According to the third Stalnaker condition $\neg B\text{Available}(\text{John})$ belongs to the auto-epistemic theory. This enables to generate, using modus ponens,

$$\neg \text{Available}(\text{John}).$$

Starting now with the set

$$\Delta' = \Delta \cup \{\text{Available}(\text{John})\}$$

we do not have anymore the $wff \neg B\text{Available}(\text{John})$ in the auto-epistemic theory, since

$$\Delta \vdash \text{Available}(\text{John})$$

consequently, the new theory will contain $B\text{Available}(\text{John})$.

In auto-epistemic logic, a *stable expansion* of a set of premises Δ is defined as an auto-epistemic theory, Ω , that contains Δ , and is stable and grounded on Δ , that is, as the set of consequences of

$$\Delta \cup \{B\alpha : \alpha \in \Omega\} \cup \{\neg B\alpha : \alpha \notin \Omega\}.$$

From a set of premises Δ , the possible sets of beliefs that an ideal rational agent may hold, from Δ are given by the stable expansions of Δ . Note that there may be more than one stable expansion of a set Δ . As an example, from [Moore 88, p. 110], consider the set of premises

$$\Delta = \{\neg BP \rightarrow Q, \neg BQ \rightarrow P\}.$$

Any auto-epistemic theory that contains these premises, if P does not belong to the theory, then Q belongs to the theory, and the other way around. This means that a stable expansion contains either P or Q , but not both.

As a final remark, we should note that an agent following auto-epistemic logic is omniscient about what it knows and about what it does not know (if $\alpha \in \Omega$, then $B\alpha \in \Omega$; if $\alpha \notin \Omega$, then $\neg B\alpha \in \Omega$).

There are two alternatives to define the semantics of auto-epistemic logic, both proposed by Moore. The first semantics proposed [Moore 83, 85], is based on the notion of auto-epistemic model; the second one [Moore 84], is based on the notion of possible worlds from modal logic.

4.4 Circumscription

Circumscription was introduced in [McCarthy 80], was generalized in [McCarthy 84], and has been explored by many researchers. Circumscription is not a non-monotonic logic but rather an attempt to add to classical logic a way of “jumping to conclusions” and to infer certain properties about the objects that satisfy certain relations. The idea underlying circumscription is to state that all the objects that satisfy a certain property are those for which it is possible to prove that property. For example, circumscribing the

property of “being a block” corresponds to assuming that all objects about which we cannot prove that they are blocks, are not blocks.

Let α be a first-order *wff* containing the n -ary predicate letter P . Let $\alpha(\Phi)$ be the result obtained by replacing all occurrences of P in α by the predicate Φ . The circumscription of P in α , with the predicate Φ , is the schemata

$$(\alpha(\Phi) \wedge \forall x[\Phi(x) \rightarrow P(x)]) \rightarrow \forall x[P(x) \rightarrow \Phi(x)],$$

where x is a vector of n elements, $x = (x_1, \dots, x_n)$. Notice that, if the antecedent of the implication is false, nothing can be concluded with circumscription. The interesting case happens when $\alpha(\Phi)$ is true (that is, when Φ shares with P the property of satisfying α), and having the property Φ implies having the property P . In this case, circumscription allows us to conclude that Φ and P are equivalent. All instances of this schema, together with α , may be used to derive new information. The parameter Φ may be replaced by *any* expression containing an n -ary predicate.

The above schema states that the only entities (x) that satisfy P are those that *have to* satisfy it, assuming the proposition α . Since the schema corresponds to an implication, if we assume both antecedents we may infer the consequent. The first antecedent, $\alpha(\Phi)$ corresponds to the assumption that Φ satisfies the conditions required by P , and the second antecedent $\forall x[\Phi(x) \rightarrow P(x)]$ corresponds to the assumption that the entities that satisfy Φ are a subset of the entities that satisfy P . The conclusion states that the converse is true, which makes Φ and P extensionally equivalent.

Let us suppose that α was the following proposition (from [McCarthy 80, p. 32]):

$$Block(A) \wedge Block(B) \wedge Block(C),$$

that states that A , B , and C are blocks. This proposition contains the unary predicate “*Block*”. The circumscription of *Block* in α produces the schema:

$$((\Phi(A) \wedge \Phi(B) \wedge \Phi(C)) \wedge \forall x[\Phi(x) \rightarrow Block(x)]) \rightarrow \forall x[Block(x) \rightarrow \Phi(x)].$$

In order to proceed, we have to choose Φ in such a way that guarantees that Φ corresponds to all blocks that exist. If we choose

$$\Phi(x) = (x = A \vee x = B \vee x = C)$$

and we replace $\Phi(x)$ in the schemata that defines circumscription, and take into account that α , the antecedent of the implication, is satisfied, we can infer that

$$\forall x[Block(x) \rightarrow (x = A \vee x = B \vee x = C)]$$

which states that the only blocks that exist are A , B , and C .

The difficulty in the use of circumscription is the choice of the predicate Φ . This choice has to be done in such a way to guarantee that it defines (circumscribes) the entities in which we are interested. For complex situations, the choice of this predicate is very difficult, which prevents the use of circumscription in systems for automatic reasoning.

4.5 Summary

In this chapter, we addressed the study of non-monotonic logics, with special stress on default logic, a logic whose conclusions may be invalidated in the presence of additional information. Default logic is an example of a class of logics, called non-monotonic logics, that are important to model our reasoning pattern in face of incomplete information. We argued that in traditional logic it is not possible to perform this type of reasoning and presented arguments in favor of the need of non-monotonic logics.

Default logic extends the rules of inference of classical logic with default rules. Rules that are triggered, not only by what has been derived, but also by what can be derived. This second aspect is responsible for the non-monotonic behavior of the logic. We show that in these logics the set of theorems cannot be computed in a constructive way and needs some form of “guessing” what will be in the final set of theorems.

We described other approaches to non-monotonic logics: circumscription (that adds to classical logic a mechanism that enables jumping to conclusions) and auto-epistemic logic (that uses a modal operator).

4.6 Bibliographic notes and historical remarks

Non-monotonic reasoning¹⁵ is an area of AI that has attracted considerable interest ([Bobrow 80], [Ginsberg 87], [Perlis 87], [Reiter 88]). There are two important issues associated with non-monotonic reasoning: the formalization of reasoning and the process of belief revision. The first issue addresses the ways of formalizing the reasoning pattern that we discussed and the reasoning process associated with it. The second addresses the issue

¹⁵This designation was introduced by [Minsky 75].

of how to update information about the world when additional information clashes with previously produced conclusions (this aspect is addressed in Chapter 4).

Default logic was introduced by [Reiter 80] and was revised by [Reiter and Criscuolo 81]. The semantics for default logic was provided by [Etherington 87, 88]. A good introduction to default logic, describing, in parallel, other formalisms for non-monotonic reasoning is presented in [Besnard 89]. The book by [Lukasiewicz 90] presents a detailed study of the various approaches to non-monotonic reasoning, in particular, default logic, auto-epistemic logic, and circumscription.

The example presented in page 69 is due to [Lukasiewicz 90, pp. 169–170] and the example presented in page 74 is due to [Reiter and Criscuolo 81].

Auto-epistemic logic was proposed by [Moore 83, 85, 88].

Since non-monotonic logics correspond to a radical change to the behavior of a logical system, we should expect that objections should be raised against them. Here, we present an overview some of these. A more complete discussion of these objections can be found in [Ginsberg 87, p. 11] and [Etherington 88, pp. 63–73].

1. *Objections to the use of logic to model non-monotonic reasoning.*

This objection suggests the use of probabilistic and fuzzy notions, instead of logical notions, to model statements such as “Typically employed persons are available to work” [Cheeseman 85], [Zadeh 85]. In these approaches, propositions are associated with numerical values (in general, real numbers in the interval $[0,1]$) that define the degree of belief on those statements.

To answer this objection, we should keep in mind that it is very difficult, even impossible, to assign numerical values to propositions. Furthermore, some conclusions obtained by non-monotonic logic formalisms may have a degree of plausibility near to zero but nevertheless are plausible and interesting [Reiter 88, pp. 472–473].

2. *Objection regarding the lack of decidability of logic.*

A logic is semi-decidable if there is a mechanical procedure that, given any propositions, $\alpha_1, \dots, \alpha_n$, and β , is guaranteed to terminate in a finite amount of time if $\{\alpha_1, \dots, \alpha_n\} \vdash \beta$ and may never terminate if $\{\alpha_1, \dots, \alpha_n\} \not\vdash \beta$. First-order logic is semi-decidable, which means

that we are guaranteed to have an answer if a given formula is a theorem. The objection raised to non-monotonic logics (in [Israel 80], [Reiter 80] and [Perlis 87]) is the fact that they are not decidable: The application of a rule of inference in default logic, for example, requires the verification of the consistency of the conclusion; in other words, we have to prove that the negation of the conclusion is not derivable from the premises.

This aspect is, in fact, a problem that has two practical consequences: On the one hand, the implementations of non-monotonic reasoning systems tend to be very slow, since they require multiple consistency checks (in theory, these may never return an answer to certain questions); on the other hand, a system that uses non-monotonic reasoning may produce wrong answers once in a while (or execute forever, which is even worse).

3. *Objection to the multiple sets of theorems.*

The fact that the set of theorems of a non-monotonic logic is not unique (this corresponds to the existence of multiple extensions in default logic) has led [Kramosil 75], [Sandewall 72], and [Reiter 78a] to object to these formalisms. In particular, [Reiter 78a] states that “such behavior is clearly unacceptable. At least we should require a non-monotonic theory to satisfy a Church-Rosser property: independently of the order of application of the rules of inference, the set of premises must be unique”.

This is an objection that no longer makes sense. In fact people have understood the need of multiple extensions: thinking of an extension as the set of beliefs that a rational agent may have, given the premises and the suggestions in the default rules, it is just natural that more than one of these sets may exist.

4. *Objections to qualification and expressive power.*

The inability to distinguish, in non-monotonic logics, between what is known and what is suggested by the default rules has been pointed out by [Nutter 83] as a negative aspect of these logics.

The answer to this objection is given, in part, by the use of Belief Revision Systems, described in the next chapter, and by the work of [Cravo and Martins 91, 93].

4.7 Exercises

1. Show that the default theory

$$(\{\frac{:P}{\neg Q}, \frac{:Q}{\neg P}\}, \emptyset)$$

has two extensions: $\Omega_1 = Th(\{\neg P\})$ and $\Omega_2 = Th(\{\neg Q\})$.

2. Show that $Th(\{Q, R, \neg P\})$ is an extension of the default theory

$$(\{\frac{Q:P}{P}, \frac{R:\neg P}{\neg P}\}, \{Q, R\}).$$

3. Show that the following default theory¹⁶ has no extensions:

$$(\{\frac{:P}{\neg P}\}, \emptyset).$$

- (a) Using the syntactic definition of extension.
- (b) Using the semantic definition of extension.

4. Compute the models of the extensions of the closed normal default theory $T = (R, \Delta)$, where R has the following default rules:

$$r_1 = \frac{:P}{P}$$

$$r_2 = \frac{:Q}{Q}$$

and

$$\Delta = \{\neg Q \vee \neg P\}.$$

5. Compute the models of the extensions of the closed normal default theory $T = (R, \Delta)$, where R has the following default rules:

$$r_1 = \frac{:P}{P}$$

$$r_2 = \frac{:Q}{Q}$$

$$r_3 = \frac{:R}{R}$$

and

$$\Delta = \{P \rightarrow (\neg Q \wedge \neg R)\}.$$

¹⁶From [Reiter 80, p. 91].

6. Compute the models of the extensions of the default theory $T = (R, \Delta)$, where R has the following default rules:

$$r_1 = \frac{P : (R \wedge \neg S)}{R}$$

$$r_2 = \frac{Q : S}{S}$$

and

$$\Delta = \{P, Q\}.$$

7. The default rule

$$\frac{\alpha(x) : \beta_1(x), \dots, \beta_m(x)}{\gamma(x)}$$

is interpreted in the following way, “from $\alpha((x) \cdot \{x/t_1\})$, if it is consistent to suppose $\beta_1((x) \cdot \{x/t_1\})$, $\dots$, $\beta_m((x) \cdot \{x/t_1\})$, then, we can derive $\gamma((x) \cdot \{x/t_1\})$ ”.

Explain the reason why we need the *uffs* $\beta_1((x) \cdot \{x/t_1\})$, $\dots$, $\beta_m((x) \cdot \{x/t_1\})$, as justifications for this rule, rather than their conjunction $\beta_1((x) \cdot \{x/t_1\}) \wedge \dots \wedge \beta_m((x) \cdot \{x/t_1\})$.

8. Compute the models of the extensions of the default theory, using an implicit representation of models:

$$(\{r_1, r_2, r_3\}, \emptyset)$$

where

$$r_1 = \frac{: P \wedge \neg Q}{\neg Q}$$

$$r_2 = \frac{: Q \wedge \neg R}{\neg R}$$

$$r_3 = \frac{: R \wedge \neg P}{\neg P}.$$

9. Compute the models of the extensions of the default theory, using an implicit representation of models:

$$(\{r_1, r_2\}, \{P, Q\})$$

where

$$r_1 = \frac{P : R \wedge \neg S}{R}$$

$$r_2 = \frac{Q : S}{S}.$$

10. Compute the models of the extensions of the default theory $(\{r_1, r_2\}, \{P\})$, where

$$r_1 = \frac{P : Q}{Q}$$

$$r_2 = \frac{P : Q \wedge \neg Q}{R},$$

using an implicit representation of models.

11. Compute the models of the extensions of the default theory, using an implicit representation of models:

$$(\{r_1, r_2, r_3\}, \{P\})$$

where

$$r_1 = \frac{P : Q}{Q}$$

$$r_2 = \frac{Q : R}{R}$$

$$r_3 = \frac{Q : \neg R}{\neg R}.$$

12. Compute the models of the extensions of the default theory, using an implicit representation of models:

$$(\{r_1, r_2\}, \{P \vee Q\})$$

where

$$r_1 = \frac{: \neg P}{\neg P}$$

$$r_2 = \frac{: \neg Q}{\neg Q}.$$

13. Using default logic, represent the following information:

- (a) Typically, secretaries are women;
- (b) Typically, women have children.

What can be concluded from the fact that Mary is a woman?

14. Consider the statements above, plus the information that “Typically, women over 50 have no more children”. What can you infer from the fact that Mary is a 55-year old secretary? Are all the inferences produced acceptable? How can the problem be corrected?
15. Describe two examples, from the real world, where default rules are used. represent these rules in default logic.

Chapter 5

Belief Revision

‘Always speak the truth — think before you speak — and write it down afterwards.’
Lewis Carroll, *Through the Looking Glass*

Let us recall the basic assumption about an AI program that we discussed in Chapter 1. Most AI programs maintain a model of their environment, which is updated to reflect the perceived changes in the environment. A representation of this model is stored in a knowledge base that contains propositions (or statements) about the environment (represented in some formal language, for example, first order logic), and the program draws inferences from the information in the knowledge base.

We also discussed in Section 2.5, that the knowledge base does not necessarily contain propositions that are universally true, but rather propositions about a certain application domain (proper axioms). For this reason, the set of propositions in the knowledge base under consideration, at a given instant of time is called the *beliefs* of the program, that is, the set of propositions that the program believes in.

Based on the definition of a rational conclusion presented in page 60, we can define a *belief* as a proposition that the program considers as if it were true. New beliefs may be added to the knowledge base either through observation (propositions whose origin is the outside world) or through inference using

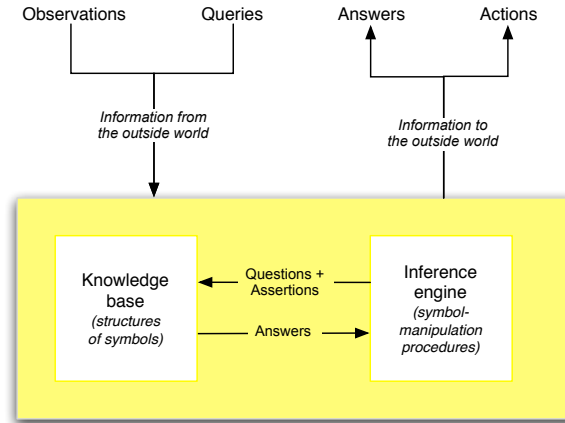


Figure 5.1: Detailed architecture of a knowledge-based system.

the beliefs in the knowledge base. We take the position that in a knowledge-based system, all propositions that are generated during inference are always added to the knowledge base.

In Figure 4.1, we present a more detailed view of the architecture of a knowledge-based system that was shown in Figure 1.3:

- The *knowledge base* is composed of symbol structures, using some formal representation language. These symbol structures are a representation of statements about the world.
- The *inference engine* is composed of procedures that manipulate the symbol structures in the knowledge base.
- New information about the outside world (labeled as “observations”) is added to the knowledge base.
- The interaction with the system is performed through queries that come from the outside world.
- An outside world query causes the inference engine to question the knowledge base, checking whether the desired information is explicitly available or if there are statements in the knowledge base that enable to answer the query through inference.

The answer to a query from the outside world may generate an inference process through which the inference engine asks additional questions to the knowledge base and adds new assertions to the knowledge base as the result of inference.

5.1 The need for belief revision

Programs that follow the model that we have described have to face the problem of changing their beliefs, this is called the *belief revision problem*. The revision may have to be done because new observations lead to conflicts with the beliefs in the knowledge base or because new beliefs produced during inference show that some of the beliefs in the knowledge base are not plausible or are mutually inconsistent.

The belief revision problem falls within the general issue of the study of systems with defeasible conclusions. The work in this area follows three different, but complementary, lines:

1. The study of logical systems in which some of the conclusions can be invalidated in face of future evidence, *non-monotonic logics* (see Chapter 3);
2. The study of the aspects which are necessary to incorporate in a computer program in order for it to change its beliefs — this aspect is addressed in this chapter;
3. The study of the dynamics of belief changes, that corresponds to the study of the formal relationships between the sets of propositions before the revision and the sets of propositions after the revision. This work falls under the general designation of *belief revision theories*.

When we addressed non-monotonic logics, we saw that additional information leads to the computation of the extensions of the new theory, that is, the computation of the consequences of the new set of premises — the new set of beliefs.

The use of this approach in a knowledge-based program would correspond to the removal from the knowledge base of *all* derived beliefs and the start of the inference from scratch. This approach is not desirable because there are certainly many conclusions from the original set of premises that are

still derivable from the second set. Also, since the inference process is computationally expensive, we do not want to throw away work that was done. Instead, we need to have a mechanism through which the *only* beliefs that are removed are those that are no longer derivable.

Belief revision systems are computer systems that allow for this behavior. Although there are different kinds of belief revision systems, they all share the following aspects:

- they keep the consistency of a set of beliefs,
- they only keep beliefs as long as there are reasons to believe in them,
- they explain the reasons for holding a belief,
- they identify contradictions, and
- they identify possible culprits for contradictions.

For a belief revision system, the word “belief” is taken to denote *justified belief*. A proposition is believed either because the system was told so — this corresponds to an observation (these beliefs are called premises) — or because it depends on believed propositions.

An essential aspect associated with belief revision systems corresponds to the record of the dependencies (or justifications) between beliefs. If it would be impossible to distinguish beliefs that depend upon a premise and beliefs that do not, then any belief may be questioned when the premise is removed; if it is not possible to identify the premises that underlie a contradiction, then every premise in the knowledge base may be questioned when a contradiction occurs.

Therefore, belief revision systems require the change of the basic architecture of a knowledge-based system to include a record of the justifications of the information in the knowledge base, as well as the addition of special procedures to compute dependencies among propositions and to handle the belief changes (Figure 4.2).

Depending on the process used to record dependencies among beliefs, two main types of belief revision systems may be considered:

1. In *justification-based systems* each belief is associated with the beliefs that were immediately used in its derivation;

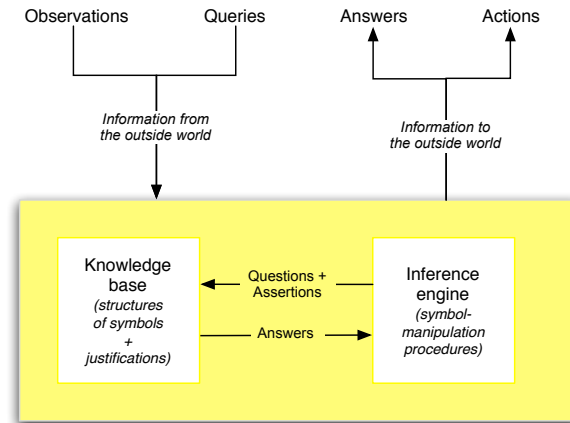


Figure 5.2: Revised architecture of a knowledge-based system.

2. In *assumption-based systems* each belief is associated with the premises that underlie its derivation.

5.2 Justification-based systems

In a justification-based system, each belief is associated with the beliefs that immediately originated it. In the general case, a belief may depend on the presence of other beliefs as well as on the absence of certain beliefs (see Chapter 3).

The general form of a justification associates each belief to two sets of beliefs: (1) the set that should be believed for the belief to be held, and (2) the set that should not be believed for the belief to be held. This is schematically represented as shown in Figure 4.3: beliefs are represented by nodes (circles or ovals) and dependencies among beliefs are represented by linking nodes through a “justification symbol”. The meaning of the justification in Figure 4.3 is that the belief in the statement corresponding to node c is justified by the belief in the statements represented by nodes $a_1, \dots, a_n$ and by the lack of belief in the statements represented by nodes $a_{n+1}, \dots, a_m$.

Let us consider the example presented at the outset of Chapter 3 regarding the availability of employed persons. Let us suppose that the system believes that “typically, an employed person is available to work”, “if a person is sick,

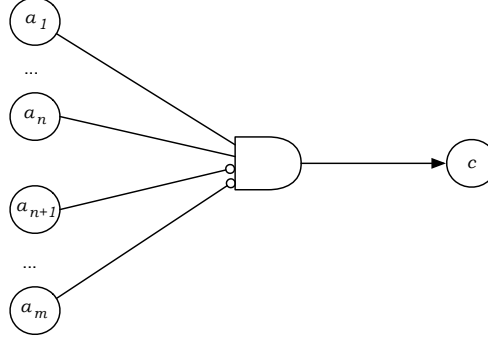


Figure 5.3: Graphical representation of dependencies among beliefs.

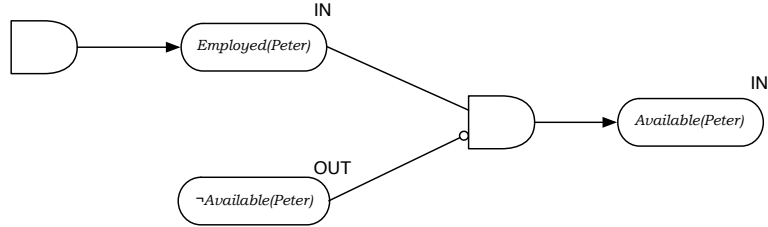


Figure 5.4: Justifications to believe that “Peter is available to work”.

then he/she is not available to work”, and “Peter is an employed person”. Using default logic, these statements would be represented as:

$$\frac{Employed(x) : Available(x)}{Available(x)},$$

$$\forall x [Sick(x) \rightarrow \neg Available(x)],$$

$$Employed(Peter).$$

With this information, the system could be led to believe that “Peter is available to work” and justify this belief with the belief that “Peter is an employed person” and the lack of belief that “Peter is not available to work”.

In Figure 4.4 we show the justification of the belief that “Peter is available to work”. In this figure, the word **IN** next to a proposition means that the proposition is believed, and the word **OUT** next to a proposition means that the proposition is not believed. Besides the justifications for derived

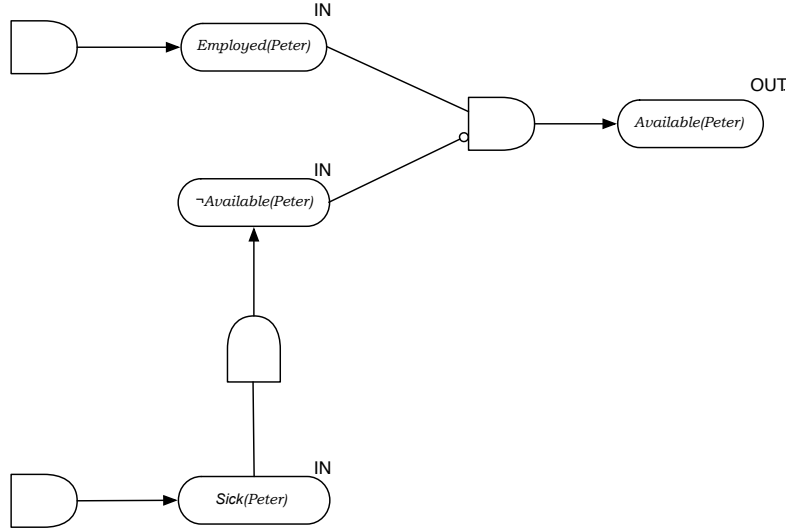


Figure 5.5: Justifications not to believe that “Peter is available to work”.

beliefs, as it is the case for the justification of the belief that “Peter is available to work”, there are justifications for propositions that are the result of observations (or that are told to the system as basic beliefs), as in the case of “Peter is an employed person”; these are called *premises*. Premises are always believed. We should also notice that in Figure 4.4 there is no justification to believe that “Peter is not available to work”, and, for this reason, it is not believed.

If we now add the premise that “Peter is sick”, which enables the derivation that “Peter is not available to work”, the belief that “Peter is available to work” is no longer held. The justifications for this new situation are shown in Figure 4.5.

We should note the lack of explicit representation (in Figures 4.4 and 4.5) of the information¹

$$\frac{Employed(x) : Available(x)}{Available(x)}$$

and

$$\forall x [Sick(x) \rightarrow \neg Available(x)].$$

¹ Although this information is somehow implicitly represented in the justification structure.

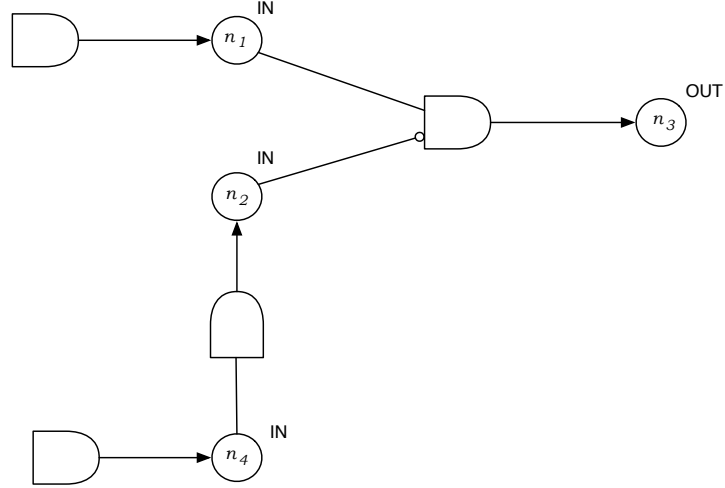


Figure 5.6: Dependencies supporting the disbelief in node n_3 .

In fact, a typical belief revision system only considers atomic propositions and their negation; all propositions using connectives or quantifiers are used by the system and cannot be disbelieved (exceptions to this approach are discussed in Sections 4.5.1 and 4.5.2).

5.2.1 JTMS

We all know the importance of abstraction in Computer Science. An *abstraction* corresponds to a separation of concerns, enabling us to focus on a particular aspect of a problem, while disregarding the others.

In belief revision systems, the main concern is on the computation of the dependencies among beliefs, in finding out reasons to believe or to disbelieve a certain proposition, and in the revision of the beliefs when a contradiction is detected. For this reason, it is usual to separate the dependency-recording mechanism and the computation of the belief-labels of the propositions from the inference mechanism that produces new propositions from other propositions.

The separation of the dependency-recording mechanisms from the inference engine lead to the study of belief revision systems in isolation. This separation of concerns, not only allows to concentrate on the issue at hand, but

also enables to construct belief revision systems that can then be coupled to knowledge based systems. This study lead to the introduction of the designation *Truth Maintenance System*,² TMS, for short. When a TMS is studied as a justification-based system, it is known as a *Justification-based Truth Maintenance System*, or JTMS, for short.

Since a TMS is only concerned with dependencies among beliefs, ignoring their contents, it is usual to replace those beliefs by identifiers (typically of the form $n_1, n_2, \dots$), which are associated with objects called *nodes*. There is a one-to-one function, T , that translates between beliefs in the knowledge base and nodes in the TMS. For example, the dependencies of Figure 4.5 can be represented as shown in Figure 4.6, using the translation:

$$T(\text{Employed}(\text{Peter})) = n_1,$$

$$T(\neg \text{Available}(\text{Peter})) = n_2,$$

$$T(\text{Available}(\text{Peter})) = n_3.$$

$$T(\text{Sick}(\text{Peter})) = n_4.$$

Using this perspective, the architecture of a system that uses a TMS is shown in Figure 4.7.³ There are two components in this architecture, the problem solver and the TMS. The designation *problem solver* is used to stress that this component may be more generic than just an inference engine associated with a knowledge base. For example it may correspond to a general search algorithm.

- The problem solver is responsible for the following tasks:
 - It retrieves from the TMS the information about which propositions are believed;
 - It produces new propositions using inference;
 - It supplies the TMS with the results of the inference, alongside with the reasons that justify the results of the inference.
 Observations are also supplied by the problem solver to the TMS with appropriate justifications;

²This designation was introduced by Jon Doyle in 1977 [Doyle 77], although Doyle, himself, later on recognized that these systems have nothing to do with truth and should be more adequately designated by *Reason Maintenance Systems*. The original designation, however, prevailed.

³This figure was adapted from [Reinfrank 89, p. 8].

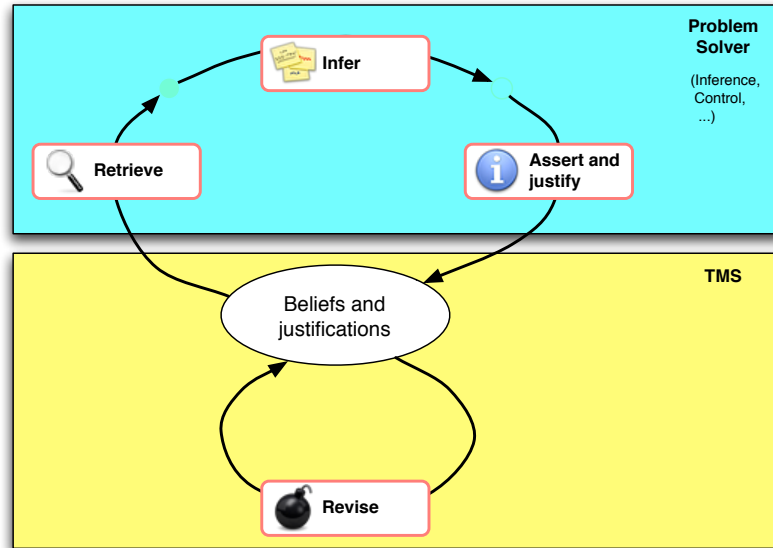


Figure 5.7: Architecture of a TMS-based system.

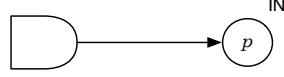
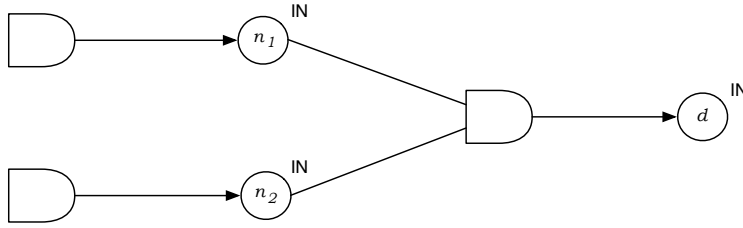
- The role of the TMS corresponds to record the dependencies, as supplied by the problem solver.

When the TMS detects that the beliefs are inconsistent, it is responsible for the revision of the beliefs, changing the beliefs in order to get rid of the inconsistency.

From this description, it should be clear that a TMS does not have deductive capabilities. It receives information from the program that works with the application domain, the *problem solver*, and supplies the problem solver with the information about which propositions should be believed.

In summary, a TMS works with nodes and justifications. Nodes stand for beliefs, and justifications support belief in nodes by relating them to other nodes. There are labels, associated with nodes, that indicate the belief status of the associated node. It should also be noticed that the existence of labels is due to computational efficiency, since their value can always be computed given the current nodes and justifications.

In a graphical form, nodes are represented by circles (or ovals) and justifications are represented by linking nodes through a “justification symbol”. The

Figure 5.8: Representation of the premise p .Figure 5.9: Representation of the derived node d .

general case of a justification is shown in Figure 4.3, meaning that the belief in node c is justified by the belief in nodes $a_1, \dots, a_n$ and by the lack of belief in nodes $a_{n+1}, \dots, a_m$. In this justification, node c is called the *consequent* and nodes $a_1, \dots, a_n, a_{n+1}, \dots, a_m$ are called the *antecedents*. Among the antecedents, nodes $a_1, \dots, a_n$ are called the *monotonic antecedents* and nodes $a_{n+1}, \dots, a_m$ are called the *non-monotonic antecedents*.

Justifications enable the classification of nodes into three categories:

- *Premises* correspond to nodes for which there is a justification with no antecedents. According to the meaning of a justification, a premise is always believed. Figure 4.8 shows an example of a premise, node p , with the corresponding IN label.
- *Derived nodes* correspond to believed node whose justification contains a non-empty set of antecedents. Figure 4.9 shows an example of a derived node, d .
- *Assumptions* correspond to believed nodes whose justification contains a non-empty set of non-monotonic antecedents. Notice that an assumption is a particular case of a derived node. Figure 4.10 shows an example of an assumption, node a .

Nodes may have more than one justification, each one representing a different reason to believe in the node. The decision to associate a justification

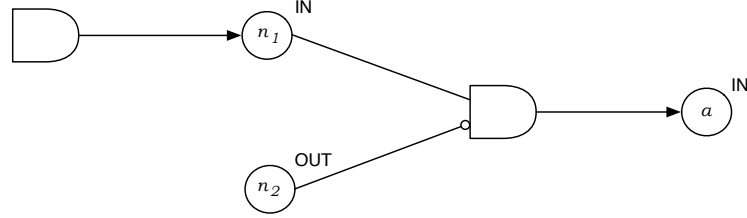
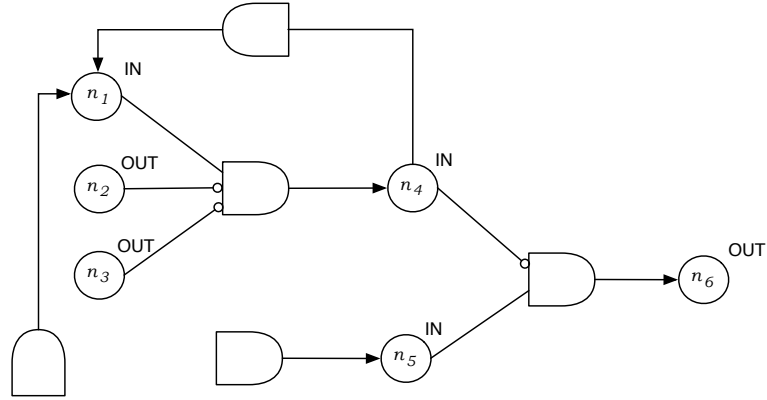
Figure 5.10: Representation of the assumption a .

Figure 5.11: Example of dependencies and labelling.

to a given node is taken outside the TMS by the problem solver. One of the tasks of the JTMS corresponds to associate nodes with labels, IN or OUT. Whenever a justification is added to a node, the JTMS computes the labels for all the nodes that are affected by the addition of the justification.

As an example, let us assume that the JTMS contains the dependencies shown in Figure 4.11, alongside with the corresponding labelling. Before we go on, there are two remarks that we would like to make regarding these dependencies:

- Node n_1 has two justifications. It is both justified as a premise and is justified as a derived node.
- There is a circular justification involving nodes n_1 and n_4 .

Let us now assume that node n_3 is justified as a premise. The addition of

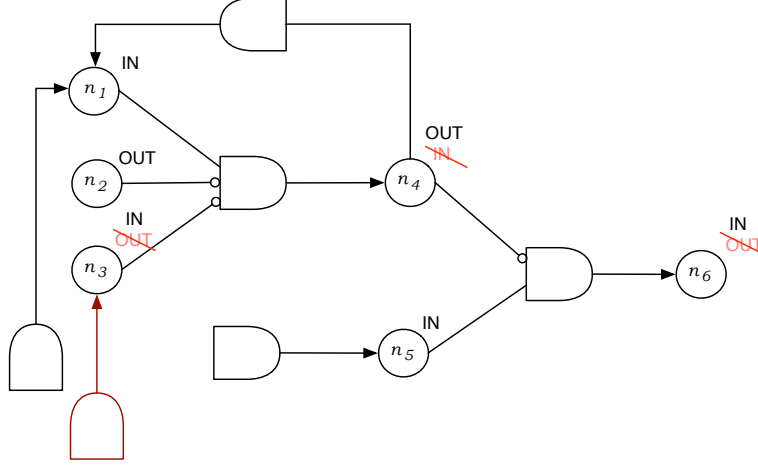


Figure 5.12: Dependencies and labelling after the addition of the premise n_3 .

this justification leads to a revision of the labelling of the nodes as shown in Figure 4.12:

- Node n_3 becomes believed (it is now a premise), which makes node n_4 disbelieved, which, in turn, makes node n_6 believed.
- The justification of node n_1 based on node n_4 does not change the belief status of node n_1 , since it still has a justification that supports the belief in this node.

In the process of computation of the labelling of nodes, there are two kinds of loops involving justifications that are of particular importance. First of all, let us define what a loop is. A *loop* is a sequence of nodes $(n_1, \dots, n_m)$ such that $n_1 = n_m$ and for each two nodes in the sequence n_i, n_{i+1} ($1 \leq i \leq m-1$) there is a justification that contains n_i in its antecedents and whose consequent is n_{i+1} .

- An *odd loop*, is a loop in which the condition “for each two nodes in the sequence n_i, n_{i+1} ($1 \leq i \leq m-1$) there is a justification that contains n_i in its *non-monotonic antecedents*⁴ and whose consequent is n_{i+1} is verified an *odd* number of times.

⁴Note that this is a particular case of the definition of loop above.

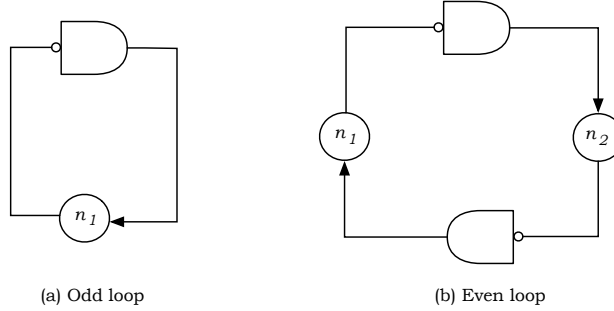


Figure 5.13: Odd and even loops.

Figure 4.13a shows the simplest odd loop. An odd loop, without additional information, corresponds to a belief that cannot be decided whether or not it holds (it cannot be assigned a labelling).

- An *even loop*, is a loop in which the condition “for each two nodes in the sequence n_i, n_{i+1} ($1 \leq i \leq m - 1$) there is a justification that contains n_i in its *non-monotonic antecedents* and whose consequent is n_{i+1} is verified an *even* number of times.

Figure 4.13b shows the simplest even loop. An even loop, without additional information, corresponds to two possible states of belief (it may have two alternative labellings).

Considering the justifications shown in Figure 4.13b, if node n_1 is not believed, then node n_2 is believed, and if node n_2 is not believed, then node n_1 is believed.

From what we said about even loops, it should be noticed that a labelling may not be unique as illustrated in Figure 4.14. In this figure we show, next to each node, its two possible labels. One labelling corresponds to all labels with white background, and the other labelling corresponds to all labels with shadowed background.

5.2.2 Interaction between the JTMS and the problem solver

The JTMS communicates with the problem solver, receiving information about new nodes and new justifications and supplying the problem solver with information about which nodes should be believed, taking into account

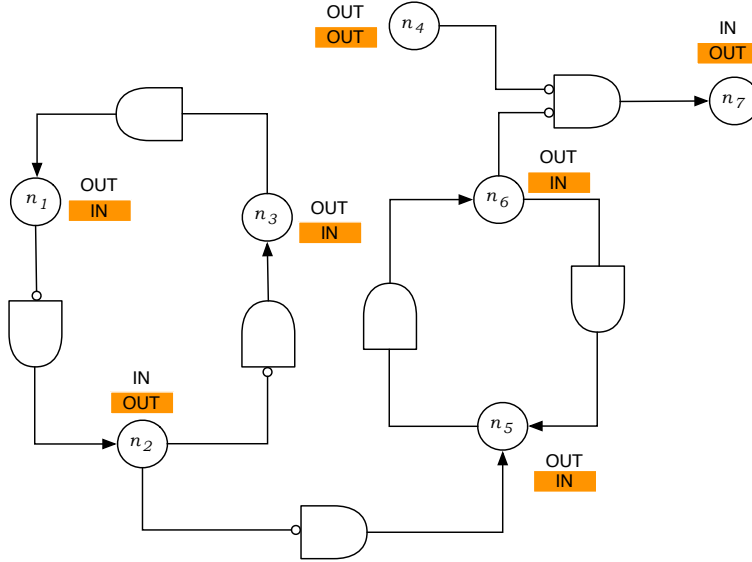


Figure 5.14: Two possible labellings for nodes.

the existing justifications (see Figure 4.7). The operations that can be performed by the JTMS are divided into three categories:

1. *Inspection operations.*

These operations correspond to an inspection of the network maintained by the TMS and are triggered by questions from the problem solver. These questions are of three types:

(a) Should node n be believed?

This question is answered by retrieving the label associated with node n .

(b) Why should node n be believed?

This question is answered by supplying the nodes that should be believed and the nodes that should not be believed so that node n is believed.

(c) What premisses underlie node n ?

This question is answered by supplying all the premisses that should be believed in order for node n to be believed.

2. *Modification operations.*

These operations modify the network maintained by the TMS. They are originated by information supplied by the problem solver. This information can be of five different types:

- (a) Add node n .
This adds a new node, with identifier n and without any justification.
- (b) Add premise p .
This adds the premise with identifier p to the network. If node p already exists it is justified as a premise.
- (c) Justify node c , based in the belief in nodes $a_1 \dots a_n$ and in the lack of belief in nodes $a_{n+1} \dots a_m$.
The effect of this operation is to add the proper justification to node c and the addition of any non-existing nodes mentioned in the antecedents of this justification.
- (d) Add contradiction p .
This adds the node with identifier p to the dependency network, marking it as a contradiction node. A *contradiction node* is a special kind of node whose belief must be avoided.
- (e) Remove premise p .
The effect of this is the removal of the justification of p as a premise.

3. *Updating operations.*

These operations are executed whenever a change is performed upon the dependency network. Their goal is to recompute the labels associated with the nodes in the network in such a way that they are consistent with the existing justifications.

5.2.3 Removal of contradictions

One of the goals of a TMS is to maintain consistent sets of beliefs. Whenever the TMS discovers that its beliefs are inconsistent, that is, whenever a contradiction node is believed, it should take action in order to restore consistency. This change can either correspond to the removal of a premise or to the addition of a justification to a node that is not believed.

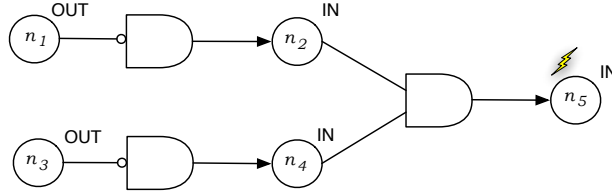


Figure 5.15: A contradiction is detected.

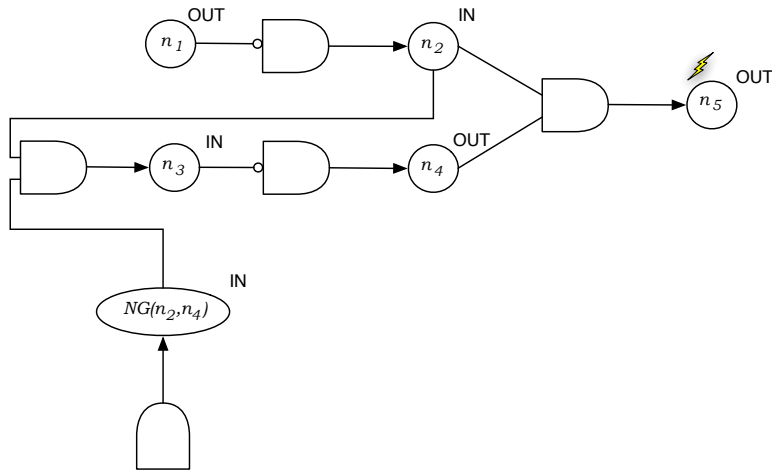


Figure 5.16: The contradiction is removed.

Suppose that we have the network shown in Figure 4.15. According to its only labelling the nodes believed are n_2 , n_4 , and n_5 , which included the (only) contradiction node of this network, n_5 . A contradiction node is graphically indicated with a spark above the node.

In order to obtain a consistent set of beliefs we should label node n_5 **OUT**. This can only be achieved if one of n_2 or n_4 is labeled **OUT**, which, again can only be done if we label one of the nodes n_1 or n_3 as **IN**.

Suppose that the TMS decides to label n_3 **IN**. We can only label this node **IN** if we can provide a justification for it. This is done by creating a new node, called a *nogood* (whose identifier is $NG(n_2, n_4)$), that corresponds to a premise and justifies the belief in n_3 based on the fact that n_2 is **IN** (Figure 4.16).

Often more than one iteration, and the addition of a nogood, are needed in order to reach a network with a consistent set of beliefs.

5.3 Dependency networks

One of the goals of the work in TMSes is the specification of the dependencies among beliefs and, based on such specification, the computation of which beliefs do hold and which beliefs do not hold. One way of doing this, is to use a formal system called dependency networks, described in this section.

Let $\mathcal{L}$ be the language used to define the identifiers for nodes, a *dependency network* is a triple $[N, C, J]$ where,

1. $N \subset \mathcal{L}$, represents the nodes in the dependency network,
2. $C \subset \mathcal{L}$, represents a distinguished set of nodes, the *contradiction nodes*, and
3. $J \subset 2^{\mathcal{L}} \times 2^{\mathcal{L}} \times \mathcal{L}$, is a set of justifications.

A justification is of the form $\langle A_M \mid A_{Nm} \mid c \rangle$ in which $A_M \subset \mathcal{L}$, $A_{Nm} \subset \mathcal{L}$, and $c \in \mathcal{L}$. A_M is called the set of *monotonic antecedents*, A_{Nm} is called the set of *non-monotonic antecedents*, and c is the *conclusion*. We define the functions:

$$\begin{aligned} ma(\langle A_M \mid A_{Nm} \mid c \rangle) &= A_M \\ nma(\langle A_M \mid A_{Nm} \mid c \rangle) &= A_{Nm} \\ conc(\langle A_M \mid A_{Nm} \mid c \rangle) &= c \end{aligned}$$

The justification $\langle A_M \mid A_{Nm} \mid c \rangle$ supports the belief in c if all nodes in A_M are believed and no node in A_{Nm} is believed. We say that $\langle A_M \mid A_{Nm} \mid c \rangle$ justifies node c .

Justifications divide nodes into two categories:

1. *Premises*, which correspond to nodes for which there is a justification with no antecedents.

Given the dependency network $[N, C, J]$ and a node $p \in N$, we say that p is a premise if $\langle \emptyset \mid \emptyset \mid p \rangle \in J$.

2. *Conclusions*, which correspond to nodes which have a justification with at least one non-empty antecedent.

Given the justification $\langle \{a_1, \dots, a_n\} \mid \{a_{n+1}, \dots, a_m\} \mid c \rangle$, node c is believed if nodes $a_1 \dots a_n$ are all believed and none of the nodes $a_{n+1} \dots a_m$ is believed.

5.3.1 Computation of labels for nodes

Given the dependency network $[N, C, J]$ we want to determine sets $B \subseteq N$ whose members it is reasonable to believe, given the justifications J . The computation of reasonable beliefs is somewhat similar to the computation of an extension of a non-monotonic logic and cannot be formalized in a constructive way. The requirements imposed upon a reasonable set of beliefs are:

1. It is *well founded*, i.e., the beliefs are either premises or are supported by a justification whose monotonic antecedents are all believed (using well founded arguments) and whose non-monotonic antecedents are all disbelieved;
2. It is *closed*, i.e., the consequence of a justification whose monotonic antecedents are all believed and whose non-monotonic antecedents are all disbelieved is believed;
3. It is *not contradictory*, that is $B \cap C = \emptyset$.

Algorithms for computing labels for nodes were developed by [Doyle 79] and [Goodwin 82] (who developed a labelling algorithm that detects odd loops and always terminates, which is not the case with Doyle's algorithm).

Given the dependency network, $[N, C, J]$, labelling algorithms correspond to the computation of the values of the total function $L : N \mapsto \{\text{IN}, \text{OUT}\}$.

The following actions are taken as a result of any *modification operations* (see Section 4.2.2).

1. Add node n . As a result, $[N, C, J]$ is transformed into $[N \cup \{n\}, C, J]$.
2. Add premise p . As a result, $[N, C, J]$ is transformed into $[N \cup \{p\}, C, J \cup \{\langle \emptyset \mid \emptyset \mid p \rangle\}]$.

3. Justify node c , based in the belief in nodes $a_1 \dots a_n$ and in the lack of belief in nodes $a_{n+1} \dots a_m$. The effect of this operation is to transform $[N, C, J]$ into $[N \cup \{a_1, \dots, a_n, a_{n+1}, \dots, a_m, c\}, C, J \cup \{<\{a_1, \dots, a_n\} \mid \{a_{n+1}, \dots, a_m\} \mid c>\}]$.
4. Add contradiction p . As a result, $[N, C, J]$ is transformed into $[N \cup \{p\}, C \cup \{p\}, J]$.
5. Remove premise p . As a result, $[N, C, J]$ is transformed into $[N, C, J - \{<\emptyset \mid \emptyset \mid p>\}]$.

5.4 Assumption-based systems

An assumption-based TMS (ATMS for short) associates nodes with the assumptions underlying their derivation. In these systems the labels of nodes are no longer IN or OUT but rather the sets of assumptions underlying their derivation.

In this section, we present a simplified version of an ATMS that does not consider justifications with non-monotonic antecedents. The ATMS considers the distinction between *premises*, which are nodes that hold universally, and *assumptions* made by the problem solver and which can be retracted. A premise corresponds to the same concept as presented in the TMS; an assumption corresponds to a node without a justification.

The big advantage of the ATMS over the JTMS stems from the additional flexibility it provides in dealing with multiple possible states of affairs. In fact, by labelling nodes with the sets of assumptions under which they hold we no longer have a single state of belief (as is the case in JTMS, whose state of belief contains all nodes whose label is IN) but rather a number of possible states that equals the cardinality of the power set of the assumptions introduced. This allows a comparison of the results obtained under different choices (i.e., different assumptions), the existence of alternative solutions to a problem, the detection of the possible culprits for a contradiction, and the recovery from contradictions.

The disadvantages of the ATMS over JTMS are the additional difficulty involved in representing justifications with non-monotonic antecedents (most versions of the ATMS do not allow for non-monotonic justifications) and the

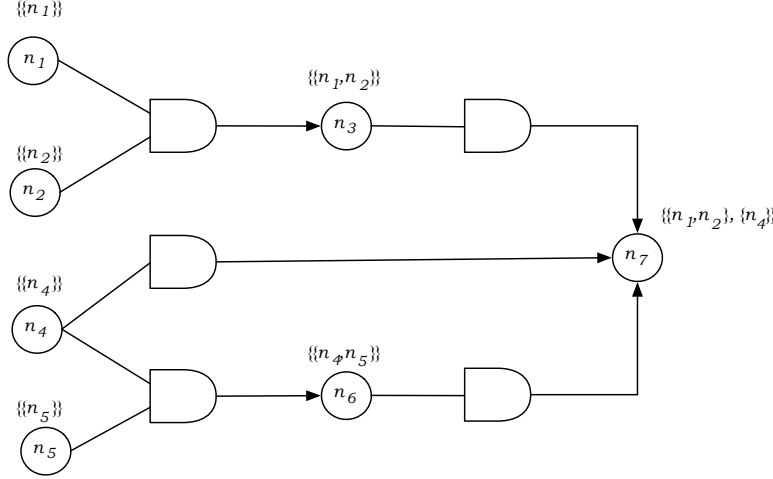


Figure 5.17: ATMS labelling.

control over the behavior of the problem solver.⁵

Since ATMS only considers justifications with empty non-monotonic antecedents, we abbreviate the justification $\langle A_M \mid \emptyset \mid c \rangle$ to $\langle A_M \parallel c \rangle$, and we call it a *monotonic justification*.

5.4.1 Computation of labels for nodes

The goal in computing ATMS labels is to find *minimal* sets of assumptions that underlie the derivation of each node. This computation is done by propagating and combining labels, starting with the labels of the premises and assumptions:

1. The premise $\langle \emptyset \parallel p \rangle$ is labeled with $\{\emptyset\}$.
2. The assumption $\langle a \parallel a \rangle$ is labeled with $\{\{a\}\}$.
3. The label of c , resulting from the justification $\langle \{a_1, \dots, a_n\} \parallel c \rangle$ is a set containing all the combinations of the labels of nodes $a_1, \dots, a_n$, keeping only minimal sets.

⁵See [de Kleer and Williams 86], [Dressler 90], and [Forbus and de Kleer 88].

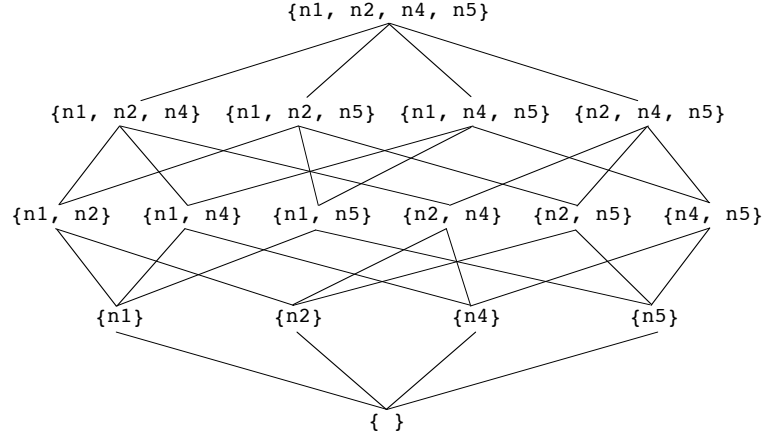


Figure 5.18: Environment lattice for assumptions n_1 , n_2 , n_4 , and n_5 .

In Figure 4.17 we show the ATMS labels for a network. Notice that the label of n_7 , computed directly from the labels of n_3 , n_4 , and n_6 , is $\{\{n_1, n_2\}, \{n_4\}, \{n_4, n_5\}\}$. Since $\{n_4, n_5\}$ contains $\{n_4\}$, and we want to keep minimal sets, the set of premises $\{n_4, n_5\}$ is discarded from the label.

The power set lattice of the assumptions provides a good way to visualize the space of possible combinations of assumptions. Figure 4.18 shows this lattice for the network of Figure 4.17. The edges between sets of assumptions represent subset/superset relations. For example, node n_6 of Figure 4.17 will be believed under all sets of assumptions which, in the lattice, are above $\{n_4, n_5\}$.

5.4.2 Interaction between the ATMS and the problem solver

The communication between the ATMS and the problem solver is basically similar to the one described for JTMS except that with the ATMS we must keep in mind that we no longer have a single state of belief. This interaction uses the following operations:

1. *Inspection operations*

- (a) Should node n be believed, given the assumptions $a_1, \dots, a_n$?
This question is answered by retrieving the label associated with

node n and checking whether in this label there is a set contained in $\{a_1, \dots, a_n\}$.

- (b) Why should node n be believed? This question is answered in the same way by JTMS and ATMS.
- (c) What assumptions underlie node n ? This question is answered by supplying the label associated with node n .

2. *Modification operations*

- (a) Add node n . Similar to JTMS.
- (b) Add premise p . Similar to JTMS.
- (c) Add contradiction p . Similar to JTMS.
- (d) Justify node c based on the belief in nodes $a_1 \dots a_n$ (note the lack of non-monotonic antecedents).

3. *Updating operations.*

These operations are executed whenever a modification operation is performed upon the dependency network. Their goal is to recompute the labels associated with the nodes in the network which are affected by the operation.

5.4.3 Removal of contradictions

In ATMS contradictions are removed by removing from the nodes' labels those sets of assumptions that are discovered to be inconsistent. Suppose, for example, that in the network presented in Figure 4.17, we discover that n_6 is a contradiction node. Since the label of node n_6 is $\{\{n_4, n_5\}\}$ this set of assumptions is discovered to be inconsistent. When this happens, all the sets of assumptions which are supersets of $\{n_4, n_5\}$ are marked as nogoods (shaded areas in the lattice of Figure 4.19) and these sets are removed from the labels of all nodes in the network. In particular, the label of node n_6 (the contradictory node) will be $\emptyset$.

5.5 Other approaches

In this section we list two alternative approaches to TMS-like systems. The commonality among them is that they both support reasoning by the TMS,

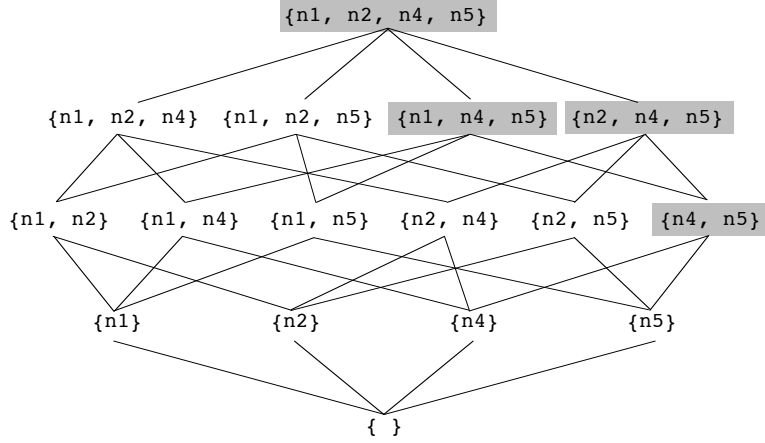


Figure 5.19: Inconsistent sets of premises.

rather than passing this task to the problem solver, as is the case with JTMS and ATMS.

5.5.1 Multiple Belief Reasoner

Multiple Belief Reasoner (MBR) is an ATMS-like system in which the problem solver and the TMS are merged into one single system. MBR is based on a logic called SWM* [Martins 91].⁶

SWM* deals with knowledge states. A *knowledge state*, written $[[KB, KIS]]$, is a pair containing a knowledge base (KB) and a set of known inconsistent sets (KIS). The knowledge base contains propositions (written as wffs) associated with an indication of the dependencies between a particular wff and other wffs in the knowledge base; the set of known inconsistent sets is a set containing the sets of premises in the KB known to be inconsistent.

The knowledge base is a set of supported wffs. A *supported wff* consists of a wff and an associated pair, its *support*, containing an origin tag and an origin set. The *origin tag* indicates how the supported wff was placed in the knowledge base (i.e., whether it was supplied by an outside system or was generated during deduction). The *origin set* is a set of premises; it contains all and only the premises that were actually used in the derivation of that

⁶A successor to SWM [Martins and Shapiro 83, 88].

wff.

The growth of knowledge states is dictated by the rules of inference of SWM* which are grouped into two sets:

1. *Pure logic rules*, which correspond to traditional inference rules. They have the effect of adding new supported wffs to the KB . This addition may be done in two different ways: a new supported wff is introduced from the outside (this corresponds to a premise — a premise in SWM* is called a *hypothesis*) or a supported wff is derived from other supported wffs in the KB . These rules transform $[[KB, KIS]]$ into $[[KB', KIS]]$ where $KB \subset KB'$.
2. *Computational rules* are rules that update the information about sets known to be inconsistent. These rules are obligatorily applied whenever a new inconsistent set is discovered, in which case a new set is added to KIS . These rules transform $[[KB, KIS]]$ into $[[KB, KIS']]$, where $KIS \subset KIS'$.

Based on SWM*'s features the behavior of an ATMS-like system called MBR (for Multiple Belief Reasoner) is defined. MBR uses two notions: context and belief space. A context is defined to be a set of premises. A context determines a belief space, which is the set of all premises defining the context and all the wffs in the KB that were derived exclusively from them. A belief space is represented by $\ll [[KB, KIS]], C \gg$, where C is a set of wffs that corresponds to premises in the KB ($(\forall H \in C)[\langle H, hyp, \{H\} \rangle \in KB]$). The belief space determined by a context is the subset of all the wffs in KB whose origin set is contained in the context. Any operation performed by MBR (query, addition, etc.) is associated with a context, the current context which defines the current belief space. A proposition is said to be believed if it belongs to the current belief space.

SWM* does not consider non-monotonic justifications, but there is an extension of it, SWMC, which does [Cravo and Martins 91]. The language of SWMC is an extension of the language of SWM*. It contains a new quantifier, the default quantifier which allows it to express default rules. It also has a distinguished predicate, *Applicable*, that, among other things, allows it to express exceptions to default rules.

MBR has been used in the SNePS system (discussed in Chapter 5).

5.5.2 Logic-based TMS

Logic-based TMS (LTMS) is based on the work of [McAllester 78]. In this kind of TMS, relations among propositions are represented by clauses. These clauses can be used to deduce the truth value of any of the involved propositions. The description presented here is a simplification of McAllester's work (a detailed introduction to LTMS, together with a discussion of its implementation, can be found in [Charniak, Riesbeck, McDermott, and Meehan 87]).

LTMS works with nodes, truth states, terms, clauses, and supports. *Nodes* represent propositions supplied by the problem solver. Any atomic proposition is represented by a node with no meaning to LTMS. A proposition involving logical connectives is also represented by a node, but in this case LTMS "understands" the meaning of the logical connectives that appear in the proposition; each node has an associated *truth state*. This truth state may be TRUE (believed to be true), FALSE (believed to be false), or UNKNOWN (no belief either way). It is said that a node has a truth value when its truth state is either TRUE or FALSE; A *term* is a pair containing one node and one truth value, TRUE or FALSE, a term is TRUE if the node has the truth value of the term; A *clause* is a set of terms one of which must be TRUE, for the clause to be TRUE. For example, the clause $((A. \text{FALSE})(B. \text{TRUE}))$ states that either A is FALSE or B is TRUE. A clause represents the relationship between the truth values of the nodes appearing in the clause.

A clause may be used by LTMS to deduce a truth value for any of the nodes it contains, and in this case the clause will act as a *justification* for the deduced truth value. A clause may be given multiple interpretations. For example, the clause $((A. \text{FALSE})(B. \text{TRUE}))$ may be interpreted either as $A \rightarrow B$ or, equivalently, $\neg B \rightarrow \neg A$. If this clause was used to deduce the truth value TRUE for node B (given the truth value TRUE of node A), it would act as a justification for the truth value of B. In a JTMS the corresponding justification would be $\langle \{A\} \mid \emptyset \mid B \rangle$.

There are two differences worth pointing out, between a JTMS justification and a LTMS clause: (1) An LTMS clause is a special kind of clause, a Horn clause. This means that only one term in the clause has TRUE as its second component. (2) A JTMS justification is not symmetrical, i.e. the label (IN/OUT) of its consequent may be concluded from the labels of its antecedents, but the label of the consequent does not allow JTMS to conclude anything about the label of any one of the antecedents, while in

LTMS justifications are symmetrical: the truth value of any node in a clause may be computed from the truth values of the other terms in the clause.

At any time the problem solver can add a clause and supply it to LTMS. Whenever this happens a check is immediately made to see if it is possible to derive new information. This will happen whenever all the terms of the clause are FALSE, except for one, which is UNKNOWN. In this case, this term must be TRUE (since the truth value of the clause is TRUE), and LTMS can conclude the truth value of this term's node. The other way a truth value can be added to a node is by deducing it from a clause. In this case the support of the truth value is the clause used to deduce it. Whenever a truth value is added, a check is made to see if this enables the deduction of some other truth value, from a clause. Obviously, the only clauses that are relevant are the ones that contain a term that has been given a truth value. The procedure used to compute truth values for clauses is called *boolean constraint propagation*.

The truth value of a node can be removed, meaning that the truth state of that node changes from TRUE or FALSE to UNKNOWN. The removal of a truth value corresponds to a removal of information. There are several ways in which the truth value of a node can be removed. One way for a truth value to be removed is when the problem solver decides to retract a premise or an assumption. This decision may be taken either because the problem solver decides that a premise is no longer known, or to resolve a contradiction (a contradiction is a clause whose terms are all false).

We said that LTMS “understands” the meaning of the logical connectives. To make this possible each of the logical symbols has an associated axiom set which is used to generate clausal constraints. For example, the axioms for implication are:

1. $((P \rightarrow Q \text{ . FALSE}) (P \text{ . FALSE}) (Q \text{ . TRUE}));$
2. $((P \rightarrow Q \text{ . TRUE}) (P \text{ . TRUE}));$
3. $((P \rightarrow Q \text{ . TRUE}) (Q \text{ . FALSE})).$

5.6 Applications

The capability of determining the source of contradictions coupled with the possibility of changing beliefs are essential features of any intelligent system.

In general, any system that follows the paradigm described at the outset of this chapter can use (and benefit from) TMS techniques. However, there are some problem areas in which these techniques are of paramount importance, in particular (other application areas can be found in [Martins 90]):

1. *Reasoning based on partial information, default assumptions and potentially inconsistent data.* This kind of reasoning is likely to produce contradictions and a system that resorts to it should be able to recover from them. This aspect is discussed in [Cravo and Martins 91], [Etherington 87], and [Morris 87a].
2. *Constraint Satisfaction.* This is a problem area that involves making choices and their retraction when a violation of constraints is detected. Again, the capability of discovering the source of the violation and its removal can be guided by TMS techniques. There is a good tradition in the application of TMSes to this area, see, for example, [Dhar and Ranganathan 90].
3. *Diagnosis.* The use of TMS techniques, in particular the recording of dependencies, is important to detect possible causes of malfunctioning. Among others, [de Kleer and Williams 87] and [Struss 88] discuss this aspect.
4. *Planning.* The applications of TMSes to planning pertain to the aspect of reasoning about change, i.e., reasoning about the propositions that change whenever an action is performed. Among others, the following papers discuss this aspect [Drummond 87], [Janlert 87], [Morris 87b], and [Pinto-Ferreira and Martins 90].
5. *Search.* The ATMS introduces a new approach to search in which several alternatives are considered and switching between nodes in the search tree is done by changing the premises under consideration. Among others, [Forbus and de Kleer 88] discuss this aspect.

5.7 Bibliographic notes and historical remarks

Several papers, although they do not specifically address TMSes, were influential in their development. Their study helps in understanding the general setting under which TMSes were introduced. TMSes have their roots in the frame problem [McCarthy and Hayes 69], the problem of deciding which

conditions change and which conditions do not change in a system when it undergoes some modification. Among the approaches to this problem [Fikes 75] proposes a system that uses dependencies among propositions and [Hayes 75] argues about the need for TMS-style systems.

In the mid 1970s [Stallman and Sussman 77] designed EL, a system in which dependencies of propositions are permanently recorded. The recording of dependencies has particular importance in finding faulty assumptions responsible for the generation of contradictions and for explanation of the program's beliefs. Their work had two major influences in AI: It opened a new perspective to the handling of alternatives (dependency-directed backtracking), and it influenced the creation of systems that, using dependencies, handle and can recover from contradictions (EL was a direct influence in the development of Doyle's TMS.). At the same time, and independently from Doyle's work, [London 78] describes a planning system relying on the use of dependencies. A reasoning system, called AMORD [de Kleer, Doyle, Steele, and Sussman 77], was an influential system on the development of the ATMS.

Justification-based systems historically corresponds to the first kind of belief revision system developed, as the result of the work of Jon Doyle [Doyle 79]. The terminology "assumption-based" was introduced by [de Kleer 84, 86] although similar ideas had been investigated earlier by [Martins and Shapiro 83]. Algorithms for computing ATMS labels can be found in [de Kleer 86] and [Reinfrank 89]. A description of the contradiction removal in ATMS can be found in [de Kleer 86]. A non-monotonic ATMS was developed by [Dressler 88].

During the last few years there has been some effort in providing foundations for TMSes and relating them with other non-monotonic formalisms: [Reinfrank 89] discussed the relationship between TMS and non-monotonic logics. [Fujiwara and Honiden 89] and [Reinfrank, Dressler and Brewka 89] have shown the equivalence between the reasonable beliefs of a dependency network and the strongly grounded extensions of autoepistemic logic. [Brown 88] provided a formal semantics to TMS by offering a logic that characterizes some models of TMSes. [Doyle 83] was the first attempt to present a formal description of a TMS. [Elkan 90] provided a characterization of the inferences performed by a non-monotonic TMS in terms of logic programming with stable set semantics and autoepistemic logic. [Popchev, Zlatareva, and Mircheva 90] provides a logical theory of truth maintenance reasoning subsuming plausible reasoning that distinguishes between true and

plausible arguments. They propose a behavior in presence of inconsistencies that corresponds to blocking the propagation of the contradiction rather than trying to get rid of it.

An annotated bibliography on Truth Maintenance Systems can be found in [Martins 90]. Introduction and overviews of Truth Maintenance Systems can be found in [Reinfrank 89] and [Smith and Kelleher 88].

Further details on the identification of possible culprits for a contradiction and the removal of the contradiction can be found in [Stallman and Sussman 77] who describe dependency-directed backtracking, and in [Doyle 79].

Dependency networks were first proposed by [Doyle 83] and are nicely presented in [Reinfrank 89].

The work in *belief revision theories* can be consulted in [Gärdenfors 88] and [Williams and Rott 01].

5.8 Exercises

1. Discuss the relationship between non-monotonic logics and belief revision systems.
2. Compare ATMS and JTMS regarding the following aspects: (a) Ease of dealing with multiple alternatives; (b) Ease of representing non-monotonic justifications.
3. Consider the following information regarding a dependency network:
 - A is a premise
 - B depends on A and on the absence of C
 - C depend on D
 - D depends on the absence of B
 - E depends on B
 - E depends on the absence of F
 - F depends on the absence of E
 - (a) Draw the dependency network corresponding to the information above, using a JTMS.
 - (b) Compute the labels of your dependency network.

- (c) What kinds of loops are present in the network and what are their properties?
- (d) Could the information above be represented using an ATMS? Why?
4. Consider the JTMS network shown in Figure 4.20. Compute all admissible labellings for this network.

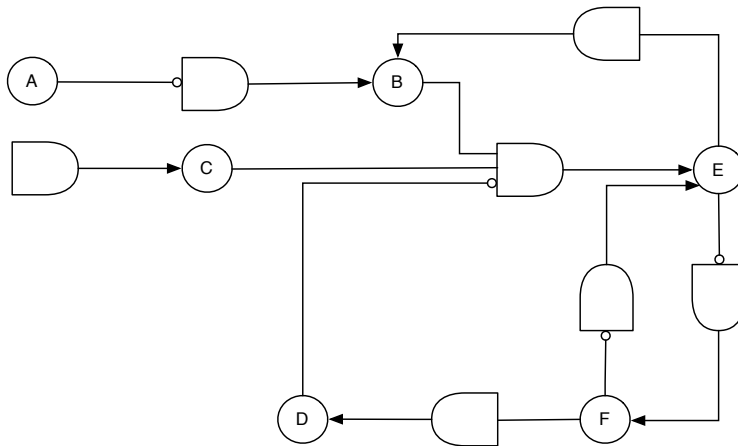


Figure 5.20: Sample dependency network.

5. Consider the dependency network $[\{n_1, n_2, n_3, n_4, n_5\}, \{n_5\}, \{<\emptyset \mid \{n_2\} \mid n_1>, <\emptyset \mid \{n_4\} \mid n_3>, <\{n_1, n_3\} \mid \emptyset \mid n_5>\}]$. Provide a revision of this network that removes the contradiction node.

Chapter 6

Semantic Networks

“When you follow two separate chains of thought, Watson, you will find some point of intersection which should approximate to the truth.”

Connan Doyle, *The Disappearance of Lady Frances Carfax*

Semantic networks correspond to an approach to knowledge representation that attempts to capture the idea that concepts used in the human mind are associated, or linked, with other concepts. The designation “*semantic network*”, as it is used now, is best thought of as the name for a family of representational schemes rather than a single representation formalism. Semantic networks were initially proposed by researchers associated with psychology and linguistics who attempted to characterize the nature of human understanding.

In AI, semantic networks originated from the study of the organization of human semantic memory, or memory for word concepts. Semantic networks were conceived as “*a representational format [that] can permit the meanings of words to be stored, so that humanlike use of these meanings is possible*”.¹

A semantic network can be thought of as a diagram that represents relationships between concepts in some domain. Although such relationships exist in the human mind, the diagram is just an attempt to represent the way we

¹From [Quillian 68, p. 227].

think about (and store) these concepts and their relationship.

Semantic networks are knowledge representation schemes involving *nodes* and *arcs* that link nodes.² Nodes either represent *objects* or *concepts* and arcs represent *relations between nodes*. The arcs are directed and labelled; thus, mathematically, a semantic network is a *labelled directed graph*. In print, nodes are usually represented by circles or boxes and arcs are drawn as labelled arrows between the nodes. In its simplest form, a semantic network is a collection of undifferentiated nodes and labelled arcs. The structure of the network defines the meaning of the concepts it represents. From this perspective, concepts have no meaning in isolation, they only exhibit meaning when viewed relative to the other concepts to which they are connected by arcs. Thus, in semantic networks, *structure is everything*.

Let us consider, as an informal example, the meaning of the word “aircraft”. We know that aircrafts are vehicles and have wings. A vehicle is something whose purpose is to provide transportation.

From this small example, we can see that, in general, concepts are defined in terms of other concepts. In our example, the concepts of “vehicle”, “transportation”, and “wing” have all been used to define the concept “aircraft”. If we do not understand some of these defining concepts, then we cannot fully understand what an aircraft is (except, perhaps, in some sort of intuitive sense). Thus, we understand concepts through their relationships to other concepts. When we learn about a concept, we put together in our minds the ways in which it is related to other concepts which we already know about. These relationships between concepts might be modified as we learn more about them.

Semantic networks address the representation of relationships between concepts through the use of diagrams of some kind — a diagrammatic model may encapsulate relationships between elements in a clear way, can be rearranged to provide a new or better structure and, once others have understood the conventions used in the diagram, it is efficient in communicating knowledge to others.

Figure 5.1 corresponds to our first attempt to represent the concept of an aircraft. We use a rather naive representation that suits well enough the notions that we want to convey in this first discussion. The node labelled **aircraft** is linked to the node labelled **vehicle** with an arc labelled **is-a**.³ This

²These arcs are sometimes also called *links* or *arrows*.

³The arc labelled **is-a** is very common in some kinds of semantic networks; sometimes

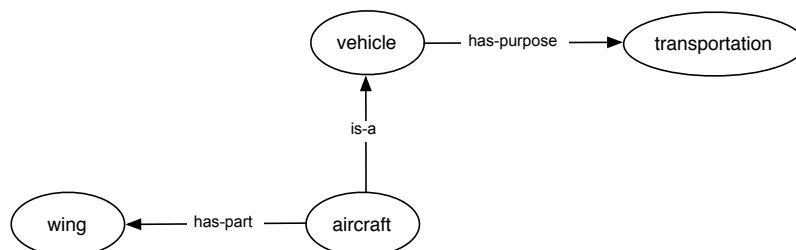


Figure 6.1: An aircraft is a vehicle with wings. Vehicles are means of transport.

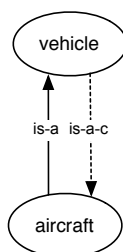


Figure 6.2: Converse arc between vehicle and aircraft.

arc introduces the relation that aircrafts are vehicles. Another node, with the label **wing**, pointed from the node labelled **aircraft** by the arc with label **has-part** represents that aircrafts have wings. The figure also shows that the purpose of vehicles is transportation.

A characteristic worth mentioning about the node-arc representation is the existence of the implicit “inverse” of all relationships represented by arcs. If there is an arc going from one node to another, this implies that there is also a converse arc, corresponding to the inverse relation, from the second node to the first. For example, in Figure 5.1, there will be an arc, whose label we may denote by **is-a-c**, that goes from **vehicle** to **aircraft** (see Figure 5.2). Converse arcs are not explicitly represented in our figures.

We can add more information to our diagram, namely that aircrafts may be commercial and military and that CS-TJF is a commercial aircraft. The result of this addition corresponds to three additional nodes, linked to other

the arc **a-k-o**, standing for “a kind of” is used instead.

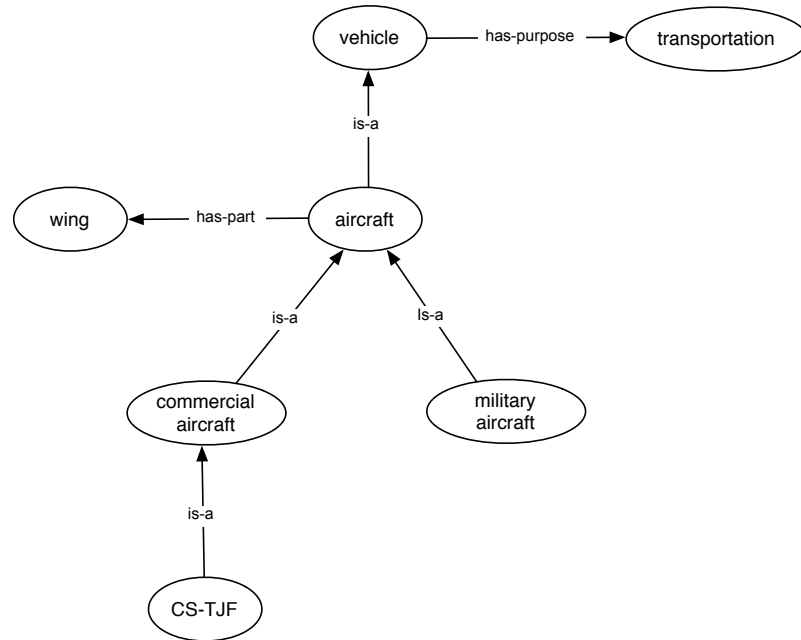


Figure 6.3: Commercial and military aircrafts are aircrafts. CS-TJF is a commercial aircraft.

nodes using the *is-a* link.

The resulting network is shown in Figure 5.3. In our intuitive sense, we should be able to understand that CS-TJF has wings and that its purpose is transportation. This conclusion can be reached by “composing” arcs by placing them end-to-tail. This composition creates a new, implicit, *is-a* link from the node CS-TJF to the node *aircraft*. By composing arcs, new relationships between nodes can be found and described. This process is sometimes called *path-based inference* and the terminology introduced stems from a branch of mathematics called *Category Theory*.⁴ We must make sure, however, that, when these representations are used by a computer, the same intuitive results can be produced by the computer itself.

It is now important to note a few points of potential semantic confusion in our small network:

⁴See, for example, [Poigné 92].

- The nodes in this network are of different kinds.

The node labelled **aircraft** represents the generic concept of an aircraft; it can be thought of as possessing properties common to all aircrafts. The node labelled **CS-TJF** represents an individual instance of the node **aircraft**; it represents a particular aircraft.

- The meaning of the arcs needs to be considered as well. It should be apparent that not all arcs are alike.

Some arcs express “type” information about the node at their end; the arc **is-a** asserts that a node is of a certain class, for example **CS-TJF** is a commercial aircraft and commercial aircrafts are aircrafts.

In our simple and naive representation, we have been using the arc labelled **is-a** to represent both set membership (**CS-TJF** is a commercial aircraft) and to denote subset containment (commercial aircrafts are aircrafts). This may be a serious source of confusion for the procedures that generate inference from the network structures (the inference engine).

Other arcs represent properties of the nodes they emanate from; the arc **composed-of** asserts that the node at its end has a given component or property.

The important aspect to retain from this example is that the natural simplicity with which we deal with diagrammatic representations involving nodes and labelled arcs may easily lead to an informal treatment of the representations, which, not surprisingly, potentially originates several confusions produced by the procedures that manipulated these representations. For these reasons, one of the first aspects that one should address when dealing with semantic networks is a clear definition of the “semantics” of the semantic network.

There is a large number of knowledge representation formalisms that can be classified as semantic networks. The commonality among them is the idea that knowledge is represented as a labelled directed graph composed of nodes and labelled arcs. Under this category, fall representations such as those shown in Figures 5.4,⁵ and 5.5.⁶ As illustrated by these figures, arcs do not need to have an explicit label, but they can be represented by different arc shapes that implicitly identify the corresponding label.

⁵Adapted from [Quillian 68, p. 236].

⁶Adapted from [Schank 75, p. 50].

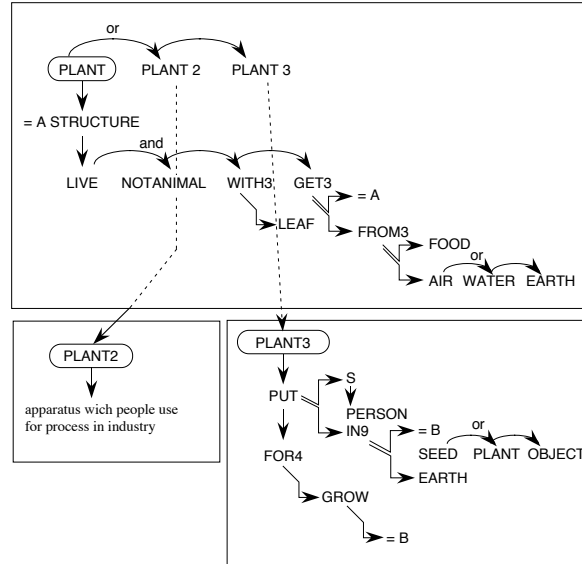


Figure 6.4: Meanings of the word “plant”.

6.1 Basic principles of semantic networks

When we start to deal with knowledge representation using semantic networks, we have to consider two distinct aspects:

1. The kinds of nodes and arcs that can be used and the rules for their combination — the *syntax of the network*.
2. The meaning of the different kinds of arcs and node structures — the *semantics of the network*.

Most semantic networks are based on the notion of attribute-value pair, the attribute is represented by the name of the arc that points to the corresponding value.

For example, let us consider the sentence “Mary is a 30-year old student with brown hair” that can be represented as shown in Figure 5.6. In this figure, the node labelled **Mary** denotes a particular person; the arcs that emanate from this node denote attributes of that person.

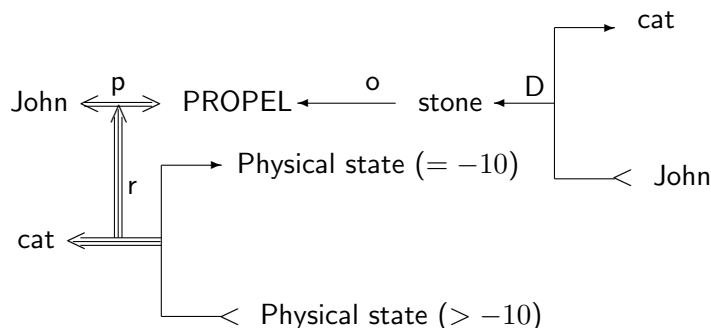


Figure 6.5: John threw a stone to the cat and killed him.

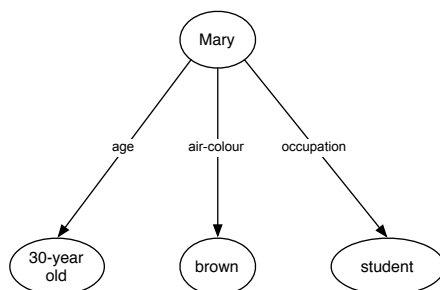


Figure 6.6: Mary is a 30-year old student with brown hair.

If we now want to represent that “Mary is more than 25-years old”, we could be tempted to use the representation shown in Figure 5.7. However, when we do this, the meaning of our representation changes! The “thing” at the end of an arc is no longer a value, but becomes a proposition that is true for the value. With this new representation, there are now two interpretations for the node at the end of an arc, either a *value* or a *predicate* associated with the value.

With this representation, it is not possible to use only two nodes to represent that “John is older than Mary” since in the predicate associated with John’s age we would have to represent Mary’s age. In order to solve this problem, we can use *intensional entities* to represent, respectively, “John’s age” and “Mary’s age” (nodes A2 and A1, respectively, in Figure 5.8).⁷ In this case,

⁷ *Intensional entities* can be roughly defined as entities that correspond to a given meaning, rather than representing the *actual* world entity that corresponds to that meaning — the *extensional* entity. Intensional entities are the objects of thought, whereas the exten-

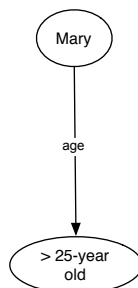


Figure 6.7: Mary is more than 25-years old.

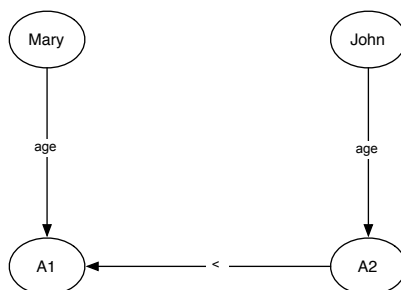


Figure 6.8: John is older than Mary.

we don't represent the extension of these intensional objects, that is, their value. The proposition that "John is older than Mary" would be represented as shown in Figure 5.8.

Under this perspective, we can revise the representation shown in Figure 5.7, and represent the proposition that "Mary is more than 25-years old" by the network shown in Figure 5.9. In this way, we have a unique meaning for the node that an arc points to, the value of the property associated with the arc.

Semantic networks are used in a wide variety of domain areas, among which natural language understanding, where arcs are used to represent verb usage.

sional entities correspond to the objects in the real-world. In our example, we introduce a node that corresponds to John's age, without saying what that age really is. For a more detailed description on the discussion between intensional and extensional entities you should refer to [Shapiro and Rapaport 87] or to [Dennett 81].

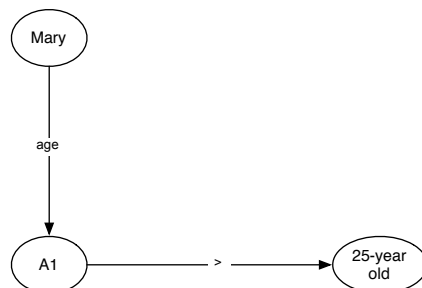


Figure 6.9: Mary is more than 25-years old (revised representation).

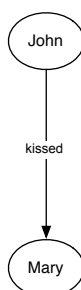


Figure 6.10: John kissed Mary.

For example, according to what we have been discussing, the sentence “John kissed Mary” could be represented as shown in Figure 5.10. However, if we use this representation, the semantics of our notation changes again. An arc not only represents the attributes of a node, but also relationships between a node and other nodes.

If we want to use both kinds of representations, to state, for example, that “John is more than 30-years old and kissed Mary” (Figure 5.11) we had to tell the system that manipulates the network (the inference engine) that the arcs used in the representation are of different nature, and thus have to be treated differently by the procedures that are responsible for the inference.

In order to avoid this situation, we have to find another meaning for arcs. The most commonly used alternative is based on the work of linguists⁸ on cases associated with verbs. A *case* is a particular role that a noun phrase

⁸In particular, on the work of [Fillmore 68].

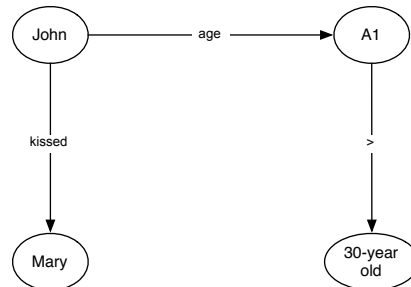


Figure 6.11: John is older than 30 and kissed Mary.

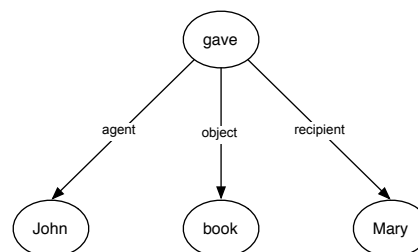


Figure 6.12: John gave Mary a book.

or other participant takes in the action or activity expressed by the verb of a sentence.

For example, the sentence “John gave Mary a book” has a giving action, where John is the agent, Mary is the recipient, and the book is the object. This can be represented by the network of Figure 5.12. Because, in any act of giving, the underlying case relationships remain the same — there is always someone who gives, something given, and someone to whom it is given — we might see the representation of Figure 5.12 as the filling in, for the sentence “*John gave Mary a book*”, of a more abstract schema that also provides an interpretation for numerous other sentences. The set of arcs that is used to represent a given statement is called a *case frame*. We might then say that the meaning of each of these sentences is “give”, together with specific values filling the nodes at the end of the case links.

We should notice, however, that the node labelled **gave** in Figure 5.12 does not represent a general characteristic of the verb “to give”, but rather a particular instance of the giving action. It is more correct to represent this

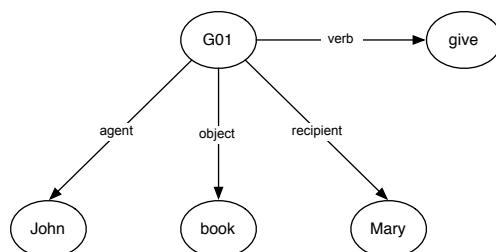


Figure 6.13: Alternative representation for “John gave Mary a book”.

instance by a new node (that identifies the instance) and to make explicit that it corresponds to a giving action. In Figure 5.13, G01 represents an instance of giving and give represents the verb, in general.

The use of case frames in a semantic network gives rise to a new interpretation for the nodes and arcs in the network:

1. There are nodes that represent *objects*, as before. These nodes, in our example, are John, Mary, and book;
2. There are nodes that represent *propositions*, in our example, G01.

The arcs that emanate from this new kind of nodes, are different, in nature, from the other arcs:

1. The arcs we had before (for example, in figures 5.6 and 5.7) represent statements. Their presence in the networks corresponds to a proposition about the two nodes that they connect. These are called *assertional* arcs, since they make assertions about the nodes that they relate;
2. The arcs that emanate from nodes denoting propositions represent the “parts” of a proposition. For example, the meaning of G01, in Figure 5.13, is given by *all* arcs that emanate from it, not only by a subset of these arcs. These new arcs are called *structural*. We can argue that these arcs represent the statement that the agent of G01 is John, the recipient of G01 is Mary, and so on. In this representation G01 has no meaning other than what is ascribed by the structural links to other nodes.

Another important aspect is that we must be careful in making interpretations on the basis of a clear understanding of the semantics of the notation and not on the basis of the informal semantics implied by the names of the nodes and arcs.

Although we have been looking at semantic networks as a pictorial representation, there is a formal textual representation of semantic networks. Under this perspective, semantic networks can be seen as a formal language for representing knowledge, including well-defined rules of inference to produce new knowledge from existing knowledge.

6.2 SNePS: A case study

In this section, we consider in detail, a propositional semantic network, called SNePS (for Semantic Network Processing System), developed by Stuart C. Shapiro and his research group. The goal of building SNePS was to produce a network that would be able to represent virtually everything that is used in natural language. For this reason, SNePS offers high expressive power.

SNePS uses structural arcs, as described at the end of the previous section. SNePS is called a *propositional* semantic network because each proposition is represented by a node and not by an arc. Thus, assertional arcs do not exist in SNePS.

In SNePS, nodes represent concepts, and arcs represent *non-conceptual* relations between concepts.⁹ Arcs are only used to structure concepts. The fact that concepts are represented by nodes means that, in SNePS, we can only talk about the information represented by nodes and we cannot refer to the information represented by arcs. Arcs are considered to be part of the syntactic structure of the node from which they emanate.

SNePS imposes certain restrictions on the representation it uses. It is important to understand these restrictions, because they distinguish SNePS from a general labelled, directed graph, and from many other semantic network formalisms.

⁹This means that the “arc” information does not correspond to a concept in the network.

6.2.1 Arcs in SNePS

In SNePS, there are two kinds of arcs, direct and converse arcs. For each relation represented by a direct arc, there is an inverse relation represented by a converse arc. Converse arcs have the same name as the corresponding direct arcs, but are followed by a “-”. Converse arcs will not be shown in our diagrams. So, direct arcs are the arcs that appear in the graphical descriptions of the network.

The labels of the arcs used in SNePS are defined by the person who is using the network to represent knowledge, much in the same way as the person that writes a logic-based representation “invents” the names of the predicates. However, in order to enable the inference engine to interpret the propositions involving SNePS’ logical connectives, there are some pre-defined arcs in SNePS (as described in Section 5.2.4).

6.2.2 Nodes in SNePS

In SNePS, nodes represent concepts. The notion of a concept is vague; it corresponds to anything that we can talk about. Among other things, a concept can be a proposition. The benefit of representing propositions by nodes is that propositions about propositions can be represented with no limits.¹⁰

There are two aspects regarding the concepts represented in SNePS:

1. Nodes represent intensional concepts;
2. Each concept is represented by a unique node and each node represents a unique concept. This statement means, among other things, that it is not possible to have in SNePS two different nodes corresponding to the same intensional concept. This is known as the *uniqueness principle*. This principle guarantees that nodes will be shared whenever possible.

If there is a direct arc from node m to node n , it is said that m *immediately dominates* n . If there is a path of direct arcs from node m to node n , it is said that m *dominates* n . There are no cycles composed only of direct arcs,

¹⁰This is one difference from the first-order logic representation, where no propositions about propositions can be represented.

so no node dominates itself. If there is a direct arc, r , from node m to node n , it is said that n “is the value of” $r(m)$.

According to their position in the network, nodes can be atomic or structured.

1. A node is said to be *atomic* (also said, *unstructured*) if it dominates no other node. Atomic nodes can be further divided into constant and variable nodes:

- (a) *Atomic constant nodes*, also called *base nodes*, represent non-structured concepts, and correspond, in a sense, to the constants of first-order logic.

Atomic constant nodes are represented, in our notation, by a rectangle, inside of which there is the identifier of the node.¹¹ These nodes are labelled by a string that stands for the name of an individual concept, for the name of a relation, for the name of a property, and so on.

The label of an atomic constant node may also be an identifier of the form Bn , where n is a positive integer.¹²

- (b) *Atomic non-constant nodes*, also called *variable nodes*, represent variables, and correspond, in some sense, to the variables in first-order logic.¹³

Variable nodes are represented by a circle, inside of which there is the name of the node. The label of variable nodes is automatically generated by SNePS and corresponds to an identifier of the form Vn , where n is a positive integer.

2. A node is said to be *structured* (also said, *molecular*) if it dominates other node(s). Structured nodes are represented by a circle, inside of which there is the label of the node. Structured nodes can be further divided into constant and variable nodes:

- (a) *Constant structured nodes* represent either propositions or structured concepts. These nodes correspond either to *wffs* in logic without free variables or to complex terms.

¹¹This is not a standard notation within SNePS.

¹²These nodes are automatically generated by SNePS’ inference system, when introducing constants — see Section 5.2.8.

¹³In SNePS, these nodes may also represent arbitrary propositions.

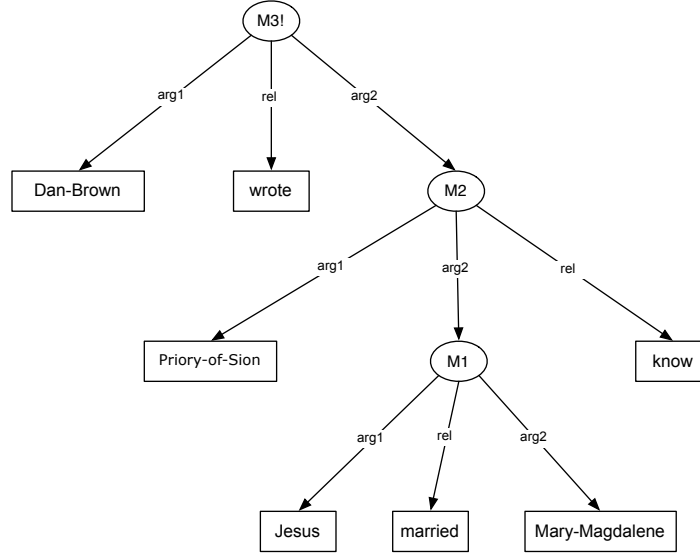


Figure 6.14: Dan Brown wrote that the Priory of Sion knows that Jesus married Mary Magdalene.

Constant structured nodes are automatically labelled in SNePS by an identifier of the form Mn , where n is a positive integer. This identifier may be followed by the character “!” — in this case, we say the node is *asserted*.

- (b) *Non-constant structured nodes*, also called *pattern nodes*, dominate at least a node corresponding to a free variable. These nodes correspond to *wffs* in logic with free variables.

Non-constant structured nodes are labelled in SNePS by an identifier of the form Pn , where n is a positive integer.

As a first example, let us assume that **arg1**, **rel**, and **arg2** are labels of direct arcs. The intended meaning of a case frame with these arcs is that the relation pointed by the arc **rel** is verified by the nodes pointed out by arcs **arg1** (the first argument of the relation) and **arg2** (the second argument of the relation). Let us assume, furthermore, that **Dan-Brown**, **wrote**, **Priory-of-Sion**, **knows**, **Jesus**, **married**, and **Mary-Magdalene** are atomic constant nodes.

In the representation shown in Figure 5.14, **M3!**, **M2**, and **M1** are constant structured nodes; node **M1** immediately dominates the nodes labelled **Jesus**,

married, and Mary-Magdalene; node M3! dominates, among others, M2, Priory-of-Sion, married, and Mary-Magdalene. Node M3! is the only asserted node in this small network (it is the only node whose identifier ends with the character “!”).

Without going into the details of the representation:

- node M1 corresponds to the proposition that “Jesus married Mary Magdalene”;
- node M2 corresponds to the proposition that “the Priory of Sion knows that Jesus married Mary Magdalene”;
- node M3! corresponds to the proposition that “Dan Brown wrote that the Priory of Sion knows that Jesus married Mary Magdalene”.

From the description of this intended meaning, we can see that in SNePS¹⁴ we have a higher expressive power than in first-order logic, since we can write propositions *about* propositions.

For the purpose of reasoning, propositions that are asserted in the network must be distinguished from other propositions that are represented in SNePS but not asserted. Asserted propositions correspond to nodes whose name is of the form $Mn!$. The exclamation mark indicates nodes in which SNePS “believes”; in this case, we say that the node is *asserted* in the network.¹⁵ In the example shown in Figure 5.14, only node M3 is asserted. We will refer to this (and other asserted nodes) either by M3 or by M3!, without distinction.

Related to the representation in Figure 5.14:

- SNePS does not believe that “Jesus married Mary Magdalene”. Although SNePS does not believe otherwise, either — this means that the closed world assumption (see Chapter 3) is not enforced in SNePS;
- SNePS does not believe that “the Priory of Sion knows that Jesus married Mary Magdalene”;

¹⁴And in most semantic networks.

¹⁵In fact, the distinction between asserted and non-asserted nodes is more complex because it resorts to ATMS-based context-shadowing operations (see Chapter 4). However, for the purpose of this chapter, we will stick to the definition above.

- but SNePS believes that “Dan Brown wrote that the Priory of Sion knows that Jesus married Mary Magdalene” — the corresponding node is asserted in the network.

In summary, we have the following types of nodes in SNePS:

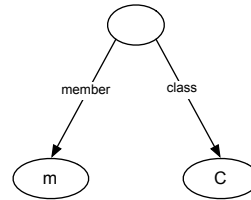
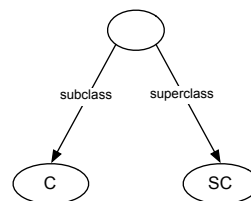
node	<i>atomic</i>	{	<i>constant</i>	constant individual relations, concepts, and properties (labelled by a string or by an identifier of the form Bn)
			<i>variable</i>	arbitrary individuals or arbitrary propositions (labelled by an identifier of the form Vn)
	<i>structured</i>	{	<i>constant</i>	structured concepts, structured individuals, structured properties, or deduction rules (labelled by an identifier of the form Mn)
			<i>variable</i>	parts of propositions (labelled by an identifier of the form Pn)

6.2.3 Knowledge representation in SNePS

SNePS has no pre-defined arcs, except for those related to the logical connectives (part of these are described in Section 5.2.4). Therefore, the first step towards representing knowledge in SNePS corresponds to the choice and definition of the arcs that will be used.

Since arcs in SNePS are *structural*, to define a piece of knowledge in SNePS we have to use a “case frame” of arcs that emanate from a node and correspond to the relation or proposition that the node represents. Although there are no fixed guidelines for the definition of these case frames (they depend on the intended application), there are some commonly used representations, some of which are described in this section.

Three basic statements that appear in most applications are the statements

Figure 6.15: *m* is a member of class *c*.Figure 6.16: *C* is a subclass of class *SC*.

that an individual is a member of a class, that a class is a subclass of another class, and that an object has a given property.

- The first statement is usually represented by the case frame **member / class** (Figure 5.15), where the arc labelled **member** points to the node that corresponds to the concept about which we are asserting membership and the arc labelled **class** points to the node that corresponds to the class.
- The second statement is usually represented by the case frame **subclass / superclass** (Figure 5.16), where the arc labelled **subclass** points to the node that corresponds to the subclass and the arc labelled **superclass** points to the node that corresponds to the superclass.
- The third statement is usually represented by the case frame **object / property** (Figure 5.17), where the arc labelled **object** points to the node that corresponds to the object and the arc labelled **property** points to the node that corresponds to the property.

Using the first two case frames, we show in Figure 5.18, the representation that CS-TJF is a commercial aircraft (node M1!) and that the class of

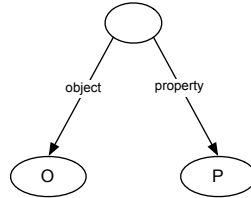


Figure 6.17: Object O has the property P.

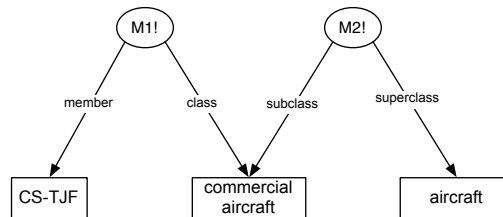


Figure 6.18: CS-TJF is a commercial aircraft and commercial aircrafts are aircrafts.

commercial aircraft is a subclass of aircraft (node M2!). It is also important to notice that, from this information alone, SNEPS is not able to know that CS-TJF is an aircraft, because the “operational semantics” associated with these case frames is not part of the network yet. The “operational semantics” may be specified through the use of the connectives described in Section 5.2.4 or the path-based operations described in Section 5.2.7.

It is also common to need to explicitly represent arbitrary relations in SNEPS. This can be done using the case frame `rel / arg1 / ... / argn`, where the arc labelled `rel` points to the node that corresponds to the relation and the arc labelled `argi` ($1 \leq i \leq n$) points to the node that corresponds to the i -th argument of the relation (Figure 5.19).

This case frame can be used to represent any relation, including the membership, subset, and property relations that we discussed before. In Figure 5.20 we show an alternative representation for “CS-TJF is a commercial aircraft”, using the `rel / arg1 / ... / argn` case frame. The representational difference between the use of the case frame of Figure 5.20 and the case frame presented in Figure 5.18 is the fact that by using the case frame of Figure 5.20, the membership relation itself becomes a concept in the network, and thus some-

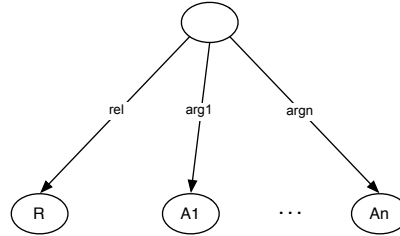
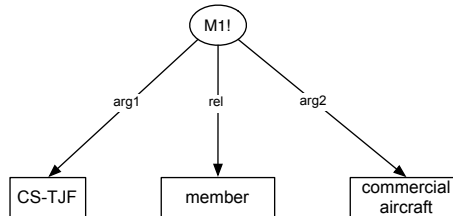
Figure 6.19: Case frame for a n -ary relation.

Figure 6.20: Alternative representation for “CS-TJF is a commercial aircraft”.

thing we can talk about, whereas with the representation of Figure 5.18, the membership relation is implicit in the structure of the arcs and thus cannot be referred to by SNePS.

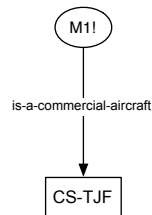


Figure 6.21: Bad representation for “CS-TJF is a commercial aircraft”.

We could also be tempted to use the representation shown in Figure 5.21 to represent that “CS-TJF is a commercial aircraft”.¹⁶ However, this representation is a step backwards when compared with the representation of Figure 5.18, since the membership in the class of commercial aircraft is no

¹⁶I am grateful to António Leitão for suggesting this representation.

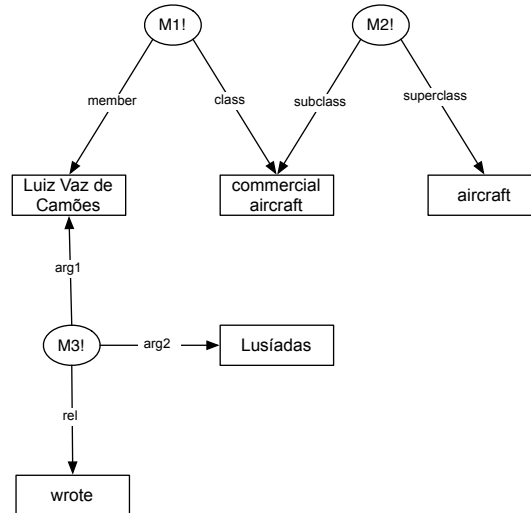


Figure 6.22: “Luiz Vaz de Camões is a commercial aircraft” and “Luiz Vaz de Camões wrote Lusíadas”.

longer a concept, but rather a relation between two nodes. With the representation of Figure 5.21, we cannot talk about the class of commercial aircraft nor about the relationship that CS-TJF bears with it.

From this discussion, it may seem that the explicit representation of relations, since it corresponds to a more fine-grained representation, should be preferred over the representation of relations through case frames that do not use the explicit concept of relation. You should be aware, however, such a generalized use of the case frame `rel / arg1 / ... / argn` for the representation of relations has a negative effect on the efficiency of the inference process, because the pattern matcher will be overloaded with information (basically, all the nodes that represent relations can match with a generic relation).

With the knowledge that we already have about the basic representation in SNePS, we now represent some additional information. CS-TJF is the licence plate of a particular aircraft that belongs to the fleet of TAP Portugal. This aircraft is named “Luiz Vaz de Camões”, after the most famous Portuguese poet, whose masterwork is a book called “Lusiádas”. If, instead of the licence plate of this aircraft, we use its actual name, we may produce the network shown in Figure 5.22 that also states that “Luiz Vaz de Camões

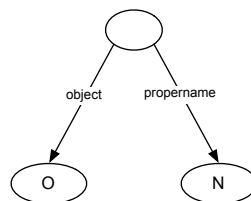


Figure 6.23: Object O has name N.

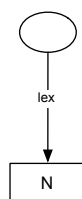


Figure 6.24: N is the external representation of a node.

wrote Lusíadas”.

Due to the uniqueness principle, the node labelled Luiz Vaz de Camões is shared between nodes M1 and M3. This representation is clearly wrong, since we will be able to infer that “the person who wrote Lusíadas is a commercial aircraft”. The problem here is that we are mixing the name that is used to refer to an entity with the entity itself — they are both represented by the same node.

In SNePS it is common to use one, among two alternative case frames, to deal with this problem:

1. The case frame **object / propername** (Figure 5.23), where the arc labelled **object** points to the node that represents the entity that we are naming and the arc labelled **propername** points to the node that corresponds to the name of that entity, asserts that the object has a given proper name;
2. The case frame just with the arc **lex** (Figure 5.24), from the node that represents the entity that we are naming to the node that corresponds to the name of that entity.

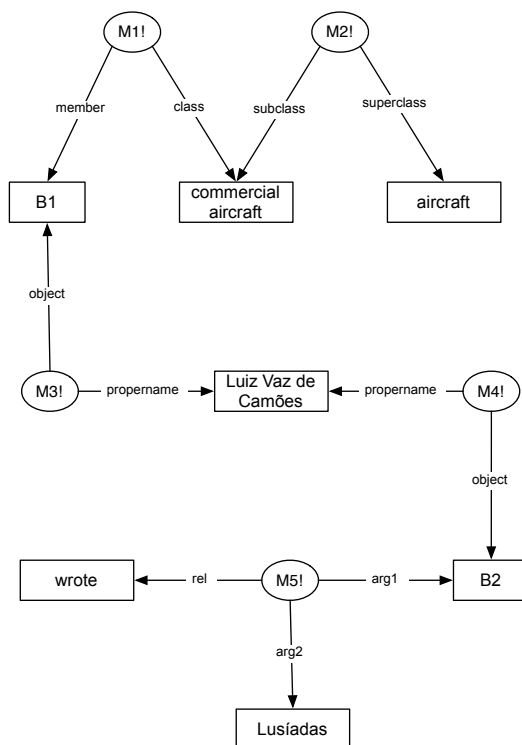


Figure 6.25: Proper representation of “Luiz Vaz de Camões is a commercial aircraft” and “Luiz Vaz de Camões wrote Lusiadas”.

Although these case frames may seem equivalent, the second is mostly used in applications of natural language, where we want to represent the lexemes that correspond to words represented in the network. In this book we use the representation in Figure 5.23.

With this new case frame, we are able to correct the problem existing in Figure 5.22 and rewrite the network as shown in Figure 5.25. In this figure, base node B1 represents the commercial aircraft that is named “Luiz Vaz de Camões” and base node B2 represents the person that wrote Lusiadas.

We said at the outset that SNEPS nodes represent intensional concepts. *Intensional entities* correspond to a given meaning — are the objects of thought —, whereas *extensional entities* correspond to objects in the real-world.

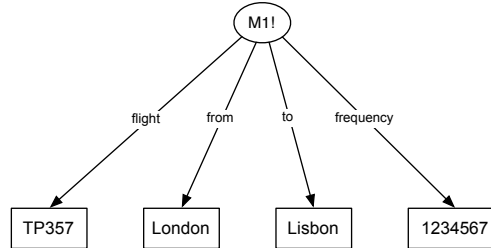


Figure 6.26: TP357 daily connects London to Lisbon.

In order to elaborate on the issue of intensional and extensional entities, let us introduce some representations. In Figure 5.26, we show the representation that flight TP357 connects London to Lisbon on a daily frequency. In timetables of airline companies, the frequency is represented by a sequence of digits, ranging from 1 (corresponding to Monday) to 7 (corresponding to Sunday), indicating the days of the week the flight operates. Thus, a flight whose frequency is 1234567 is operated daily.

In Figure 5.26, we use the case frame *flight* / *from* / *to* / *frequency* to represent the statement that a given flight number connects the origin with the destination at a given frequency.

Every day, there is an instance of flight TP357, connecting London to Lisbon. In Figure 5.27, we represent the particular instance of this flight on April 15, 2005.

In this figure, we use the case frame *flight* / *flight-id* / *date* to represent the flight identification of a flight on a given day. In the same figure we also

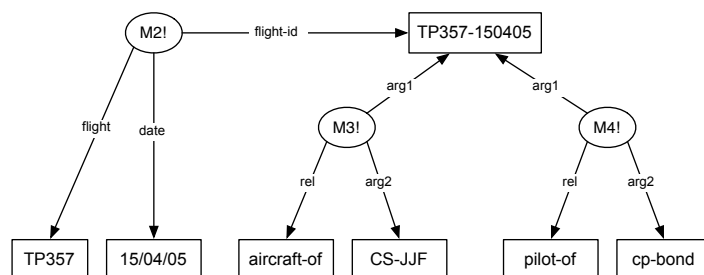


Figure 6.27: TP357 flight on April 15, 2005.

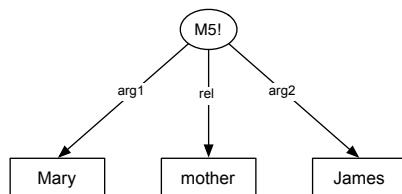


Figure 6.28: James is Mary's son.

represent that the aircraft used in this instance of flight TP357 is CS-TJF and the pilot of this flight is Captain Bond.

Let us assume, furthermore, that we know of a person, named James, who is Mary's son (Figure 5.27). What has this to do with the flight example? You may ask. Well, it happens that James is Captain Bond!

The question now raised, is how to represent together networks of figures 5.27 and 5.28. Should we merge nodes **cp-bond** and **James** into a single node or shall we keep them separate? This is exactly where the issue of intensional and extensional entities arises.

Under the perspective that SNEPS nodes represent intensional concepts, these two nodes should be different. Node labelled **James** corresponds to Mary's son. This person should give Mary a present on Mothers' day. Node labelled **cp-bond** is the captain of flight TP357-150405. This person has some duties as such, none of them corresponds to giving Mary a present on Mothers' day. Nevertheless, these two intensional entities correspond to the same extensional entity, the same physical person. If these nodes were merged into a single node, we would lose the possibility of stating, for example, that "Peter didn't know that James was Captain Bond".

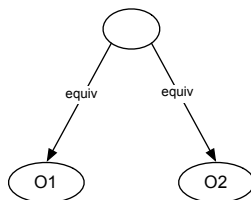


Figure 6.29: O1 and O2 correspond to the same object.

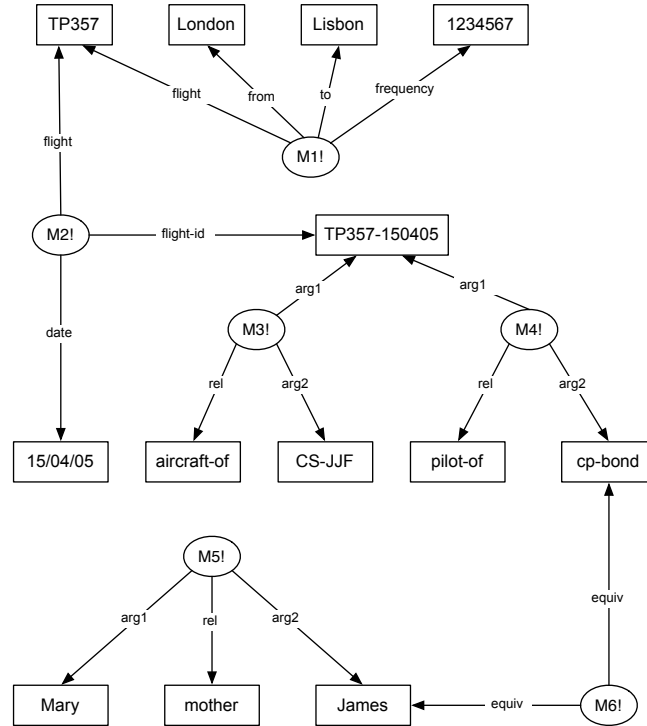


Figure 6.30: Captain Bond and James are the same person.

It is often needed to state that two intensional entities correspond to the same object in the world. In order to enable the representation of statements of this type, SNePS uses the case frame *equiv* / *equiv* (Figure 5.29), where the arcs labelled *equiv* point to the nodes about which we are stating extensional equivalence. In Figure 5.30, we show the network that asserts that *cp-bond* and *James* are the same extensional entity.

Before closing this section, we present in Table 5.1 a summary of the standard case frames used in SNePS.

6.2.4 Logical connectives in SNePS

An important aspect of semantic networks concerns how to represent logical relations between propositions. One obvious alternative is to use the logical connectives of classical logic, which have the advantage of having

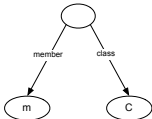
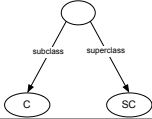
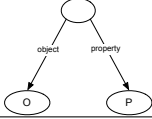
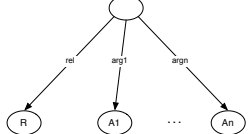
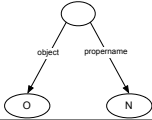
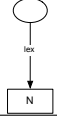
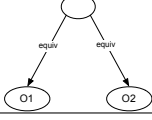
Case frame	Meaning
	m is a member of class c
	C is a subclass of class SC
	Object O has the property P
	General n -ary relation
	Object O has name N
	N is the external representation of a node
	O1 and O2 correspond to the same object

Table 6.1: Summary of standard case frames.

the guarantee that the inferences produced are sound, well known, and well understood.

However, the choice of logical connectives depends on other things, such as ease of representation, ease of proving properties about the system, proofs of equivalence with other logical systems, and so on. If the number of logical connectives is small, it is easy to prove properties *about* the system but it is difficult to prove properties *in* the system; if the number of logical connectives is large, the opposite holds.

In the choice of logical connectives for SNePS, the main guidelines were adequate expressive power and naturalness of the representation. The set of connectives traditionally used in first-order logic presents some disadvantages from the network representation point of view. First, all the connectives, except negation, are binary and therefore expressing sentences about sets of propositions becomes cumbersome. For example, having to choose between representing

$$(P \vee Q) \vee R$$

and

$$P \vee (Q \vee R)$$

is unnecessary and unfairly distinguishes one of the disjuncts.

On the other hand, suppose, for example, that given three propositions, P , Q and R , we wanted to express the fact that exactly one of them is true. Using the traditional logical connectives ($\vee$ and $\wedge$) this would be done as:¹⁷

$$(((P \wedge \neg Q) \wedge \neg R) \vee ((\neg P \wedge Q) \wedge \neg R)) \vee ((\neg P \wedge \neg Q) \wedge R) \quad (6.1)$$

which is a lengthy and difficult-to-read *uff*. Sentences involving more than three propositions (for example, exactly five out of ten propositions are true) are even more complicated to express. Since this type of sentence often occurs in some of the intended applications of SNePS, the need was felt to find a simpler and more powerful way of defining such sentences.

The foregoing discussion led to the definition of a set of non-standard connectives. From the logic point of view, non-standard connectives take sets of propositions as arguments. Non-standard connectives are defined as syntactical combinations of the standard logical connectives and can be seen as an

¹⁷In each of the conjunctions, we had to make a choice of associations because logical connectives are binary.

easy and convenient way to express complex propositions — they correspond to *syntactic sugar* for the standard logical connectives.

When we consider the logic underlying the reasoning of SNePS, the introduction of non-standard connectives leads to a revision of the definition of the language of the logic, with the introduction of the following rules:

1. *Logical symbols*:¹⁸ $\wedge \Rightarrow$ $\vee \Rightarrow$ ${}_n\mathbb{X}_i^j$ (with, $i, j, n \in \mathbb{N}$) $\forall$ $\exists$
 - (a) the symbol $\wedge \Rightarrow$ is read *and-entailment*;
 - (b) the symbol $\vee \Rightarrow$ is read *or-entailment*;
 - (c) the symbol ${}_n\mathbb{X}_i^j$ is read *and-or*;
 - (d) the symbols $\forall$ and $\exists$ have the same reading as in first-order logic.
2. *Well-formed formulas (wffs)*:¹⁹
 - (a) Predicates are *wffs*;
 - (b) If $\alpha_1, \dots, \alpha_n, \beta_1, \dots, \beta_m$ are *wffs* then $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$ and $\{\alpha_1, \dots, \alpha_n\} \vee \Rightarrow \{\beta_1, \dots, \beta_m\}$ are *wffs*;
 - (c) If $\alpha_1, \dots, \alpha_n$ are *wffs*, $0 \leq i \leq j \leq n$ then, ${}_n\mathbb{X}_i^j \{\alpha_1, \dots, \alpha_n\}$ is a *wff*;
 - (d) If α is a *wff* containing zero or more occurrences of the individual variable x , then $\forall x[\alpha]$ and $\exists x[\alpha]$ are *wffs*;
 - (e) Nothing else is a *wff*.

In SNePS, there are more non-standard connectives than those described here (see, for example, Section 5.6), however, for the aims of our presentation these connectives are enough.

The logic underlying SNePS is based on relevance logic (see Section 2.2). In our presentation, we simplify the statement of the rules of inference, leaving out the indication of the identifier set.

Rules for and-entailment. *And-entailment* is a generalization of entailment to take two sets of propositions, a set of antecedents and a set of consequents.

¹⁸The first three symbols are called *non-standard connectives*.

¹⁹On purpose, we avoided the definition of terms.

In the *woff* $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$, $\alpha_1, \dots, \alpha_n$ are called the *antecedents* and $\beta_1, \dots, \beta_m$ are called the *consequents*. This *woff* means that the conjunction of the antecedents entails the conjunction of the consequents. For example, the following *woff* contains an example of the use of and-entailment:

$$\forall x, y, z [\{Member(x, y), Subclass(y, z)\} \wedge \Rightarrow \{Member(x, z)\}].$$

The *woff* $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$ is considered as being a syntactical abbreviation for the *woff* $(\alpha_1 \Rightarrow \dots \Rightarrow (\alpha_n \Rightarrow (\beta_1 \wedge \dots \wedge \beta_m)))$,²⁰ in which the order of the antecedents is irrelevant.

And-entailment introduction. The rule of and-entailment introduction, written $\wedge \Rightarrow I$, corresponds to a generalization of the $\Rightarrow I$ rule (see page 49). The $\wedge \Rightarrow I$ rule applies n times the rule of $\Rightarrow I$.

If $\beta_1 \wedge \dots \wedge \beta_m$ is derived, inside nested hypothetical proofs initiated by the hypotheses $\alpha_1, \dots, \alpha_n$, we may derive $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$ outside the hypothetical proofs initiated by the hypotheses $\alpha_1, \dots, \alpha_n$:

$$\begin{array}{rcl}
 o & \left| \begin{array}{l} \alpha_1 \\ \vdots \end{array} \right. & \text{Hyp} \\
 \vdots & & \\
 p & \left| \begin{array}{l} \left| \alpha_n \right. \\ \vdots \end{array} \right. & \text{Hyp} \\
 \vdots & & \\
 q & \left| \beta_1 \wedge \dots \wedge \beta_m \right. & \\
 q+1 & \{ \alpha_1, \dots, \alpha_n \} \wedge \Rightarrow \{ \beta_1, \dots, \beta_m \} & \wedge \Rightarrow I, (o, \dots, p, q)
 \end{array}$$

The actual statement of this rule is more complex than we have presented. On the one hand, in the logic underlying SNePS, alongside with each *woff*, there is a record of the hypotheses that were actually used in the derivation of that *woff*,²¹ and, therefore, the full statement of the $\wedge \Rightarrow I$ rule requires that all of $\alpha_1, \dots, \alpha_n$ were *actually used* in the derivation of $\beta_1 \wedge \dots \wedge \beta_m$; on the other hand, there is a notion of *context* shadowing operation, associated with an ATMS-like formalism.

²⁰Where $\Rightarrow$ is the relevance logic entailment.

²¹An equivalent to the identifier set of Section 2.2.

These two aspects combined avoid the explicit representation of the nested structures of subproofs as we have been using up to now. For the purposes of this chapter, our standard representation is sufficient for understanding the rules of inference.

And-entailment elimination, written $\wedge \Rightarrow E$, states that in a proof that contains $\alpha_1, \dots, \alpha_n$, and $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$, we may derive $\beta_1 \wedge \dots \wedge \beta_m$:

o	α_1	
$\vdots$	$\vdots$	
p	α_n	
q	$\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$	
$q + 1$	$\beta_1 \wedge \dots \wedge \beta_m$	$\wedge \Rightarrow E, (o, \dots, p, q)$

You may have been lead to think that it might be more natural to have the inference rule “from $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$ and $\alpha_1 \wedge \dots \wedge \alpha_n$ we may derive $\beta_1 \wedge \dots \wedge \beta_m$ ”. If this were the case, then the rule of $\wedge \Rightarrow I$ should have been stated something as “if in a proof initiated by the hypothesis $\alpha_1 \wedge \dots \wedge \alpha_n$ we have the *wff* $\beta_1 \wedge \dots \wedge \beta_m$, then we can derive $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$, immediately outside the proof initiated by the hypothesis $\alpha_1 \wedge \dots \wedge \alpha_n$ ”.

Rules for or-entailment. Or-entailment is another generalization of entailment to take two sets of propositions, a set of antecedents and a set of consequents.

In the *wff* $\{\alpha_1, \dots, \alpha_n\} \vee \Rightarrow \{\beta_1, \dots, \beta_m\}$, $\alpha_1, \dots, \alpha_n$ are called the *antecedents* and $\beta_1, \dots, \beta_m$ are called the *consequents*. This *wff* means that the disjunction of the antecedents entails the conjunction of the consequents. For example, the following *wff* contains an example of the use of or-entailment:

$$\forall x [\{Commercial_aircraft(x), Military_aircraft(x)\} \vee \Rightarrow \{Aircraft(x)\}].$$

The *wff* $\{\alpha_1, \dots, \alpha_n\} \vee \Rightarrow \{\beta_1, \dots, \beta_m\}$ is considered as a syntactical abbreviation of n *wffs*, $\alpha_i \Rightarrow (\beta_1 \wedge \dots \wedge \beta_m)$, for $1 \leq i \leq n$.

Or-entailment introduction, written $\vee \Rightarrow I$. The rule for the introduction of or-entailment takes into account that $\{\alpha_1, \dots, \alpha_n\} \vee \Rightarrow \{\beta_1, \dots, \beta_m\}$ is an abbreviation for $(\alpha_1 \Rightarrow (\beta_1 \wedge \dots \wedge \beta_m)) \wedge \dots \wedge (\alpha_n \Rightarrow (\beta_1 \wedge \dots \wedge \beta_m))$. For

this reason, the introduction of $\{\alpha_1, \dots, \alpha_n\} \vee\Rightarrow \{\beta_1, \dots, \beta_m\}$ requires the proof of the n *wffs* $\{\alpha_i\} \wedge\Rightarrow \{\beta_1, \dots, \beta_m\}$, for $1 \leq i \leq n$.

Formally, if, for each i such that $1 \leq i \leq n$, we have a proof initiated by the hypothesis α_i , inside of which the *wff* $\beta_1 \wedge \dots \wedge \beta_m$ has been derived,²² then we can infer $\{\alpha_1, \dots, \alpha_n\} \vee\Rightarrow \{\beta_1, \dots, \beta_m\}$ in a proof that immediately contains all the hypothetical proofs initiated by the hypotheses α_i ($1 \leq i \leq n$).

l_1	α_1	Hyp
$\vdots$	$\vdots$	
l_k	$\beta_1 \wedge \dots \beta_m$	
$\vdots$	$\vdots$	
l_n	α_n	Hyp
$\vdots$	$\vdots$	
l_r	$\beta_1 \wedge \dots \wedge \beta_m$	
$l_r + 1$	$\{\alpha_1, \dots, \alpha_n\} \vee\Rightarrow \{\beta_1, \dots, \beta_m\}$	$\vee\Rightarrow\text{I}, ((l_1, l_k), \dots, (l_n, l_r))$

Or-entailment elimination, written $\vee\Rightarrow\text{E}$, states that from $\{\alpha_1, \dots, \alpha_n\} \vee\Rightarrow \{\beta_1, \dots, \beta_m\}$ and any α such that $\alpha \in \{\alpha_1, \dots, \alpha_n\}$ we may infer $\beta_1 \wedge \dots \wedge \beta_m$.

p	$\{\alpha_1, \dots, \alpha_n\} \vee\Rightarrow \{\beta_1, \dots, \beta_m\}$	
q	α_i ($1 \leq i \leq n$)	
$q + 1$	$\beta_1 \wedge \dots \wedge \beta_m$	$\vee\Rightarrow\text{E}, (p, q)$

Rules for and-or. *And-or* is a connective that generalizes $\neg$ (not), $\wedge$ (and), $\vee$ (or), as well as other connectives that are common in classical logic, such as *exclusive or*, *nand*, and *nor*. Furthermore, the generalization is applied to a set of propositions.

In the *wff* ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$, $\alpha_1, \dots, \alpha_n$ are called its *arguments*. The proposition represented by the *wff* ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$, $0 \leq i \leq j \leq n$ asserts that there is a relevant connection between the propositions represented by *wffs* $\alpha_1, \dots, \alpha_n$ such that at least i and at most j of them must simultaneously be true. Given ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$,

²²In the full statement of these rules, the derivation of $\beta_1 \wedge \dots \wedge \beta_m$ must use the hypothesis α_i .

- if $n - i$ of its arguments are known to be false, then the remaining i have to be true;
- if j of its arguments are known to be true then the remaining $n - j$ have to be false.

For example,

$$\begin{aligned} {}_2\mathbb{X}_1^2\{P, Q\} &= P \vee Q, \\ {}_2\mathbb{X}_2^2\{P, Q\} &= P \wedge Q, \\ {}_1\mathbb{X}_0^0\{P\} &= \neg P. \end{aligned}$$

As another example, using and-or, *wff* 5.1 is written as

$${}_3\mathbb{X}_1^1\{P, Q, R\}.$$

In order to present the rules of inference associated with *and-or*, we need to introduce the following notation:

1. Let $C_i^n(\alpha_1, \dots, \alpha_n)$ represent an ordered set²³ of all the i -combinations of the elements of the set $\{\alpha_1, \dots, \alpha_n\}$.

We represent the k -th element of $C_i^n(\alpha_1, \dots, \alpha_n)$ by ${}_k c_i^n(\alpha_1, \dots, \alpha_n)$, and the set $\{\alpha_1, \dots, \alpha_n\} \setminus {}_k c_i^n(\alpha_1, \dots, \alpha_n)$ by ${}_k \bar{c}_i^n(\alpha_1, \dots, \alpha_n)$.

For notational convenience, when the arguments of ${}_k c_i^n$ and ${}_k \bar{c}_i^n$ are left unspecified they default to $\alpha_1, \dots, \alpha_n$.

2. If f is a function of one argument, we write $f\langle\{\alpha_1, \dots, \alpha_n\}\rangle$ to denote the set $\{f(\alpha_1), \dots, f(\alpha_n)\}$. For example,

$$\neg\langle\{\alpha_1, \dots, \alpha_n\}\rangle = \{\neg\alpha_1, \dots, \neg\alpha_n\}.$$

And-or introduction (written $\mathbb{XI}$). The introduction of ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$, requires the following two sets of conditions to be verified:

²³Using some ordering criteria which we don't care about.

1. If $n-i$ arguments whatsoever are known to be false, then all the others have to be true.

This can be stated as

$$\forall k, 1 \leq k \leq \binom{n}{n-i} [\neg \langle k c_{n-i}^n \rangle \wedge \Rightarrow \langle k \bar{c}_{n-i}^n \rangle]. \quad (6.2)$$

Conditions 5.2, may be stated in the following way:

For each k such that $1 \leq k \leq \binom{n}{n-i}$, the *wff* $\alpha_1 \wedge \dots \wedge \alpha_i$, where $\{\alpha_1, \dots, \alpha_i\} = \langle k \bar{c}_{n-i}^n \rangle$, was derived inside hypothetical proofs initiated by the hypotheses $\neg \beta_1, \dots, \neg \beta_{n-i}$, where $\{\beta_1, \dots, \beta_{n-i}\} = \langle k c_{n-i}^n \rangle$.

This means that that $\{\neg \beta_1, \dots, \neg \beta_{n-i}\} \wedge \Rightarrow \{\alpha_1, \dots, \alpha_i\}$.

2. If j arguments whatsoever are known to be true, then all the others have to be false.

This can be stated as

$$\forall k, 1 \leq k \leq \binom{n}{j} [\langle k c_j^n \rangle \wedge \Rightarrow \neg \langle k \bar{c}_j^n \rangle]. \quad (6.3)$$

Conditions 5.3, may be stated in the following way:

For each q such that $1 \leq q \leq \binom{n}{j}$ the *wff* $\neg \alpha_1 \wedge \dots \wedge \neg \alpha_{n-j}$, in which $\{\alpha_1, \dots, \alpha_{n-j}\} = \langle q \bar{c}_j^n \rangle$, was derived inside hypothetical proofs initiated by the hypotheses $\beta_1, \dots, \beta_j$, where $\{\beta_1, \dots, \beta_j\} = \langle q c_j^n \rangle$.

This means that $\{\beta_1, \dots, \beta_j\} \wedge \Rightarrow \{\neg \alpha_1, \dots, \neg \alpha_{n-j}\}$.

The $\mathbb{X}\mathbb{I}$ rule states that, given n , i and j , if conditions 1 and 2 (above) are verified, then, we can derive ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$. Therefore, the $\mathbb{X}\mathbb{I}$ rule requires the verification of the $\binom{n}{n-i}$ entailments listed under 1 and of the $\binom{n}{j}$ entailments listed under 2.

And-or elimination, written $\mathbb{X}\mathbb{E}$, allows the following inferences:

1. From ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$, $\neg \beta_1, \dots$, and $\neg \beta_{n-i}$, where $\{\beta_1, \dots, \beta_{n-i}\} \subset \{\alpha_1, \dots, \alpha_n\}$, for each $\alpha \in \{\alpha_1, \dots, \alpha_n\} \setminus \{\beta_1, \dots, \beta_{n-i}\}$ we can derive α .
2. From ${}_n\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$, $\beta_1, \dots$, and β_j , where $\{\beta_1, \dots, \beta_j\} \subset \{\alpha_1, \dots, \alpha_n\}$, for each $\alpha \in \{\alpha_1, \dots, \alpha_n\} \setminus \{\beta_1, \dots, \beta_j\}$ we can derive $\neg \alpha$.

Rules for the universal quantifier. The *universal quantifier* is represented by $\forall$. The proposition $\forall x[\alpha(x)]$ means that every constant, c , that can replace x makes the proposition $\alpha(x)$ true.

Universal introduction, written $\forall I$.

If $\beta(t)$, was derived in a hypothetical proof that was initiated by the hypothesis $\alpha(t)$ which uses a term (t) that was never used in the system prior to α 's introduction, then we can deduce $\forall x[\alpha(x) \vee \Rightarrow \beta(x)]$ (in which α and β are free for x), in the proof immediately containing the hypothetical proof initiated by the hypothesis $\alpha(t)$.

Universal elimination, written $\forall E$.

From $\forall x[\alpha(x)]$, for any individual symbol, c , we can deduce $\alpha(c)$.

Rules for the existential quantifier. The proposition $\exists x[\alpha(x)]$ means that there is at least one constant that can replace x and make the proposition $\alpha(x)$ true. The *existential quantifier* is not implemented in the current version of SNEPS.

Existential introduction, written $\exists I$.

From $\alpha(c)$ in which c is an individual constant, we can deduce $\exists x[\alpha(x)]$.

Existential elimination, written $\exists E$.

From $\exists x[\alpha(x)]$ and any individual constant c which was never used before, we can deduce $\alpha(c)$. The individual constant c is called *Skolem constant*.²⁴

If an existential quantifier exists within the scope of a universal quantifier, there is the possibility of the existentially quantified variable to depend on the universally quantified variable. In this case, in order to eliminate the existential quantifier, the corresponding variable should be replaced by a term made of a new function name whose argument is the universally quantified variable. This function is called *Skolem function*. For example, in the *wff* $\forall x[\exists y[P(x, y)]]$ we may use the Skolem function $f(x)$ to eliminate the existential quantifier, obtaining $\forall x[P(x, f(x))]$.

²⁴Named after the mathematician Thoralf Skolem (1887–1963).

6.2.5 Network representation of the connectives

When we consider the logical connectives in SNePS, we have, not only to consider what inference rules are associated with the connectives, but we also have to consider how to represent in the network a proposition corresponding to the connective.

In order to define the network representation of a proposition involving a connective, we have to address three questions:

1. How is a proposition corresponding to the connective represented?

Taking into account that SNePS is a propositional semantic network, a proposition corresponding to a connective is represented by a structured node that relates the nodes corresponding to the individual propositions.

2. How are the individual propositions involved in a connective represented?

This question is answered by the introduction of case frames as described in Section 5.1.

3. How to represent the connective itself?

This question is answered by the kinds of arcs that emanate from the structured node representing the connective. Case frames are defined in SNePS, individually, for each of the connectives — there are pre-defined arcs that emanate from the node that corresponds to the connective, and whose semantics is known by SNePS's inference engine.

And-entailment is represented in the network by a node with arcs labelled **&ant** pointing to the nodes corresponding to the antecedents and with arcs labelled **cq** pointing to the nodes corresponding to the consequents (Figure 5.31).

In Figure 5.31, and in the figures that will follow, a triangle below a node represents the network structure of the proposition that is written below it.

Or-entailment is represented by a node with arcs labelled **ant** pointing to the nodes corresponding to the antecedents, and arcs labelled **cq** pointing to the nodes corresponding to the consequents (Figure 5.32).

The proposition $\mathbb{X}_i^j\{\alpha_1, \dots, \alpha_n\}$ is represented in SNePS by a node with arcs labelled **arg** pointing to the nodes corresponding to the propositions that

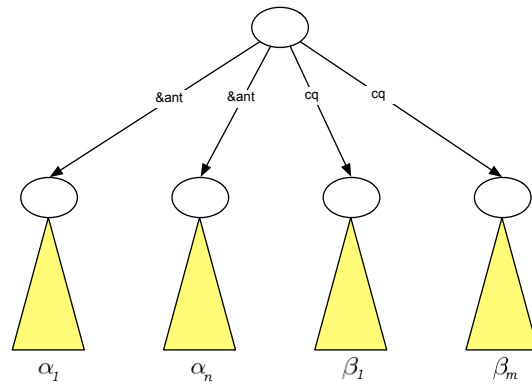


Figure 6.31: Representation of $\{\alpha_1, \dots, \alpha_n\} \wedge \Rightarrow \{\beta_1, \dots, \beta_m\}$.

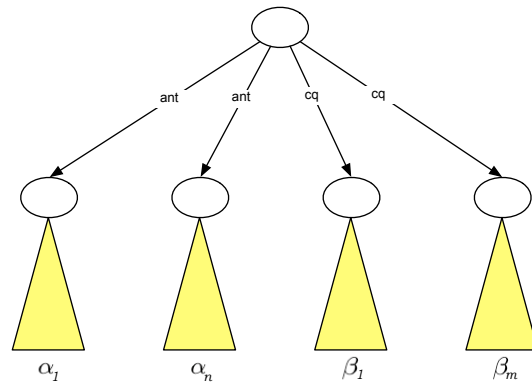
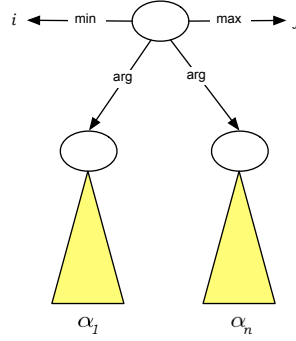


Figure 6.32: Representation of $\{\alpha_1, \dots, \alpha_n\} \vee \Rightarrow \{\beta_1, \dots, \beta_m\}$.

Figure 6.33: Representation of $\bigvee_i^j \{\alpha_1, \dots, \alpha_n\}$.Figure 6.34: Representation of $\forall x[\alpha(x)]$.

are the arguments of the *and-or*, and arcs labelled *min* and *max* pointing to the parameters i and j , respectively. There is no need for an explicit representation of n , since this corresponds to the number of arcs labelled *arg*. In Figure 5.33 we show the representation of *and-or* in SNePS.

The universal quantifier is represented in SNePS by an arc labelled *forall* that links the node corresponding to the proposition where the quantification is stated to the nodes that correspond to the quantified variables (Figure 5.34).

To represent the effects of the existential quantifier, the current version of SNePS makes use of Skolem functions: whenever an existentially quantified variable y is bound within the scope of a universally quantified variable x , y can be replaced by the Skolem function $f(x)$, provided that f is not used anywhere else. In this case, the existential quantifier that binds y can be eliminated.

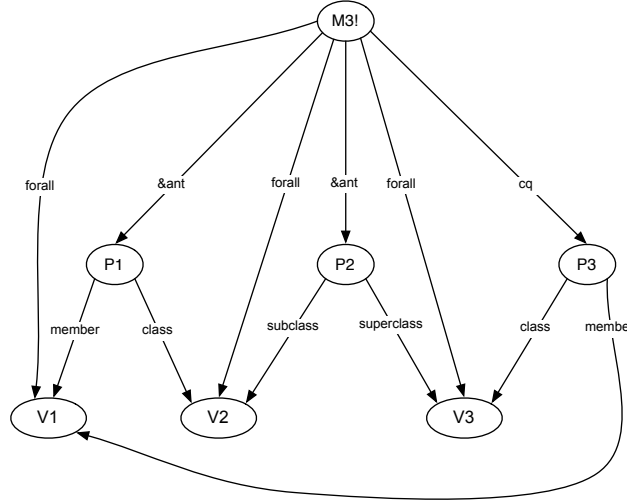


Figure 6.35: Transitivity of membership.

6.2.6 Putting it all together

In this section, we discuss representations in SNePS involving both user-defined case frames and the pre-defined case frames used to represent logical connectives and quantifiers. Every node in SNePS that corresponds to a connective or that contains a quantifier is called a *rule node* or just a *rule*.

We start by defining rules that specify some of the properties of membership and subclass relations. The first statement that we represent asserts that if an element is member of a class, then it is also a member of any superclass of that class. Using *and-entailment*, this statement can be expressed as follows:

$$\forall m, c_1, c_2 [\{Member(m, c_1), Subclass(c_1, c_2)\} \wedge \Rightarrow \{Member(m, c_2)\}].$$

The representation of this statement in SNePS is shown in Figure 5.35. This figure makes use of both user-defined case frames (**member** / **class** and **subclass** / **superclass**) and system-defined case frames to represent the rule. Node M3! states that for all V1, V2, and V3, if V1 is a member of class V2 and V2 is a subclass of V3, then V1 is a member of class V3.

With the rule in Figure 5.35 and the network shown in Figure 5.18, SNePS is now capable of knowing that CS-TJF is an aircraft.

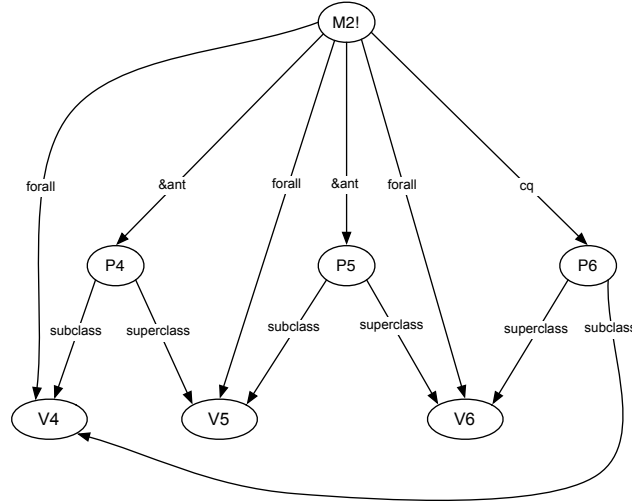


Figure 6.36: Transitivity of subclass.

If we intend to use the class-subclass relation, we should also state that if a class is a subclass of another class, which, in turn, is a subclass of a third class, then the first class is a subclass of the third class. This is a statement of the transitivity relationship that holds between the class-subclass relation. This statement can be expressed as:

$$\forall c_1, c_2, c_3 [\{Subclass(c_1, c_2), Subclass(c_2, c_3)\} \wedge \Rightarrow \{Subclass(c_1, c_3)\}].$$

The representation of this statement in SNePS is shown in Figure 5.36.

If we adopt the previous representation for asserting the transitivity of a relation, we would have to write similar rules for *all* the relations (case frames) represented in the network that satisfy the transitivity property. An alternative way of representing the transitivity of the subclass relation is to explicitly state that the relation is transitive, and to have a rule that states what it means for a relation to be transitive (this representation is shown in Figure 5.37):²⁵

$$\begin{aligned} &\forall R [\{Transitive(R)\} \\ &\quad \vee \Rightarrow \\ &\quad \{\forall c_1, c_2, c_3 [\{R(c_1, c_2), R(c_2, c_3)\} \wedge \Rightarrow \{R(c_1, c_3)\}]\}]. \end{aligned}$$

²⁵Notice that this logic-like formula is not a *wff* of first-order logic.

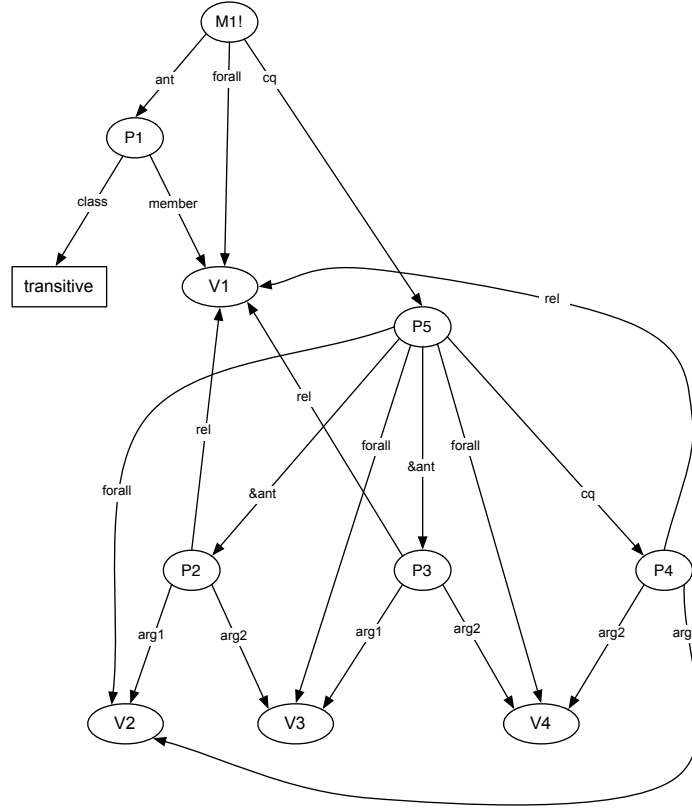


Figure 6.37: Meaning of a transitive relation.

Stating that a relation is transitive requires the possibility of talking *about* the relation. Since the case frames *member* / *class* and *subclass* / *superclass* do not allow us to explicitly talk about the relation involved, the approach used in Figure 5.37 requires the use of the case frame *rel* / *arg1* / ... / *argn*, as discussed in page 148.

Due to the fact that, in SNEPS nodes represent propositions (among other things), we are able to write propositions *about* propositions. In this way, the logic underlying the reasoning of SNEPS is a higher-order logic.

The advantage of this second approach of representation is that it can be applied to *any* transitive relation. Thus, a SNEPS' user, after defining the network structure of Figure 5.37, just has to state that a certain relation is transitive to obtain the desired behavior. On the other hand, this second

approach may overload the work of the inference engine as discussed in page 149.

6.2.7 Inference in SNePS

There are two types of inference in SNePS, node-based inference and path-based inference.

Node-based inference. Node-based inference allows a node or a structure of nodes to be inferred from the existence of instances of patterns of nodes. Node-based inference relies on a pattern matching operation that takes a node (either a constant node or a pattern node) and locates all the nodes in the network that match with it.

There are two main concepts involved in the implementation of SNePS inference engine (known as SNIP the SNePS inference package), pattern-matching and the use of procedural (or active) versions of rules:

- The *pattern-matching process* is given a piece of the network (either to be deduced in backward inference or to be added in forward inference) and locates relevant rules in the network;
- The rules located by the pattern matcher are then *compiled* into a set of processes which are given to a multi-processing system for execution. The *multi-processing system* used by SNePS, called MULTI, is a LISP-based system mainly consisting of a simple evaluator, a scheduler, and system primitives.

In node-based inference, SNIP looks for nodes that correspond to propositions, using the non-standard connectives, and matches nodes against components of these propositions.

For example, given the network of Figure 5.38:

- node M2! matches node P1 with the substitution {V1/CS-TJF, V2/commercial-aircraft} (the nodes that match are linked, in Figure 5.38, by a dashed line);
- node M3! matches node P2 with the substitution {V2/commercial-aircraft, V3/aircraft}.

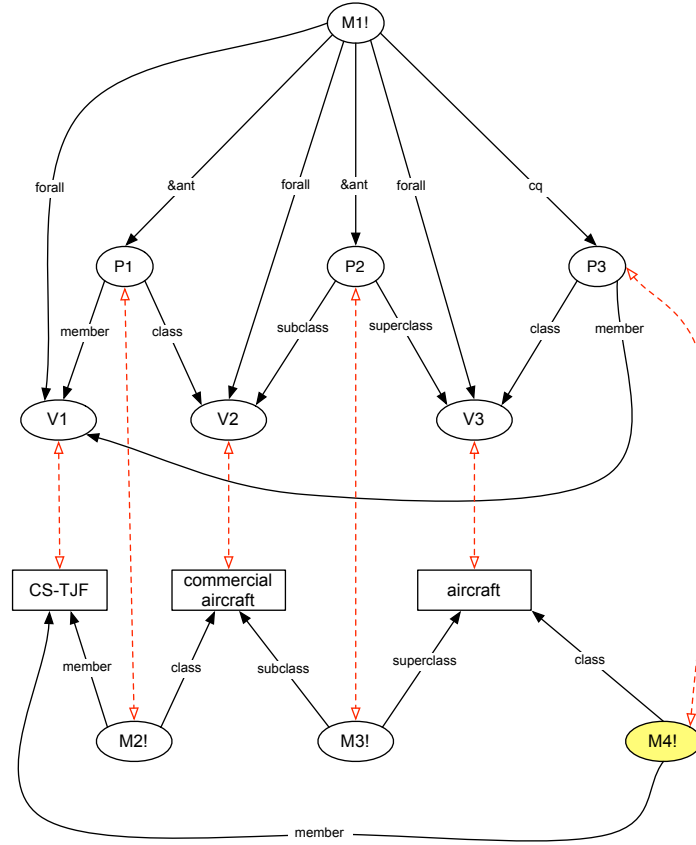


Figure 6.38: Network used in node-based inference.

This allows the inference of an instance of the consequent of the proposition represented by node M1! (P3) with the substitution $\{V1/CS-TJF, V3/aircraft\}$; this new node, M4!, is shaded in Figure 5.38.

Node-based inference may either work in *backward inference* mode, trying to prove instances of consequents, or in *forward inference* mode, trying to find consequences of antecedents.

The SNEPS inference system (in what concerns node-based inference) has the following characteristics:

- it allows both backward and forward inference to be performed;

- every rule in the network may be either used in backward inference, forward inference, or both;
- when a rule is used it is activated and remains active until explicitly de-activated by the user;
- the activated rules are assembled into a set of processes, called an *active connection graph* (*acg*), which carry out the inferences;
- the *acg* also stores all the results generated by the activated rules; if during some deduction, the inference system needs some of the rules activated during a previous deduction, it uses their results directly instead of re-deriving them.

Path-based inference. Path-based inference allows an arc, or a path of arcs, between two nodes to be inferred from the existence of a specified path of arcs between the same two nodes.

The result obtained in the previous section could have been obtained in a rather different way: suppose that we have nodes M2! and M3! of Figure 5.38 and that we tell SNePS that whenever the sequence of arcs²⁶

member- / ! / class / subclass- / ! / superclass

is found from a node x to a node y, then it can infer the existence of the sequence of arcs **member- / ! / class** from node x to node y; under these circumstances SNePS could obtain the same results as before.

However, the way this result is obtained is very different from the previous derivation: rather than relying on the existence of instances of patterns of nodes, we rely on the existence of a sequence of arcs (which we call a *path*) from a node to another; this is *path-based inference*.

6.2.8 Building and using SNePS networks

The use of SNePS networks can be done through SNePSUL, the SNePS User Language. This is the standard command language for using SNePS. SNePSUL

²⁶Note the use of the converse arcs.

is a LISP-like language, entered by the user at the top-level of SNePS' read-eval-print loop. For more information about how to use SNePSUL refer to [Shapiro and Rapaport 03].²⁷

Before we start presenting SNePSUL, we have to introduce a few definitions:

```

<letter> ::= A | B | C | D | E | F | G | H | I | J | K | L | M |
           N | O | P | Q | R | S | T | U | V | X | Y | W | Z |
           a | b | c | d | e | f | g | h | i | j | k | l | m |
           n | o | p | q | r | s | t | u | v | x | y | w | z

<digit> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0

<dash> ::= -

<letter or digit> ::= <letter> | <digit>

<char> ::= <letter> | <digit> | <dash>

<string> ::= <char>+

<relation> ::= <letter or digit> |
               <letter or digit>+ <string> <letter or digit>+

<node> ::= <node identifier> | <node function>

<node set> ::= <node>+ | ( <node>+ )

<node identifier> ::= <constant node> | <variable node>

<constant node> ::= <string> | B<digit>+ | #<string>

<variable node> ::= $<string> | V<digit>+ | *<string>

<node function> ::=

```

²⁷ SNePS can be obtained at <http://www.cse.Buffalo.EDU/sneps/>.

```

    (build {<relation> <nodeset>}+) |
    (assert {<relation> <nodeset>}+) |
    (add {<relation> <nodeset>}+) |
    (activate <nodeset>) |
    (!<node>) |
    (find {<path> <nodeset>}+) |
    (findassert {<path> <nodeset>}+) |
    ?<string> |
    (deduce [<number>] {<relation> <nodeset>}+) |
    (deducetrue [<number>] {<relation> <nodeset>}+) |
    (deducefalse [<number>] {<relation> <nodeset>}+)

<unipath> ::= <relation> | <relation>-

<path> ::= < unipath > |
    (converse <path>) |
    (compose <path>*) |
    (kstar <path>) |
    (kplus <path>)

```

Definition of arcs

In this section, “relation” and “arc label” are used interchangeably.

Whenever an arc labelled *R* goes from node *x* to node *y*, SNePS also creates an arc labelled *R-* to go from *y* to *x*. Relation names ending in the character “-” are reserved for “converse arc” labelling and are not allowed in the non-terminal symbol <relation>.

(define <relation>⁺)
 Defines each <relation> to be an arc label. A message is provided if a relation has previously been defined.

(undefine <relation>⁺)
 Undefines each relation. This command is most useful in correcting typographical errors.

Initially, SNePS has a set of relations defined as if the following command had been executed:²⁸

```
(define forall min max ant &ant cq arg)
```

Building networks

²⁸There are many more predefined relations (arcs) in SNePS, however, we do not deal with them in this chapter.

The commands in this section add information to the network, either in the form of a node or a node and some arcs. All the commands presented take an optional argument that corresponds to a context. In this chapter, we will not use contexts, thus this argument is not even shown.

It is not possible to add just an arc to the network. Isolated nodes cannot be added to the network, so the commands **#** and **\$** can only be used within the lexical context of a **build**, **assert**, or **add** command.

(build {<relation> <nodeset>}⁺)

Creates a node in the network with an arc labelled **<relation>** to each node in the associated **<nodeset>**, and returns a singleton set containing the node built.

The node created by **build** is unasserted.

If the node to be built would look exactly like an already existing node, then no node is built, but a singleton set containing the extant node is returned.

(assert {<relation> <nodeset>}⁺)

Similar to **build**, but creates the node as an asserted node.

(add {<relation> <nodeset>}⁺)

Similar to **assert**, but, in addition, triggers forward inference with the node added.

(activate <nodeset>)

Finds all the asserted nodes that dominate the nodes in **<nodeset>** and triggers forward inference on them.

(!<node>)

A postfix command that asserts **<node>** and returns a singleton set containing node.

#<string>

A macro command that creates a new base node, assigns a singleton set containing the new node as the value of the SNePSUL variable **<string>**, and returns that set.

This may not be used at the top-level SNePSUL read-eval-print loop, since that would create an isolated node.

\$<string>

A macro command that creates a new variable node, assigns a singleton set containing the new node as the value of the SNePSUL variable `<string>`, and returns that set.

This may not be used at the top-level SNePSUL read-eval-print loop, since that would create an isolated node.

***<string>**

A macro command that returns the value of the SNePSUL variable `<string>`.

Displaying the network

The commands in this section are various ways of printing, or otherwise displaying, the information in the network.

(dump {<nodeset>}*)

Prints the name of each node in the `<nodeset>` along with all arcs going from it or into it, and the nodes that each arc points to or from. The SNePSUL variable `*nodes` corresponds to all the nodes in the network.

(describe {<nodeset>}*)

Similar to `dump`, but describes only the molecular and pattern nodes in the `<nodeset>`.

Describes all molecular and pattern nodes dominated by nodes it describes; describes any node at most once — the second and later times, only the node's name is printed.

Retrieving information

The commands in this section find nodes in the network, and return them.

(find {<path> <nodeset>}+)

Returns the set of nodes such that each node in the set has every specified `<path>` going from it to at least one node in the accompanying `<nodeset>`. Returns all appropriate nodes.

(findassert {<path> <nodeset>}+)

Similar to `find` but only returns asserted nodes.

?<symbol>

May be used in any `find` function in place of a `<nodeset>`, to stand for “any node.” The scope of these symbols is the outermost `find` function and all embedded `find` functions.

After return of the outermost `find` function, `<symbol>` will be a SNePSUL variable whose value will be the set of nodes it matched.

(deduce [<number>] {<relation> <nodeset>}⁺)

Is like **findassert**, but uses backward inference on any deduction rules. Returns all inferred nodes that satisfy the specification, and inferred nodes that satisfy the negation of the specification

The <number> argument is optional.

- If <number> is omitted, then **deduce** continues until no more answers can be derived;
- If <number> is a single integer, it specifies the total number of answers requested;
- If <number> is zero, no inference is done — only answers already in the network are returned. In this case, **deduce** is the same as **find**;
- Otherwise, <number> must be a list of two numbers, (n_{pos} n_{neg}), and deduction terminates after at least n_{pos} positive and n_{neg} negative instances are derived.

(deducetrue [<number>] {<relation> <nodeset>}^{*})

Is like **deduce**, but only returns inferred nodes that satisfy the specification.

(deducefalse [<number>] {<relation> <nodeset>}^{*})

Is like **deduce**, but only returns inferred nodes that satisfy the negation of the specification.

Path-based inference

Path-based inference allows an arc between two nodes to be inferred from the presence of a path of arcs between them. The various versions of **find** as well as **deduce** will use any path-based inference rules that have been defined.

A path-based inference rule is of the form $\langle \text{relation} \rangle \leftarrow \langle \text{path} \rangle$. Where $\langle \text{relation} \rangle$ is a direct arc and $\langle \text{path} \rangle$ is a path of arcs as defined below. A path-based inference rule is defined by the SNEPSUL command **define-path**.

(define-path {<relation> <path>}⁺)

This command defines the path-based inference rule,

$$\forall n_1, n_2 [\langle \text{path} \rangle (n_1, n_2) \rightarrow \langle \text{relation} \rangle (n_1, n_2)]$$

If a path of arcs specified by $\langle \text{path} \rangle$ is in the network going from node n_1 to node n_2 , then the single arc labelled by $\langle \text{relation} \rangle$ is inferred as going from node n_1 to node n_2 . No relation may have more than one path-based inference rule for it at any time.

(undefine-path {<relation> <path>}*)

Deletes the given path-based inference rules.

The following is a partial version of the path-defining functions.²⁹ In this description, it is important to keep in mind the distinctions between $\langle \text{relation} \rangle$, $\langle \text{unitpath} \rangle$, and $\langle \text{path} \rangle$, since there are places where it matters.

(converse <path>)

If P is a $\langle \text{path} \rangle$ from node x to node y then **(converse P)** is a $\langle \text{path} \rangle$ from y to x .

(compose <path>*)

If $n_1, \dots, n_n$ are nodes and P_i is a $\langle \text{path} \rangle$ from node n_i to node n_{i+1} ($1 \leq i \leq n-1$), then **(compose $P_1 \dots P_{n-1}$)** is a path from n_1 to n_n . If the symbol “!” appears between P_{i-1} and P_i , then n_i must be asserted.

(kstar <path>)

If path P composed with itself zero or more times is a path from node x to node y , then **(kstar P)** is a path from x to y .

(kplus <path>)

If path P composed with itself one or more times is a path from node x to node y , then **(kplus P)** is a path from x to y .

6.2.9 Examples

In this section, we present some examples of the use of SNePS. When interacting with SNePSUL we are in a read-eval-print loop that reads SNePSUL commands, evaluates them and prints the results. The SNePSUL prompt is a star (*). In these interactions, user’s input is written in lower-case characters.

²⁹This description was taken from [Shapiro et. al. 89].

Example of node-based inference. The following example is based on the networks presented in figures 5.18 and 5.35.

Welcome to SNePS-2.6 [PL:0a 2002/09/30 22:37:46]

Copyright (C) 1984--2002 by Research Foundation of State University of New York.

SNePS comes with ABSOLUTELY NO WARRANTY!

Type '(copyright)' for detailed copyright information.

Type '(demo)' for a list of example applications.

```
* (define member class subclass superclass)           Arcs are defined.
(MEMBER CLASS SUBCLASS SUPERCLASS)

* (assert member cs-tjf                               The first node is built.
    class commercial-aircraft)
(M1!)

* (describe m1)                                       M1! is described.
(M1! (CLASS COMMERCIAL-AIRCRAFT)
    (MEMBER CS-TJF))
(M1!)

* (describe
    (assert subclass commercial-aircraft
        superclass aircraft))                       Another node is built and
(M2! (SUBCLASS COMMERCIAL-AIRCRAFT)                  described.
    (SUPERCLASS AIRCRAFT))
(M2!)

* (describe
    (assert
        forall ($x $y $z)
            &ant ((build member *x class *y)
                (build subclass *y
                    superclass *z))
            cq (build member *x class *z)))           A rule node is built.
(M5! (FORALL V3 V2 V1)
    (&ANT (P2 (SUBCLASS V2) (SUPERCLASS V3))
        (P1 (CLASS V2) (MEMBER V1))))
(CQ (P3 (CLASS V3) (MEMBER V1))))
(M5!)

* (deduce member $who class $what)                   Who belongs to what class?

I wonder if
((P4 (CLASS V5) (MEMBER V4)))
holds within the BS defined by context
DEFAULT-DEFAULTCT
```

```
I know
((M1! (CLASS (COMMERCIAL-AIRCRAFT))
      (MEMBER (CS-TJF)))))
```

CS-TJF is a commercial aircraft.

```
I wonder if
((P2 (SUBCLASS V2) (SUPERCLASS V3)))
holds within the BS defined by context
DEFAULT-DEFAULTCT
```

Deduction rule M3! has been found and SNePS looks for instances of its antecedents.

```
I wonder if
((P1 (CLASS V2) (MEMBER V1)))
holds within the BS defined by context
DEFAULT-DEFAULTCT
```

```
I know
((M2! (SUBCLASS (COMMERCIAL-AIRCRAFT))
      (SUPERCLASS (AIRCRAFT)))))
```

Instances are found.

```
I know
((M1! (CLASS (COMMERCIAL-AIRCRAFT))
      (MEMBER (CS-TJF)))))
```

```
Since
((M3!
  (FORALL (V3 <- AIRCRAFT)
    (V2 <- COMMERCIAL-AIRCRAFT)
    (V1 <- CS-TJF))
  (&ANT
    (P2 (SUBCLASS (V2 <- COMMERCIAL-AIRCRAFT))
      (SUPERCLASS (V3 <- AIRCRAFT)))
    (P1 (CLASS (V2 <- COMMERCIAL-AIRCRAFT))
      (MEMBER (V1 <- CS-TJF)))))
  (CQ
    (P3 (CLASS (V3 <- AIRCRAFT))
      (MEMBER (V1 <- CS-TJF))))))
and
((P2 (SUBCLASS (V2 <- COMMERCIAL-AIRCRAFT))
  (SUPERCLASS (V3 <- AIRCRAFT))))
and
((P1 (CLASS (V2 <- COMMERCIAL-AIRCRAFT))
  (MEMBER (V1 <- CS-TJF))))
I infer
((P3 (CLASS (V3 <- AIRCRAFT))
  (MEMBER (V1 <- CS-TJF))))
(M4! M1!)
```

An instance of deduction rule M3! is found, with the substitution {V3 / AIRCRAFT, V2 / COMMERCIAL-AIRCRAFT, V1 / CS-TJF} (rule of $\forall E$). Instances of the antecedents, with the same substitution are found.

The consequent is deduced.

```
* (describe m4 m1)
(M4! (CLASS AIRCRAFT)
      (MEMBER CS-TJF))
(M1! (CLASS COMMERCIAL-AIRCRAFT)
      (MEMBER CS-TJF))
(M4! M1!)
```

Results are described.
CS-TJF is both a commercial
aircraft (a statement
that was asserted) and an
aircraft (a statement that was
derived).

Example of path-based inference. The following example is based on the network presented in Figure 5.18.

```
* (define member class subclass superclass)
(MEMBER CLASS SUBCLASS SUPERCLASS)

* (describe
  (assert member cs-tjf
    class commercial-aircraft))
(M1! (CLASS COMMERCIAL-AIRCRAFT)
      (MEMBER CS-TJF))
(M1!)

(describe
  (assert subclass commercial-aircraft
    superclass aircraft))
(M2! (SUBCLASS COMMERCIAL-AIRCRAFT)
      (SUPERCLASS AIRCRAFT))
(M2!)

* (define-path
  class
  (compose class subclass- ! superclass))
CLASS implied by the path
(COMPOSE CLASS SUBCLASS- ! SUPERCLASS)
CLASS- implied by the path
(COMPOSE SUPERCLASS- ! SUBCLASS CLASS-)
```

Arcs are defined.

CS-TJF is a commercial aircraft
and commercial aircrafts
are aircrafts.

A path is defined.

```

* (describe
  (deduce member $what class aircraft))

I wonder if
((P1 (CLASS (AIRCRAFT)) (MEMBER V1)))
holds within the BS defined by context
DEFAULT-DEFAULTCT

I know
((M1! (CLASS (COMMERCIAL-AIRCRAFT))
      (MEMBER (CS-TJF))))

(M3! (CLASS AIRCRAFT)
      (MEMBER CS-TJF))

(M3!)

```

Notice that node M3! was not in the original network. This node has just been created by path-based inference. This was done by the network match function, when looking for nodes that match the pattern (P1 (CLASS (AIRCRAFT)) (MEMBER V1)).

6.3 Other kinds of semantic networks

6.3.1 Conceptual graphs

Conceptual Graphs correspond to a family of semantic networks, based on the work of the mathematician Charles S. Pierce (1839–1914). Informally, a Conceptual Graph is a structure of concepts and conceptual relations where every arc links a concept node and a conceptual relation node.

Formally, there is an abstract syntax that specifies conceptual graphs as mathematical structures without making any commitments to any concrete notation or implementation. There are multiple ways to write a Conceptual Graph, among which, a *display form* uses graphical displays; a *linear form* is a representation for conceptual graphs without requiring graphical displays. Figure 5.39 shows the display form of a conceptual graph that represents the sentence “CS-TJF flew from London to Lisbon”.

In Conceptual Graphs, concepts are represented by rectangles — in Figure 5.39, [Go], [Aircraft: CS-TJF], [City: London], [City: Lisbon], and [Flew]. Conceptual relations are represented by circles or ovals — in Figure 5.39, (Agnt) relates [Go] to the agent CS-TJF, (Dest) relates [Go] to the destination Lisbon, and so on.

For n -ary relations, the arcs are numbered from 1 to n ; for binary relations, the number 1 may be replaced by an arrowhead pointing to the relation, and

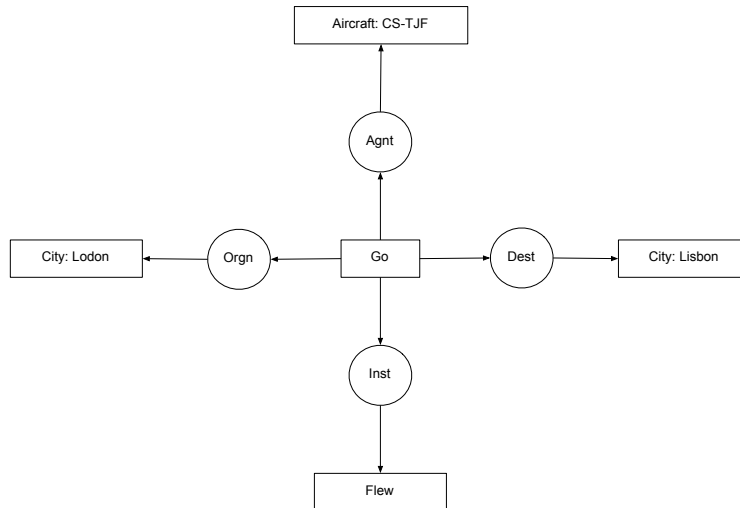


Figure 6.39: CS-TJF flew from London to Lisbon.

the number 2 may be replaced by an arrowhead pointing from the relation. An arrow pointing to a circle may be read “has $a(n)$ ”; an arrow pointing away may be read “which is $a(n)$ ”.

The linear form for Conceptual Graphs is intended as a more compact notation than the display form, but with good human readability. It is exactly equivalent in expressive power to the abstract syntax and the display form. Following is the linear form for Figure 5.39:

```

[Go]-
  (Agnt)->[Aircraft: CS-TJF]
  (Orgn)->[City: London]
  (Dest)->[City: Lisbon]
  (Inst)->[Flew].
  
```

In this form, concepts are represented by square brackets and conceptual relations are represented by parentheses. A hyphen at the end of a line indicates that the relations attached to the concept are continued on subsequent lines.

6.3.2 Semantic primitives

Associated with each representational formalism there is the problem of deciding the vocabulary to be used in the formalism. When we use logic, we have to decide the names of the predicates, functions, and constants to use; if we use SNePS we have to decide the case frames to adopt for our representation.

The work in semantic primitives addresses this problem. A *semantic primitive* is a representational symbol that is used without definition. The idea to use semantic primitives is based on the atomic elements of chemistry or a primitive basis in mathematics (used for an axiomatization). A set of primitives should satisfy the following conditions:

1. *Finiteness.* A small number of primitives should be defined. The number of primitives should be much smaller than the number of entities that can be described through a combination of primitives.
2. *Comprehension.* The set of primitives must be able to describe all entities of the domain.
3. *Independence.* Primitives should not be definable in terms of other primitives of the same theory (the primitives correspond to a minimal set).
4. *Canonicity.* The analysis of an entity of the domain, in terms of the primitives, should be unique and not ambiguous (existence of a canonical form).

One of the approaches to semantic primitives, Conceptual Dependency, provides a representation for actions, using a small number of primitives. Each primitive is associated with a certain number of inferences.

In order to achieve a canonical form, Conceptual Dependency use two types of classifications: the classification of concepts into categories (for example, actions, location in space and time) and the grouping of concepts into conceptualizations, each one of them associated with a certain number of cases.³⁰

Concepts are classified in the following categories:

³⁰In Fillmore's sense.

1. *Picture Producers* (PPs), that correspond to physical objects. for example, John, book, cat. Special cases of PPs correspond to forces of Nature, for example, the wind.
2. *Actions* (ACTs), that correspond to the primitive actions that can be performed by an agent. There are eleven primitive actions, classified into four groups:
 - (a) *Physical ACTs*. This group contains the following actions, described with their associated meanings:
 - i. PROPEL. To apply a force to a physical object;
 - ii. MOVE. To move a body part;
 - iii. INGEST. To transfer something inside a living being;
 - iv. EXPEL. To transfer something from inside a living being;
 - v. GRASP. To physically grasp an object.
 - (b) *ACTs characterized by changing states*. This group contains the following actions, described with their associated meanings:
 - i. PTRANS. To modify the location of a physical object;
 - ii. ATRANS. To modify an abstract relation, for example, change of ownership.
 - (c) *ACTs used as instrument for others*. This group contains the following actions, described with their associated meanings:
 - i. SPEAK. To produce sounds;
 - ii. ATTEND. To capture a stimuli.
 - (d) *mental ACTs*. This group contains the following actions, described with their associated meanings:
 - i. MTRANS. To transfer information;
 - ii. MBUILD. To construct information from previous information.

Each ACT entails a certain number of inferences. For example, X PTRANSed Y FROM W to Z, gives rise to the following three inferences: (1) Y is located in Z; (2) Y is no longer in W; (3) If (Z = X) or (Z is human and asked for the PTRANS) the Z is happy.

3. *Locations* (LOCs). Every physical action has a location that modifies the conceptualization of the action.
4. *Times* (Ts). Every action has an associated time.

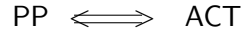


Figure 6.40: PPs can act.



Figure 6.41: PPs may be described by attributes.

5. *Action Aiders* (AAs). Modify the actions, there are few AAs. For example, to throw has an associated velocity.
6. *Picture Aiders* (PAs). Represent attributes of objects, for example, color and size. These attributes have associated values.

In order to represent knowledge, concepts are grouped into conceptualizations. Each conceptualization has an action, an actor (the agent that executes the action), an object (the thing over which the action applies), a recipient (that receives the result of the action), a direction, and a state.

Figures 5.40 through 5.43 show some of the ways of combining conceptual categories. The arrows used in these figures correspond to labelled arcs in the semantic network. The legal ways of combining the concepts correspond to the syntax of the network; each conceptualization has a meaning that is part of the semantics of the network.

Figure 5.5, presented earlier in this chapter, shows the use of primitives and conceptualizations in order to produce the conceptual dependency representation of “John threw a stone to the cat and killed him”.

Conceptual Dependency is a form of a semantic network. Whereas in tra-

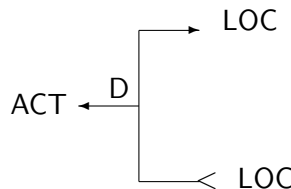


Figure 6.42: ACTs have a direction.

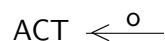


Figure 6.43: ACTs may have objects.

ditional semantic networks the primitives are structural and are used to agglomerate concepts (in SNePS these primitives are defined by the user, with the exception of those needed to represent logical connectives and quantifiers), in the conceptual dependency the primitives are both structural and conceptual, and they are used to create more complex concepts.

6.4 Concluding remarks

Semantic networks, as a knowledge representation formalism, present both advantages and disadvantages.

Their main advantage is the possibility of explicitly stating associations between concepts. In this way, all the concepts that are related with another concept are linked to it. In this way we can define structured objects.

The main disadvantage of semantic networks is that, if care is not taken with the semantics of the representation, the network may generate wrong inferences, since the interpretation of the semantic structure depends on the program that manipulates it. In other words, if distinctions are not made in the representation and treated correctly by reasoning algorithms, then unexpected consequences will arise from inconsistencies between interpretations assumed by the person that is building information into the network and interpretations assumed by the designers of the algorithms that operate on the representations.

6.5 Bibliographic notes and historical remarks

Semantic networks were first proposed, as a knowledge representation formalism in AI in 1966, by M.R. Quillian (reprinted in [Quillian 68]), who was influenced by the work of Bertram Raphael two years earlier (reprinted in [Raphael 68]). A formalism similar to semantic networks was proposed in the late 19th century by the mathematician Charles S. Peirce [Roberts 73].

The idea of a semantic network — that is, of a network of associatively linked concepts — is much older. According to [Anderson and Bower 73, p. 9], this idea can be traced all the way back to Aristotle.

By the end of the 1960s and the beginning of the 1970s, there was a growing interest in semantic networks. As semantic networks started to address more complex problems and tried to represent a larger scope of domains, the explanation of the meaning of the structures used became more and more difficult. In the beginning of the 1970s researchers attempted at precisely defining the meaning of the representations in semantic networks (the “semantics” of semantic networks). Among the important papers in this area are [Woods 75] and [Shapiro 71]. The paper by [Brachman 79] presents a good summary of these attempts. The discussion presented in Section 5.1 is based on the work of [Woods 75]. The book by [Cercone and McCalla 87a] presents a collection of papers on knowledge representation, discussing fundamental issues of semantic networks as well as applications.

An excellent account of semantic networks can be found in [Lehmann 92b] and in [Sowa 00].

SNEPS has been developed since 1971 by Stuart C. Shapiro and its group. SNEPS has undergone several versions, the most important of which are SNEPS 1.0 [Shapiro 79a], [Shapiro and Rapaport 87] and SNEPS 2.1 [Shapiro and Martins 90]. A detailed description of the historical evolution of SNEPS is presented in [Shapiro and Rapaport 92]. The *uniqueness principle* was introduced by [Maida and Shapiro 82]. A complete description of the case frames used in SNEPS is presented in [Shapiro and Rapaport 87] and [Shapiro *et al.* 93]. A detailed discussion about the disadvantages of the standard connectives and the reasons for the introduction of SNEPS’s non-standard connectives is presented in [Shapiro 79b].

Active connection graphs were introduced by [McKay and Shapiro 81]. SNEPS’s inference engine, called MULTI, was developed by [McKay and Shapiro 80]. *Path-based inference* in SNEPS is described on [Shapiro 78], [Srihari 81], and [Cravo and Martins 90].

The work of Charles S. Pierce has deeply influenced the development of Conceptual Graph. Conceptual Graphs were developed by John Sowa [Sowa 84, 91, 00]. A description of Pierce’s work that influenced Conceptual Graphs can be found in [Roberts 73].

The most important approaches to semantic primitives were done by Schank

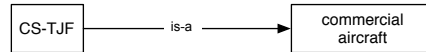


Figure 6.44: Impossible representation in SNePS.

and his research group, at Yale University [Schank 75], [Schank and Abelson 77], [Schank and Riesbeck 81], and by Wilks, in the University of Essex [Wilks 73, 77].

6.6 Exercises

1. Explain the distinction between assertional and structural arcs. Explain the meaning of each of these types of arcs and provide networks that use each one of them.
2. Consider the alternative representations for class membership shown in Figures 5.15 and 5.20.
 - (a) Discuss the advantages and drawbacks of each of them in what respects the expressive power of the representation.
 - (b) Discuss the advantages and drawbacks of each of them regarding the possibility of defining rules that define the properties of the membership relation.
3. SNePS does not allow the representation shown in Figure 5.44 for the statement “CS-TJF is a commercial aircraft”. Explain the reason for this decision.
4. Explain the difference between the representations of figures 5.21 and 5.44. Why is the representation of Figure 5.21 legal?
5. What does ${}_2\mathbb{X}_1^1\{\alpha, \beta\}$ represent?
6. What does ${}_n\mathbb{X}_0^0\{\alpha_1, \dots, \alpha_n\}$ represent?
7. Using SNePS, represent, in a graphical form, the following statements: Anna knows that John bought a dog. The dog is a German Shepherd. John paid 300 Euros for the dog. Anna also knows that Stu sold his dog. Anna believes that John bought Stu’s dog.

8. Using SNePS, represent, in a graphical form, the following statements: John and Mary are lawyers. Charles believes that all the lawyers are rich. Charles knows John's address but he does not know that he and Mary live together.
9. Using SNePS, represent, in a graphical form, the following statements: John is a professor and earns 20,000 Euros a year. His friends believe that he earns much more. They do not know that he earns as much as a police officer.
10. Represent in SNePS the concept of a reflexive relation.
11. Represent in SNePS the concept of a symmetric relation.
12. Represent in SNePS the concept of an equivalence relation.
13. In SNePS, there is a non-standard connective called *numerical-entailment* that generalizes both *and-entailment* and *or-entailment*. *Numerical-entailment*, represented by $i \rightarrow$ (with, $i \in \mathbb{N}$), takes two sets of propositions, the antecedents and the consequents. The proposition

$$\{\alpha_1, \dots, \alpha_n\} i \Rightarrow \{\beta_1, \dots, \beta_m\}$$

($1 \leq i \leq n$) states that the conjunction of i antecedents $\alpha_1, \dots, \alpha_n$ entails the conjunction of the consequents $\beta_1, \dots, \beta_m$.

- (a) Explain why *numerical-entailment* generalizes both *and-entailment* and *or-entailment*.
- (b) Find out situations where the use of the *numerical-entailment* would be appropriate.
14. In SNePS, there is a non-standard connective called *thresh*. *Thresh*, represented by ${}_n\Theta_i^j$ (with, $i, j, n \in \mathbb{N}$) takes as arguments a set of propositions. The proposition

$${}_n\Theta_i^j\{\alpha_1, \dots, \alpha_n\}$$

($1 \leq i \leq j \leq n$) means that either fewer than i or more than j of its n arguments are true. Find out situations where the use of the *thresh* would be appropriate.

15. Another non-standard connective existing in SNePS, is the *numerical quantifier*. The *numerical quantifier* is represented by ${}_n\exists_i^j$ (with, $i, j, n \in \mathbb{N}$). The proposition

$${}_n\exists_i^j x [\{\alpha_1(x), \dots, \alpha_k(x) : \beta(x)\}]$$

$(1 \leq i \leq j \leq n)$ means that there are n constants that when replaced by x make $\alpha_1(x) \wedge \dots \wedge \alpha_k(x)$ true. However, at least i and at most j of them also satisfy $\beta(x)$. Numerical quantification enables us to represent propositions such as “there are exactly two numbers such that $x^2 + 4 = 4x$ ”; “there are at least two numbers such that $z + 2 < 6$ ”.

Using the family tree represented in Figure 2.6, write a proposition using the numerical quantifier that will allow to infer that Jackie Bouvier is not Lisa Simpson’s mother. Hint: remember that every person has exactly one mother and that Marge B. Simpson is Lisa Simpson’s mother.

16. Represent in SNePS the statements of the following puzzle (from [Summers 72]):

Freeman knows five women: Ada, Bea, Cyd, Deb and Eve.

- (a) *The women are in two age brackets: three women are under 30 and two women are over 30.*
- (b) *Two women are teachers and the other three women are secretaries.*
- (c) *Ada and Cyd are in the same age bracket.*
- (d) *Deb and Eve are in different age brackets.*
- (e) *Bea and Eve have the same occupation.*
- (f) *Cyd and Deb have different occupations.*
- (g) *Of the five women, Freeman will marry the teacher over 30.*

Using natural deduction, try to answer the following question: *Who will Freeman marry?*

Chapter 7

Frames

“When I have heard some slight indication of the course of events, I am able to guide myself by the thousand of other similar cases which occur to my mind”

Connan Doyle, *The Red-headed Leage*
(Sherlock Holmes)

Frames were introduced, as a knowledge representation formalism, to simulate the organization of knowledge in “chunks” corresponding to stereotyped situations. A frame could be viewed as a skeleton of a structure which represents a complex object or situation and which corresponds to an understanding of a typical representation of that object or situation.

A frame is activated when we encounter the specific situation described by the frame. The frame is used to supply interpretations and to make predictions about that situation.

For example, when we enter the office of a computer scientist, we activate frames that enable us to predict what we expect to find:

- Walls, floor, and ceiling, and, usually, one or more windows (Figure 6.1).

These predictions are made because we are entering a *room*. The office of a computer scientist is a specialization of a frame corresponding to a room.

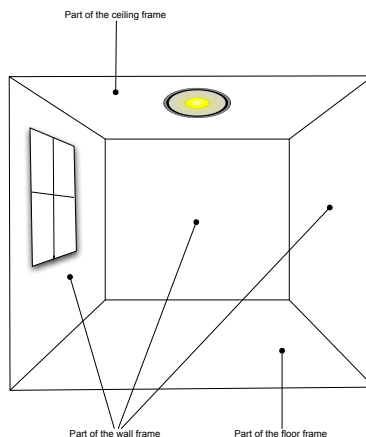


Figure 7.1: Room frame.

We would be very surprised, if, when opening the door, were found ourselves on a beach or in a room with no ceiling;

- A desk, chairs, and one or more bookshelves (Figure 6.2).

These predictions are made because we are entering a room corresponding to an *office*.

Again, we would be surprised in the face of a dishwasher or a bath-tub. The existence of these objects would not surprise us if we would be entering a room whose type was unknown to us.

- A computer and computer science books (Figure 6.3).

These predictions are made because we are entering a *computer-scientist office*.

We would be surprised to find, instead, medical instruments (these would be associated with the office of a medical doctor).

In a similar way, we would make predictions about what to expect to find on the walls, floor or ceiling.

Each of the concepts involved in this example — room, office, computer-scientist office, wall, floor, ceiling, and so on — corresponds to a frame, which, in turn, may refer to other frames. For example, the frame corresponding to a room refers to the frames corresponding to wall, ceiling and floor.

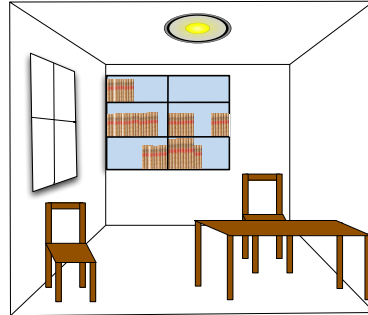


Figure 7.2: Office frame (specialization of room).

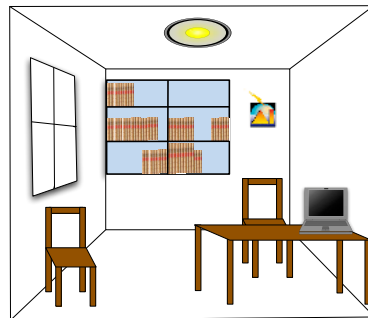


Figure 7.3: Computer-scientist office frame (specialization of office).

The organization of knowledge using frames reduces the computational effort to process knowledge, because, once a frame is selected, all the relevant knowledge is associated with the frame itself. A *frame* is, thus, similar to an object in an object-oriented language — the object agglomerates all operations that manipulate it.

7.1 Basic concepts

Frames correspond to chunks of knowledge describing conceptual entities with associated descriptions. Frames organise knowledge in a *hierarchical* way.

Going back to our example about aircrafts, we show, in Figure 6.4, part of

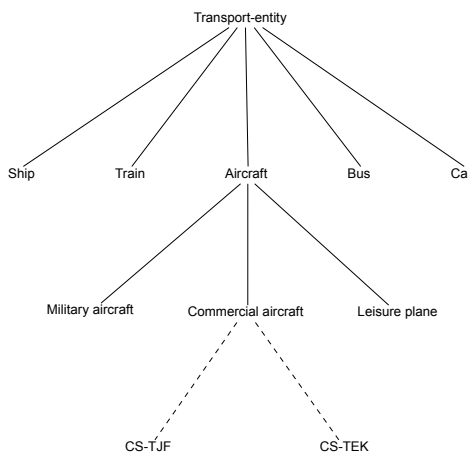


Figure 7.4: Classes and members of transportation entities.

a frame hierarchy corresponding to information pertaining transportation entities. Each name (identifier) in this figure corresponds to a frame.

There are frames that correspond to *classes* and there are frames that correspond to *instances*. Solid-line connections between frames correspond to class/subclass relations; dashed-line connections between frames correspond to class/member relations.

For example, **Commercial-aircraft** is a subclass of **Aircraft** (which contains two other subclasses, **Military-aircraft** and **Leisure-plane**); both **CS-TJF** and **CS-TEK** are instances of **Commercial-aircraft**.

Each frame is associated with a certain number of *attributes* (or *slots*). For example, a **Transport-entity** may be characterised by a transport type, **t-type** (either passengers or goods), a transport place, **t-place**, (either land, sea, or air), an **owner**, and a date of construction, **date**. These attributes may have associated values; it is also possible for the value associated with an attribute to be *unknown*.

Under this perspective, we may be tempted to consider a frame hierarchy as a special type of semantic network, where the identifiers of the frames correspond to nodes. Nodes are either linked by an arc labelled **is-a** for class/subclass relation or by an arc labelled **instance** for a member/class relation. Attributes may correspond to labelled arcs that connect the nodes in the hierarchy with their corresponding values.

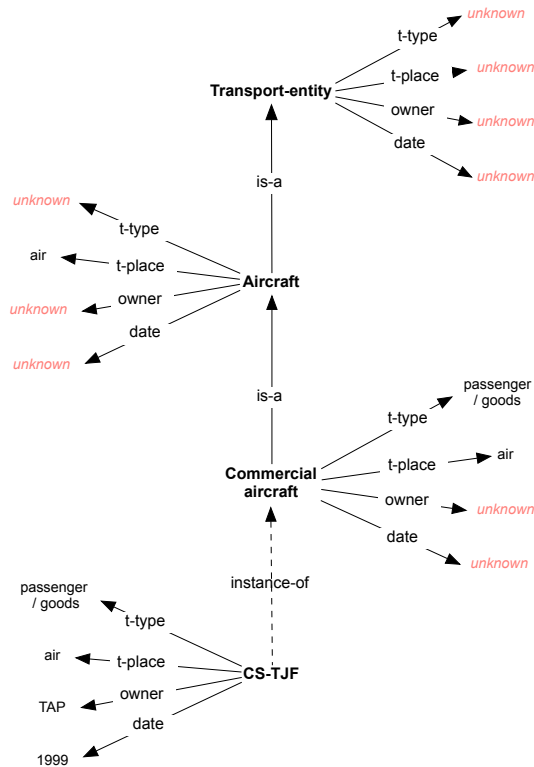


Figure 7.5: Attributes related to Transport-entity.

We show in Figure 6.5, the information associated with part of the frame hierarchy of Figure 6.4. We can see that all attributes of **Transport-entity** have an *unknown* value; however, the frame **Aircraft** has the attribute **t-place** with value **air**. At first, frames seem to differ from semantic networks only in the sense that a frame “contains” all the information that is related to it.

A useful characteristic of hierarchical representations, such as frames, is that they provide a natural structure for property inheritance. For example, since a **Commercial-aircraft** is an **Aircraft**, then those properties that hold for aircrafts should be “inherited” by commercial aircrafts and need not be explicitly specified for the **Commercial-aircraft** frame; instead they may be derived via an inheritance procedure. This greatly reduces the amount of information that must be explicitly stored, although there may be an increase in the time required to access knowledge through the inheritance

mechanism.

Property inheritance introduces a first difference between frames and semantic networks — in frames, the attributes, and their values, are propagated “down” to all other frames of the hierarchy, following the *is-a* and *instance* links. This means that the attributes of a particular frame may either be “inherited” from the frames higher up in the hierarchy or may correspond to attributes that do not exist in higher frames but are introduced in the class that corresponds to the frame.

The inheritance mechanism leads into the idea of *defaults*. Another aspect associated with frames is that attributes may have a “default value” which is the value that is taken whenever no explicit value is specified. For example, since the most common transportation entity is a car, we may state that, if nothing is said about a transportation entity, we may assume that its transport place is *land* — thus “land” is the *default value* for the transport place of a transportation entity.

In addition, the values of a particular attribute need not necessarily be filled with a value but, rather, they may indicate a procedure to run to obtain a value — this is known as *procedural attachment* or *attached procedure*. Procedural attachment is useful when there is a high cost associated with computing a particular value, when the value changes with time, or when the expected access frequency is low — instead of computing the value for each instance, the values are computed only when needed or when some other values change.

In Figure 6.6 we show a detailed view of two frames corresponding to the classes *Transport-entity* and *Aircraft*. This figure is based on the hierarchy shown in Figure 6.5 and uses the notation that is common to the graphical representation of frames — a frame is represented by a box; the name of the frame is shown on the left-hand side, above the box; the attributes and their corresponding values are shown inside the box.

An *Aircraft* is characterized by the following attributes:

- *Transport-type*, which is inherited from the class *Transport-entity*, and whose value may either be *passenger* or *goods*.
- *Transport-place*, which is inherited from the class *Transport-entity* and whose value may be either *land*, *sea*, or *air*.

The attribute *Transport-place* has *land* as default value, that is, if we

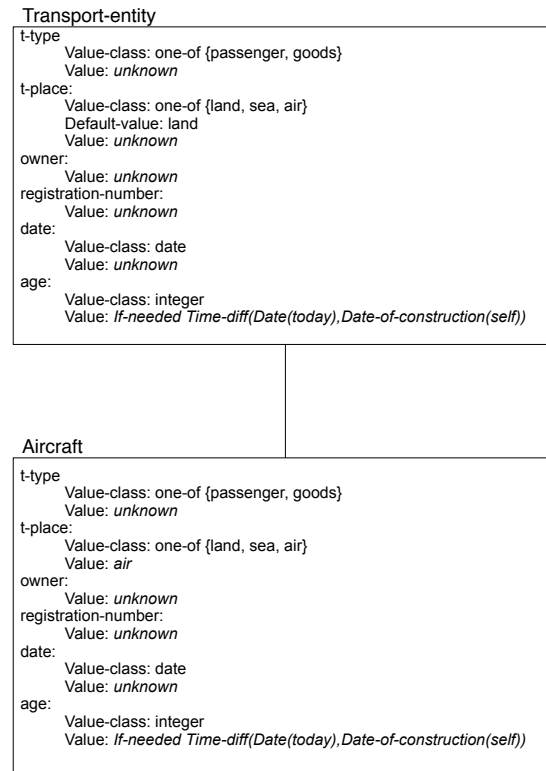


Figure 7.6: Two frames in the transport hierarchy.

do not know the transport place of a specific transport entity, we may assume that it is land.

The attribute **Transport-place** in the **Aircraft** frame has *air* as its value. In this case, the default value coming from **Transport-entity** is overridden.

- **Owner**, which is inherited from the class **Transport-entity**.
- **Registration-number**, which is inherited from the class **Transport-entity**.
- **Date** (of construction), which is inherited from the class **Transport-entity** and whose value must be a **date**.
- **Age**, which is inherited from the class **Transport-entity** and whose value must be an **integer**.

The attribute **Age** is associated with a procedure (indicated by the name *if-needed*) that is executed when there is need to know the age of a particular transport entity. This procedure computes the time-difference between today's date and the date of construction of the corresponding frame.

In summary, *frames* correspond to a representation formalism, in which knowledge is hierarchically organized in *classes* and *instances*. Classes and instances are defined by a number of *attributes* (or *slots*) with certain values — frames are also known as *slot-and-filler* structures. Attributes may be associated with a value, a default value, a possible class of values, or may be associated with a procedure that is executed in certain situations. A frame “encapsulates” all the information related to it.

7.2 Reasoning with frames

There are three kinds of reasoning associated with a frame-based representation, property inheritance, procedural attachment, and rule-based reasoning. In this section we discuss the main concepts associated with each of these kinds of reasoning.

7.2.1 Property inheritance

Property inheritance is the main (sometimes the only) inference mechanism used in representations based on frames.

Property inheritance is a form of inference that explores a hierarchical structure of concepts and that consists in “transporting” the properties (attributes and their values) of the concepts in the hierarchy to the concepts that are located below them in the hierarchy (more specific concepts). We say that the more specific concepts *inherit* the characteristics of the more general concepts — the concepts located above them in the hierarchy.

Inheritance is an *inference rule* that states that the attributes and values of a frame corresponding to a class are also attributes and values of its *is-a* and *instance* descendants.

There are several issues to be considered in inheritance. The issue that is most relevant to frame systems relates to the structure of the hierarchy that

is considered. Here, there are two alternatives at stake:

- *Single inheritance.* If the hierarchy corresponds to a tree, then each of the concepts represented has *exactly* one concept immediately above it (with the exception of the root of the tree). A hierarchy of this type is shown in Figure 6.4. This kind of structure gives rise to *single inheritance*.

With single inheritance, each concept only inherits from a single concept. In this case, the computation of the properties to be inherited is simple, since this can be done by looking up in the hierarchy.

- *Multiple inheritance.* If the hierarchy corresponds to an acyclic graph, each concept represented in the hierarchy may have more than one concept immediately above it. In this case, it inherits from more than one concept.

This kind of structuring appears very often, when we represent multiple classifications of the same entity. In Figure 6.7 we show another view of the hierarchy of Figure 6.4, where we mix the hierarchical classification of transportation entities with a classification with respect to the type of use of a transportation entity — either public or private. The class **Transport-wrt-use** (read “transport *with respect to* use”) states that there are two classes in this classification, **Public** and **Private** transport. The figure also shows that **Commercial-aircrafts** are for public use.

In our example, the **Commercial-aircraft** frame inherits the attributes of **Aircraft**, as before, but also inherits attributes of **Public**, such as **transport-cost** and **transport-frequency**.

The existence of multiple tangled hierarchies gives rise to *multiple inheritance*. In multiple inheritance, the computation of the properties that are inherited can be complicated, since concepts inherit information from more than one concept. When this happens, the inherited values may be in contradiction.

In frame systems where a frame may inherit properties from multiple frames, several alternatives were studied in what concerns the way the values of the attributes are propagated. Some of these are: the union of the values of the attributes inherited, its intersection, the inheritance from the closest frame, and so on. This aspect is further discussed in Section 6.3.

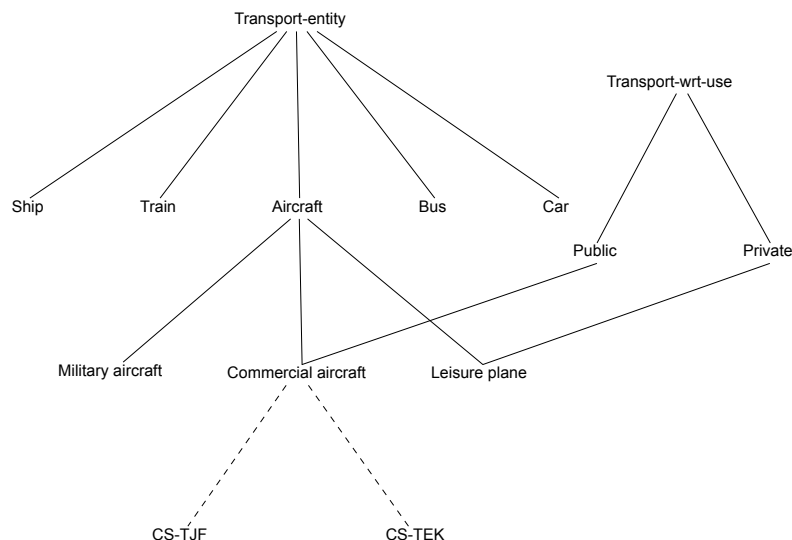


Figure 7.7: Tangled hierarchy of transportation entities.

7.2.2 Procedural attachment

Frames were the first knowledge representation formalism to mix a declarative representation with a procedural representation — they were the first formalism to conciliate the declarative vs. procedural controversy presented in Section 1.6. This approach, called *procedural attachment*, is used whenever attributes are associated with procedures.

There are two ways of activating the procedure associated with an attribute. The way these procedures are activated provides a distinction between servants and demons:

- *Servants*¹ are procedures that are executed when the value of the attribute is inspected or queried. In this way, servants correspond to a form of backward inference.

For example, based on the frame hierarchy presented in Figure 6.5, we can imagine that each transport entity has an attribute **Age** that corresponds to the age of the particular entity to which it is associated. Under this perspective, it does not make sense to associate a

¹Also called *if-needed* procedures.

constant value with this attribute, since it will change over time. Instead, this attribute should be associated with a servant that will be executed whenever we need to know the age of the transport entity. This procedure, whenever executed, computes the current age of the transport entity represented by the frame. This was already shown in Figure 6.6.

- *Demons*² are procedures that are automatically executed when the value of the attribute changes. In this way, demons correspond to a form of forward inference.

Again, referring to the hierarchy shown in Figure 6.5, we know that transport entities have an owner. The owner determines the place where the **Registration-number** is obtained. It should be clear that, whenever a transport entity changes ownership, the **Registration-number** should be updated. This can be accomplished by associating a demon with the attribute that corresponds to the **Registration-number** that will be automatically executed whenever the owner changes.

7.2.3 Rule-based reasoning

Another type of reasoning used in frame systems, but not so frequently as inheritance or procedural attachment, is rule-based reasoning. When rule-based reasoning is used in frame systems, what is typically used is a production system. A *production system* is composed of:

1. A *set of production rules*. There are many ways to write the production rules. They are all roughly equivalent. They have a *left-hand-side* (also called the *antecedent*) and a *right-hand-side* (also called the *consequent*). We consider rules of the form:

$$\text{if } \langle \text{condition} \rangle^+ \text{ then } \langle \text{action} \rangle^+$$

In this expression, $\langle \text{condition} \rangle$ is a predicate (an expression that has a logical value) and $\langle \text{action} \rangle$ is an action (it may correspond to the addition of information, to the removal of information, or to the execution of procedures).

The conditions correspond to the *left-hand-side* of the production rule; the actions correspond to the *right-hand-side* of the production rule.

²Also *if-added* procedures.

If all conditions of a production rule are true in a certain situation, the production rule is *applicable*. If the production rule is *applied*³ (also said that the production rule was *fired*), the the actions are executed.

2. A *working memory*, sometimes also called a *context* that contains declarative statements. The statements in the working memory are checked for the applicability of the production rules and are affected by their execution.

In frame systems, the working memory is composed of the existing frames, their attributes, and their values.

3. An *interpreter*, whose job is to decide what to do next. The interpreter is responsible for finding the production rules that are applicable in a given situation (that is, for selecting the production rules whose antecedents are satisfied), for selecting which of the rules should be applied (in case there is more than one applicable rule), and for executing the selected production rule.

The following are some possible strategies for resolving the application of multiple rules:

- (a) *Textual order*. This strategy assumes that the production rules are organised in some sequential way, for example, the order in which they were introduced in the system. In this case, the first rule that is applicable is the rule that is applied.
- (b) *Refractoriness*. This strategy tries not to apply the same rule twice in a row. In this way, cycles of the application of the same rule are avoided.
- (c) *Recency*. The rule that uses information that has more recently been added to the working memory is preferred over the other rules. This has the advantage of focusing the inference around new information.
- (d) *Specificity*. This strategy favours the application of more specific rules (that is either rules that have more conditions or rules that reference a smaller number of variables).

³The production rule is applicable and a decision was taken to *execute* the actions of the production rule.

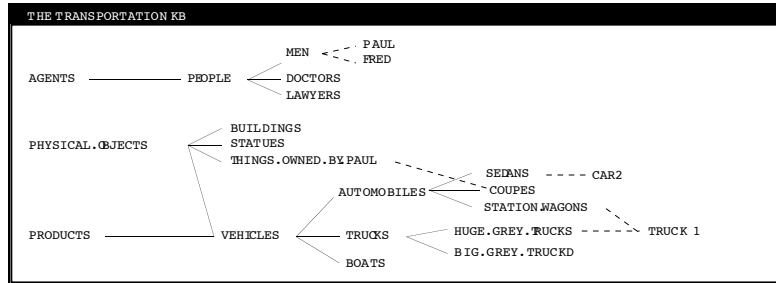


Figure 7.8: Graphical representation of a knowledge base in KEE.

7.3 KEE: A case study

As an example of a system based on frames, we discuss KEETM (Knowledge Engineering Environment)⁴ developed by IntelliCorp. KEE has been used in the development of several commercial expert systems.

7.3.1 Basics

To represent knowledge in KEE the following concepts are used:

- *Knowledge base.* A knowledge base is a collection of frames pertaining to a particular representation domain. These frames are organized in one or more frame hierarchies. For example, the representation of Figure 6.7 in KEE would be placed in a knowledge base containing two hierarchies.

In Figure 6.8, we show an example of a knowledge base in KEE.⁵

- *Unit.* A unit corresponds to a frame. There are two kinds of units, classes and instances.
 - *Classes* correspond to generic entities (classes of entities) and may have both superclasses and subclasses.
 - *Instances* correspond to individual entities.

⁴KEE is a trademark of IntelliCorp.

⁵From [Fikes and Kehler 85, p. 906].

```

(OUTPUT)TheAUTOMOBILESunit
Unit: AUTOMOBILES in knowledge base
transportation
Superclasses:VEHICLES
Subclasses:STATION.WAGONS, COUPES, SEDANS
Member of (CLASSES in KB GENERICUNITS)

MemberSlot:*COLORfromPHYSICAL.OBJECTS
Inheritance: OVERRID.VALUES
Cardinality.Max: 1
Values: Unknown

MemberSlot:*HEIGHTfromPHYSICAL.OBJECTS
Inheritance: OVERRID.VALUES
ValueClass: INTEGER
Cardinality.Min: 1
Cardinality.Max: 1
Comment: "Height in inches."
Values: Unknown

MemberSlot:*LENGTHfromPHYSICAL.OBJECTS
Inheritance: OVERRID.VALUES
ValueClass: INTEGER
Cardinality.Min: 1
Cardinality.Max: 1
Comment: "Length in inches."
Values: Unknown

MemberSlot:*LOCATIONfromPHYSICAL.OBJECTS
Inheritance: OVERRID.VALUES
Cardinality.Min: 1
Cardinality.Max: 1
Values: Unknown

MemberSlot:*OWNERfromPHYSICAL.OBJECTS
Inheritance: OVERRID.VALUES
ValueClass: AGENTS
Cardinality.Max: 1
Values: Unknown

```

Figure 7.9: Units and slots in a class frame in KEE.

- *Slot.* A slot corresponds to an attribute. Slots describe particular aspects of a frame. There are two types of slots, member slots and own slots.
 - *Member slots* are associated with classes and represent characteristics of the elements of the class. For this reason, member slots are passed down to subclasses and members.

For example, if we had a unit in KEE corresponding to the class “Aircraft” of Figure 6.6, the slot “Owner” would be a member slot, since it corresponds to a characteristic of the elements of this class. This slot would be propagated to all subclasses and to all members of “Aircraft”.

In Figure 6.9, we show an example of units and slots in a class frame in KEE.⁶
 - *Own slots* describe attributes that are specific to the unit where they appear (either a class or a member) and are not passed down to the subclasses and members. Instances only have own slots.

⁶From [Fikes and Kehler 85, p. 906].

The CAR2 unit	
Unit: CAR2 in knowledge base	
transportation	
Member of: SEDANS	
OwnSlot: COLOR from PHYSICAL.OBJECTS	
Inheritance: OVERRIDE.VALUES	
Values: Unknown	
OwnSlot: HEIGHT from PHYSICAL.OBJECTS	
Inheritance: OVERRIDE.VALUES	
ValueClass: INTEGER	
Cardinality.Min: 1	
Cardinality.Max: 1	
Comment: "Height in inches."	
Values: Unknown	
OwnSlot: LENGTH from PHYSICAL.OBJECTS	
Inheritance: OVERRIDE.VALUES	
ValueClass: INTEGER	
Cardinality.Min: 1	
Cardinality.Max: 1	
Comment: "Length in inches."	
Values: Unknown	
OwnSlot: LOCATION from PHYSICAL.OBJECTS	
Inheritance: OVERRIDE.VALUES	
Cardinality.Min: 1	
Cardinality.Max: 1	
Values: Unknown	
OwnSlot: OWNER from PHYSICAL.OBJECTS	
Inheritance: OVERRIDE.VALUES	
ValueClass: AGENTS	
Cardinality.Max: 1	
Values: Unknown	
OwnSlot: PRODUCER from PRODUCTS	
Inheritance: OVERRIDE.VALUES	

Figure 7.10: Units and slots in an instance frame in KEE.

Returning to the representation in Figure 6.6, we could define an own slot in the unit “Aircraft” that would contain the largest aircraft, for example the Airbus A-380. This slot describes a property that is associated with the *class* of Aircraft itself, not with the individual aircraft that belong to the class.

In Figure 6.10, we show an example of units and slots in an instance frame in KEE.⁷

- *Facets*. Facets characterize particular aspects of attributes. Facets are in a sense “attributes of attributes”.

Examples of facets that are pre-defined in KEE are

- *inheritance role* that describes the way the value of the attribute is propagated to the units down in the hierarchy;
- *value class* that specifies the kinds of values that may appear in the slot;
- *values* that represents the value (or values) associated with the slot;

⁷From [Fikes and Kehler 85, p. 906].

- *maximal cardinality* that specifies the maximum number of values that the slot may have;
- *minimal cardinality* that specifies the minimum number of values that the slot may have.

In KEE a given unit may belong to more than one frame hierarchy and thus KEE has to deal with the problem of multiple inheritance. To handle this problem, each slot in KEE has a facet that describes how the values of the slot are propagated to the units that are below it in the hierarchy. The possible values for this facet, among others, are:

1. *Override*. This qualification is only applicable to attributes that may only have one value. In this case, the value of the attribute that has specifically been assigned to a particular unit, takes precedence over any inherited value. This means that whenever a value is assigned to a particular slot, it overrides the inherited value.
2. *Override.Values*. This qualification is applicable to attributes that may have more than one value. It is similar to *Override*.
3. *Union*. In this case, the value of the slot is the union of all the values inherited from the superclasses with an eventual local value.
4. *Unique*. This qualification blocks inheritance.
5. *Method*. An arbitrary procedure may be specified to compute the result of inheritance.

7.3.2 TELLANDASK

TELLANDASK is a language available within KEE that enables the reference to units and to their attributes and values. TELLANDASK is used by the production rules of the reasoning component of KEE and may also be used to add or retract information to/from the knowledge bases.

TELLANDASK supplies a set of functions to interface with the knowledge base that enable either to add information to the knowledge base or to retrieve information that is stored there. These functions rely on the use of LISP-like expressions that reference entities or relations in the knowledge bases.

We do not intend to specify the complete set of expressions allowed by TELLANDASK, but rather to give an idea of what can be expressed with the language. TELLANDASK expressions can be classified into three groups:

1. *Basic expressions.* These expressions allow to refer to relations between units, to attributes of units, and to facets of attributes.

Some of the relevant basic expressions are:

- (a) *Class/subclass relation.* This relation is expressed by

(SUBCLASS.OF <subc> <superc>),⁸

where <subc> and <superc> correspond to units in KEE. The expression states that <subc> is a subclass of <superc>.

For example, taking into account the hierarchy represented in Figure 6.4, the basic expression

(SUBCLASS.OF Commercial-aircraft Aircraft)

states that “Commercial-aircraft” is a subclass of “Aircraft”.

- (b) *Member/class relation.* This relation is expressed by

(IN.CLASS <mem> <class>),

where <mem> and <class> correspond to units in KEE. The expression states that <mem> is a member of the class <class>.

For example, taking into account the hierarchy represented in Figure 6.4, the basic expression

(IN.CLASS CS-TJF Commercial-aircraft)

states that “CS-TJF” is a member of the class “Commercial-aircraft”.

- (c) *Value of a member slot.* This can be expressed by

(MEMBER.VALUE <slot> <unit> <value>),

where <unit> corresponds to the name of a unit in KEE, <slot> corresponds to the name of a slot in <unit>, and <value> corresponds to the value of that slot.

For example, taking into account the hierarchy represented in Figure 6.6, the basic expression

⁸An alternative to write this expression is (ALL <subc> ARE <superc>). This is called Natural Language form (NL-form, for short). All TELLANDASK expressions have an equivalent NL-form.

(MEMBER.VALUE Transport-place Aircraft Air)

states that the “Transport-place” of “Aircraft” is “Air”.

- (d) *Value of an own slot.* This can be expressed by

(OWN.VALUE <slot> <unit> <value>),

where <unit> corresponds to the name of a unit in KEE, <slot> corresponds to the name of a slot in <unit>, and <value> corresponds to the value of that slot.

For example, the basic expression

(OWN.VALUE Owner CS-TJF TAP)

states that the “Owner” of “CS-TJF” is “TAP”.

- (e) *Minimum cardinality of member slot.* This can be expressed by the relation

(MEMBER.MIN.CARD <slot> <unit> <integer>),

where <unit> corresponds to the name of a unit in KEE, <slot> corresponds to the name of a slot in <unit>, and <integer> corresponds to the minimal cardinality of that slot.

- (f) *Maximum cardinality of member slot.* This can be expressed by the relation

(MEMBER.MAX.CARD <slot> <unit> <integer>),

where <unit> corresponds to the name of a unit in KEE, <slot> corresponds to the name of a slot in <unit>, and <integer> corresponds to the maximal cardinality of that slot.

2. *Composite expressions.* These expressions allow the combination, through logical operators, of basic expressions. Composite expressions are defined in the following way:

- (a) Any basic expression is a composite expression;
- (b) If <comp exp₁> ... <comp exp_n> ($n \geq 2$) are composite expressions, then, (AND <comp exp₁> ... <comp exp_n>) is a composite expression;
- (c) If <comp exp₁> ... <comp exp_n> ($n \geq 2$) are composite expressions, then, (OR <comp exp₁> ... <comp exp_n>) is a composite expression;

- (d) If `<comp exp>` is a composite expression, then, `(NOT <comp exp>)` is a composite expression;
 - (e) If `<comp exp1>` and `<comp exp2>` are composite expressions, then, `(EQUAL <comp exp1> <comp exp2>)` is a composite expression.
3. *Commands.* These expressions allow the addition and retraction of information to the knowledge base, as well as the inspection and inference of the information in the knowledge base. Commands are applied to composite expressions.

There are three commands, **ASSERT**, **RETRACT**, and **QUERY**.

- (a) **(ASSERT <comp exp>)**
If `<comp exp>` is a composite expression, then the command **(ASSERT <comp exp>)** introduces in the knowledge base the statement corresponding to `<comp exp>`;
- (b) **(RETRACT <comp exp>)**
If `<comp exp>` is a composite expression, then the command **(RETRACT <comp exp>)** removes from the knowledge base the statement corresponding to `<comp exp>`;
- (c) **(QUERY <comp exp>)**
If `<comp exp>` is a composite expression (eventually containing free variables),⁹ then the command **(QUERY <comp exp>)** generates backward inference, using the rules described in Section 6.3.3, trying to derive instances of `<comp exp>`.

7.3.3 Reasoning with KEE

KEE provides a rule-based reasoning component. Its inference engine, called **RULESYSTEM**, allows both forward inference and backward inference.

The rules in KEE are a generalization of the production rules presented in Section 6.2.3. Rules in KEE are represented in units (frames), which are organised into a knowledge base — thus, in KEE, there is a special knowledge base that only contains rules. This means that rules may be organized into several hierarchies of classes and instances.

A rule in KEE is defined by the following syntax:

⁹A variable is represented in KEE by an identifier preceeded by “?”.

```

<KEE rule> ::= (IF <cond> THEN <concl> ACTION <act>) |
               (IF <cond> THEN <concl>) |
               (IF <cond> ACTION <act>)

```

were,

```

<cond> ::= <condition> | (<condition>+)

```

```

<concl> ::= <conclusion> | (<conclusion>+)

```

```

<act> ::= <action> | (<action>+)

```

- A <condition> is either a TELLANDASK expression or a LISP form that evaluates to T, NIL, or UNKNOWN. The <condition> is satisfied just in case it evaluates to T.
- A <conclusion> corresponds to a command in KEE.
- An <action> corresponds to a LISP form.

Each rule has its own variables. If the same variable name is used in two rules, it is considered to correspond to two different variables. A rule's variables are assigned values during the evaluation of the rule's <cond>; all variables appearing either in the <concl> or <act> parts of a rule must have values when the rule is applied.

A rule is *invoked* (or *applied*), when its <cond> evaluates to T. In this case, all the <conclusion> in the <concl> part of the rule are asserted and all the <action> in the <act> part of the rule are executed.

The following is an example of a rule in KEE:

<pre> (IF (AND (IN.CLASS ?x Commercial-aircraft) (OWN.VALUE Owner ?x TAP)) THEN (ASSERT (OWN.VALUE Quality ?x Good)) ACTION (set-goal make-trip ?x)) </pre>	<pre> If, a commercial aircraft is owned by TAP, then it has a good quality; we should set as goal to make a trip on it. </pre>
---	--

Inference is started with the following commands:

```

(ASSERT <comp exp> <rule class>)

```

Adds the <comp exp> to the knowledge base and performs forward inference with it, using the rules belonging to the class <rule class>.

(QUERY <comp exp> <rule class>)

Performs backward inference with the query <comp exp>, using the rules belonging to the class <rule class>.

Since rules in KEE are organised in knowledge bases, there is a special class, the *rule class*, that contains all the KEE rules. This enables the definition of some slots in these classes, that are inherited by all the rules, that enable to control the reasoning. Some of these slots are:

- **SELECT.DERIVATION.NODE.** Specifies the type of search that is to be used during inference, for example depth-first search (the default value) or breadth-first search;
- **RESOLVE.CONFLICT.** Specifies conflict resolution strategies regarding the application of the rule.

7.4 Formal semantic of frames

Most frame systems are constructed on the basis of a program with the behaviour that we have been describing, without much concerns about the semantics of the representation. The lack of formal semantics of frames lead to approaches to translate frames into logic. We provide one such approach that uses default logic.

The examples provided alongside with this translation are based on the frames shown in figures 6.4 and 6.6.

1. Each frame is associated with a constant in first-order logic.

For example, there will be constants named *transport-entity*, *aircraft*, *commercial-aircraft*, *CS-TJF*, and *CS-TEK*.

2. Each attribute (either member or own attribute) is associated with a constant in first-order logic.

For example, there will be constants named *t-type*, *t-place*, *owner*, and *age*.

3. Each value is associated with a constant in first-order logic. There is a special constant, *UNK*, for *unknown*.

For example, there will be constants named *passenger*, *goods*, *air*, and *TAP*.

4. The value of each member attribute is related to the corresponding frame by the first-order logic predicate M .

The predicate $M(n, f, v)$ states that in frame f , there is a member attribute whose name is n with the value v .

For example, $M(t-place, aircraft, air)$.

5. The value of each own attribute (attribute that is not propagated by inheritance) is related to the corresponding frame by the first-order logic predicate O .

The predicate $O(n, f, v)$ states that in frame f , there is an own attribute whose name is n with the value v .

For example, $O(largest, aircraft, A-380)$.

6. Besides user-defined own attributes, there are special own attributes, both associated with frames that correspond to classes and with frames that correspond to instances. These attributes relate a frame with other frames directly above or below it in the hierarchy:

- (a) Frames that correspond to classes have the special own attributes *Subclass* and *SubclassOf* that relate the frame with its subclasses and superclasses.

The predicate $Subclass(f_s, f)$ means that f_s is a subclass of f ; the predicate $SubclassOf(f, f_s)$ means that f has f_s as one of its subclasses.

For example,

$Subclass(commercial-aircraft, aircraft),$

$SubclassOf(aircraft, commercial-aircraft),$

$SubclassOf(aircraft, military-aircraft),$

$SubclassOf(aircraft, leisure-plane).$

- (b) Frames that correspond to instances have the special own attributes *Member* and *MemberOf* that relate the frame with its superclasses.

The predicate $Member(m, f)$ means that m is a member of class f ; the predicate $MemberOf(f, m)$ means that f has m as one of its members.

For example,

$Member(CS-TJF, commercial-aircraft),$

$MemberOf(\text{commercial-aircraft}, CS-TJF),$
 $MemberOf(\text{commercial-aircraft}, CS-TEK).$

7. The propagation of the attributes from a class to a subclass is specified by the following default rule:

$$\frac{Subclass(c_s, c) \wedge M(n, c, v) : M(n, c_s, v) \wedge \neg Ex(n, c_s)}{M(n, c_s, v)} \quad (7.1)$$

where the predicate Ex indicates that there is an exception of the propagation of the attribute n to the subclass c_s .

8. The propagation of the values of the attributes of a class to a member (or instance) are specified by the following default rule:¹⁰

$$\frac{Member(e, c) \wedge M(n, c, v) : O(n, e, v) \wedge \neg OwnExc(n, e)}{O(n, e, v)} \quad (7.2)$$

where the predicate $OwnExc$ corresponds to an exception of the propagation of the attribute n to the member e .

9. Exceptions to the propagation of attributes may be specified either by the predicates Ex and $OwnExc$ above or by the explicit indication that a given attribute (both member and own) of a frame can only have one value:

$$\forall n, f, v_1 [M(n, f, v_1) \rightarrow \forall v_2 [v_1 \neq v_2 \rightarrow \neg M(n, f, v_2)]] \quad (7.3)$$

$$\forall n, f, v_1 [O(n, f, v_1) \rightarrow \forall v_2 [v_1 \neq v_2 \rightarrow \neg O(n, f, v_2)]] \quad (7.4)$$

10. In order to enable reasoning about membership and subclass relations, the following two *wffs* should also be considered:

$$\forall e, c_1, c_2 [(Member(e, c_1) \wedge Subclass(c_1, c_2)) \rightarrow Member(e, c_2)] \quad (7.5)$$

$$\forall c_1, c_2, c_3 [(Subclass(c_1, c_2) \wedge Subclass(c_2, c_3)) \rightarrow Subclass(c_1, c_3)] \quad (7.6)$$

11. Restrictions imposed upon the values of attributes are translated to first-order formulas. For example, the fact that t-type is either passenger or goods is translated to:

¹⁰Remember that a frame that corresponds to an instance only has own attributes.

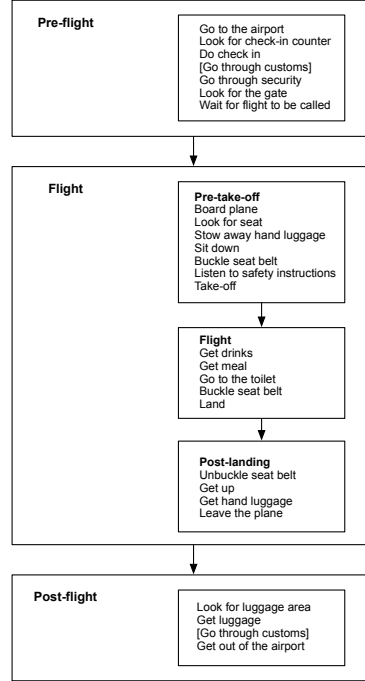


Figure 7.11: Flight script.

$$\forall f, s [M(f, t\text{-}type, s) \rightarrow (s = passenger \vee s = goods)].$$

Under this translation, a knowledge base using frames is translated into the default theory (R, Δ) , where R contains the default rules given by expressions 6.1 and 6.2 and Δ contains *wffs* 6.3, 6.4, 6.5, and 6.6, plus all the *wffs* that are obtained by the translation specified above.

This translation is simple, but it presents the problem that it does not specify a translation for procedural attachment. Two alternative solutions can be used to solve this problem:

1. The development of an algorithm to translate a program into logic statements;
2. To give up from this translation and to define a procedure as the value that it produces.

7.5 Scripts

Frames are used to represent the structure of objects. A frame-like representation, called a *script* is used to represent activities — sequences of events or actions. Scripts are representations for often performed action sequences, and these sequences are causally linked. Like frames, scripts are often hierarchically organised. A script is essentially a pre-packaged inference chain relating to a specific routine situation.

A script is made up of scenes, each scene is activated by some condition. Scripts have default values, which are the most often or expected reference, for the actors, actions, and events in the script.

Among other attributes, scripts are characterised by the following aspects:

- the name of the script (such as *flight script*);
- the track (such as *commercial flight* or *military flight*);
- a set of roles (such as *stewardess*);
- a set of entry conditions (such as that *a person wants to travel*);
- a set of results (such as that *the person is located in another place*);
- a list of scenes (such as *pre-flight*, *flight*, and *post-flight*). Scenes are broken down into actions.

In Figure 6.11, we show a script that corresponds to the actions taken by someone who takes a flight.

There were strong and weak scripts. *Strong scripts* have expectations for order or temporal sequence. *Weak scripts* have no expectations for order. Thus, an activity that is represented in memory by a strong script is one that people have a clear and distinct expectation about the appropriate order in which to perform the action sequence, or follow the script. An activity with a weak script, on the other hand, is one that people do not have definite order expectations for, so the actions may be performed in an improvised order without changing the outcome.

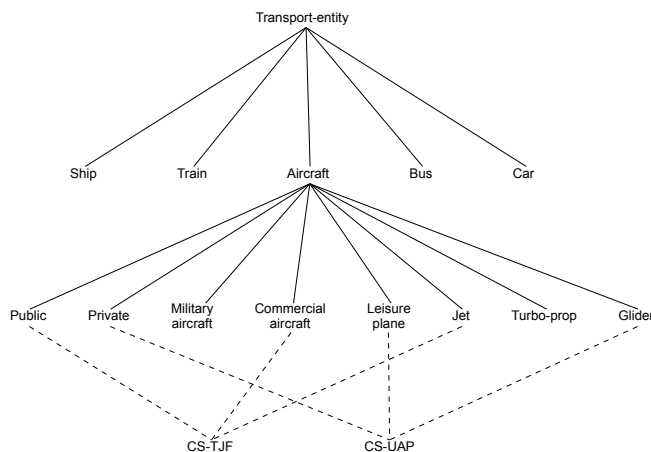


Figure 7.12: Multiple classifications for Aircraft.

7.6 Concluding remarks

The main advantage of frames is the natural way in which they allow the representation of domains where the information is structured as a hierarchy of classes. The other advantage of frames corresponds to the property inheritance mechanism, that is a natural and efficient way of producing inferences.

A problem with frames is that often we need to introduce multiple classifications for the same entity. For example, an aircraft may be classified according to its type, power, and use. Mixing all these classifications into a single hierarchy (Figure 6.12) may give rise to a big mess. In order to solve this problem, it is usual to introduce a *fictitious level* in the hierarchy that enables to make the appropriate distinctions. In Figure 6.13, we introduce three frames, *Aircraft-wrt-type*, *Aircraft-wrt-power*, and *Aircraft-wrt-use* that enable to separate different classes of ships with respect to different classifications.

7.7 Bibliographic notes and historical remarks

Frames were proposed by Marvin Minsky [Minsky 75]. A similar idea had been proposed in psychology by [Bartlett 32]. An overview of frames as a

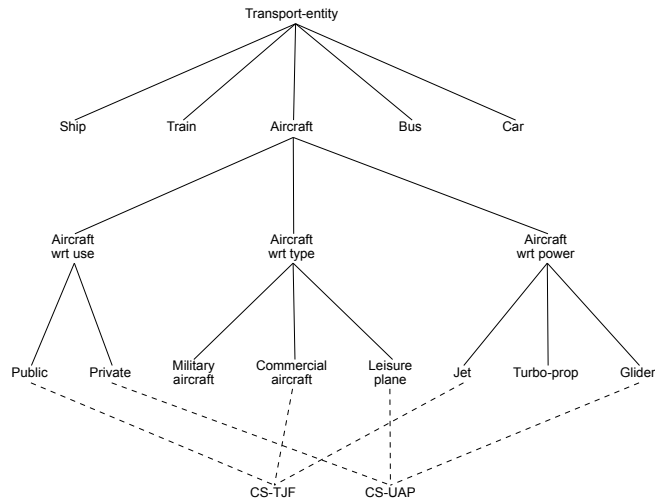


Figure 7.13: Classes of **Aircraft** according to different classifications.

knowledge representation formalism is presented in [Maida 92]; it discusses their historical evolution, their connection with psychology, their connection with object-oriented languages, and the problems associated with inheritance.

There are several languages to represent knowledge using frames, for example, KRL [Bobrow and Winograd 77], FRL [Roberts and Goldstein 77], UNITS [Stefik 79], and KEE [Fikes and Kehler 85], [IntelliCorp 85], [Filman 88].

Since the mid 1980s, frames originated several robust commercial tools that were used in the development of expert systems (for example, CRL, KEE, and FrameKit). By the 1990s, frames were in general use for the development of knowledge bases, and, recently, evolved into object-oriented data bases (OODBs).

Inheritance has been studied, not only in AI, but also in programming languages [Snyder 91] and databases [Borgida and Williamson 85]. In AI, inheritance has been one concern in knowledge representation since the early beginnings, and was addressed by [Quillian 68], [Collins and Quillian 70, 72], and [Minsky 75]. Formal methods for inheritance are described in [Carpenter and Thomason 90], [Horty and Thomason 88, 90], [Horty 91], and [Thomason 92]. [Carpenter and Thomason 90] argue for the need of inheritance and

describes the properties that translational formalisms used in inheritance should present. [Touretzky 86] presents the first work that deals with inheritance from the topological approach. Describes, in detail, the problems associated with inheritance networks and compares the topological approach with the translational approach. [Eklund 91] discusses the problems associated with multiple inheritance and the corresponding solutions. Provides a clear and detailed overview on inheritance systems.

[Fikes and Kehler 85] is a paper that describes the role of frames in representation and reasoning associated with Expert Systems. It uses KEE as the underlying frame system. [Nado and Fikes 87] describes a frame language oriented to the definition of inheritance with exceptions. Describes the algorithm used for inheritance.

Production systems were first proposed by [Post 43] and several variations of this formalism are being used in AI (see, for example, [Waterman and Hayes-Roth 78]). Chapter 3 of the book [Barr and Feigenbaum 81] presents a good perspective on the concepts and applications related with production systems. Strategies for resolving the application of multiple production rules are presented in [McDermott and Forgy 78].

The formal semantics for frames were first proposed by Pat Hayes, who attempted to define the semantics of a frame language (FRL) by describing a translation of FRL into first-order logic [Hayes 79].

Scripts were proposed by [Schank and Abelson 77] to cope with representational problems found in the conceptual dependency theory (see Section 5.3.2).

7.8 Exercises

1. Explain why frame systems have been criticised for their weak expressive power.
2. What are the advantages of frame systems?
3. Taking into account the family example presented in Section 2.5.1, represent in KEE kinship relations.
4. Represent, in KEE, knowledge about the courses offered in a given University. In particular, the following aspects should be taken into account:

- (a) Courses can be regular lecture courses, seminars, or independent study;
- (b) Regular lecture courses have lectures and may have recitations;
- (c) Each course has a syllaby that fits within a scientific subject;
- (d) Each course has a required grading work that can either be an exam, one or more projects, or both;
- (e) Seminars and independent study do not have exams;
- (f) Each course is associated with a set of faculty members;
- (g) There is a faculty member that is in charge of each course.

Chapter 8

Description Logics

‘Not very nice *alone*,’ he interrupted,
quite eagerly: ‘but you’ve no idea what
a difference it makes, mixing it with
other things’
Lewis Carroll, *Through the Looking
Glass*

Description logics is a designation for a collection of knowledge representation formalisms that aims at the unification of features of several other formalisms, namely, frame-based systems, semantic networks, and object-oriented representations. The word “logic” in this designation attempts to convey that description logics are defined based on a logic formalism, with a model-theoretic semantics.

The development of the ancestor formalism of description logics was motivated by the arbitrary way in which frames deal with the representation of knowledge. When we represent knowledge in frames, we define a partial order on frames. Each frame is characterized by a certain number of attributes that are propagated to the frames that are located lower down in the hierarchy. In spite of this, in a frame, the value of a given attribute may be changed arbitrarily at any point. From this point of view, two problems are identified:

1. The partial order relations between frames are established by the person who builds the representation, and there are no external criteria to define when these relations are correct.

This makes the semantics of the links between frames purely opera-

tional — the “semantics” just defines the way attributes are propagated, without any external criteria to define when relations should be established.

2. The fact that an inherited attribute can arbitrarily be changed, prevents the enforcement of the properties (or characteristics) that one wants to assign to the classes that frames represent.

For example, considering the frames in Figure 6.13, the intended meaning of the class *Jet* is that the entities in this class are powered by “jet” engines. However, there is nothing in the frame system that prevents us from changing the type of power of *CS-TJF* to “glider”.¹

Based on these problems, a new representation formalism emerged, using a clear distinction between *structural links*, links that relate the components of a structured concept, and *assertional links*, links that represent assertions about concepts, associating them with individuals.²

Another aspect of this formalism is to concentrate on the way of *defining concepts*. For this reason, it does not address certain aspects of a representation that clash with the notion of “definition”, in particular the notions of default value and overriding inherited values.

A third aspect is the distinction between intensional and extensional knowledge. We already discussed the distinction between intensional and extensional concepts.³ *Intensional knowledge* pertains a given meaning; *extensional knowledge* is specific to a particular problem domain. In description logics, there is a separation between the representation of knowledge regarding these two types of knowledge.

Description logics represent the knowledge about an application domain by defining the concepts of the domain (what is called its *terminology*), and by using those concepts to specify properties of the objects in the domain (what is called its *description*).

¹This problem was solved in KEE (see Section 6.3) with an implementation feature, but the problem remains on most other frame systems.

²The need to distinguish between these two kinds of links is discussed in Section 5.1.

³See Sections 5.1 and 5.2.2.

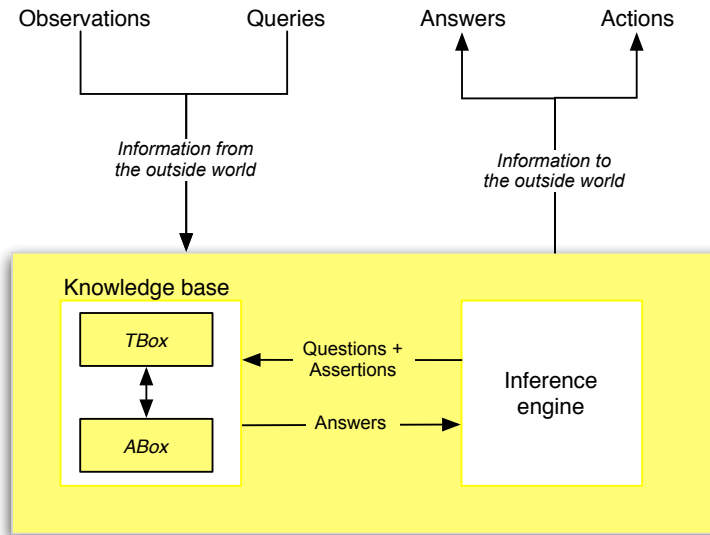


Figure 8.1: Architecture of a description logics-based system.

8.1 Basic principles

In a description logics-based system, the knowledge base is composed by two parts, a *TBox* and an *ABox* (see Figure 7.1):

1. The *TBox* contains a representation of concepts in terms of other concepts.

This component defines the terminology of concepts and is called the *terminological representation component* (hence, the name “*TBox*”).

The *TBox* contains intensional knowledge in the form of a *terminology*, defining the meaning of the terms of a domain.

2. The *ABox* contains the representation of the objects existing in the world and the representation of the links between them and the existing concepts.

This component contains extensional knowledge that is specific to the individuals of the domain and is called the *assertional representation component* (hence the name “*ABox*”).

The *ABox* is used to make assertions about the world.

The *TBox* and the *ABox* are usually defined using different representation languages, thus, description logics have also been known as *hybrid systems*.

8.1.1 TBox – The terminological component

The terminological component represents knowledge based on a taxonomy of concepts. The semantics of the taxonomy is defined independently of the algorithms that manipulate the data structures that correspond to the taxonomy. The semantics defines when a concept is a generalization or a specialization of another concept.

Concepts are divided into generic and individual concepts:

1. *Generic concepts* represent classes. Generic concepts are, in turn, divided into defined and primitive concepts:

- (a) *Defined concepts* are concepts whose meaning is fully defined by their description.

Defined concepts are introduced by specifying the *necessary and sufficient conditions* for a given entity to be classified as belonging to the class corresponding to the concept.

For example, *Woman* may be defined as *Human and Female*.

- (b) *Primitive concepts* are concepts whose meaning cannot be defined in an exhaustive way;⁴ they are considered associated with incomplete definitions.

Primitive concepts are introduced through the specification of the *necessary conditions* for a given entity to be considered as belonging to the class that the concept represents. These specifications constrain the class of the entities that give the necessary properties for the concept but do not fully define it.

For example, *Dog* may be defined as a *Mammal that barks*.

2. *Individual concepts* represent intensional individuals. Individual concepts may be considered as representing classes with a single element. These concepts have no assertional impact and are used to describe potential individuals. In their description, there is no claim about their existence.

⁴Primitive concepts are also known as *natural concepts* or *natural kinds*.

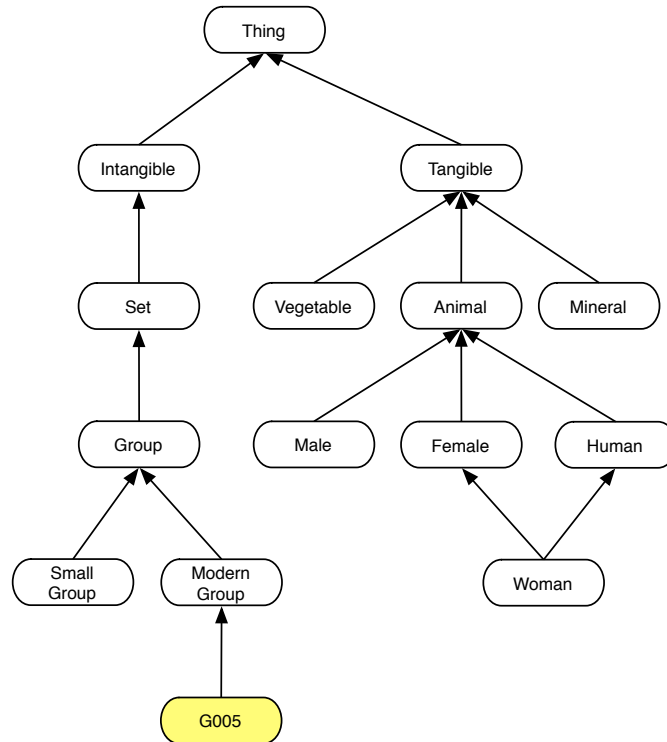


Figure 8.2: Taxonomy of concepts.

In order to define concepts, two kinds of specifications are mainly used, taxonomic specifications and role specifications. Some description logics also use structural conditions.

Taxonomic specifications. Taxonomic specifications relate a concept with other concepts that are either more general or more specific than the given concept.

In Figure 7.2, we show a hierarchy that contains generic and individual concepts.⁵ Concepts are graphically represented by labeled nodes, and the taxonomic relations between concepts are represented by directed unlabeled arcs that point from more specific concepts to more general concepts, and that correspond to assertional links. Individual concepts are represented by

⁵This figure was adapted from [Nebel 90], Figures 1.2 and 3.1.

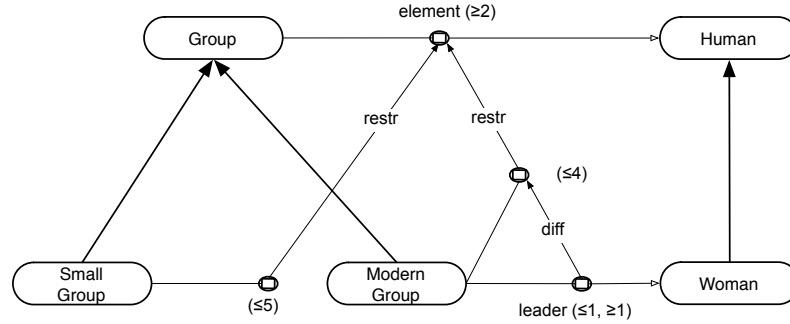


Figure 8.3: Roles relating concepts.

nodes with a shaded background.

An important aspect of this taxonomy is that the meaning of a concept depends *only* on the meaning of the concepts that are above it in the taxonomy and on the information associated with that particular concept.

Role specifications. Role specifications correspond to potential relations between the members of the class corresponding to a concept and other concepts. Roles try to capture a common generalization of the notions of attribute, part, constituent, aspect, parameter, argument, grammatical case, and so on.

Roles, together with restrictions, differentiate concepts and specify the ways a concept differs from the intersection of its immediate superconcepts in the taxonomy. Among other things, a role may be associated with restrictions on the possible values it may assume, and with the number of possible associations. Roles are graphically represented by circles with an inscribed square, and the restrictions to the values a role may assume are represented by an arc from the role to the possible concepts that may be values of the role.

As an example, let us consider the concepts defined in Figure 7.3,⁶ which are based on the taxonomy of Figure 7.2. Informally, this figure states that a **Group** contains, at least, two elements that are **Human**; a **SmallGroup** is a **Group** with, at most, five elements; and that a **ModernGroup** is a group with, at most, four elements, one of which, the **leader**, is a **Woman**.

⁶This figure was adapted from [Nebel 90], Figure 3.2.

Figure 7.3 represents, among other things, that a **Group** has a role called **element**, that the elements of a group are **Human**, and that the number of elements is greater than or equal to 2.

Roles may be hierarchically organized through two different relations:

1. *A role may be a more constrained version of another role.*

In Figure 7.3, the role **element** associated with the concept **SmallGroup** has an additional restriction on the role **element** associated with the concept **Group**. This restriction states that the number of elements of a **SmallGroup** must be less than or equal to 5. In this case, the links between roles are labeled with the label **restr**.

2. *A role may be a differentiated version of another, more general, role.*

For example, in Figure 7.3, the role **leader** is a differentiated version of the role **element**. In this case, links between roles are labeled with the label **diff**.

Roles associated with a concept are also applicable to all specializations of that concept. In this respect, concepts inherit roles of the most general concepts to which they are connected. This kind of inheritance is different from the inheritance found in frames, since the inherited attribute corresponds to a defining characteristic of a concept. More specific concepts may add restrictions to these characteristics but may not modify them.

The stress placed on the definition of concepts is the main idea behind description logics. The most important consequence of this aspect is the possibility of reasoning about relations between concepts, based on their description. This aspect of reasoning is called *terminological reasoning* and gave rise to the automatic classification of the concepts in a hierarchy.

Structural conditions. Another possible specification that may be included in the terminological component is called structural conditions. *Structural conditions*⁷ define relationships between roles and constrain the potential values that a role may assume.

The roles used in the definition of a concept are, in general, not independent of one another. These interdependencies are specified through structural conditions.

⁷Structural conditions are not included in most description logics.

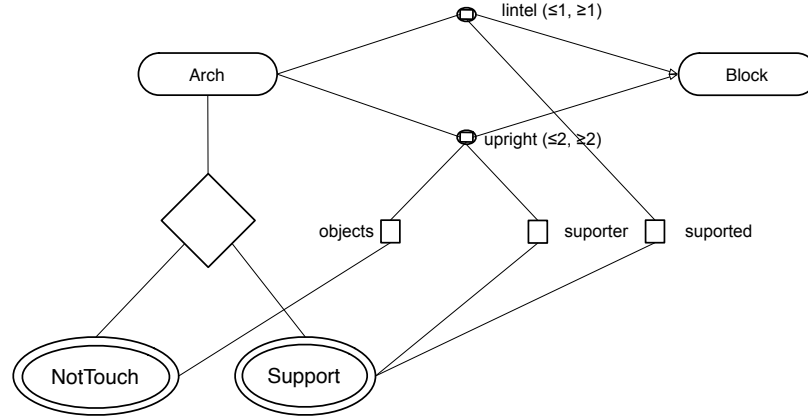


Figure 8.4: Example of structural conditions.

In general, structural conditions correspond to parameterized descriptions of the individual relations that have to be satisfied by instances of the concept. These relations are called *parameterized entities*, because there is an instantiation of the relation that is different for each instantiation of the concept.

Graphically, these relations are represented by a double circle (enclosing the name of the relation) linked to a diamond-shaped lozenge, which is, in turn, linked to the concept. The label inside a double circle corresponds to a concept represented in the system and specifies a correspondence between the roles of that concept and the roles of the concept being described. This correspondence is done, graphically, through the indication of the name of the role (of the concept corresponding to the double circle) associated with a square.

In Figure 7.4, we show the description of the **Arch** concept. An **Arch** has two roles, **lintel** (that has only one object that fills its role) and **upright** (for which there are two objects that fill its role); both of these roles must be filled by **Block**. Structural conditions regarding the **Arch** concept, state that, given a particular arch, its **uprights** must satisfy the **NotTouch** relation, and the **uprights** must satisfy the relation **Support** in relation to the **lintel** (see also Figure 7.5).

There are two kinds of structural conditions:

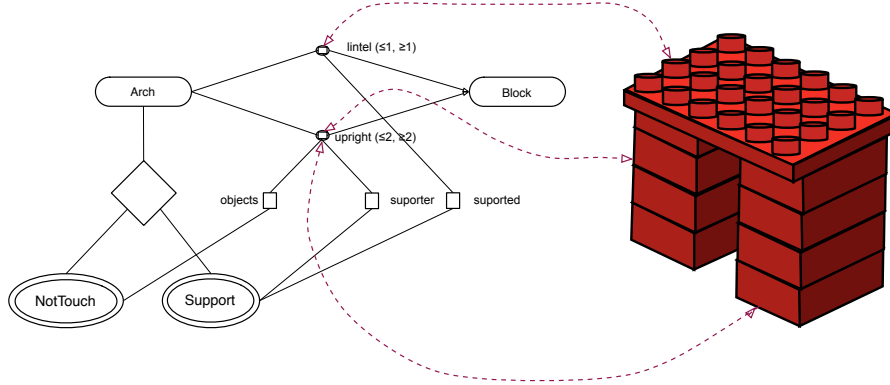


Figure 8.5: Some relations of the structural conditions with an object.

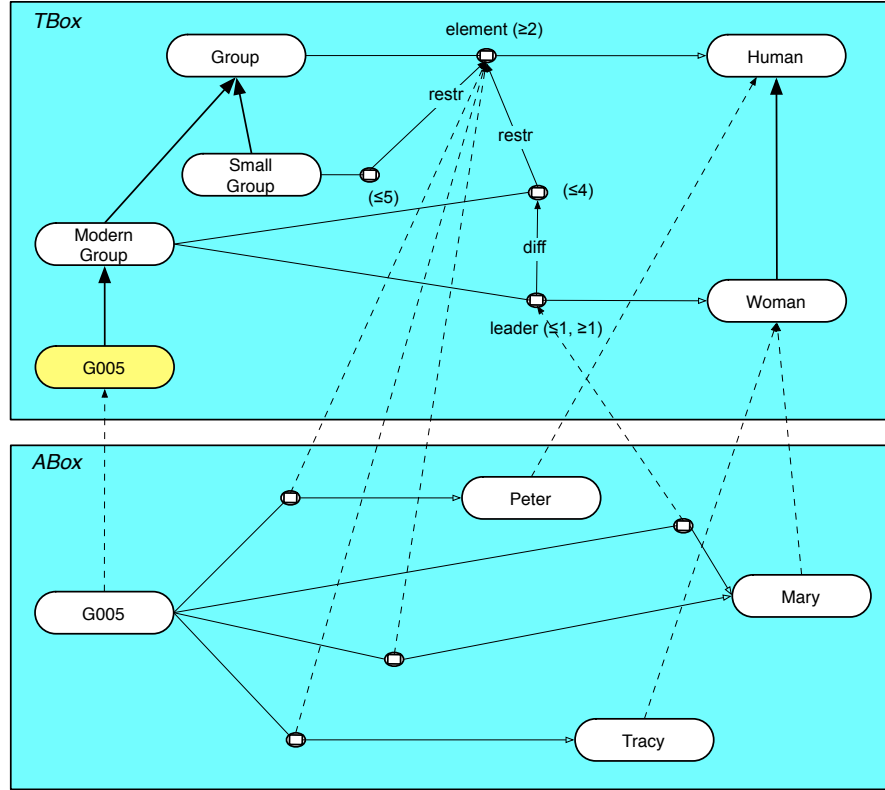
1. General conditions, or *structural relations*, which enable the specification of the way roles and concepts interrelate with one another.
The interdependencies between the components of an arc are examples of structural relations.
2. Particular conditions, or *role value maps*, which enable the specification of subset and equality relations between the possible fillers of a role.

8.1.2 ABox – The assertional component

The assertional component contains a description of the world — a set of assertions about the objects in the world and their interrelations.

A description of an object in the assertional component is the assertion that a given object is an instance of a concept. With a description relation, we can assert that an object has certain values as fillers for its roles or that it has a certain number of values as fillers for these roles without having to specify those values.

As an example, suppose that based on the concepts shown in Figure 7.3, we define that G005 is a **Modern Group**, whose elements are Peter, Tracy, and Mary; Mary is the leader. Since we are making assertions about objects in the world, this will be represented in the assertional component (*ABox*), with connections to the relevant concepts in the terminological component

Figure 8.6: Connection between the *TBox* and the *ABox*.

(*TBox*). The complete representation is shown in Figure 7.6.

8.2 Terminological formalism

In this section, we describe several features of languages that are used in the terminological formalism.

In the specifications of concepts (both generic and individual) and in the specification of roles, we can use two approaches. We can provide a name for the concept (or role), in which case, we have an atomic concept (or role); we can define the concept (or role) as combination of other concepts or roles, in which case we have a structured concept (or role). In short:

- An *atomic concept* (*atomic role*) is a concept (role) that is denoted by a name — the name of the concept (role).

Atomic concepts are further divided into *primitive* and *defined*.

- A *Structured concept* (*structured role*) is specified using a combination of other concepts (roles).

The combination of concepts (roles) resorts to a set of constructs that enable to create concepts (roles) from other concepts (roles).

In the specification of the terminological formalism we use a formal language — the *terminological language*.

The language presented in this section uses a syntax that resembles other logical formalisms — this is called the *abstract syntax*. This language uses two disjoint alphabets, one to denote *atomic concepts* and another to denote *atomic roles*. Structured concepts and structured roles, are created by the application of constructs, called *descriptions*, to other concepts or roles. We use the designation *term* to denote either a concept or a role.

Another language to express description logics expressions, known as *concrete syntax*, uses a LISP-like syntax that can actually be used from within a computer (see Sections 7.5 and 7.7).

Concepts correspond to unary predicate symbols in first-order logic and roles correspond to binary predicate symbols in first-order logic. In description logics, there is no explicit representation of function symbols.

Description logics (and their predecessor formalisms) are concerned with the definition of a formal semantics. Alongside with the description of the abstract syntax, we present the semantics of the language constructs, making precise the notions being introduced.

The idea behind the definition of the semantics for a description logic is the existence of a *universe of discourse*, D , in the same sense as it is introduced for the semantics of first-order logic (see Section 2.2.2). Each concept represents a certain number of entities in the universe of discourse (a subset of D); each role represents a certain number of pairs of these entities (a subset of $D \times D$).

If we try to make a parallel between this description and the semantics of first-order logic, each concept corresponds to a unary relation (is part of the

set of relations),⁸ and each role corresponds to a binary relation (is also part of the set of relations).

One aspect of the semantics of a description logic corresponds to the association of each concept and each role with the relations that they denote. The sets associated with concepts and roles are called their *extensions*. The extension of a concept (or the extension of a role) is defined by an extension function (E), a function that maps terms into extensions.

Let D be the universe of discourse, let C be a set of concepts, and let R be a set of roles. The extension function, E , is such that:

1. $E : C \mapsto 2^D$
Each concept is associated with a subset of D ;
2. $E : R \mapsto 2^{D^2}$
Each role is associated with a subset of ordered pairs of D .

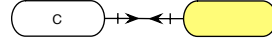
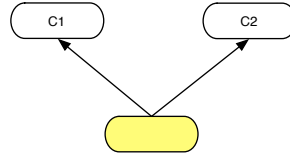
The pair $[D, E]$ is called a *semantic structure*.

8.2.1 Concept descriptions

We start by providing the syntactic definition of concepts:

1. *Atomic concepts*.
 - (a) $\top$ is an atomic concept, called the *universal concept*.
The universal concept contains all concepts.
 - (b) $\perp$ is an atomic concept, called the *inconsistent concept*.
The inconsistent concept contains no concept.
 - (c) Any name may be an atomic concept.
 - (d) Nothing more is an atomic concept.
2. *Structured concepts*, or just *concepts*.
 - (a) An atomic concept is a concept.

⁸This is not quite true, because a unary relation is a set of *one*-tuples containing entities of the universe of discourse, whereas a concept is a set of entities of the universe of discourse.

Figure 8.7: Graphical representation of $\neg C$.Figure 8.8: Graphical representation of $C_1 \sqcap C_2$.

- (b) If C is an atomic concept, then $\neg C$ is a concept.

The construct $\neg C$ is read *concept complement*. It is also known as concept negation to emphasize the relationship to logic. Figure 7.7 shows the graphical representation of concept complement.

For example, if **Human** is a concept, the construct $\neg \text{Human}$ denotes the concept of non-human.

- (c) If C_1 and C_2 are concepts, then $C_1 \sqcap C_2$ is a concept.

The construct $C_1 \sqcap C_2$ is read *intersection of concepts*. It is also known as concept conjunction to emphasize the relationship to logic. Figure 7.8 shows the graphical representation of the intersection of concepts.

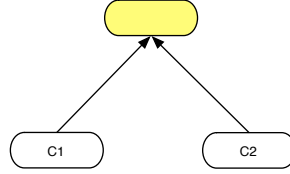
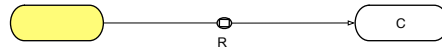
For example, if **Female** and **Human** are concepts, the construct $\text{Female} \sqcap \text{Human}$ denotes the females that are humans.

- (d) If C_1 and C_2 are concepts, then $C_1 \sqcup C_2$ is a concept.

The construct $C_1 \sqcup C_2$ is read *union of concepts*. It is also known as concept disjunction to emphasize the relationship to logic.

For example, if **Man** and **Woman** are concepts, the construct $\text{Man} \sqcup \text{Woman}$ denotes the the set of all men and women. Figure 7.9 shows the graphical representation of the union of concepts.

- (e) If R is a role and C is a concept, then $\forall R.C$ is a concept. This construct, called *value restriction*, requires that all the individuals that are in the relationship R with the concept being described, belong to the concept C . Figure 7.10 shows the graphical representation of value restriction.

Figure 8.9: Graphical representation of $C1 \sqcup C2$.Figure 8.10: Graphical representation of $\forall R.C$.

For example, assuming that **element** is a role and **Human** is a concept, the construct $\forall \text{element}.\text{Human}$ defines the concept containing all entities that have an **element** relation with humans.

- (f) If R is a role, then $\exists R.\top$ is a concept.

This construct, called *limited existential quantification*, specifies all the individuals that are in the relationship R with something. Figure 7.11 shows the graphical representation of limited existential quantification.

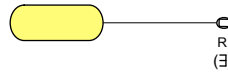
For example, assuming that **leader** is a role, the construct $\exists \text{leader}.\top$ defines the concept containing all the entities have a **leader** relation with something.

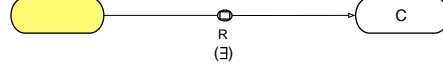
- (g) If R is a role and C is a concept, then $\exists R.C$ is a concept.

This construct, called *full existential quantification*, specifies all the individuals that are in the relationship R with the concept C . Figure 7.12 shows the graphical representation of limited existential quantification.

For example, the construct $\exists \text{leader}.\text{Woman}$ defines the concept containing all the entities have a **leader** which is a **Woman**.

- (h) If R is a role and n is a non-negative integer, then $\geq n R$ is a

Figure 8.11: Graphical representation of $\exists R.\top$.

Figure 8.12: Graphical representation of $\exists R.C$.Figure 8.13: Graphical representation of $\geq n R$.

concept.

This construct, called *number restriction*, restricts the cardinality of the sets of fillers of role R . Figure 7.13 shows the graphical representation of number restriction.

For example, the construct ≥ 2 *element* establishes that there are at least two elements.

- (i) If R is a role and n is a positive integer, then $\leq n R$ is a concept. This construct, called *number restriction*, restricts the cardinality of the sets of fillers of role R . The representation for this construct is similar to the representation of $\geq n R$.
- (j) If $i_1, \dots, i_n$ are individuals, then $\{i_1, \dots, i_n\}$ is a concept. We provide no graphical representation for this construct. This construct enables the explicit specification of a concept as being a set of individuals.
- (k) Nothing else is a concept.

The concepts described above have the following meanings:⁹

$$E(\top) = D \quad (8.1)$$

$$E(\perp) = \emptyset \quad (8.2)$$

$$E(\neg C) = D \setminus E(C) \quad (8.3)$$

$$E(C_1 \sqcap C_2) = E(C_1) \cap E(C_2) \quad (8.4)$$

$$E(C_1 \sqcup C_2) = E(C_1) \cup E(C_2) \quad (8.5)$$

$$E(\forall R.C) = \{x \in D : \forall y [(x, y) \in E(R)) \rightarrow (y \in E(C))\} \quad (8.6)$$

⁹Where $\| S \|$ represents the cardinality of the set S .

$$E(\exists R.\top) = \{x \in D : \exists y [(x, y) \in E(R)]\} \quad (8.7)$$

$$E(\exists R.C) = \{x \in D : \exists y [(x, y) \in E(R) \wedge y \in E(C)]\} \quad (8.8)$$

$$E(\geq n R) = \{x \in D : \parallel \{y : (x, y) \in E(R)\} \parallel \geq n\} \quad (8.9)$$

$$E(\leq n R) = \{x \in D : \parallel \{y : (x, y) \in E(R)\} \parallel \leq n\} \quad (8.10)$$

$$E(\{i_1, \dots, i_n\}) = \{E(i_1) \cup \dots \cup E(i_n)\} \quad (8.11)$$

8.2.2 Role descriptions

Similarly to what we did with concepts, we start by providing the definition of roles:

1. *Atomic roles.*

- (a) $\top$ is an atomic role, called the *universal role*. The universal role contains all the roles.
- (b) Any name may be an atomic role.

2. *Structured roles, or just roles.*

- (a) An atomic role is a role.
- (b) If R is an atomic role, then $\neg R$ is a role.
- (c) If R_1 and R_2 are roles, then $R_1 \sqcup R_2$ is a role.

The construct $R_1 \sqcup R_2$ is read *union of roles*.

As an example, if `hasSon` and `hasDaughter` correspond to atomic roles that specify the relation between a person and his son or daughter, respectively, the construct `hasSon` $\sqcup$ `hasDaughter` corresponds to the role that specifies the relation between one person and his children.

- (d) If R_1 and R_2 are roles, then $R_1 \sqcap R_2$ is a role.

The construct $R_1 \sqcap R_2$ is read *role intersection*.

- (e) If R_1 and R_2 are roles, then $R_1 \odot R_2$ is a role.

This construct, called *role composition*, corresponds to the composition of roles R_1 and R_2 .

For example, `hasChild` $\odot$ `hasChild` corresponds to the role that specifies the relation between one person and his grand children.

- (f) If R is a role, then R^+ is a role.

This construct, the *transitive closure*, corresponds to the closure of R composed with itself.

The transitive closure of R is defined as,

$$R^+ = \bigsqcup_{i \geq 1} R^i \quad (8.12)$$

where R^i is defined in the following way

$$R^i = \begin{cases} R & \text{if } i = 1 \\ R \odot R^{i-1} & \text{if } i > 1 \end{cases}$$

For example, hasChild^+ corresponds to the role that specifies the relation between one person and his descendants.

- (g) If R is a role, then R^- is a role.

This construct corresponds to the *inverse relation*.

For example, hasChild^- corresponds to the role that specifies the relation between one person and his parents.

- (h) Nothing else is a role.

The description of roles has the following meaning:

$$E(U) = D^2 \quad (8.13)$$

$$E(\neg R) = D^2 \setminus E(R) \quad (8.14)$$

$$E(R_1 \sqcap R_2) = E(R_1) \cap E(R_2) \quad (8.15)$$

$$E(R_1 \sqcup R_2) = E(R_1) \cup E(R_2) \quad (8.16)$$

$$E(R_1 \odot R_2) = \{(x, y) \in D^2 : \exists z [(x, z) \in E(R_1) \wedge (z, y) \in E(R_2)]\} \quad (8.17)$$

$$E(R^+) = \bigcup_{i \geq 1} E(R^i) \quad (8.18)$$

$$E(R^-) = \{(x, y) \in D^2 : (y, x) \in E(R)\} \quad (8.19)$$

8.2.3 Knowledge bases — the TBox

The *TBox* is a set of terminological axioms. *Terminological axioms* make statements about how concepts or roles are related to each other.

There are two types of terminological axioms, equalities and inclusions:

- If C_1 and C_2 are concepts, and if R_1 and R_2 are roles, then *equalities* are axioms of the form:

$$C_1 \equiv C_2 \quad (8.20)$$

$$R_1 \equiv R_2 \quad (8.21)$$

If the left-hand side of an equality is an atomic concept or an atomic role, then the equality is called a *definition*.

A definition associates the name that appears in its left-hand side with the description that appears in its right-hand side and is the way to associate symbolic names with complex descriptions.

- If C_1 and C_2 are concepts, and if R_1 and R_2 are roles, then *inclusions* are axioms of the form:

$$C_1 \sqsubseteq C_2 \quad (8.22)$$

$$R_1 \sqsubseteq R_2 \quad (8.23)$$

If the left-hand side of an inclusion is an atomic concept or an atomic role, then the inclusion is called a *specialization*.

A finite set of definitions is called a *terminology* or *TBox*. There are two important assumptions usually made about description logics terminologies:

- Only one definition for a concept (or role) is allowed. That is, for every atomic concept, there is at most one axiom whose left-hand side is the name of the concept;
- Definitions are acyclic in the sense that concepts are neither defined in terms of themselves nor in terms of other concepts that indirectly refer to them.¹⁰ This restriction implies that every defined concept can be expanded in a unique way into a complex expression containing only atomic concepts.

¹⁰This restriction does not hold for all description logics.

As an example, the following is a terminology for the concepts presented in Figure 7.3 and in part of Figure 7.2:

$$\text{Woman} \equiv \text{Female} \sqcap \text{Human} \quad (8.24)$$

$$\text{Group} \equiv \text{Set} \sqcap \forall \text{element. Human} \sqcap \geq 2 \text{ element} \quad (8.25)$$

$$\text{SmallGroup} \equiv \text{Group} \sqcap \leq 5 \text{ element} \quad (8.26)$$

$$\begin{aligned} \text{ModernGroup} \equiv \text{Group} \sqcap \exists \text{leader. Woman} \sqcap \leq 1 \text{ leader} \sqcap \\ \sqcap \geq 1 \text{ leader} \sqcap \leq 4 \text{ element} \end{aligned} \quad (8.27)$$

$$\text{leader} \sqsubseteq \text{element} \quad (8.28)$$

8.2.4 Models and semantic structures

Given a semantic structure $[D, E]$ and an axiom in a terminology, we say that the semantic structure is a model of the axiom, written $\models_{[D, E]}$, if it satisfies the following conditions:

$$\models_{[D, E]} (C_1 \equiv C_2) \text{ if and only if } E(C_1) = E(C_2) \quad (8.29)$$

$$\models_{[D, E]} (C_1 \sqsubseteq C_2) \text{ if and only if } E(C_1) \subseteq E(C_2) \quad (8.30)$$

$$\models_{[D, E]} (R_1 \equiv R_2) \text{ if and only if } E(R_1) = E(R_2) \quad (8.31)$$

$$\models_{[D, E]} (R_1 \sqsubseteq R_2) \text{ if and only if } E(R_1) \subseteq E(R_2) \quad (8.32)$$

A semantic structure $[D, E]$ is a model of a terminology $\mathcal{T}$, written $\models_{[D, E]} \mathcal{T}$, if it is a model of all the axioms of a terminology.

8.2.5 Terminological languages

Description logics differ from one another, in part, by the expressive power that is provided by the terminological language.

The minimal terminological language with practical interest, called $\mathcal{AL}$ (attributive language), enables intersection of concepts, negation of atomic concepts, restriction on values and limited existential quantification.

Many other extensions exist, each characterized by a calligraphic letter. The name of the languages is formed by adding letters to the minimal language. In general, description logics languages have a name of the form

$$\mathcal{AL}[\mathcal{U}][\mathcal{E}][\mathcal{N}][\mathcal{C}][\mathcal{H}][\mathcal{R}]$$

where,

- $\mathcal{U}$ denotes the existence of union of concepts;
- $\mathcal{E}$ denotes the existence of full existential quantification;
- $\mathcal{N}$ denotes the existence of number restrictions;
- $\mathcal{C}$ denotes the existence of negation of structured concepts;
- $\mathcal{H}$ denotes the existence of role hierarchies;
- $\mathcal{R}$ denotes the existence of intersection of roles;
- $\mathcal{R}^+$ denotes the existence of transitive roles.

From the semantic point of view, not all these languages are distinct. The language with union, full existential quantification, and number restrictions, $\mathcal{ACUEN}$, has enough expressive power to write all $\mathcal{AL}$ -languages.

8.3 Assertional formalism

The *ABox* contains extensional knowledge about the domain of interest, expressed as a set of assertions about individuals. The *ABox* describes a specific state of affairs about an application domain in terms of concepts and roles. Some of the concepts and roles in the *ABox* may be defined names in the *TBox*.

In the *ABox*, we introduce individuals, by giving them names, and assert properties of these individuals. If a , b , and c are individual names, C is a concept, and R is a role, in the *ABox* we may introduce the following assertions:

$$C(a) \tag{8.33}$$

$$R(b, c) \tag{8.34}$$

The first assertion, called *concept assertion*, states that a belongs to C ; the second assertion, called *role assertion*, states that c is a filler of the role R for b .

The *ABox* is a finite set of concept assertions and role assertions. We should note that these assertions follow the syntax of first-order logic.

One of the basic assumptions made in the *ABox* is called the *unique name assumption* — distinct names denote distinct individuals.

Taking into account the *ABox* presented in Figure 7.6, we can define the following assertions:

$$\text{ModernGroup}(\text{G005}) \quad (8.35)$$

$$\text{Woman}(\text{Mary}) \quad (8.36)$$

$$\text{Woman}(\text{Tracy}) \quad (8.37)$$

$$\text{Human}(\text{Peter}) \quad (8.38)$$

$$\text{element}(\text{G005}, \text{Mary}) \quad (8.39)$$

$$\text{element}(\text{G005}, \text{Tracy}) \quad (8.40)$$

$$\text{element}(\text{G005}, \text{Peter}) \quad (8.41)$$

$$\text{leader}(\text{G005}, \text{Mary}) \quad (8.42)$$

8.3.1 Interpretations

Let Δ be a description of the world (an *ABox*), let D be a universe of discourse, let O be the set of individuals defined by Δ , let C a set of concepts, and let R be a set of roles.

An interpretation function, I , is a total function such that:

1. $I : O \mapsto D$
Each individual is associated with an entity of the universe of discourse;
2. $I : C \mapsto 2^D$
Each concept is associated with a subset of D ;
3. $I : R \mapsto 2^{D^2}$
Each role is associated with a subset of ordered pairs of D .

The pair $[D, I]$ is called an *interpretation*. Given an assertion α ($\alpha \in \Delta$), we say that the interpretation $[D, I]$ satisfies α , written $\models_{[D, I]} \alpha$, if and only if the following conditions are verified:¹¹

¹¹Note the similarities with the conditions presented in Section 2.4.2.

$$\models_{[D,I]} C(o) \text{ if and only if } I(o) \in I(C) \quad (8.43)$$

$$\models_{[D,I]} R(o_1, o_2) \text{ if and only if } (I(o_1), I(o_2)) \in I(R) \quad (8.44)$$

An interpretation, $[D, I]$ is a model of a description of the world, Δ , written as $\models_{[D,I]} \Delta$, if and only if it satisfies all assertions of Δ .

Let us notice that the terminological formalism and the assertional formalism share symbols:

- the terminological formalism constrains the application of terms to subsets of D ;
- the assertional formalism associates individuals with objects of the universe of discourse, these associations must be consistent with the restrictions in the terminological formalism.

A *joint interpretation* of a terminology and a description of the world requires a relationship between the models of the description of the world with the semantic structure of the terminology. Whereas the semantic structure of the terminology can be considered as a specification of the *possible* relations introduced by the way we organize our knowledge, world descriptions correspond to partial descriptions of the way the world in fact is, and must respect the relations required by the terminology.

A model $[D, I]$ of a description of the world, Δ , *respects the terminology* $\mathcal{T}$ if and only if there is a semantic structure $[D, E]$ of $\mathcal{T}$ such that the restrictions of the functions I and R with respect to the individual concepts and roles defined in Δ are identical:

$$\forall C [I(C) = E(C)] \quad (8.45)$$

$$\forall R [I(R) = E(R)] \quad (8.46)$$

8.4 Reasoning in description logics

8.4.1 Reasoning in the terminological component

In general, the terminological component allows the following kinds of reasoning, subsumption, coherence (or satisfiability), classification, and completion.

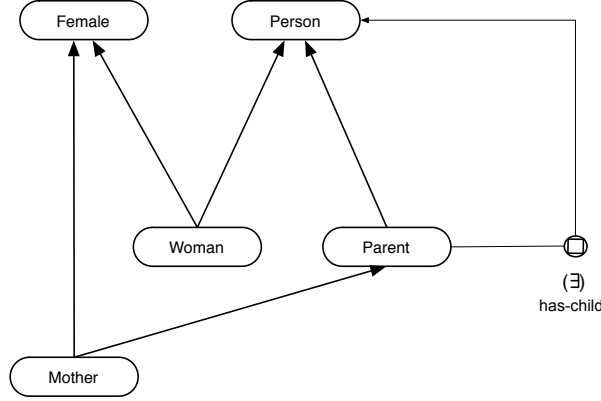


Figure 8.14: Example of a simple representation.

Subsumption. We say that a concept subsumes another if and only if the set of entities that it represents contains the set of entities represented by the other.

In a formal way, concept D *subsumes* concept C with respect to a terminology $\mathcal{T}$,¹² if and only if $E(C) \subseteq E(D)$ for every model $[D, E]$ of $\mathcal{T}$. If D subsumes C , we write $\mathcal{T} \models_{[D, E]} C \sqsubseteq D$.

Before, we discuss the details of the computation of subsumption, we will consider the following example, whose graphical representation is shown in Figure 7.14. Let $\mathcal{T}$ be the terminology defined by axioms 7.47 to 7.49.

$$\text{Woman} \equiv \text{Female} \sqcap \text{Person} \quad (8.47)$$

$$\text{Mother} \equiv \text{Female} \sqcap \text{Parent} \quad (8.48)$$

$$\text{Parent} \equiv \text{Person} \sqcap \exists \text{has-child}.\text{Person} \quad (8.49)$$

In this terminology, it is obvious, for example, that

$$\mathcal{T} \models_{[D, E]} \text{Mother} \sqsubseteq \text{Female} \quad (8.50)$$

and

$$\mathcal{T} \models_{[D, E]} \text{Woman} \sqsubseteq \text{Person}. \quad (8.51)$$

However, it may not be obvious that **Woman** subsumes **Mother**. If we expand the definition of **Parent**, used in the definition of **Mother**, to obtain

$$\text{Mother} \equiv \text{Female} \sqcap (\text{Person} \sqcap \exists \text{has-child}.\text{Person}) \quad (8.52)$$

¹²Also said that concept C is *subsumed by* concept D .

then it becomes clear that **Mother** is a more constrained concept than **Woman**: besides satisfying all the concepts that **Woman** satisfies, it has also to satisfy another concept ($\exists \text{has-child.Person}$). Thus,

$$\mathcal{T} \models_{[D,E]} \text{Mother} \sqsubseteq \text{Woman} \quad (8.53)$$

One of the algorithms for computing subsumption, called *structural subsumption algorithm*, is based on structural comparisons between concept expressions. At the heart of structural comparison is the idea that if the two concept expressions to be compared are made of sub-expressions, one can compare separately one sub-expression of a concept with all those of the other. This is what we did when comparing concepts of **Mother** and **Woman**, using axioms 7.47 and 7.52.

The structural subsumption algorithm works in two phases, in the first phase, concepts are rewritten in a normal form, then, in the second phase, their structures are compared.

The *normal form* of a concept is a description that uses a certain number of constructs applied to atomic concepts. The constructs used in the normal form depend on the terminological language, but may be restricted, for example, to negation and conjunction of concepts. The normal form is obtained by applying a sequence of transformation and simplification operations to the description of the concepts.

Examples of concept transformation operations include:¹³

$$C_1 \sqcap (C_2 \sqcap C_3) \equiv (C_1 \sqcap C_2) \sqcap C_3 \quad (8.54)$$

$$C_1 \sqcup (C_2 \sqcup C_3) \equiv (C_1 \sqcup C_2) \sqcup C_3 \quad (8.55)$$

$$\neg \neg C \equiv C \quad (8.56)$$

$$C_1 \sqcup C_2 \equiv \neg(\neg C_1 \sqcap \neg C_2) \quad (8.57)$$

$$\forall R.C_1 \sqcap \forall R.C_2 \equiv \forall R.(C_1 \sqcap C_2) \quad (8.58)$$

$$\exists R.C \equiv \neg \forall R.\neg C \quad (8.59)$$

$$\neg \top \equiv \perp \quad (8.60)$$

$$\neg \perp \equiv \top \quad (8.61)$$

¹³The proof of these transformations is left as an exercise.

Examples of concept simplification operations include:¹⁴

$$C \sqcap \neg C \equiv \perp \quad (8.62)$$

$$C \sqcup \neg C \equiv \top \quad (8.63)$$

$$C \sqcap \perp \equiv \perp \quad (8.64)$$

$$C \sqcap \top \equiv C \quad (8.65)$$

$$\forall R. \top \equiv \top \quad (8.66)$$

We present the subsumption algorithm for a subset of a description logic, the language $\mathcal{FL}^-$, consisting of intersection of concepts ($C_1 \sqcap C_2$), value restriction applied to atomic roles ($\forall R.C$), and limited existential quantification applied to atomic roles ($\exists R.\top$).

In this restricted language the normal form is computed using only the transformation corresponding to expression 7.58.¹⁵ The normal form corresponds to a conjunction of concepts in one of the forms: A , $\forall R.(A_1 \sqcap \dots \sqcap A_n)$, and $\exists R.\top$, where A , A_1 , $\dots$, A_n are atomic concepts and R is an atomic role. Since the normal form is a conjunction, the algorithm works with sets, containing the conjuncts in the normal form.

The subsumption algorithm relies on the following statement:¹⁶

$$\forall R.C \sqsubseteq \forall R.D \text{ if and only if } C \sqsubseteq D \quad (8.67)$$

The subsumption algorithm, named SUBSUM?, takes two concepts, C and D , in normal form and transformed into sets. Let $C = C_1 \sqcap \dots \sqcap C_n$ and $D = D_1 \sqcap \dots \sqcap D_m$ be the normal form expressions of concepts C and D , respectively. Let $C = \{C_1, \dots, C_n\}$ and $D = \{D_1, \dots, D_m\}$. The algorithm SUBSUM?(D, C) returns TRUE just in case $C \sqsubseteq D$.

The algorithm uses the predicates:

- ATOMIC? that receives a concept, as an argument, and returns TRUE, just in case its argument is an atomic concept;

¹⁴*Ibid.*

¹⁵The only expression applicable to this restricted language.

¹⁶Its proof is left as an exercise.

- **EXQUANT?** that receives a concept, as an argument, and returns **TRUE**, just in case its argument is a concept of the form $\exists R.T$. It tests for limited existential quantification.

The algorithm also uses the procedures:

- **PICK** that receives as argument a non-empty set and returns, randomly, one of its elements;
- **MAKESET** that receives as argument a normal form conjunction of concepts and produces the set of its conjuncts.

```

Procedure SUBSUM?( $D, C$ )
  begin
    if ( $D$ ) =  $\emptyset$ 
      RETURN(TRUE)
    else
      begin
         $d \leftarrow \text{PICK}(D)$ 
        if ATOMIC?( $d$ )  $\vee$  EXQUANT?( $d$ )
          if  $d \in C$ 
            SUBSUM?( $D \setminus \{d\}, C$ )
          else
            RETURN(FALSE)
        else (*  $d = \forall R.D'$  *)
          if ( $\forall R.C'$ )  $\in C$ 
            if SUBSUM?(MAKESET( $D'$ ), MAKESET( $C'$ ))
              SUBSUM?( $D \setminus \{d\}, C$ )
            else
              RETURN(FALSE)
          else
            RETURN(FALSE)
        end
      end
    end

```

For example, the following is a trace of the algorithm using the taxonomy presented in page 245 and assuming that **PICK** takes the first element listed in the set:

SUBSUM?({Female, Person}, {Female, Person, $\exists$ has-child. $\top$ })
 SUBSUM?({Person}, {Female, Person, $\exists$ has-child. $\top$ })
 SUBSUM?({}, {Female, Person, $\exists$ has-child. $\top$ })
 TRUE

Coherence (or satisfiability). A term is said to be coherent if and only if there is a semantic structure for which the term is not associated with the empty set.

A concept C is satisfiable with respect to $\mathcal{T}$ if there exists a model $[D, E]$ of $\mathcal{T}$ such that $E(C) \neq \emptyset$.

Satisfiability can be reduced to subsumption by the following relation

$$C \text{ is unsatisfiable if and only if } C \text{ is subsummed by } \perp \quad (8.68)$$

Classification. The process of determining subsumption relations between sets of terms.

Completion. Completion is the process through which we identify and record all the conditions that are applicable to a given term.

All description logic-based systems compute inheritance and subsumption. Most of them also perform classification. One of the research areas in description logics corresponds to the study of the balance between the expressive power of the representation language and the soundness, completeness, and computing tractability of its algorithms.

8.4.2 Reasoning in the assertional component

In the assertional component, we query the *ABox* about the properties of the represented entities.

Among these queries, there is knowledge base *consistency*, which amounts to verifying whether every concept in the knowledge base admits at least one individual; *realization*, which finds the most specific concept an individual object is an instance of; and *retrieval*, which finds the individuals in the knowledge base that are instances of a given concept. These can all be accomplished by means of instance checking.

In an *ABox*, we can ask questions about the relationships between concepts, roles and individuals. The *ABox* inference mechanism on which such questions are based is *instance checking*, the check of whether an assertion is implied by an *ABox*.

We say that an assertion α is implied by Δ , written, $\Delta \models_{[D,I]} \alpha$, if every interpretation that satisfies Δ , that is, every model of Δ , also satisfies α . If α is of the form $C(a)$, we can reduce instance checking to the consistency problem for the *ABox* because there is the following connection:

$$\models_{[D,I]} C(a) \text{ if and only if } \Delta \cup \{\neg C(a)\} \text{ is inconsistent} \quad (8.69)$$

Similarly, *concept satisfiability* can be reduced to *ABox* consistency because for every concept C we have

$$C \text{ is satisfiable if and only if } \{C(a)\} \text{ is consistent} \quad (8.70)$$

where a is an arbitrarily chosen individual name.

For example, given the world description represented in Figure 7.6, we can infer that Mary is a Woman, since she fills the role leader of Group G005 (a Modern Group).

8.4.3 Knowledge bases vs. Data bases

An analogy can be established between databases and knowledge bases in a description logic. The schema of a database is compared to the *TBox* and the instance with the data is compared to the *ABox*. However, the semantics of an *ABox* differs from the semantics of database instances: while a database instance represents exactly one interpretation, namely the one where classes and relations in the schema are interpreted by the objects and tuples in the instance, an *ABox* represents many different interpretations, namely all its models. As a consequence, absence of information in a database instance is interpreted as negative information, while absence of information in an *ABox* only indicates lack of knowledge. While the information in a database is always understood to be complete, the information in an *ABox* is, in general viewed, as being incomplete. The semantics of an *ABox* is characterized as an *open-world* semantics, while the semantics of traditional databases is characterized as a *closed-world* semantics.

Concrete syntax	Abstract syntax
(define-primitive-concept C Cd)	$C \sqsubseteq Cd$
(define-concept C Cd)	$C \equiv Cd$
(define-primitive-role R Rd)	$R \sqsubseteq Rd$
(rdef R Rd)	$R \equiv Rd$

Table 8.1: Meaning of <term introduction>.

8.5 Implementations of description logics

Many description logics implementations use a LISP-based concrete syntax. In this section, we describe languages that correspond to a superset of most languages of the description logics family. Different systems offer slight variations on the syntax that we present.

8.5.1 The terminological language

A specification in a terminological language consists in the definition of a set of concepts and a set of roles. A terminology associates names with concepts and roles. A terminology consists of one or more specifications:

<terminology> ::= <specification>⁺

<specification> ::= <term introduction> | <concept> | <role>

A <term introduction> can either correspond to the introduction of a concept or to the introduction of a role (these can, in turn, either be primitive or defined). In Table 7.1 we show the meaning of each of these constructs.

<term introduction> ::=
 (define-primitive-concept <concept name> <concept>) |
 (define-concept <concept name> <concept>) |
 (define-primitive-role <role name> <role>) |
 (rdef <role name> <role>)

<concept name> ::= <string>

<role name> ::= <string>

A <concept> may correspond to the most general concept, to the inconsistent concept, to a concept name, to the composition of other concepts,

Concrete syntax	Abstract syntax
top	$\top$
bottom	$\perp$
C	C
(not C)	$\neg C$
(and $C_1 \ C_2$)	$C_1 \sqcap C_2$
(or $C_1 \ C_2$)	$C_1 \sqcup C_2$
(all $R \ C$)	$\forall R.C$
(some R)	$\exists R.\top$
(c-some $R \ C$)	$\exists R.C$
(at-least $n \ R$)	$\geq n \ R$
(at-least $n \ R \ C$)	$\geq n \ R \sqcap \exists R.C$
(at-most $n \ R$)	$\leq n \ R$
(at-most $n \ R \ C$)	$\leq n \ R \sqcap \exists R.C$
(one-of $i_1, \dots, i_n$)	$\{ i_1, \dots, i_n \}$

Table 8.2: Meaning of <concept>.

or to the restriction on roles associated with it. In Table 7.2, we show the meaning of each of these constructs.

```

<concept> ::= *top* |
             *bottom* |
             <concept name> |
             (not <concept>) |
             (and <concept>+) |
             (or <concept>+) |
             (all <role> <concept>) |
             (some <role>) |
             (c-some <role> <concept>) |
             (at-least <minimum> <role> [<concept>]) |
             (at-most <maximum> <role> [<concept>]) |
             (one-of <individual>1 ... <individual>n)

<individual> ::= <string>

```

A <role> may either correspond to the most general role or to the composition of other roles. In Table 7.3, we show the meaning of each of these constructs.

Concrete syntax	Abstract syntax
r	U
R	R
(and-role R ₁ R ₂)	$R_1 \sqcap R_2$
(or-role R ₁ R ₂)	$R_1 \sqcup R_2$
(not-role R)	$\neg R$
(inv R)	R^-
(chain R ₁ R ₂)	$R_1 \odot R_2$

Table 8.3: Meaning of <role>.

```

<role> ::= *r* |
          <role name> |
          (and-role <role>+) |
          (or-role <role>+) |
          (not-role <role>) |
          (inv <role>) |
          (chain <role> <role>+)

```

As an example, the following concrete syntax specifications define the concepts represented in Figures 7.2 and 7.3:

```

(define-primitive-concept Intangible *top*)
(define-primitive-concept Tangible *top*)
(define-primitive-concept Set Intangible)
(define-concept Group (and Set
                           (all element Human)
                           (at-least 2 element)))
(define-concept SmallGroup (and Group
                                   (at-most 5 element)))
(define-concept ModernGroup (and Group
                                   (at-most 4 element)
                                   (at-least 1 leader)
                                   (at-most 1 leader)
                                   (all leader Woman)))
(define-primitive-concept Vegetable Tangible)
(define-primitive-concept Animal Tangible)
(define-primitive-concept Mineral Tangible)
(define-primitive-concept Male Animal)

```

Concrete syntax	Abstract syntax
(instance o C)	$C(o)$
(related o_1 o_2 R)	$R(o_1, o_2)$

Table 8.4: Meaning of <world description>.

```
(define-primitive-concept Female Animal)
(define-primitive-concept Human Animal)
(define-concept Woman (and Female Animal))
```

```
(define-primitive-role element *r*)
(define-primitive-role leader element)
```

8.5.2 The assertional language

In this section, we describe the concrete syntax of an assertional language. This language, is defined by the following BNF grammar. In Table 7.4, we show the meaning of each of these constructs.

```
<world description> ::= <description>+
<description> ::= <description of object> |
                  <description of relation>
<description of object> ::= (instance <object> <name of concept>)
<description of relation> ::= (related <object> <object> <role>)
<name of concept> ::= <string>
<name of role> ::= <string>
<object> ::= <string>
<role> ::= <string>
```

As an example, let us define the following assertions concerning the concepts represented in Figure 7.3; the result of the assertions, together with the links for the terminological representation, are shown in the bottom part of Figure 7.6:

```
(instance G005 ModernGroup)
(instance Peter Human)
```



```
(instance Mary Woman)
(instance Tracy Woman)
(related G005 Mary element)
(related G005 Peter element)
(related G005 Tracy element)
(related G005 Mary leader)
```

8.6 Knowledge modeling in description logics

One of the main tasks in the process of developing a knowledge base in a description logic is finding the right way to break the domain into concepts and their relationships. This involves the identification of the classes (the concepts) and of the relationships between them (the roles). This task is complicated because some terms may either be represented as concepts or as roles.

While the treatment of any particular term will depend on the application, there are some general guidelines to use when trying to differentiate concepts from roles.

We know that concepts correspond to unary first-order logic predicates and that roles correspond to binary relations between concepts. At first, this distinction may seem good enough to separate concepts from roles. However, since the use of nouns in natural languages corresponds both to concepts and roles, we need to look deeper in order to make this distinction. Certain nouns reflect items that have existence independent of any others (for example, human and woman); while others reflect items that depend on others for their existence (for example, sibling and cousin). A guideline for discriminating between concepts and roles is finding out whether a description can stand on its own without implying an unmentioned object related to the object in question.

However, since classes are one of the backbones of description logics representations, we usually resort to reification in order to create classes that would be most naturally represented as roles. *Reification* is the operation through which we *reify* the relation whose properties we want to talk about.¹⁷ As an example, let us consider the noun “father”. A father is someone who

¹⁷This corresponds to the creation of entities of the universe of discourse that correspond to the relation that we want to talk about and to create functions that return the relation of that property.

bears a parent-ship relation to his children. Under this perspective, “father” should be represented as a role. However, we can also think of “father” as the set of all individuals that have children – in fact there is even *father’s day* ! – so, we can also think of “father” as a concept.

The following is a sequence of steps that should be taken in order to model knowledge in description logics.¹⁸ Alongside with the description of the steps, we apply the methodology to the representation of kinship relations, using the knowledge presented in Section 2.5.1.

Step 1 – Enumerate terms. We should create a first list of the terms that will be part of the knowledge base. These terms will become individuals, concepts, or roles.

Without making any fine-grained distinctions, it is useful to write down a list of all terms we would ever care to make statements about. The key idea is to get a comprehensive list of names without worrying about overlap between individuals, concepts or any properties that the objects might have.

In our example, this step is simplified because we can easily extract the terms and predicates from the description in Section 2.5.1.

We will have the following terms:

Human, Male, Female, Parent Father, Mother, Child, Son,
Daughter, Sibling, Brother, Sister, Married, Grandparent,
Grandfather, Grandmother, Ancestor, Cousin, Hugo, Bart, Lisa,
Maggie, Herb, Homer, Marge, Selma, Paty, Mrs.Y, Abraham,
Penelope, Jackie, Mr.Bouvier

Step 2 – Distinguish concepts from roles and from individuals.

Looking at the list produced in Step 1, we should distinguish between terms that have independent existence (the concepts) and those that depend on other terms for their existence (the roles). We should decide if we need to reify some of the potential roles as concepts.

We will make a first distinction, based on the first-order logic type of entity: concepts will correspond to unary predicates, roles will correspond to binary predicates, and individuals will correspond to constants.

¹⁸This sequence was adapted from [Brachman *et al.* 91, pp. 428–449] and [Borgida and Brachman 03], where detailed versions of this methodology can be found.

- *Concepts.*

Animal Human Male Female Man Woman

- *Reified concepts.* Since we will be talking about concepts that pertain to kinship relations, we have decided to reify the following concepts that, most naturally, are thought of as relations:

Father Mother Parent Brother Sister Grandfather Grandmother
Grandparent Sibling

- *Roles.* Based in the reified concept names that we introduced in the previous step, we name the roles. Notice that many of these role names were changed from the names presented in the list produced in Step 1:¹⁹

is-related has-child is-father is-mother is-child is-son
has-son is-daughter has-daughter is-sibling is-brother
is-sister is-married is-grandparent is-grandfather
is-grandmother is-ancestor is-descendant is-cousin

- *Individuals.*

Hugo Bart Lisa Maggie Herb Homer Marge Selma Paty Mrs.Y
Abraham Penelope Jackie Mr.Bouvier

Step 3 – Develop the concept taxonomy. We should provide a classification of all concepts, imagining their extensions. The resulting taxonomy will be used as a first reference, and could be revised when definitions are given.

As the concepts are being organized in a hierarchy, we should keep in mind that if a concept C_1 is a super-concept of C_2 , then every instance of C_2 is also an instance of C_1 .

Once the representation of the knowledge base is completed, the underlying description logics-based system will be able to determine automatically that a given concept is a sub-concept of other concepts. However, when developing the domain taxonomy, it is a good idea to sketch out these relationships by hand; once the formal representation is constructed and everything is classified, we can check the resulting taxonomy against our original conception of the domain, to see if the formal representation is correct.

We establish the following taxonomic relations (Figure 7.15):

¹⁹In order to stress that roles represent relations, we prefix their name by “is-” or “has-”.

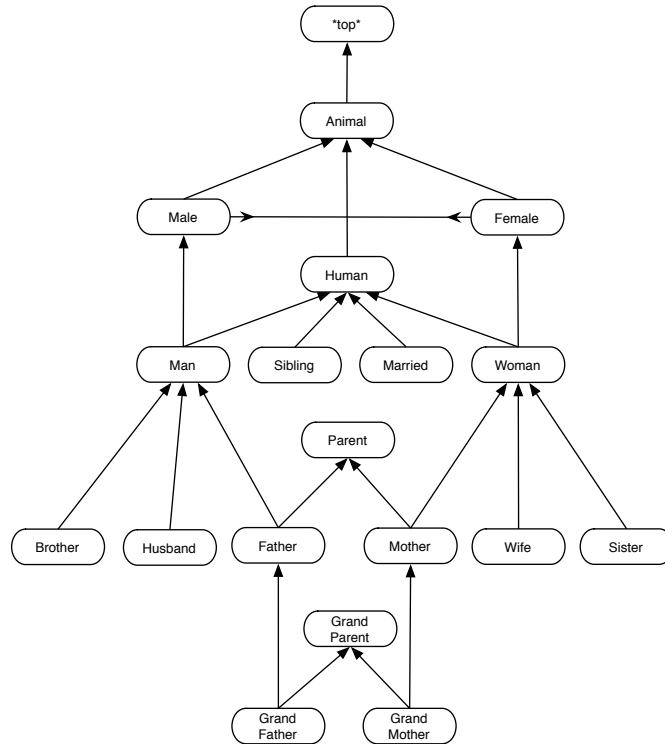


Figure 8.15: Taxonomy of concepts in the kinship example.

- An Animal is a thing (for us, the most general concept – ***top***);
- A Human is an Animal;
- A Male is an Animal;
- A Female is an Animal;
- A Man is a Male Human;
- A Woman is a Female Human;
- A Married is a Human;
- A Brother, a Husband, and a Father are Man;
- A Sister, a Wife, and a Mother are Woman;

- A Sibling is a Human;
- A Grandfather is a Father;
- A Grandmother is a Mother;
- A Parent is either a Father or a Mother;
- A Grandparent is either a Grandfather or a Grandmother.

These hierarchy relations organise the concepts as shown in the taxonomy of Figure 7.15.

Step 4 – Determine properties and parts. Once the taxonomy is laid out, we turn our attention to the internal structure of concepts. For each concept, we should write down a list of its properties. These would become roles. After this step, a revision of the taxonomy may be necessary.

In general, we may have to consider several types of roles:

- *Intrinsic properties.* These are properties that define the concept as such, for example, the children of a parent;
- *Extrinsic properties.* Properties that are not part of the definition of the concept, but are usually associated with instances of the concept;
- *Parts.* If the concept corresponds to a structured entity, we should think about the parts that compose it. These may be either physical or abstract parts;
- *Relationships to other concepts.* We should find out the relevant relationships of the role with other concepts in the knowledge base.

It is reasonable to expect that the same role will be used in many concepts. Each of the terms defined to be a role in Step 2 should be accounted for.

We can define the following properties for the concepts:

- **Animal.** Atomic primitive concept (no properties are provided);
- **Male.** Atomic primitive concept, subset of **Animal** (no properties are provided);

- Female. Atomic primitive concept, subset of Animal (no properties are provided);
- Human. Atomic primitive concept, subset of Animal (no properties are provided);
- Father. Man who has the relation has-child to Human;
- Mother. Woman who has the relation has-child to Human;
- Husband. Man who has the relation is-married to Human;
- Wife. Woman who has the relation is-married to Human;
- Brother. Man who has the relation is-sibling to Human;
- Sister. Woman who has the relation is-sibling to Human;
- Grandfather. Man who has the relation is-grandfather to Human;
- Grandmother. Woman who has the relation is-grandmother to Human.

Furthermore, we will consider the following roles:

identity	Primitive role that relates an individual with itself.
is-related	Primitive role that establishes that there is a kinship relation between two individuals.
has-son	Primitive role that relates a parent with his/her son.
is-son	Primitive role that relates a man with his parents.
has-daughter	Primitive role that relates a parent with his/her daughter.
is-daughter	Primitive role that relates a woman with her parents.
has-child $\equiv$ has-son $\sqcup$ has-daughter	Role that relates a parent with his/her children.

is-father	Primitive role that relates a man with his children.
is-mother	Primitive role that relates a woman with her children.
is-child $\equiv$ has-child ⁻	Role that relates a children with his/her parent.
is-sibling $\equiv$ (has-child ⁻ $\odot$ has-child) $\sqcap \neg$ identity	Role that relates a person with his/her brothers and sisters.
has-brother $\equiv$ (has-child ⁻ $\odot$ has-son) $\sqcap \neg$ identity	Role that relates a person with his/her brother.
has-sister $\equiv$ (has-child ⁻ $\odot$ has-daughter) $\sqcap \neg$ identity	Role that relates a person with his/her sister.
is-married	Primitive role that relates a person with his/her spouse.
is-grandfather $\equiv$ is-father $\odot$ has-child	Role that relates a man with his grandchildren.
is-grandmother $\equiv$ is-mother $\odot$ has-child	Role that relates a woman with her grandchildren.
is-grandparent $\equiv$ is-grandfather $\sqcup$ is-grandmother	Role that relates a person with his/hers grandchildren.
is-ancestor $\equiv$ has-child*	Role that relates a person with his/her descendants (children and descendants of his/her children).
is-descendant $\equiv$ is-ancestor ⁻	Role that relates a person with his/her ancestors.
is-cousin $\equiv$ has-child ⁻ $\odot$ has-sibling $\odot$ has-child	Role that relates a person with his/her cousins.

In summary, we have the following roles:

Primitive roles:

$$\text{identity} \quad (8.71)$$

$$\text{is--related} \quad (8.72)$$

$$\text{has--son} \quad (8.73)$$

$$\text{is--son} \quad (8.74)$$

$$\text{has--daughter} \quad (8.75)$$

$$\text{is--daughter} \quad (8.76)$$

$$\text{is--father} \quad (8.77)$$

$$\text{is--mother} \quad (8.78)$$

$$\text{is--married} \quad (8.79)$$

Structured roles:

$$\begin{aligned} \text{has--child} &\equiv \text{has--son} \sqcup \text{has--daughter} \\ \text{is--child} &\equiv \text{has--child}^- \\ \text{is--sibling} &\equiv (\text{has--child}^- \odot \text{has--child}) \sqcap \neg \text{identity} \\ \text{has--brother} &\equiv (\text{has--child}^- \odot \text{has--son}) \sqcap \neg \text{identity} \\ \text{has--sister} &\equiv (\text{has--child}^- \odot \text{has--daughter}) \sqcap \neg \text{identity} \\ \text{is--grandfather} &\equiv \text{is--father} \odot \text{has--child} \\ \text{is--grandmother} &\equiv \text{is--mother} \odot \text{has--child} \\ \text{is--grandparent} &\equiv \text{is--grandfather} \sqcup \text{is--grandmother} \\ \text{is--ancestor} &\equiv \text{has--child}^* \\ \text{is--descendant} &\equiv \text{is--ancestor}^- \\ \text{is--cousin} &\equiv \text{has--child}^- \odot \text{has--sibling} \odot \text{has--child} \end{aligned}$$

Step 5 – Determine restrictions. For each concept and each role that is relevant to its meaning, we should specify relevant restrictions. These include:

- *Value restrictions.* For each concept and each of its relevant roles, we should determine the class of values that can appropriately fill the role.
- *Number restrictions.* For each role, we should determine the cardinality of the role fillers.

We will consider the following restrictions:

$$\begin{aligned}\text{Wife} &\sqsubseteq \text{Woman} \sqcap \geq 1 \text{ is--married} \sqcap \leq 1 \text{ is--married} \\ \text{Husband} &\sqsubseteq \text{Man} \sqcap \geq 1 \text{ is--married} \sqcap \leq 1 \text{ is--married}\end{aligned}$$

Step 6 – Detail unrepresented value restrictions. For each value restriction specified, we should make sure that the appropriate concept exists in the taxonomy. If not, it should be added. If the concept is important in the domain being modelled, we should go through all the previous steps for the new concept.

This is not necessary in our example.

Step 7 – Distinguish essential and incidental properties. At this point, for each concept, we have a set of parent concepts (expressed in the taxonomy) and their relationships, together with a set of restrictions, such as number and value.

For each concept, we should look over this set, think about what it would mean to be a member of the class specified by the concept, and isolate the set of concepts and restrictions that would appropriately constitute a set of essential properties. These properties would be sufficient for determining membership in the concept in question. The essential properties would constitute the *definition* of the concept.

This is not necessary in our example.

Step 8 – Distinguish primitive and defined concepts. We should determine if each concept definition is complete. In order to do this, for each concept, we should ask whether the conditions determined by the above steps constitute a complete set of necessary and sufficient conditions for the concept.

We will consider the following definitions:

Primitive concepts:

$$\begin{aligned}\text{Animal} &\sqsubseteq \top \\ \text{Male} &\sqsubseteq \text{Animal}\end{aligned}$$

Female $\sqsubseteq$ Animal
 Human $\sqsubseteq$ Animal

Defined concepts:

Man $\equiv$ Human $\sqcap$ Male
 Woman $\equiv$ Human $\sqcap$ Female
 Sibling $\equiv$ Human $\sqcap \exists \text{is--sibling.Human}$
 Married $\equiv$ Human $\sqcap \exists \text{is--married.Human}$
 Father $\equiv$ Man $\sqcap \exists \text{has--child.Human}$
 Mother $\equiv$ Woman $\sqcap \exists \text{has--child.Human}$
 Husband $\equiv$ Man $\sqcap \text{Married} \sqcap \leq 1 \text{ is--married} \sqcap \geq 1 \text{ is--married}$
 Wife $\equiv$ Woman $\sqcap \text{Married} \sqcap \leq 1 \text{ is--married} \sqcap$
 $\sqcap \geq 1 \text{ is--married}$
 Brother $\equiv$ Man $\sqcap$ Sibling
 Sister $\equiv$ Woman $\sqcap$ Sibling
 Grandfather $\equiv$ Man $\sqcap \exists \text{has--child.Parent}$
 Grandmother $\equiv$ Woman $\sqcap \exists \text{has--child.Parent}$
 Parent $\equiv$ Mother $\sqcup$ Father
 Grandparent $\equiv$ Grandmother $\sqcup$ Grandfather

The result of translating the informal representation created above into a description logics will be a knowledge base of concepts, roles, and individuals. The concepts will have a set of necessary, and sometimes, necessary and sufficient conditions expressed as sets of more general concepts and restrictions.

8.7 RACER: A case study

In this section we present a system based on description logics, RACER (Renamed ABox and Concept Expression Reasoner). RACER was developed at the University of Hamburg, Germany by Ralf Möller, Volker Haarslev, and Michael Wessel. RACER is maintained by the Concordia University in

Montreal, Canada, and by the University of Applied Sciences in Wedel, Germany. RACER is available through RACER's internet page.²⁰

RACER corresponds to the description logics language $\mathcal{ALCQHI}_{R+}$. RACER handles multiple definitions and cyclic definitions of concepts. It provides an HTTP-based protocol for interconnecting description logics-based systems with interfaces and applications using an XML-based protocol. RACER also implements most of the functions specified in the Knowledge Representation System Specification.

RACER does not support role terms as specified in Table 7.3. On the other hand, it provides the possibility for the specification of a role hierarchy. A *role hierarchy* defines super- and subrole-relationships between roles. If R_1 is a superrole of R_2 , then for all pairs of individuals between which R_2 holds, R_1 must also hold.

RACER performs the following operations over the *TBox*: concept consistency, concept subsumption, computation of inconsistent concepts, and computation of the parents and children of a concept. Regarding the *ABox*, RACER performs the following operations: consistency check of an *ABox* (with respect to a *TBox*), instance testing, instance retrieval, computation of the direct types of an individual, and computation of the fillers of a role with reference to an individual.

In addition, RACER offers the possibility of reasoning using linear equations relating natural numbers, integers, real numbers, complex numbers, and strings. This enables the use of axioms of the form:²¹

```
(equivalent month-inleapyear
  (and Month
    (divisible year 4)
    (or (not-divisible year 100)
      (divisible year 400))))

(equivalent February
  (and Month
    (<= days-in-month 29)
    (or (not month-inleapyear)
      (= days-in-month 29)))
```

²⁰<http://www.sts.tu-harburg.de/~r.f.moeller/racer/>.

²¹These axioms were taken from [Haarslev and Möller 03, p. 53].

```
(or month-inleapyear
    (= days-in-month 28)))
```

This aspect will not be considered in this chapter. In the remaining part of this section, we describe part of the functionality available in RACER.²²

8.7.1 Building RACER knowledge bases

Signature. A *signature* defines the names used in a knowledge base (either in a *TBox* or in an *ABox*). In RACER it is not necessary to specify the signature, but it helps to avoid errors due to typos.

A signature specifies the atomic concepts and roles in a *TBox* and the individuals and objects in an *ABox*. A signature is defined by the following BNF expressions, where the first two (`:atomic-concepts` and `:roles`) specify aspects of the *TBox* and the last two (`:individuals` and `:objects`) specify aspects of the *ABox*.²³

```
<signature> ::= (signature [:atomic-concepts (<name of concept>+)]
                          [:roles (<role declaration>+)]
                          [:individuals (<individual>+)]
                          [:objects (<relation>+)] )
```

```
<role declaration> ::= (<name of role>
                       [:parents (<name of role>+)]
                       [:domain <name of concept>]
                       [:range <name of concept>]
                       [:inverse <name of role>]
                       [:transitive t]
                       [:reflexive t] )
```

```
<name of concept> ::= <string>
```

```
<individual> ::= <string>
```

```
<relation> ::= <string>
```

```
<name of role> ::= <string>
```

For example, given the knowledge base represented in Figure 7.6, we could

²²A complete description of this functionality can be found in [Haarslev and Möller 03].

²³Note that all the expressions in a `<signature>` are optional.

define the following signature:

```
(signature
  :atomic-concepts (Group SmallGroup ModernGroup Human
                    Woman)
  :roles ((element :range Human)
          (leader :parents (element) :range Woman))
  :individuals (G005 Peter Mary Tracy))
```

Concept axioms. The following are some of RACER's axioms for concepts. In RACER there may be more than one axiom defining the same concept. In this case, the additional axioms for the concept are added to the original concept axiom.

```
(implies <concept>1 <concept>2)
  This introduces the axiom <concept>1  $\sqsubseteq$  <concept>2.

(define-primitive-concept <concept name> <concept>)
  This axiom corresponds to syntactic sugar for the implies axiom,
  where <concept>1 is replaced by <concept name>.

(equivalent <concept>1 <concept>2)
  This introduces the axiom <concept>1  $\equiv$  <concept>2.

(define-concept <concept name> <concept>)
  This axiom corresponds to syntactic sugar for the equivalent axiom,
  where <concept>1 is replaced by <concept name>.

(disjoint <concept name>1 ... <concept name>n)
  This introduces the axiom <concept name>1  $\sqcap$  ...  $\sqcap$  <concept
  name>n =  $\perp$ .
```

Role declarations. As opposed to concept axioms, in RACER there exists just one declaration per role. There are two aspects regarding RACER's roles that are worth mentioning: roles specified as *features* (also called *attributes*) restrict a role to be functional, which means that each individual can have at most one role filler for the role; *transitive* roles are transitively closed roles.

```

(define-primitive-role <role name>
  [:transitive t]
  [:reflexive t]
  [:feature t]
  [:symmetric t]
  [:inverse <role name>]
  [:domain <concept name>]
  [:range <concept name>]
  [:parents (<role name>+)] )

```

This declaration defines a role.

It combines several aspects in the definition of a role:

- If the keywords `:transitive`, `:reflexive`, or `:feature` are specified, they declare the role to be transitive, reflexive, and a feature, respectively;
- the keyword `:symmetric` is equivalent to declaring that the role's inverse is the role itself;
- the keyword `:inverse` declares a name for the inverse of the role;
- the keywords `:domain` and `:range` declare the domain and the range of the role, respectively;
- the keyword `:parents` declares the role names that dominate the role.

```
(transitive <role name>)
```

States that a role is to be interpreted as transitive.

This declaration is redundant, since it may be obtained from `define-primitive-role`.

```
(inverse <role name>1 <role name>2)
```

Defines `<role name>1` as the name for the inverse of a `<role name>2`.

This declaration is redundant, since it may be obtained from `define-primitive-role`.

```
(domain <role name> <concept>)
```

Defines the domain for the `<role name>` to be the concept `<concept>`.

This declaration is redundant, since it may be obtained from `define-primitive-role`.

(range <role name> <concept>)

Defines the range for the <role name> to be the concept <concept>. This declaration is redundant, since it may be obtained from `define-primitive-role`.

Assertions. The following expressions enable to define the contents of the *ABox*:

(instance <object name> <concept>)

Builds a concept assertion, asserting that <object name> is an instance of <concept>.

(related <object name>₁ <object name>₂ <role>)

Builds a role assertion, asserting that two individuals (<object name>₁ and <object name>₂) are related via the role (or feature) <role>.

8.7.2 Evaluation functions

TBox

(classify-tbox)

Classifies the whole *TBox*. This function needs to be executed before queries can be asked.

(check-tbox-coherence)

This function checks if there are any unsatisfiable atomic concepts in the *TBox*. Returns a list of all atomic concepts in the *TBox* that are not satisfiable. If NIL is returned, it means that the *TBox* is coherent.

(get-concept-definition <concept name>)

Returns the definition of <concept name> compiled by RACER.

ABox

(realize-abox)

This function checks the consistency of the *ABox* and computes the most specific concepts for each individual in the *ABox*.

(compute-all-implicit-role-fillers)

Instructs RACER to compute all implicit role fillers.

8.7.3 Queries

Queries for concepts

(concept-satisfiable? <concept>)

Checks if a concept is satisfiable. Returns T or NIL.

(concept-subsumes? <concept>₁ <concept>₂)

Checks if concept <concept>₁ subsumes <concept>₂. Returns T or NIL.

(concept-equivalent? <concept>₁ <concept>₂)

Checks if concepts <concept>₁ and <concept>₂ are equivalent concepts. Returns T or NIL.

Queries for roles

(role-subsumes? <role>₁ <role>₂)

Checks if role <role>₁ subsumes <role>₂, that is, if <role>₁ is a parent role of <role>₂. Returns T or NIL.

Queries in the ABox

(check-abox-coherence)

Checks if the *ABox* is consistent. If there is a contradiction, it prints information about the culprits.

(individual-instance? <object name> <concept>)

Checks if the individual <object name> is an instance of the concept <concept> with respect to the *ABox* and its *TBox*.

8.7.4 Retrieval

TBox

(taxonomy)

Returns the whole taxonomy for the *TBox*.

(concept-synonyms <concept name>)

Returns equivalent concepts for the specified concept.

(concept-descendants <concept>)

Returns all atomic concepts in the *TBox*, which are subsumed by the specified concept.

(concept-ancestors <concept>)

Returns all atomic concepts in the *TBox*, which are subsuming the specified concept.

(describe-tbox)

Generates a description for the *TBox*.

ABox

(describe-abox)

Generates a description for the *ABox*.

(concept-instances <concept>)

Gets all individuals from the *ABox* that are instances of the specified concept.

(individual-fillers <object name> <role>)

Gets all individuals that are fillers of role <role> for the specified <object name>.

(individual-direct-types <object name>)

Gets the most specific atomic concepts of which <object name> is an instance.

(individual-types <object name>)

Gets all atomic concepts of which <object name> is an instance.

(related-individuals <role>)

Gets all pairs of individuals that are related by the relation corresponding to <role>.

8.7.5 Example

In this section we present an interaction with RACER using the example presented in Section 2.5.1 and whose model in description logics is presented in Section 7.6.

Since RACER places restrictions on the construction of roles, we had to slightly modify our previous model.

The signature of the knowledge base is defined as follows:

(signature	
:atomic-concepts	Atomic concepts in the <i>TBox</i>
(Animal Male Female Human	
Woman Man Parent Father	
Mother Child Son Daughter	
Sibling Brother Sister Married	
Grandparent Grandfather	
Grandmother Ancestor	
Cousin Aunt Uncle)	
:roles	
((is-related	<i>is-related</i> (x, y) means that
:domain Human	x bears a family relation
:range Human	to y . This is a transitive
:transitive t)	relation.
(is-son	<i>is-son</i> (x, y) means that
:domain Man	y is a son of x . It is also a
:range Human	sub-relation of
:parents (is-child))	<i>is-child</i> .
(is-daughter	<i>is-daughter</i> (x, y) means
:domain Woman	that y is a daughter of x . It
:range Human	is also a sub-relation of
:parents (is-child))	<i>is-child</i> .
(has-child	<i>has-child</i> (x, y) means
:domain Human	that x is a parent of y . It is
:range Human	also a sub-relation of
:inverse is-child	<i>is-ancestor</i> .
:parents (is-ancestor))	

```
(is-father
 :domain Father
 :range Human
 :parents (has-child))
```

is-father(x, y) means that x is the father of y . It is a sub-relation of *has-child*. The domain of this relation is the class Father.

```
(is-mother
 :domain Mother
 :range Human
 :parents (has-child))
```

A similar description holds for *is-mother*.

```
(is-sibling
 :domain Human
 :range Human
 :parents (is-related)
 :reflexive t)
```

is-sibling(x, y) means that x and y are siblings. It is a sub-relation of *is-related*. This is a reflexive relation.

```
(is-brother
 :domain Brother
 :range Man
 :parents (is-sibling))
```

is-brother(x, y) means that y is a brother of x . It is a sub-relation of *is-sibling*.

```
(is-sister
 :domain Sister
 :range Woman
 :parents (is-sibling))
```

is-sister(x, y) means that y is a sister of x .

```
(is-married
 :domain Human
 :range Human
 :reflexive t)
```

is-married(x, y) means that x is married to y . This is a reflexive relation.

```
(is-grandfather
 :domain Grandfather
 :range Human
 :parents (is-father is-grandparent))
```

is-grandfather(x, y) means that x is a grandfather of y . It is a sub-relation of *is-father* and of *is-grandparent*. The domain of this relation is the class Grandfather.

<pre>(is-grandmother :domain Grandmother :range Human :parents (is-mother is-grandparent)))</pre>	A similar description to <i>is-grandfather</i>.
<pre>(is-grandparent :domain Parent :range Human :parents (is-ancestor has-child))</pre>	<i>is-grandparent</i>(x, y) means that x is a grandparent of y. It is a sub-relation of <i>is-ancestor</i> and of <i>has-child</i>.
<pre>(is-ancestor :domain Human :range Human :inverse is-descendant :parents (is-related) :transitive t)</pre>	<i>is-ancestor</i>(x, y) means that x is an ancestor of y. This is a transitive relation. It is also a sub-relation of <i>is-related</i>.
<pre>(is-cousin :domain Human :range Human :parents (is-related) :symmetric t)</pre>	<i>is-cousin</i>(x, y) means that x and y are cousins. This is a symmetric relation.
<pre>:individuals (Hugo Bart Lisa Maggie Herb Homer Marge Selma Paty Mrs.Y Abraham Penelope Jackie Mr.Bouvier))</pre>	Individuals in the <i>ABox</i>.

The signature defines a hierarchy of roles, part of which is presented in Figure 7.16.

```
(define-primitive-concept Animal *top*)
(define-primitive-concept Human Animal)
(define-primitive-concept Male Animal)
(define-primitive-concept Female Animal)
(disjoint Female Male)
```

Disjointness of concepts is represented as shown in Figure 7.17.

```
(equivalent Woman (and Female Human))

(equivalent Man (and Male Human))
```

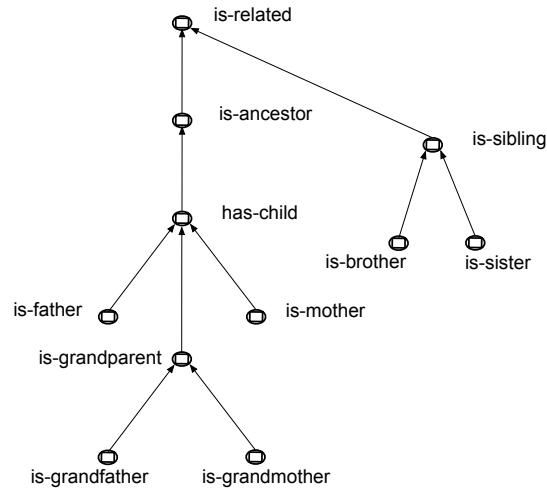


Figure 8.16: Part of the hierarchy of roles.

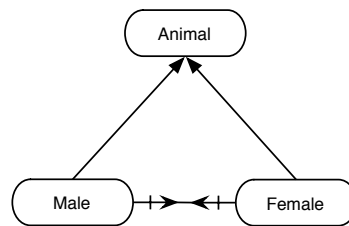


Figure 8.17: Representation of disjointness of concepts.

```

(equivalent Father
  (and Man
    (some has-child Human)))

```

A father is a man that has a relation has-child to a human. See Figure 7.18.

```

(equivalent Mother
  (and Woman
    (some has-child Human)))

```

```

(equivalent Parent
  (or Mother Father))

```

```

(equivalent Brother
  (and Man
    (some (inv has-child) Human)))

```

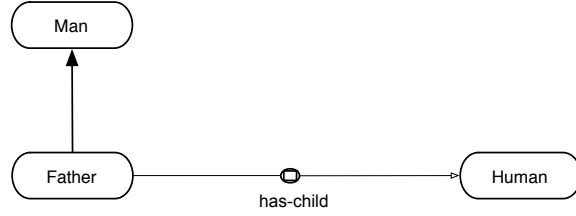


Figure 8.18: Definition of the concept Father.

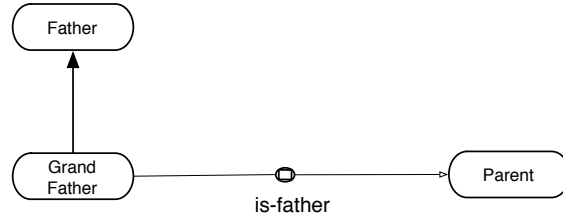


Figure 8.19: Definition of the concept Grandfather.

```
(equivalent Sister
  (and Woman
    (some (inv has-child) Human)))
```

```
(equivalent Sibling (or Brother Sister))
```

```
(equivalent Grandfather
  (and Father
    (some is-father Parent)))
```

A grandfather is a father that has a relation is-father to a parent. See Figure 7.19.

```
(equivalent GrandMother
  (and Mother
    (some is-mother Parent)))
```

```
(equivalent GrandParent
  (or GrandMother GrandFather))
```

We introduce the following individual instances and relations, based on the example shown in Figure 7.20.

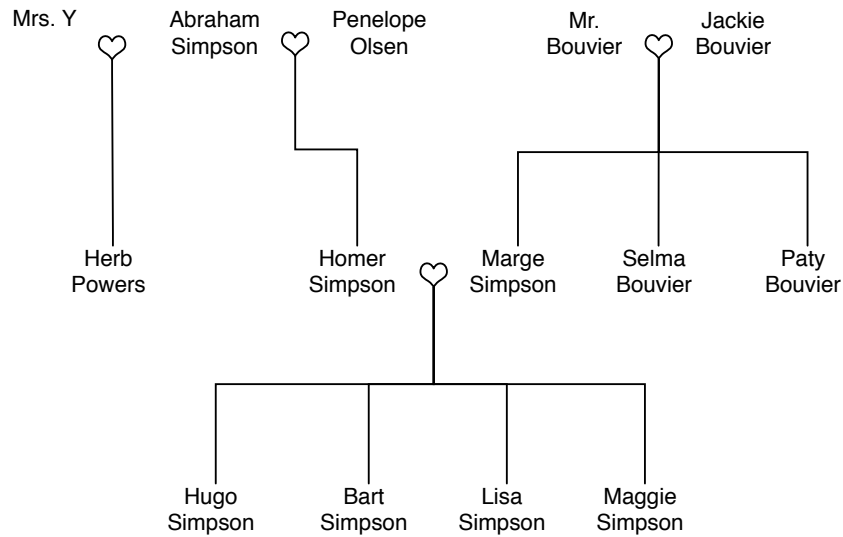


Figure 8.20: Simpsons family tree.

```
(instance Hugo Man)
(instance Bart Man)
(instance Herb Man)
(instance Homer Man)
(instance Abraham Man)
(instance Mr.Bouvier Man)
```

```
(instance Lisa Woman)
(instance Maggie Woman)
(instance Marge Woman)
(instance Selma Woman)
(instance Paty Woman)
(instance Mrs.Y Woman)
(instance Penelope Woman)
(instance Jackie Woman)
```

```
(related Abraham Penelope is-married)
(related Mr.Bouvier Jackie is-married)
(related Homer Marge is-married)
```

Individual instances and relations.

```

(related Abraham Herb is-father)
(related Abraham Homer is-father)
(related Mr.Bouvier Marge is-father)
(related Mr.Bouvier Selma is-father)
(related Mr.Bouvier Paty is-father)
(related Homer Hugo is-father)
(related Homer Bart is-father)
(related Homer Lisa is-father)
(related Homer Maggie is-father)

(related Mrs.Y Herb is-mother)
(related Penelope Homer is-mother)
(related Jackie Marge is-mother)
(related Jackie Selma is-mother)
(related Jackie Paty is-mother)
(related Marge Hugo is-mother)
(related Marge Bart is-mother)
(related Marge Lisa is-mother)
(related Marge Maggie is-mother)

```

Using the knowledge base described above, the following interaction with RACER can be obtained. There is no prompt on the system. The answers provided by RACER appear in the line following -->.

```

;;; RACER Version 1.7.19
;;; RACER: Reasoner for ABoxes and Concept Expressions Renamed
;;; Supported description logic: ALCQHIr+(D)-
;;; Copyright (C) 1998-2004, Volker Haarslev and Ralf Moeller.
;;; RACER comes with ABSOLUTELY NO WARRANTY; use at your own risk.
;;; RACER is running on Mac OS Darwin computer as node Unknown

(classify-tbox)
(check-tbox-coherence)
-->
NIL
The TBox is coherent
(no inconsistent concepts
were found).

(concept-subsumes? Sibling Brother)
-->
T
Sibling subsumes Brother

(concept-subsumes? Parent Grandparent)
-->
T
Parent subsumes
Grandparent.

(get-concept-definition GrandMother)
-->
(AND MOTHER (SOME IS-MOTHER PARENT))

```



```

(realize-abox)
(check-abox-coherence)
-->
T
ABox is coherent.

(concept-instances Father)
-->
Known instances of Father.
(ABRAHAM MR.BOUVIER HOMER)

(concept-instances GrandMother)
-->
Known instances of
Grandmother.
(PENELOPE JACKIE)

(individual-fillers Lisa
      (inv is-ancestor))
-->
Ancestors of Lisa.
(HOMER ABRAHAM MR.BOUVIER MARGE PENELOPE JACKIE)

(individual-fillers Abraham is-father)
-->
Children of Abraham.
(HERB HOMER)

(individual-direct-types Bart)
-->
((BROTHER))
(individual-types Bart)
-->
((BROTHER) (SIBLING) (HUMAN) (ANIMAL)
 (*TOP* TOP) (MAN) (MALE))

(related-individuals is-ancestor)
-->
Known instances of
the relation is ancestor.
((HOMER BART) (HOMER HUGO) (HOMER LISA)
 (HOMER MAGGIE) (ABRAHAM BART) (ABRAHAM HERB)
 (ABRAHAM HOMER) (ABRAHAM HUGO) (ABRAHAM LISA)
 (ABRAHAM MAGGIE) (MR.BOUVIER BART)
 (MR.BOUVIER HUGO) (MR.BOUVIER LISA)
 (MR.BOUVIER MAGGIE) (MR.BOUVIER MARGE)
 (MR.BOUVIER SELMA) (MR.BOUVIER PATY)
 (MARGE BART) (MARGE HUGO) (MARGE LISA)
 (MARGE MAGGIE) (MRS.Y HERB) (PENELOPE BART)
 (PENELOPE HOMER) (PENELOPE HUGO) (PENELOPE LISA)
 (PENELOPE MAGGIE) (JACKIE BART) (JACKIE HUGO)
 (JACKIE LISA) (JACKIE MAGGIE) (JACKIE MARGE)
 (JACKIE SELMA) (JACKIE PATY))

```

8.8 Bibliographic notes and historical remarks

Description logics have continuously been developed during the past 25 years, under the influence from the work of [Brachman 78]. These formalisms have also been known as *Structural Inheritance Networks*, KL-ONE-based knowledge representation languages, *terminological* knowledge representation systems, *concept languages*, and *term subsumption* languages.

The original work in what became known as description logics, was *Structural Inheritance Networks* (SI-nets, for short) [Brachman 78], a mixture of semantic network and frame system, intended to distinguish between structural relations and assertional relations. Brachman's main concern was the definition of concepts, organising them on the basis of their generality, with links that have "criteria" rather than "operational" semantics. In this way, if the implementation is faithful to its criterial semantics, one may be confident that the resulting representation is correct. SI-nets can be thought of as a kind of frame-based system with additional layers of representation conventions that carry a criterial semantics. In SI-nets, knowledge is represented as classes, subclasses, and instances. Each class (representing a concept) is associated with the characteristics that distinguish it from its superclasses.

SI-nets led to the development of KL-ONE [Brachman and Schmolze 85], the first system to deal with knowledge representation at the epistemological level (see Chapter ??). The primitives supplied by KL-ONE deal with basic relations between concepts, such as the definition that a given concept has a certain attribute, that a concept satisfies a certain relation, that a concept is defined by a set of properties, or that a concept is more specific than another one. Another characteristic of KL-ONE is the concern for the non-ambiguous definition of the meaning (semantics) of the representation. KL-ONE had a deep influence on the field of knowledge representation and gave rise to about 20 knowledge representation systems [Woods and Schmolze 92].

The success obtained by KL-ONE stems from the fact that it did not try to deal with all the necessary aspects in a knowledge representation scheme, but concentrated on the way of *defining concepts*.

The early work on the formalisms of the description logics family gave emphasis on establishing the basic terminology adopted in the modeled domain; later, the emphasis shifted to concerns about the set of concept-forming constructs admitted in the language; more recently, the shift of attention moved

towards the properties of the underlying logical systems and the designation description logics became popular.

The minimal terminological language with practical interest is called $\mathcal{AL}$ -language (attributive language) [Schmidt-Schauß and Smolka 91]. [Baader and Nutt 03, p. 49] point out that, from the semantic point of view, not all terminological languages with distinct names are in fact distinct.

A good overview of RACER can be found in [Haarslev and Möller 01]. The description of the HTTP-based protocol for interconnecting description logics-based systems with interfaces and applications using an XML-based protocol is described in [Bechhofer 00]. The functions specified in the Knowledge Representation System Specification are described in [Patel-Schneider and Swartout 93].

The book by [Baader et al. 03] provides an extensive overview on description logics, covering theory, implementation, and applications.

8.9 Exercises

1. Represent, in description logics, knowledge about the courses offered in a given University. In particular, the following aspects should be taken into account:
 - Courses can be regular lecture courses, seminars, or independent study;
 - Regular lecture courses have lectures and may have recitations;
 - Each course has a syllaby that fits within a scientific subject;
 - Each course has a required grading work that can either be an exam, one or more projects, or both;
 - Seminars and independent study do not have exams;
 - Each course is associated with a set of faculty members;
 - There is a faculty member that is in charge of the course.
2. Taking into account the information of the previous exercise, write a *TBox* that represents the courses offered by your department.
3. Represent, in description logics, knowledge about scientific papers. In particular, you should consider the following aspects:

- Papers address one or more subjects that belong to a scientific subject;
 - Papers have one or more authors that are persons;
 - Papers can either be journal papers, conference papers, or technical reports;
 - Journal papers have a limit of 100 pages and correspond to high quality work;
 - Conference papers have a limit of 10 pages and correspond to very recent work;
 - Technical reports have no page limit.
4. Suppose that you want to provide a translation of the terminological component of description logics into SNePS.

In order to do that, you should provide translations for the following aspects:

- Hierarchical relations between concepts;
 - Role relating concepts;
 - Maximum and minimum cardinality between concepts;
 - Differentiation between concepts;
 - Specialization between concepts.
5. Apply the translation provided in the previous exercise to the representation in Figure 7.3.
6. Prove that for any extension function the following expressions hold:
- (a) $C_1 \sqcap (C_2 \sqcap C_3) \equiv (C_1 \sqcap C_2) \sqcap C_3$
 - (b) $C_1 \sqcup (C_2 \sqcup C_3) \equiv (C_1 \sqcup C_2) \sqcup C_3$
 - (c) $\neg\neg C \equiv C$
 - (d) $C_1 \sqcup C_2 \equiv \neg(\neg C_1 \sqcap \neg C_2)$
 - (e) $\forall R.C_1 \sqcap \forall R.C_2 \equiv \forall R.(C_1 \sqcap C_2)$
 - (f) $\exists R.C \equiv \neg\forall R.\neg C$
 - (g) $\neg\top \equiv \perp$
 - (h) $\neg\perp \equiv \top$
7. Prove that for any extension function the following expressions hold:

- (a) $C \sqcap \neg C \equiv \perp$
- (b) $C \sqcup \neg C \equiv \top$
- (c) $C \sqcap \perp \equiv \perp$
- (d) $C \sqcap \top \equiv C$
- (e) $\forall R. \top \equiv \top$

8. Prove that $\forall R. C \sqsubseteq \forall R. D$ if and only if $C \sqsubseteq D$.

9. Represent in description logics, the following information:²⁴

- A *company* (also called a corporation) is a legal artificial entity (distinct from a natural person) which often has similar rights in law to those of a person.
- A company may have shareholders (also called *stockholders*), who may be humans, trusts or other companies.
- A company must be registered to conduct business. As part of this registration, it will also be necessary to designate the principal addresss of the company — where it may be contacted for legal process.
- A company enjoys many (or all) of the rights and obligations of individual citizens, such as the ability to own property, sign binding contracts, pay taxes, have constitutional rights and otherwise participate in society.
- Typically a company is governed by a board of directors which has a fiduciary duty to look after the interests of the company. The corporate officers such as the CEO, president, treasurer and other titled offices manage the affairs of the company.
- A shareholder or stockholder is an individual or company that legally owns one or more shares of a company. Companies listed at the stock market strive to enhance shareholder value.
- Stockholders are granted special privileges depending on the class of stock, including the right to vote on matters such as elections to the board of directors, the right to share in distributions of the company's income, the right to purchase new shares issued by the company, and the right to a company's assets during a liquidation of the company.

²⁴ Adapted from *Wikipedia*, the free encyclopedia (<http://en.wikipedia.org>).

10. Represent in description logics, the following information:²⁵

- *Transport* (or transportation, as it is called in the United States), is the movement of people and goods from one place to another. The term is derived from the Latin *trans*, meaning across, and *portare*, meaning to carry.
- Vehicles are non-living means for transportation. They are most often man-made (automobiles, motorcycles, trains, ships, aircraft), but also non-man-made means for transportations can be called vehicles, including icebergs and floating tree trunks.
- Animals, though used as means for transportation, are not called vehicles.
- Forms of transportation without vehicle or animal are walking, running, crawling, swimming and (for certain animals) flying.

11. Represent in description logics, the following information:²⁶

- A *boat* (like a ship) is a buoyant vessel designed for the purpose of transporting people and possibly goods across water. A boat is usually smaller than a ship.
- A *ship* (like a boat) is a vehicle designed for transportation across water. It is usually large enough to carry its own boats, such as lifeboats.

A rule of thumb saying is “a boat can fit on a ship, but a ship can’t fit on a boat”. The exact size at which a ship becomes a boat is often defined by local law and regulation. Submarines are always called boats.

- A boat consists of one or more buoyancy structures called *hulls* and some system of propulsion, such as, oars, paddles, a setting pole, a sail or a motor.
- The front of a ship is called the *bow*, and the rear is the *stern*. The side of the ship which is on the right when the observer is facing forward is called *starboard*; the left side is called *port*.
- Different levels of a ship are called *decks*.
- “Walls” in a ship are called *bulkheads*. Many are structural members, as well. They serve to maintain stability, prevent water

²⁵*ibid.*

²⁶*ibid.*

from flooding the entire ship in the event of a breach of the hull, and contain fire.

- Types of ships in use: cruise ship; ocean liner; tanker; auto carrier; container ship; ferry; reefer (refrigerated ship); RO-RO ship (roll on, roll off); sailing ship; tug; conventional cargo ship; tender; icebreaker.
- Ships are measured in terms of *overall length* (length along the waterline), *beam* (breadth) and *tonnage*.

12. Represent in description logics, the following information:²⁷

- An *aircraft* is any machine capable of atmospheric flight.
- There are several ways to classify an aircraft:
 - By *design*, an aircraft is classified into lighter-than-air and heavier-than-air.
 - * *Lighter-than-air* aircraft float in air in the same way that a ship floats in water, by displacing the air around the craft with a lighter gas (helium or hydrogen) or hot air. Examples of lighter-than-air aircraft include *non-steerable balloons*, such as hot air balloons, and *airships* (sometimes called dirigible balloons). The best-known type of a airship is the Zeppelin.
 - * In *heavier-than-air* aircraft, the lift is produced either by aerodynamics or by engine power.
In the case of aerodynamic lift, the aircraft is kept in the air because of aerodynamics, usually by means of wings of some kind.
With engine lift, the aircraft defeats gravity by sheer engine power.
 - By *propulsion*, an aircraft is classified into no-propulsion, piston-engine, and jet-engine.
 - * Some types of aircraft, such as the balloon or glider, *do not have any propulsion*. Balloons drift with the wind. For gliders, takeoff takes place from a high location, or the aircraft is pulled into the air by a ground-based winch or vehicle, or towed aloft by a powered “tug” aircraft.
 - * Most early aircraft used a *piston-engine* with propeller as propulsion.

²⁷ *ibid.*

- * Just prior to World War II, the first *jet engines* emerged. Different types exist, such as the ramjet, pulse jet, turbojet, and the turboprop.
- By *usage*. Three major uses for aircraft may be defined, recreational, military, and commercial.
 - * For *recreation*, almost any type of aircraft can be used, although they are usually small ones. Gliders and balloons are used almost exclusively for recreational purposes.
 - * The first use of *military* aircraft was for reconnaissance and surveillance in World War I. Soon they were adapted for attacking the ground or enemy vehicles as well, and the first bombers were born. The use of transport aircraft enabled the rapid movement of supplies, ammunition, cargo, and troops.
 - * *Commercial* aviation can be divided in passenger transport and cargo transport. For passenger transport, large planes have been developed that can transport up to 500 passengers over large distances. Commercial cargo aircraft are often similar to military transport aircraft, or might be adapted from the passenger fleets of an earlier era.

Chapter 9

Knowledge Sharing

I share no one's ideas. I have my own.
Ivan Turgenev, *Fathers and Sons*

The design and development of an appropriate knowledge base is one of the keys for the construction of an intelligent system. However, building a knowledge base remains one of the major costs in the development of an intelligent system. Most of the times, a new knowledge base has to be built from scratch which entails that most of the systems are kept in a small or medium size.

The duplication of effort in the construction of knowledge bases originates prohibitive costs and limits the capabilities of the resulting systems. For these reasons, a lot of effort has been placed in recent years in the study of how to preserve knowledge bases, how to share them, how to reuse them, and how to extend them.

There are three aspects that make the reuse of knowledge bases difficult:

1. *Heterogeneous representations.*

There is a large diversity of knowledge representation formalisms and the knowledge represented in one of the formalisms cannot be directly incorporated in another formalism.

This diversity in formalisms is unavoidable, since there is no single formalism that is better than the others, but rather, the choice of a

formalism depends largely on the aspects that are being considered.

Knowledge sharing at this level, entails the translation from one representation into another. This is a task that is actually carried out manually and there is a perceived need for the existence of tools that perform this translation task in an automatic way.

2. *Lack of conventions for communication.*

Knowledge sharing does not necessarily entail the fusion of knowledge bases. Different systems may communicate with one another provided that communication protocols are defined. There are currently no communication conventions at the knowledge base level that enable one system to query another one, nor even to understand the form in which answers are given.

3. *Lack of agreement about the knowledge level.*

Even if the problems above would have been solved, the communication between knowledge bases is made difficult, or even impossible, due to lack of agreement regarding the representation at the knowledge level: there are different and often incompatible ways of organizing and differentiating concepts and there is a different set of vocabulary used.

The work in *knowledge sharing* aims at the development of a technical infrastructure to address and solve these problems. The goal is to develop a technology that enables the development of knowledge-based systems through the selection of components from a library of reusable modules. The resulting system should also work together with other systems, using them to carry out part of the required reasoning.

9.1 KIF

In order to solve the problem of *heterogeneous representations*, programs should be developed to translate the knowledge in one formalism into another. Using this approach, the existence of n formalisms entails the development of $n^2 - n$ translators between them (Figure 9.1).

An alternative approach, that is currently being used in knowledge sharing, is to develop an intermediate language, a sort of *interlingua* (examples of which are KIF and Common Logic), and a set of translators that transform

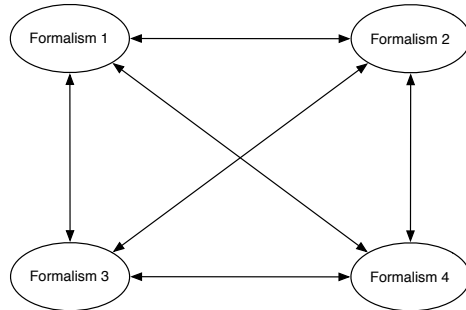


Figure 9.1: Direct translation between formalisms.

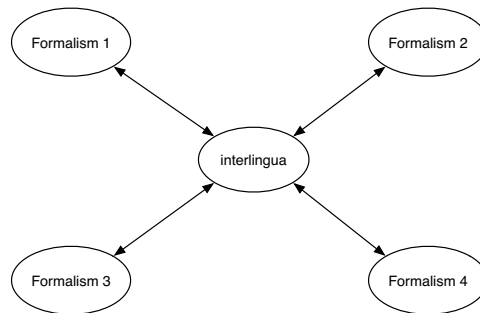


Figure 9.2: Translation using KIF.

the knowledge in different formalisms into and from this language. In this way, the existence of n formalisms requires the development of $2n$ translators (Figure 9.2).

An intermediate language that is being developed is called *Knowledge Interchange Format* (KIF) and has a role that is similar to the role of PostScript in relation with word processors and graphics: it does not aim at being a language for direct communication between Humans and machines nor a language for knowledge representation within programs, just an intermediate level to ensure the communication.

KIF is a formal language for the interchange of knowledge among disparate computer programs. KIF is not intended as a primary language for interaction with human users. KIF is also not intended to be an internal representation for knowledge within computer programs or within closely related

sets of programs. Typically, when a program reads a knowledge base in KIF, it converts the data into its own internal form. All computation is done using these internal forms. When the program needs to communicate with another program, it maps its internal data structures into KIF.

The following are the goals in the development of KIF:

1. *It should have a declarative semantics.* In other words, it should be possible to assign meaning to the structures of the language without having to use an interpreter that manipulates those structures.
2. *It should have enough expressive power.* It was agreed that the expressive power of KIF should be at least equivalent to first order logic.
3. *It should be able to represent knowledge about the representation of knowledge.* In this way, we could make explicit all the decisions about knowledge representation and we could introduce new constructs without changing the language.

As many computer-oriented languages, KIF has two varieties. In *linear* KIF, all expressions are strings of ASCII characters and, as such, are suitable for storage on serial devices and for transmission on serial media. In *structured* KIF, the legal “expressions” of the language are structured objects. Structured KIF is of special use in communication between programs operating in the same address space. There is a simple correspondence between the two varieties of KIF. For every character string, there is exactly one corresponding list structure; and, for every list structure, there is exactly one corresponding character string.

9.1.1 KIF syntax

In this section, we present a summary of the KIF language.¹ The constructs are presented using an extended BNF format.

There are three disjoint types of expressions in KIF, terms, sentences, and definitions:

¹There are many more constructs in the language, these can be found in the complete KIF specification manual [Gesenereth and Fikes 97].

Terms

Terms are used to represent entities in the world being modelled. The most important kinds of terms are individual variables, constants, functional terms, list of terms, quoted terms, and logical terms (although there are others).

$$\begin{aligned} \langle \text{term} \rangle ::= & \langle \text{variable} \rangle \mid \langle \text{constant} \rangle \mid \\ & \langle \text{functional term} \rangle \mid \langle \text{list of terms} \rangle \mid \\ & \langle \text{quoted term} \rangle \mid \langle \text{logical term} \rangle \end{aligned}$$

A **variable** is any word that starts either with the symbol ? or with the symbol @. A variable that starts with the symbol ? is called an *individual variable* and a variable that starts with the symbol @ is called a *sequence variable*.

A **constant** is any word that is not identified as a special term in KIF.

A **functional term** consists of a **constant** (that corresponds to the name of a function) followed by any number of **term** that represent the arguments of the function. There is no syntactic restriction on the number of argument terms.

$$\langle \text{functional term} \rangle ::= (\langle \text{constant} \rangle \langle \text{term} \rangle^*)$$

A **logical term** involves the use of conditional operators, whose meaning is similar to the one used in LISP.

$$\begin{aligned} \langle \text{logical term} \rangle ::= & (\text{if } \langle \text{sentence} \rangle \langle \text{term} \rangle) \mid \\ & (\text{cond } (\langle \text{sentence} \rangle \langle \text{term} \rangle)^+) \end{aligned}$$

Sentences

Sentences are used to represent statements about the world. The most important kinds of sentences are relational sentences, logical sentences, quantified sentences, equation, and inequality (although there are others).

$$\begin{aligned} \langle \text{sentence} \rangle ::= & \langle \text{relational sentence} \rangle \mid \langle \text{logical sentence} \rangle \mid \\ & \langle \text{quantified sentence} \rangle \mid \langle \text{equation} \rangle \mid \\ & \langle \text{inequality} \rangle \end{aligned}$$

A **relational sentence** consists of a **constant** (that corresponds to the name of a relation) followed by any number of **term** that represent the arguments of the relation. There is no syntactic restriction on the number of argument terms.

$$\langle \text{relational sentence} \rangle ::= (\langle \text{constant} \rangle \langle \text{term} \rangle^*)$$

A **logical sentence** enables the use of logical operators. The logical operators allowed in KIF are negation (**not**), conjunction (**and**), disjunction (**or**), implication (**=>**, that takes an arbitrary number of antecedents, as a conjunction, and a single consequent),² and equivalence (**<=>**).

$$\begin{aligned} \langle \text{logical sentence} \rangle ::= & (\text{not } \langle \text{sentence} \rangle) \mid \\ & (\text{and } \langle \text{sentence} \rangle^*) \mid \\ & (\text{or } \langle \text{sentence} \rangle^*) \mid \\ & (\text{=>} \langle \text{sentence} \rangle^* \langle \text{sentence} \rangle) \mid \\ & (\text{<=>} \langle \text{sentence} \rangle \langle \text{sentence} \rangle) \end{aligned}$$

A **quantified sentence** enables the use of the universal quantifier (**forall**) or the existential quantifier (**exists**). The quantified variables can either be unrestricted, a variable associated with a relational term that denotes a relation that restricts the domain of the variable, or a variable and a term that specifies the set over which the variable ranges.

$$\begin{aligned} \langle \text{quantified sentence} \rangle ::= & (\text{forall } (\langle \text{var specification} \rangle^+) \langle \text{sentence} \rangle) \mid \\ & (\text{exists } (\langle \text{var specification} \rangle^+) \langle \text{sentence} \rangle) \\ \langle \text{var specification} \rangle ::= & \langle \text{individual variable} \rangle^+ \mid \\ & (\langle \text{individual variable} \rangle \langle \text{constant} \rangle) \mid \\ & (\langle \text{individual variable} \rangle \text{ in } \langle \text{term} \rangle) \\ \langle \text{equation} \rangle ::= & (= \langle \text{term} \rangle \langle \text{term} \rangle) \\ \langle \text{inequality} \rangle ::= & (/= \langle \text{term} \rangle \langle \text{term} \rangle) \end{aligned}$$

Definitions

Definitions are used to define constants (entities in the world, function names or relation names). Definitions correspond to sentences that are true “by definition”, in a way that distinguishes them from sentences that express

²There is an alternative syntax for implication that is not covered in this chapter.

contingent properties of the world. They associate names (constants) with their definition. A definition has the effect of defining a set of axioms and associating them with the name of the corresponding constant. For example, we can define that a triangle is a polygon with three sides.

There are two basic categories of definitions, unrestricted definitions and restricted definitions.

`<definition> ::= <unrestricted definition> | <restricted definition>`

`<restricted definition> ::= <complete definition> | <partial definition>`

1. *Unrestricted definitions* do not impose any restriction in the way the entity is defined. Unrestricted definitions can be used to provide definitions for objects, functions, and relations.

```
<unrestricted definition> ::=
  (defobject <constant> [<string>] <sentence>*) |
  (deffunction <constant> [<string>] <sentence>*) |
  (defrelation <constant> [<string>] <sentence>*)
```

The `string` used in a definition has the role of a comment and is not used in the construction of the axiom that corresponds to the definition.

We can also define a relation `R` whose domain are the positive integers as

```
(defrelation R (=> (> ?x 0) (R ?x)))
```

This corresponds to a partial definition in the sense that it restricts the possible values of `R` but does not specify an expression that is equivalent to `R`.

2. *Restricted definitions* impose well defined ways in which the constant in the definition is being defined. Typically a knowledge representation formalism imposes restrictions in the sentences that may appear as definitions. There are two kinds of restricted definitions, complete definitions and partial definitions.

- (a) *Complete definitions* specify an expression that is equivalent to the constant being defined.

```

<complete definition> ::=
  (defobject <constant> [<string>]
    [ :-> <individual variable> ]
    := <term>) |
  (deffunction <constant> (<individual variable>*)
    [<string>]
    := <term>)|
  (defrelation <constant> (<individual variable>*)
    [<string>]
    := <sentence>)

```

We can define the function that returns the paternal grandfather of a person and the relation that defines what a bachelor is, in the following way:

```

(deffunction PATERNALGRANDFATHER (?x) :=
  (father (father ?x)))

(defrelation BACHELOR (?x) :=
  (and (man ?x)(not (married ?x))))

```

These definitions originate the following axioms:

```

(<=> (paternalgrandfather ?x) (father (father ?x)))

(= (bachelor ?x) (and (man ?x)(not (married ?x))))

```

- (b) *Partial definitions* specify an expression that restricts the possible denotations of a constant. These are conservative definitions, in the sense that the addition of a constant and the corresponding axioms to a theory that does not contain that constant does not imply any additional theorems not containing that constant. Partial definitions specify conditions that are not necessarily true.

```

<partial definition> ::=

```



```

(defobject <constant> :-> <individual variable>
  [<string>]
  :=> <sentence>) |
(defun <constant> (<individual variable>*)
  :-> <individual variable>
  [<string>]
  :=> <sentence>) |
(defrelation <constant> (<individual variable>*)
  [<string>]
  :=> <sentence>)

```

9.1.2 KIF semantics

The basis for KIF semantics is a relation between the terms and sentences of the language and a conceptualization of the world. The formalization of knowledge in KIF, as in any declarative representation, requires a *conceptualization* of the world in terms of objects, functions, and relations (see Section 2.2.2).³

In our presentation of the semantics of first-order logic in Section 2.2.2, we required that functions and relations have fixed arity. However, KIF definitions allow for functions and relations with variable arity; it is acceptable for a function or a relation to contain tuples of different sizes. For example, the $+$ function contains the tuples $(1, 1, 2)$ and $(1, 1, 1, 3)$, reflecting the fact that the sum of 1 and 1 is 2 and the fact that the sum of 1 and 1 and 1 is 3.

Different users of KIF are likely to have different universes of discourse. KIF is *conceptually promiscuous* in that it does not require every user to share the same universe of discourse. On the other hand, KIF is *conceptually grounded* in that every universe of discourse is required to include certain *basic* entities. The following basic entities must occur in every universe of discourse:

- Characters.
- Words. The words of KIF are themselves objects in the universe of discourse, along with the things they denote.
- All finite tuples of objects in the universe of discourse.

³This section was adapted from [Genesereth, Fikes, *et. al.* 92].

- $\perp$, a distinguished object that occurs as the value of various functions when applied to arguments for which the functions make no sense.

As described in Section 2.2.2, an *interpretation* is a total function I that associates the constants of KIF with the elements of a conceptualization. In order to be an interpretation, a function must satisfy the following two properties.

First, the function must map constants into concepts of the appropriate type:

- It must map object constants into objects in the universe of discourse.
If c is a constant, then $I(c) \in D$.
- It must map function constants into functions on the universe of discourse.
If f is a function letter, then $I(f) \subseteq \cup_i (D^i \times D)$.
- It must map relation constants into relations on the universe of discourse.
If r is a relation letter, then $I(r) \subseteq \cup_i D^i$.
- Notice that we allow for functions and relations of variable, finite arity.
It must map logical constants into one of the boolean values *true* or *false* (which may or may not be members of the universe of discourse).
If l is a logical constant, then $I(l) \in \{true, false\}$.

Second, I must “satisfy” the axioms provided for the different constructs existing in the language.

Note that, even with these restrictions, KIF is only a “partially interpreted” language. Although the interpretations of some constants are constrained in the definition of the language, the meanings of other constants are left open (i.e. left to the imaginations of the language users).

9.1.3 Knowledge bases in KIF

Definitions and sentences are called *forms*. A *knowledge base* in KIF is a set of forms.

In order to share knowledge bases an original knowledge base in a representation formalism is translated into a KIF knowledge base, which is, in turn, translated into the knowledge base of another formalism. The translation to KIF is not very difficult. The translation from KIF is more complicated, since the same proposition may be represented in KIF in several ways that are equivalent from the semantic point of view, but have syntactic variations. The translation from KIF involves the use of rewrite rules that are equivalence preserving and that transform unrecognizable sentences into recognizable sentences.

9.2 Knowledge Query Manipulation Language

In order to solve the problem of the *lack of conventions for communication*, work is being done towards moving away from the situation where an intelligent system is a single and monolithic entity that supplies all the functions that are needed to perform a complex task. This is based on the recognition that there is need for the integration of an intelligent system in an environment so that it can communicate with conventional programs, operating system functions, data base servers, and other intelligent systems.

From this perspective, we can see that knowledge sharing can be done at two levels:

- *Centralized sharing*, where knowledge bases are imported from one system into another.
- *Distributed sharing*, where there is no actual importation of the knowledge base, but a system remotely queries another system where the desired knowledge base resides.

The goal in distributed sharing is to develop a set of specifications that allow the interoperability among intelligent systems. To reach this goal a language, *Knowledge Query and Manipulation Language* (KQML), is being developed.

KQML is a language and a protocol for exchanging information and knowledge among systems. KQML can be used as a language for an application program to interact with an intelligent system or for two or more intelligent systems to share knowledge in support of cooperative problem solving.

KQML focuses on an extensible set of performatives, which defines the permissible operations that agents may attempt on each other's knowledge and goal stores. The *performatives* comprise a substrate on which to develop higher-level models of inter-agent interaction such as contract nets and negotiation. In addition, KQML provides a basic architecture for knowledge sharing through a special class of agent called *communication facilitators* which coordinate the interactions of other agents.

KQML is both a message format and a message-handling protocol to support run-time knowledge sharing among agents. KQML expressions consist of a content expression, encapsulated in a message wrapper, which is, in turn, encapsulated in a communication wrapper.

1. The *content* of the message contains an expression written in a language that enables knowledge to be shared, for example, in KIF;
2. The *message wrapper* identifies the speech act that the sender attaches to the content, such as an assertion, a query, or a command;
3. The *communication wrapper* describes the lower level communication parameters such as the identity of the sender and recipient, a unique identifier associated with the communication, and an indication of whether the communication must be synchronous or asynchronous.

9.3 Ontologies

In order to solve the problem of *lack of agreement about the knowledge level*, work is being done regarding the problem of how to share the contents of formal knowledge bases. Sharing knowledge bases involves much more than having a common formalism (KIF) and a communication protocol (KQML). The most fundamental question is understanding the nature of what needs to be held in common between communicating agents.

If we consider how knowledge bases are designed, we came down to the conceptualization of the world (the identification of the entities, relations and functions that are important for performing some task). This conceptualization makes commitments about distinctions, granularity, and so on.

In philosophy, *Ontology*, the most fundamental branch of metaphysics, is the study of being or existence as well as the basic categories thereof.

Aristotle described Ontology as “the science of being *qua* being.” The word *qua* means “with regard to the aspect of”. Thus, ontology is the science of being with regard to the aspect of being, or the study of beings insofar as they exist. More precisely, ontology concerns determining what categories of being are fundamental and asks whether, and in what sense, the items in those categories can be said to “be”.

Different philosophers made different lists of the fundamental categories of being. The following is a sample of ontological questions:

1. What are the fundamental categories of being?
2. What is existence?
3. What are physical objects?
4. Is it possible to give an account of what it means to say that a physical object exists?
5. What are an object’s properties or relations and how are they related to the object itself?
6. Is existence a property?
7. When does an object go out of existence, as opposed to merely changing?

Knowledge representation has a long tradition in philosophy-related work, in which it refers to the subject of existence. For an AI system, what “exists” is what can be represented.

The work on knowledge sharing borrowed the term “ontology” from philosophy. An *ontology* is a specification of a conceptualization. Recall from Section 2.2.2 that a conceptualization is an abstract, simplified view of the world that we wish to represent for some purpose.⁴ Thus, an ontology is a description of the concepts and relationships that can exist for an agent or a community of agents. This definition is consistent with the current usage of the word “ontology”, as set of concept definitions, but more general. And it has a different sense of the word than its use in philosophy.

⁴Nicola Guarino [Guarino 98] argues that the notion of conceptualization presented in Section 2.2.2 needs to be revised so that an ontology can deal with *extensional* relations rather than *intensional* relations.

More important than the definition itself, is the purpose that an ontology is for: ontologies are designed for the purpose of enabling knowledge sharing and reuse. From a pragmatic standpoint, an ontology can be seen as a set of definitions of formal vocabulary. In this context, an ontology is a specification used for making ontological commitments. An *ontological commitment* is an agreement to use a vocabulary (an agreement of how to ask queries and to make assertions) in a way that is consistent (but not complete) with respect to the theory specified by an ontology. Every knowledge base is committed to some conceptualization, either explicitly or implicitly. We build knowledge-based systems that commit to ontologies. We design ontologies so we can share knowledge with and among those knowledge-based systems.

We say that a knowledge-based systems commits to an ontology if its observable actions are consistent with the definitions in the ontology. The idea of ontological commitments is based on the knowledge level perspective (see Section ??). Knowledge is attributed to systems (or agents) by observing their actions; a system “knows” something if it acts as if it had the information and is acting rationally to achieve its goals.

In the context of AI, we can describe the ontology of a program by defining a set of representational terms. An ontology is the attempt to formulate an exhaustive and rigorous conceptual schema within a given domain, typically, a hierarchical data structure containing all the relevant entities and their relationships and rules (axioms, theorems) within that domain. In such, an ontology definitions associate the names of entities in the universe of discourse (classes, relations, functions, or other objects) with human-readable text describing what the names mean, and formal axioms that constrain the interpretation and well-formed use of these terms — an ontology is the statement of a logical theory.

There are several types of ontologies (Figure 9.3):

- An ontology which is not tied to a particular problem domain but attempts to describe general entities — such as time, space, matter, object, event, action — is known as a *top-level ontology*, a *foundation ontology*, or an *upper ontology*.
- An ontology that describes the vocabulary of a a generic domain — such as chemistry, medicine, transportation — is known as a *domain ontology*.
- An ontology that describes the vocabulary of a a generic task or ac-

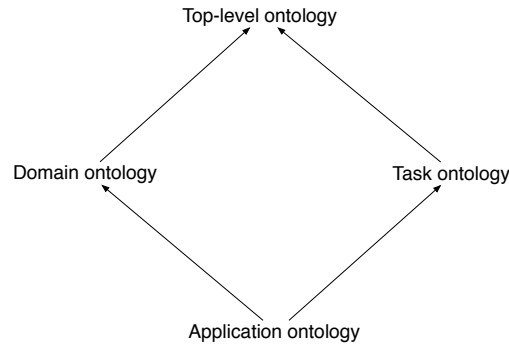


Figure 9.3: Classification of ontologies.

tivity — such as diagnosing, communicating, selling — is known as a *task ontology*.

- An *application ontology* describes the concepts relating to a particular domain and typically includes specializations of both domain and task ontologies.

It seems reasonable to attempt at having a *unified* top-level ontology for large communities of users. There are several top-level ontologies currently available:

- Cyc is a well-known and quite comprehensive ontology. It corresponds to a proprietary system under development since 1985,⁵ consisting of a top-level ontology and several domain-specific ontologies. A subset of the Cyc ontology has been released for free under the name OpenCyc.⁶
- WORDNET, a freely available database originally designed as a semantic network based on psycholinguistic principles, was expanded by addition of definitions and is now also viewed as a dictionary. It qualifies as a top-level ontology by including the most general concepts as well as more specialized concepts, related to each other, not only by the subsumption relations, but by other semantic relations as well, such as part-of and cause. However, it has not been formally axiomatized so as to make the logical relations between the concepts precise.

⁵[Lenat and Guha 90].

⁶<http://opencyc.org>.

- SUMO (Suggested Upper Merged Ontology) is another attempt to define a top-level ontology, created by an IEEE working group and freely available. SUMO is a top-level ontology intended for a variety of computer information processing systems. It was developed by Teknowledge Corporation and is one candidate for the IEEE “Standard Upper Ontology”.

9.3.1 Ontology languages

To be useful, ontologies must be expressed in a concrete notation. An *ontology language* is a formal language by which an ontology is built. There have been a number of languages for ontologies, both proprietary and standards-based:

- The Cyc project had its own ontology language based on first-order logic, called CycL.
- KIF is, among other things, an ontology language.
- DAML+OIL⁷ is a semantic markup language for Web resources. It builds on earlier W3C standards such as RDF and RDF Schema, and extends these languages with richer modelling primitives. DAML+OIL provides modelling primitives commonly found in frame-based languages. The language has a clean and well defined semantics.
- OWL⁸ is a language for making ontological statements, developed as a follow-on from RDF and RDFS, as well as earlier ontology language projects include OIL and DAML.
- Ontolingua.⁹ Provides a mechanism for defining ontologies that are portable over representation systems. Definitions written in a standard format for predicate calculus are translated by a system called Ontolingua into specialized representations, including frame-based systems as well as relational languages. This allows to share and reuse ontologies, while retaining the computational benefits of specialized implementations.

⁷<http://www.daml.org/2001/03/reference>.

⁸<http://www.w3.org/TR/2004/REC-owl-features-20040210>.

⁹<http://www.ksl.stanford.edu/software/ontolingua/>.

9.3.2 Vocabulary vs. taxonomy vs. ontology

People use the word *ontology* to mean different things, glossaries and data dictionaries, thesauri and taxonomies, and formal ontologies and inference. A *formal ontology* is a controlled vocabulary expressed in an ontology representation language. This language has a grammar for using vocabulary terms to express something meaningful within a specified domain of interest. The grammar contains formal constraints on how terms in the ontology's controlled vocabulary can be used together.

A *controlled vocabulary* is a list of terms that have been enumerated explicitly. This list is controlled by and is available from a controlled vocabulary registration authority. All terms in a controlled vocabulary have an unambiguous, non-redundant definition. This is a design goal that may not be true in practice. It depends on how strict the controlled vocabulary registration authority is regarding registration of terms into a controlled vocabulary. At a minimum, the following two rules should be enforced:

1. If the same term is commonly used to mean different concepts in different contexts, then its name is explicitly qualified to resolve this ambiguity.
2. If multiple terms are used to mean the same thing, one of the terms is identified as the preferred term in the controlled vocabulary and the other terms are listed as synonyms or aliases.

A *thesaurus* is a networked collection of controlled vocabulary terms. A thesaurus uses associative relationships in addition to parent-child relationships. The expressiveness of the associative relationships in a thesaurus vary and can be as simple as “related to term” as in term *A* is related to term *B*.

A *taxonomy* is a collection of controlled vocabulary terms organized into a hierarchical structure. Each term in a taxonomy is in one or more parent-child relationships to other terms in the taxonomy.

There may be different types of parent-child relationships in a taxonomy, but good practice limits all parent-child relationships to a single parent to be of the same type.

Some taxonomies allow multiple hierarchies, which means that a term can have multiple parents. This means that if a term appears in multiple places in a taxonomy, then it is the same term. Specifically, if a term has children

in one place in a taxonomy, then it has the same children in every other place where it appears.

9.4 Time ontology

Time goes, you say? Ah, no!
Alas, Time stays, we go.
Austin Dobson, *The Paradox of Time*,
1877

In this section we present, as an example, part of the time ontology. Rather than using a specialized ontology language, we present the time ontology using first-order logic. Time is related to almost all aspects of our lives and is also omnipresent in most of the applications. For this reason, the time ontology is one of the top-level ontologies that underlie most of the applications.

We know that an ontology corresponds to a definition of terms, thus, formally, when presenting the time ontology, we should start by listing all the terms used in the ontology. However, since this list would make our presentation hard to read, we decided to list at the outset just the terms corresponding to the main concepts and relations, presenting the other terms alongside with the text.

- *Temporal entities, instants and intervals* (see Section 9.4.1). There are three main classes (defined as predicates of one argument) in the ontology:

- *Temporal_entity*
- *Instant*
- *Interval*

Intervals, are divided into several types:

- *Proper_interval*
- *Halfinf_interval*
- *Inf_interval*
- *Neginf_interval*

- *Posinf_interval*
- *Relations between instants and intervals* (see Section 9.4.1). The following main relations link instants and intervals:
 - *Begins*
 - *Ends*
 - *Inside*
 - *Time_between*
- *Directionality of time* (see Section 9.4.2). There are two main predicates that define the directionality of time:
 - *Before*
 - *After*
- *Relations between intervals* (see Section 9.4.3). Relations between time intervals are of paramount importance to define the time ontology. The following are the main relations that can be defined among intervals:
 - *Int_equals*
 - *Int_before*
 - *Int_meets*
 - *Int_overlaps*
 - *Int_starts*
 - *Int_during*
 - *Int_finishes*
 - *Int_after*
 - *Int_met_by*
 - *Int_overlapped_by*
 - *Int_started_by*
 - *Int_contains*
 - *Int_finished_by*
- *Concatenation of intervals* (see Section 9.4.6).
 - *Concatenation*
- *Aggregates of temporal entities* (see Section 9.4.7).

- *Temp_seq*
- *Equiv_tseq*
- *I_inside*
- *Disjoint_tseqs*
- *Temporal units* (see Section 9.4.8).
 - *Temporal_unit*
 - *Calendar_unit*
 - *Clcock_unit*
 - *duration* (function)
 - *Has_un*

The following constants make up temporal units:

- **Second**
- **Minute**
- **Hour**
- **Day**
- **Week**
- **Month**
- **Year**

9.4.1 Instants and intervals

The main class in the ontology is *temporal entity*, denoted by the one-argument predicate *Temporal_entity*.¹⁰ There are only two subclasses of *Temporal_entity*, *Instant* and *Interval*. Intervals are temporal entities with extent and instants are point-like in that they have no interior points.

To make the ontology axioms easier to read, we use the variable *T* to denote either a temporal entity or an interval and we use the variable *t* to denote an instant. We should notice that this is just a convention, making no commitment about the meaning of the variables.

$$\forall t [Instant(t) \rightarrow Temporal_entity(t)] \quad (9.1)$$

¹⁰From now on, we talk about a class by referring to the predicate that identifies the members of the class.

$$\forall T [Interval(T) \rightarrow Temporal_entity(T)] \quad (9.2)$$

$$\forall x [Temporal_entity(x) \rightarrow (Instant(x) \vee Interval(x))] \quad (9.3)$$

Two basic relations are defined between instants and temporal entities, *Begins* and *Ends*:

$$Begins : Instant \times Temporal_entity \mapsto \{true, false\}$$

$Begins(t, T) = t$ is the beginning instant of temporal entity T

$$Ends : Instant \times Temporal_entity \mapsto \{true, false\}$$

$Ends(t, T) = t$ is the ending instant of temporal entity T

The beginning and end of an instant is itself and conversely.

$$\forall t [Instant(t) \leftrightarrow Begins(t, t)] \quad (9.4)$$

$$\forall t [Instant(t) \leftrightarrow Ends(t, t)] \quad (9.5)$$

The beginning and end of a temporal entity, if they exist, are unique.

$$\begin{aligned} \forall t_1, t_2, T [(Temporal_entity(T) \wedge \\ Begins(t_1, T) \wedge \\ Begins(t_2, T)) \rightarrow (t_1 = t_2)] \end{aligned} \quad (9.6)$$

$$\begin{aligned} \forall t_1, t_2, T [(Temporal_entity(T) \wedge \\ Ends(t_1, T) \wedge \\ Ends(t_2, T)) \rightarrow (t_1 = t_2)] \end{aligned} \quad (9.7)$$

A *proper interval* is an interval whose beginning and end are not identical:

$$\forall T [Proper_interval(T) \rightarrow Interval(T)] \quad (9.8)$$

$$\begin{aligned} \forall T [Proper_interval(T) \leftrightarrow \forall t_1, t_2 [(Begins(t_1, T) \wedge \\ Ends(t_2, T)) \rightarrow \\ (t_1 \neq t_2)]] \end{aligned} \quad (9.9)$$

There are intervals with no end (called *positively infinite* intervals) and intervals with no beginning (called *negatively infinite* intervals). These intervals correspond to *half-infinite* intervals.¹¹

$$\forall T [Posinf_interval(T) \rightarrow Interval(T)] \quad (9.10)$$

$$\forall T [Neginf_interval(T) \rightarrow Interval(T)] \quad (9.11)$$

$$\forall T [Posinf_interval(T) \rightarrow \neg \exists t [Ends(t, T)]] \quad (9.12)$$

$$\forall T [Neginf_interval(T) \rightarrow \neg \exists t [Begins(t, T)]] \quad (9.13)$$

$$\forall T [Halfinf_interval(T) \leftrightarrow (Posinf_interval(T) \vee Neginf_interval(T))] \quad (9.14)$$

An interval is *infinite* if it is both positively and negatively infinite.

$$\forall T [Inf_interval(T) \leftrightarrow (Posinf_interval(T) \wedge Neginf_interval(T))] \quad (9.15)$$

Figure 9.4 shows, using a description logic-like notation, the classes in the time ontology.

The predicate *Inside* is a relation between an instant and an interval.¹²

$$Inside : Instant \times Interval \mapsto \{true, false\}$$

$$Inside(t, T) = t \text{ is inside the interval } T$$

The predicate *Time_between* is a relation among a temporal entity and two instants, the beginning and end points of the temporal entity.

$$Time_between : Temporal_entity \times Instant \times Instant \mapsto \{true, false\}$$

$$Time_between(T, t_1, t_2) = T \text{ is the interval between instants } t_1 \text{ and } t_2$$

¹¹An alternative approach to handle infinite intervals is to reify the notion of infinity, by the introduction of two constants, *Positive_Infinity* and *Negative_Infinity*, corresponding to $+\infty$ and $-\infty$, respectively, and to introduce two predicates *Posinf* and *Neginf* which are true of only these points. Half-infinite intervals are then intervals that have one of these as an endpoint.

¹²The *Inside* predicate is not intended to include beginnings and ends of intervals.

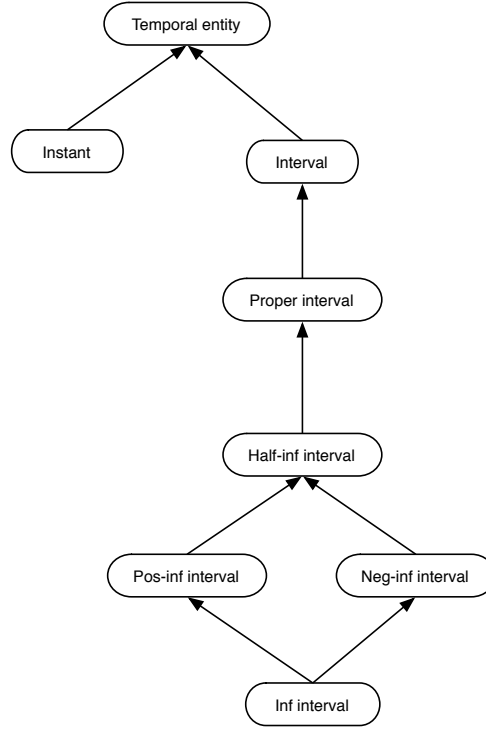


Figure 9.4: Classes in the time ontology.

$$\begin{aligned}
 \forall t_1, t_2 \ [(t_1 \neq t_2) \rightarrow \exists T \ [Time_between(T, t_1, t_2) \\
 \leftrightarrow \\
 (Begins(t_1, T) \wedge Ends(t_2, T))]] \quad (9.16)
 \end{aligned}$$

In characterizing clock and calendar terms (see Section 9.4.8) it is useful to have a relation between instants and intervals that says that the instant is inside or is the beginning of the interval:

$$\forall t, T \ [Begins_or_in(t, T) \leftrightarrow (Begins(t, T) \vee Inside(t, T))] \quad (9.17)$$

9.4.2 Directionality of time

In order to formalise the directionality of time, *Before* and *After* relations are defined over temporal entities:

$$Before : Temporal_entity \times Temporal_entity \mapsto \{true, false\}$$

$$Before(T_1, T_2) = T_1 \text{ is before } T_2$$

$$After : Temporal_entity \times Temporal_entity \mapsto \{true, false\}$$

$$After(T_1, T_2) = T_1 \text{ is after } T_2$$

The predicate *Before* is considered to be basic to instants and derived for intervals.

$$\begin{aligned} \forall T_1, T_2 [Before(T_1, T_2) \leftrightarrow \exists t_1, t_2 [Ends(t_1, T_1) \wedge \\ Begins(t_2, T_2) \wedge \\ Before(t_1, t_2)]] \end{aligned} \quad (9.18)$$

After is defined in terms of *Before*.

$$\forall T_1, T_2 [After(T_1, T_2) \leftrightarrow Before(T_2, T_1)] \quad (9.19)$$

Before is anti-reflexive, anti-symmetric and transitive.

$$\forall T_1, T_2 [Before(T_1, T_2) \rightarrow (T_1 \neq T_2)] \quad (9.20)$$

$$\forall T_1, T_2 [Before(T_1, T_2) \rightarrow \neg Before(T_2, T_1)] \quad (9.21)$$

$$\forall T_1, T_2, T_3 [(Before(T_1, T_2) \wedge Before(T_2, T_3)) \rightarrow Before(T_1, T_3)] \quad (9.22)$$

The end of an interval is not before the beginning of the interval.

$$\begin{aligned} \forall t_1, t_2, T [(Interval(T) \wedge \\ Begins(t_1, T) \wedge \\ Ends(t_2, T)) \rightarrow \neg Before(t_2, t_1)] \end{aligned} \quad (9.23)$$

The beginning of a proper interval is before the end of the interval.

$$\begin{aligned} \forall t_1, t_2, T [(Proper_interval(T) \wedge \\ Begins(t_1, T) \wedge \\ Ends(t_2, T)) \rightarrow Before(t_1, t_2)] \end{aligned} \quad (9.24)$$

If one instant is before another, then there is a time between them (see the definition of *Time_between* on page 392).

$$\begin{aligned} \forall t_1, t_2 [(&Instant(t_1) \wedge \\ &Instant(t_2) \wedge \\ &Before(t_1, t_2)) \rightarrow \exists T [Time_between(T, t_1, t_2)]] \end{aligned} \quad (9.25)$$

If an instant is inside a proper interval, then the beginning of the interval is before the instant, which is before the end of the interval. This is the main property of *Inside* (see the definition of *Inside* on page 392).

$$\begin{aligned} \forall T, t, t_1, t_2 [(&Proper_interval(T) \wedge \\ &Inside(t, T) \wedge \\ &Begins(t_1, T) \wedge \\ &Ends(t_2, T)) \rightarrow (Before(t_1, t) \wedge \\ &Before(t, t_2))] \end{aligned} \quad (9.26)$$

An instant inside a positive half-infinite interval has infinitely many instants after it; an instant inside a negative half-infinite interval has infinitely many instants before it.

$$\begin{aligned} \forall t_1, T [(&Posinf_interval(T) \wedge \\ &Inside(t_1, T)) \rightarrow \exists t_2 [After(t_2, t_1) \wedge \\ &Inside(t_2, T)]] \end{aligned} \quad (9.27)$$

$$\begin{aligned} \forall t_1, T [(&Neginf_interval(T) \wedge \\ &Inside(t_1, T)) \rightarrow \exists t_2 [Before(t_2, t_1) \wedge \\ &Inside(t_2, T)]] \end{aligned} \quad (9.28)$$

Total order in time. For most applications, time is linearly or totally ordered.

Axiom 9.29 does not belong to the basic time ontology in order to allow models of time with branching futures, among other aspects. It corresponds to an option that should be used whenever we want to impose total linear order on time.

Since Axiom 9.29 does not belong to the basic time ontology, we say that the time ontology is *silent* about whether time is totally ordered.

$$\begin{aligned} \forall t_1, t_2 [(Instant(t_1) \wedge \\ Instant(t_2)) \rightarrow (Before(t_1, t_2) \vee \\ (t_1 = t_2) \vee \\ After(t_1, t_2))] \end{aligned} \quad (9.29)$$

9.4.3 Interval relations

The axioms between intervals are known as *interval calculus*. The following relations are defined between intervals:

$$\begin{aligned} Int_equals &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_before &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_meets &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_overlaps &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_starts &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_during &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_finishes &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_after &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_met_by &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_overlapped_by &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_started_by &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_contains &: Proper_interval \times Proper_interval \mapsto \{true, false\} \\ Int_finished_by &: Proper_interval \times Proper_interval \mapsto \{true, false\} \end{aligned}$$

The meanings of these relations are obvious and their definitions are left as an exercise.

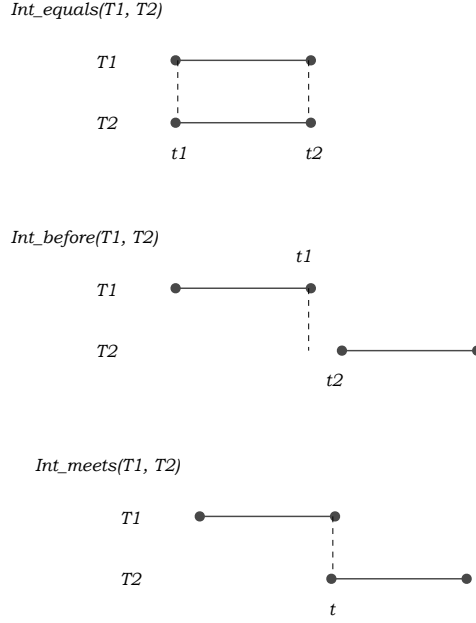


Figure 9.5: Interval calculus (Part 1).

The definitions of interval relations, in terms of *Before*, among their beginning and end points, when they exist, are given by the following axioms (see Figures 9.5 and 9.6)¹³:

$$\begin{aligned} \forall T_1, T_2 [Int_equals(T_1, T_2) \leftrightarrow & (\forall t_1 [Begins(t_1, T_1) \leftrightarrow \\ & Begins(t_1, T_2)] \wedge \\ & \forall t_2 [Ends(t_2, T_1) \leftrightarrow \\ & Ends(t_2, T_2)])] \end{aligned} \quad (9.30)$$

$$\forall T_1, T_2 [Int_before(T_1, T_2) \leftrightarrow Before(T_1, T_2)] \quad (9.31)$$

$$\forall T_1, T_2 [Int_meets(T_1, T_2) \leftrightarrow \exists t [Ends(t, T_1) \wedge Begins(t, T_2)]] \quad (9.32)$$

$$\begin{aligned} \forall T_1, T_2 [Int_overlaps(T_1, T_2) \leftrightarrow & \exists t_2, t_3 [Ends(t_2, T_1) \wedge \\ & Begins(t_3, T_2) \wedge t_2 < t_3] \end{aligned}$$

¹³In these figures, a proper interval is represented by a line ending with solid-coloured circles, representing the beginning and end points of the interval.

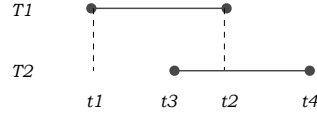
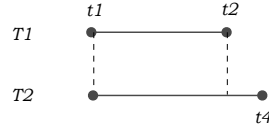
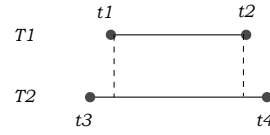
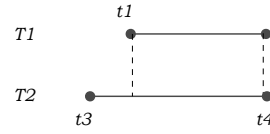
Int_overlaps(T1, T2)*Int_starts(T1, T2)**Int_during(T1, T2)**Int_finishes(T1, T2)*

Figure 9.6: Interval calculus (Part 2).

$$\begin{aligned}
 & \text{Before}(t_3, t_2) \wedge \\
 & \forall t_1 [\text{Begins}(t_1, T_1) \rightarrow \\
 & \quad \text{Before}(t_1, t_3)] \wedge \\
 & \forall t_4 [\text{Ends}(t_4, T_2) \rightarrow \\
 & \quad \text{Before}(t_2, t_4)] \quad (9.33)
 \end{aligned}$$

$$\begin{aligned}
 \forall T_1, T_2 [\text{Int_starts}(T_1, T_2) \leftrightarrow & \exists t_2 [\text{Ends}(t_2, T_1) \wedge \\
 & \forall t_1 [\text{Begins}(t_1, T_1) \leftrightarrow \\
 & \quad \text{Begins}(t_1, T_2)] \wedge \\
 & \forall t_4 [\text{Ends}(t_4, T_2) \rightarrow \\
 & \quad \text{Before}(t_2, t_4)]]] \quad (9.34)
 \end{aligned}$$

$$\begin{aligned}
\forall T_1, T_2 [Int_during(T_1, T_2) \leftrightarrow & \exists t_1, t_2 [Begins(t_1, T_1) \wedge \\
& Ends(t_2, T_1) \wedge \\
& \forall t_3 [Begins(t_3, T_2) \rightarrow \\
& \quad Before(t_3, t_1)] \wedge \\
& \forall t_4 [Ends(t_4, T_2) \rightarrow \\
& \quad Before(t_2, t_4)]]] \quad (9.35)
\end{aligned}$$

$$\begin{aligned}
\forall T_1, T_2 [Int_finishes(T_1, T_2) \leftrightarrow & \exists t_1 [Begins(t_1, T_1) \wedge \\
& \forall t_3 [Begins(t_3, T_2) \rightarrow \\
& \quad Before(t_3, t_1)] \wedge \\
& \forall t_4 [Ends(t_4, T_2) \leftrightarrow \\
& \quad Ends(t_4, T_1)]]] \quad (9.36)
\end{aligned}$$

In the development of interval calculus, it is assumed that any intervals that are *Int_equals* are identical. This means that intervals are uniquely determined by their beginning and end points. This property is called *extensional collapse*:

$$\forall T_1, T_2 [Int_equals(T_1, T_2) \leftrightarrow (T_1 = T_2)] \quad (9.37)$$

The inverse interval relations are defined in the following way:

$$\forall T_1, T_2 [Int_after(T_1, T_2) \leftrightarrow Int_before(T_2, T_1)] \quad (9.38)$$

$$\forall T_1, T_2 [Int_met_by(T_1, T_2) \leftrightarrow Int_meets(T_2, T_1)] \quad (9.39)$$

$$\forall T_1, T_2 [Int_overlapped_by(T_1, T_2) \leftrightarrow Int_overlaps(T_2, T_1)] \quad (9.40)$$

$$\forall T_1, T_2 [Int_started_by(T_1, T_2) \leftrightarrow Int_starts(T_2, T_1)] \quad (9.41)$$

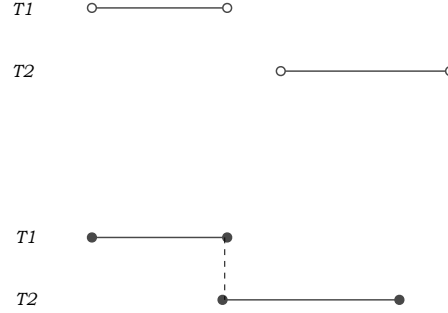
$$\forall T_1, T_2 [Int_contains(T_1, T_2) \leftrightarrow Int_during(T_2, T_1)] \quad (9.42)$$

$$\forall T_1, T_2 [Int_finished_by(T_1, T_2) \leftrightarrow Int_finishes(T_2, T_1)] \quad (9.43)$$

It is convenient to have the relation *I_before* that generalizes over several interval and instant relations, covering both *Int_before* and *Int_meets* for proper intervals (Figure 9.7).

$$I_before : Interval \times Interval \mapsto \{true, false\}$$

$$\begin{aligned}
\forall T_1, T_2 [I_before(T_1, T_2) \leftrightarrow & (Before(T_1, T_2) \vee \\
& (Proper_interval(T_1) \wedge \\
& Proper_interval(T_2) \wedge \\
& Int_meets(T_1, T_2)))] \quad (9.44)
\end{aligned}$$

Figure 9.7: Cases in I_before relation.

It is also useful to have a relation I_inside that generalizes over all temporal entities and aggregates of temporal entities (see Section 9.4.7, expression 9.64). We first define the predicate I_inside_i that generalizes over instants and intervals and covers Int_starts , $Int_finishes$, and Int_equals as well as Int_during for intervals.

$$I_inside_i : Temporal_entity \times Interval \mapsto \{true, false\}$$

We break the definition into several cases:

$$\begin{aligned} \forall T_1, T_2 [I_inside_i(T_1, T_2) \leftrightarrow & ((T_1 = T_2) \vee \\ & (Instant(T_1) \wedge \\ & Proper_interval(T_2) \wedge \\ & Inside(T_1, T_2)) \vee \\ & (Proper_interval(T_1) \wedge \\ & Proper_interval(T_2) \wedge \\ & (Int_starts(T_1, T_2) \vee \\ & Int_during(T_1, T_2) \vee \\ & Int_finishes(T_1, T_2) \vee \\ & Int_equals(T_1, T_2)))))] \end{aligned} \quad (9.45)$$

The predicate $Nonoverlap$ is defined for proper intervals intersecting in at most an instant.

$$Nonoverlap : Proper_interval \times Proper_interval \mapsto \{true, false\}$$

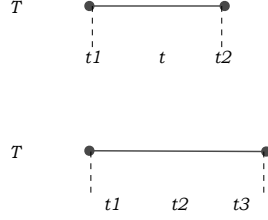


Figure 9.8: Properties of convexity.

$$\begin{aligned}
 \forall T_1, T_2 [Nonoverlap(T_1, T_2) \leftrightarrow & (Int_before(T_1, T_2) \vee \\
 & Int_after(T_1, T_2) \vee \\
 & Int_meets(T_1, T_2) \vee \\
 & Int_met_by(T_1, T_2))] \quad (9.46)
 \end{aligned}$$

In addition, it is useful to have a predicate for specifying “starts or is-during”.

$$Starts_or_during : Proper_interval \times Proper_interval \mapsto \{true, false\}$$

$$\begin{aligned}
 \forall T_1, T_2 [Starts_or_during(T_1, T_2) \leftrightarrow & (Int_starts(T_1, T_2) \vee \\
 & Int_during(T_1, T_2))] \quad (9.47)
 \end{aligned}$$

9.4.4 Convexity

Convexity implies that intervals are contiguous with respect to *Before*, in that an instant between two other instants inside an interval is also inside the interval. Convexity is defined by the following two axioms (see Figure 9.8):¹⁴

$$\begin{aligned}
 \forall T, t, t_1, t_2 [(Begins(t_1, T) \wedge \\
 & Ends(t_2, T) \wedge \\
 & Before(t_1, t) \wedge \\
 & Before(t, t_2)) \rightarrow Inside(t, T)] \quad (9.48)
 \end{aligned}$$

¹⁴Convexity may be stronger than some users will want if they are modeling time as a partial ordering. This is related with Axiom 9.29.

$$\begin{aligned}
\forall T, t_1, t_2, t_3 [& (Before(t_1, t_2) \wedge \\
& Before(t_2, t_3) \wedge \\
& Inside(t_1, T) \wedge \\
& Inside(t_3, T)) \rightarrow Inside(t_2, T)] \quad (9.49)
\end{aligned}$$

9.4.5 Density

In most applications, it is useful to have the property of *density*, that is, the property that between any two distinct instants there is a third distinct instant. The following axiom is optional in the ontology and should be used in the cases where density of time is required:

$$\begin{aligned}
\forall t_1, t_2 [& (Instant(t_1) \wedge \\
& Instant(t_2) \wedge \\
& Before(t_1, t_2)) \rightarrow \exists t [Instant(t) \wedge \\
& Before(t_1, t) \wedge \\
& Before(t, t_2)]] \quad (9.50)
\end{aligned}$$

If the ontology should support theories in which time consists of discrete instants, Axiom 9.50 should be left out, in which case it is said that the ontology is *silent* about whether time is dense.

9.4.6 Concatenation of intervals

In most applications, we look at intervals as composed of several other, smaller intervals. For example, a day is composed of 24 1-hour intervals, a year is composed of 12 months.

We say that a proper interval I is a concatenation of a set S of proper intervals if and only if S covers all of I , and all members of S are subintervals of I and are mutually disjoint.

$$Concatenation : Proper_interval \times 2^{Proper_interval} \mapsto \{true, false\}$$

In order to make the axiom that defines *Concatenation* easier to read, we define the predicates *Disjoint_members*,¹⁵ *Left_aligned*, *Right_aligned*,¹⁶

¹⁵*Member* is a predicate that tests set membership.

¹⁶Note that this definition takes into account that *Begins_or_in* covers only *Beginning_of* and *Inside*.

and *Full_contained*.

$$\begin{aligned} \forall S, i_1, i_2 [Disjoint_members(i_1, i_2, S) \leftrightarrow \\ ((Member(i_1, S) \wedge \\ Member(i_2, S)) \rightarrow (Nonoverlap(i_1, i_2) \vee \\ (i_1 = i_2)))] \end{aligned} \quad (9.51)$$

$$\begin{aligned} \forall I, S [Left_aligned(I, S) \leftrightarrow \\ \forall t [Begins_or_in(t, I) \rightarrow \exists i [Begins_or_in(t, i) \wedge \\ Member(i, S)]]] \end{aligned} \quad (9.52)$$

$$\begin{aligned} \forall I, S [Right_aligned(I, S) \leftrightarrow \\ \forall t [Begins_or_in(t, I) \rightarrow \exists i [(Ends(i) = t) \wedge \\ Member(i, S)]]] \end{aligned} \quad (9.53)$$

$$\begin{aligned} \forall I, S [Full_contained(I, S) \leftrightarrow \\ \forall i [Member(i, S) \rightarrow (Int_starts(i, I) \vee \\ Int_during(i, I) \vee \\ Int_finishes(i, I)]] \end{aligned} \quad (9.54)$$

$$\begin{aligned} \forall I, S [Concatenation(I, S) \leftrightarrow (Proper_interval(I) \wedge \\ \forall i_1, i_2 [Disjoint_members(i_1, i_2, S)] \wedge \\ Left_aligned(I, S) \wedge \\ Right_aligned(I, S) \wedge \\ Full_contained(I, S)]] \end{aligned} \quad (9.55)$$

The following properties can be proved for *Concatenation*:¹⁷

$$\forall I, S [Concatenation(I, S) \rightarrow \exists! i [Member(i, S) \wedge Int_starts(i, I)]] \quad (9.56)$$

$$\begin{aligned} \forall I, S [Concatenation(I, S) \rightarrow \exists! i [Member(i, S) \wedge \\ Int_finishes(i, I)]] \end{aligned} \quad (9.57)$$

¹⁷The uniqueness ($\exists!$) follows from *Nonoverlap*.

Except for the first and last elements of S , every element of S has elements that precede and follow it. These properties depend on the property of convexity.¹⁸

$$\begin{aligned}
& \forall I, S [Concatenation(I, S) \\
& \quad \rightarrow \\
& \quad \forall i_1 [Member(i_1, S) \rightarrow (Int_finishes(i_1, I) \vee \\
& \quad \quad \exists! i_2 [Member(i_2, S) \wedge \\
& \quad \quad \quad Int_meets(i_1, i_2)])] \quad (9.58)
\end{aligned}$$

$$\begin{aligned}
& \forall I, S [Concatenation(I, S) \\
& \quad \rightarrow \\
& \quad \forall i_2 [Member(i_2, S) \rightarrow (Int_starts(i_2, I) \vee \\
& \quad \quad \exists! i_1 [Member(i_1, S) \wedge \\
& \quad \quad \quad Int_meets(i_1, i_2)])] \quad (9.59)
\end{aligned}$$

9.4.7 Aggregates of temporal entities

A temporal aggregate is a set of temporal entities, but it has further structure. The relation *I_before* imposes a natural order on some sets of temporal entities, and we use the predicate *Temp_seq* to describe those sets.

A temporal sequence is a set of temporal entities totally ordered by the *I_before* relation. A temporal sequence has no overlapping temporal entities.

$$Temp_seq : 2^{Temporal_entity} \mapsto \{true, false\}$$

$$\begin{aligned}
\forall S [Temp_seq(S) \leftrightarrow & \forall t [Member(t, S) \rightarrow Temporal_entity(t)] \wedge \\
& \forall t_1, t_2 [(Member(t_1, S) \wedge Member(t_2, S)) \\
& \quad \rightarrow \\
& \quad ((t_1 = t_2) \vee \\
& \quad I_before(t_1, t_2) \vee \\
& \quad I_before(t_2, t_1))] \quad (9.60)
\end{aligned}$$

The same temporal sequence may be broken up into a set of intervals in many different ways. It is useful to be able to talk about temporal sequences

¹⁸*ibid.*

that are equivalent in the sense that they cover the same regions of time. Sequences S_1 and S_2 are equivalent temporal sequences when any temporal entity inside one is also inside the other.

$$Equiv_tseq : 2^{Temporal_entity} \times 2^{Temporal_entity} \mapsto \{true, false\}$$

$$\begin{aligned} \forall S_1, S_2 [Equiv_tseq(S_1, S_2) \leftrightarrow & (Temp_seq(S_1) \wedge \\ & Temp_seq(S_2) \wedge \\ & \forall T [Temporal_entity(T) \\ & \rightarrow \\ & (\exists t_1 [Member(t_1, S_1) \wedge \\ & I_inside_i(T, t_1)]) \leftrightarrow \\ & (\exists t_2 [Member(t_2, S_2) \wedge \\ & I_inside_i(T, t_2)])])] \end{aligned} \quad (9.61)$$

A minimal temporal sequence is one that is minimal in that its intervals are maximal, so that the number of intervals is minimal. We can view a week as a week or as 7 individual successive days; the first would be minimal.

We can go from a non-minimal to a minimal temporal sequence by concatenating intervals that meet. S is a *minimal temporal sequence* when any two distinct intervals in S have a temporal entity not in S between them.

$$Min_tseq : 2^{Temporal_entity} \mapsto \{true, false\}$$

$$\begin{aligned} \forall S [Min_tseq(S) \leftrightarrow & \forall t_1, t_2 [Member(t_1, S) \wedge Member(t_2, S) \\ & \rightarrow \\ & ((t_1 = t_2) \vee \\ & \exists t [I_before(t_1, t) \wedge \\ & I_before(t, t_2) \wedge \\ & \neg Member(t, S)])] \end{aligned} \quad (9.62)$$

A temporal sequence S_1 is a *minimal equivalent temporal sequence* to temporal sequence S , if S_1 is minimal and equivalent to S .

$$\forall S_1, S [Min_equiv_tseq(S_1, S) \leftrightarrow (Min_tseq(S_1) \wedge Equiv_tseq(S_1, S))] \quad (9.63)$$

We now generalize I_inside_i to the predicate I_inside , which covers both temporal entities and temporal sequences. A temporal entity is I_inside a temporal sequence if it is I_inside_i one of the elements of its minimal equivalent temporal sequence.

$$\begin{aligned}
\forall T, S [I_inside(T, S) \leftrightarrow & ((Temporal_entity(T) \wedge \\
& Temporal_entity(S) \wedge \\
& I_inside_i(T, S)) \vee \\
& (Temporal_entity(T) \wedge \\
& Temp_seq(S) \wedge \\
& \exists S_1, T_1 [Min_equiv_tseq(S_1, S) \wedge \\
& Member(T_1, S_1) \wedge \\
& I_inside_i(T, T_1)])] \quad (9.64)
\end{aligned}$$

We define the relation $Disjoint_tseqs$ between two temporal sequences. Temporal sequences S_1 and S_2 are disjoint if there is no overlap between the elements of one and the elements of the other.

$$\begin{aligned}
\forall S_1, S_2 [Disjoint_tseqs(S_1, S_2) \leftrightarrow & (Temp_seq(S_1) \wedge \\
& Temp_seq(S_2) \wedge \\
& \neg \exists t, t_1, t_2 [Member(t_1, S_1) \wedge \\
& Member(t_2, S_2) \wedge \\
& I_inside(t, t_1) \wedge \\
& I_inside(t, t_2)])] \quad (9.65)
\end{aligned}$$

The first temporal entity in a temporal sequence is the one that is I_before any of the others.

$$\begin{aligned}
\forall t, s [First_tseqs(t, s) \leftrightarrow & (Temp_seq(s) \wedge \\
& Member(t, s) \wedge \\
& \forall t_1 [Member(t_1, s) \rightarrow \\
& ((t_1 = t) \vee \\
& I_before(t, t_1))]]] \quad (9.66)
\end{aligned}$$

The predicate $Last_tseq$ is defined similarly.

$$\begin{aligned}
\forall t, s [Last_tseq(t, s) \leftrightarrow & (Temp_seq(s) \wedge \\
& Member(t, s) \wedge
\end{aligned}$$

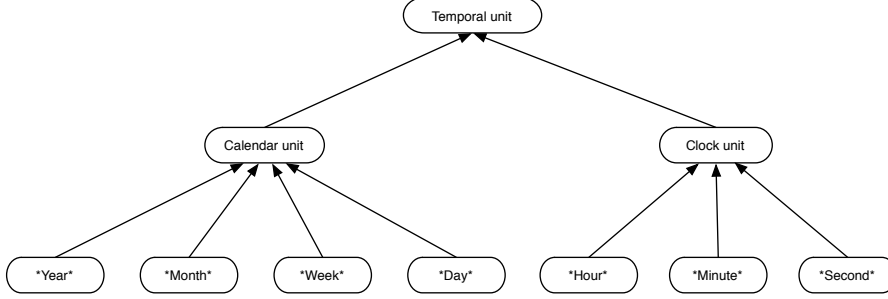


Figure 9.9: Structure of temporal units.

$$\forall t_1 [Member(t_1, s) \rightarrow ((t_1 = t) \vee I_before(t_1, t))]] \quad (9.67)$$

9.4.8 Temporal units

Temporal units are a set of entities. *Temporal units* are defined as follows:

$$\begin{aligned} \forall U [Temporal_unit(U) \leftrightarrow & (U = *Second* \vee \\ & U = *Minute* \vee \\ & U = *Hour* \vee \\ & U = *Day* \vee \\ & U = *Week* \vee \\ & U = *Month* \vee \\ & U = *Year*)] \end{aligned} \quad (9.68)$$

There are two kinds of temporal units, *calendar units* and *clock units* (Figure 9.9). The difference between calendar and clock units is related to the way the numbering of units of an interval is done.

$$\forall U [Temporal_unit(U) \leftrightarrow (Calendar_unit(U) \vee Clock_unit(U))] \quad (9.69)$$

$$\begin{aligned} \forall U [Calendar_unit(U) \leftrightarrow & (U = *Day* \vee \\ & U = *Week* \vee \\ & U = *Month* \vee \\ & U = *Year*)] \end{aligned} \quad (9.70)$$

$$\begin{aligned} \forall U [Clock_unit(U) \leftrightarrow & (U = *Second * \vee \\ & U = *Minute * \vee \\ & U = *Hour*)] \end{aligned} \quad (9.71)$$

Durations are treated as functions on intervals and temporal units. The function *duration* maps *Intervals* $\times$ *Temporal_units* into reals.¹⁹

$$duration : Intervals \times Temporal_units \mapsto \mathbb{R} \cup \{\infty\}$$

The concept of *duration* can be extended to temporal sequences in a straightforward manner.

$$duration : Temporal_seq \times Temporal_units \mapsto \mathbb{R} \cup \{\infty\}$$

If a temporal sequence is the empty set, $\emptyset$, its duration is zero.

$$\forall u [Temporal_unit(u) \rightarrow duration(\emptyset, u) = 0] \quad (9.72)$$

The duration of a singleton set is the duration of the temporal entity in it.²⁰

$$\begin{aligned} \forall t, u [(Temporal_entity(t) \wedge \\ Temporal_unit(u)) \rightarrow (duration(\{t\}, u) = \\ duration(t, u))] \end{aligned} \quad (9.73)$$

The duration of the union of two disjoint temporal sequences is the sum of their durations.

$$\begin{aligned} \forall s_1, s_2, u [(Temp_seq(s_1) \wedge \\ Temp_seq(s_2) \wedge \\ Disjoint_tseqs(s_1, s_2) \wedge \\ Temporal_unit(u)) \rightarrow (duration(s_1 \cup s_2, u) = \\ = (duration(s_1, u) + \\ + duration(s_2, u)))] \end{aligned} \quad (9.74)$$

A uniform temporal sequence is one all of whose members are of equal duration.

$$\forall S [Uniform_tseq(S) \leftrightarrow \forall t_1, t_2, u [Member(t_1, S) \wedge$$

¹⁹Because of infinite intervals, the range must also include infinity.

²⁰We need to use the predicate *Disjoint_tseqs* to ensure that there is no overlap between intervals in s_1 and intervals in s_2 .

$$\begin{aligned}
& Member(t_2, S) \wedge \\
& Temporal_unit(u)] \\
& \rightarrow \\
& (duration(t_1, u) = duration(t_2, u))]
\end{aligned} \tag{9.75}$$

Temporal units are *composed of* smaller temporal units (see the definition of *Concatenation* in Section 9.4.6). A larger temporal unit is a concatenation of smaller temporal units. The predicate *Has_un* specifies the number of smaller unit intervals that concatenate to a larger interval.

$$Has_un : integer \times Temporal_unit \times Proper_interval \mapsto \{true, false\}$$

$Has_un(n, U, I) = I$ is the concatenation of n unit intervals using unit U .

For example, if I is some month of June, then $Has_un(30, *Day*, I)$ would be true. *Has_un* is defined as follows:

$$\begin{aligned}
\forall n, U, I [Has_un(n, U, I) \leftrightarrow \exists S [(|| S || = n) \wedge \\
\forall i [Member(i, S) \rightarrow \\
(duration(i, U) = 1)] \wedge \\
Concatenation(I, S)]
\end{aligned} \tag{9.76}$$

The following axioms state the relations between successive temporal units:

$$\forall T [Minute(T) \rightarrow Has_un(60, *Second*, T)] \tag{9.77}$$

$$\forall T [Hour(T) \rightarrow Has_un(60, *Minute*, T)] \tag{9.78}$$

$$\forall T [Day(T) \rightarrow Has_un(24, *Hour*, T)] \tag{9.79}$$

$$\forall T [Week(T) \rightarrow Has_un(7, *Day*, T)] \tag{9.80}$$

$$\forall T [Year(T) \rightarrow Has_un(12, *Month*, T)] \tag{9.81}$$

Clock and calendar intervals

Before we go on, we present the external representation of a time interval. Since an interval contains the starting point but does not contain the end point (see page 392), we use the representation $[a, b[$ to denote the interval

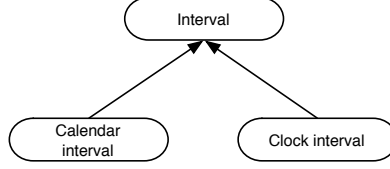


Figure 9.10: Calendar and clock intervals.

between points a and b . For example, $[5:14, 5:17[$ is an interval, for which we can state that $\text{duration}([5:14, 5:17[, *Minute*) = 3$.

A distinction is made between *clock intervals* and *calendar intervals* because they differ in how they number their unit intervals.

1. *Clock intervals* start numbering their time by zero. For hours, minutes, and seconds, we start numbering with zero. For example, the first minute of the hour $[10:00, 11:00[$ is 10:00. We number hours, minutes, and seconds for the number just completed.
2. *Calendar intervals* start numbering their time by one. For days, weeks, months, and years, we start numbering with one. For example, the first day of June is June 1st. We number days, weeks, months, and years for the number we are working on.

A *day*, as a *calendar interval*, begins at and includes midnight and goes until, but does not include, the next midnight. Including the beginning but not the end of a calendar interval in the interval may strike some as arbitrary. But we get a cleaner treatment if, for example, all times of the form 12: xx a.m., including 12:00 a.m. are part of the same hour and day, and all times of the form 10:15: xx , including 10:15:00, are part of the same minute.

$$\forall T [Clock_int(T) \rightarrow Interval(T)] \quad (9.82)$$

$$\forall T [Calendar_int(T) \rightarrow Interval(T)] \quad (9.83)$$

$$\forall T [Interval(T) \rightarrow (Clock_int(T) \vee Calendar_int(T))] \quad (9.84)$$

We define the predicate *Clk_int* for numbering clock intervals and the predicate *Cal_int* for numbering calendar intervals.

$$Clk_int : Interval \times integer \times Temporal_unit \times Interval \mapsto \{true, false\}$$

$Cal_int : Interval \times integer \times Temporal_unit \times Interval \mapsto \{true, false\}$

$Clk_int(I_1, n, U, I_2) = I_1$ is the n^{th} clock interval of type U in I_2 .

$Cal_int(I_1, n, U, I_2) = I_1$ is the n^{th} calendar interval of type U in I_2 .

For example, $Clk_int(10:03, 4, *Minute*, [10:00, 11:00])$ and $Cal_int(13_June_2005, 13, *Day*, June\ 2005)$.

To state properties of both clock and calendar intervals, we relate the two predicates with the axiom:

$$\forall I_1, n, U, I_2 [Cal_int(I_1, n, U, I_2) \leftrightarrow Clk_int(I_1, n - 1, U, I_2)] \quad (9.85)$$

Thus, if the larger unit has N smaller units, the argument n in Clk_int runs from 0 to $N - 1$, whereas in Cal_int , n runs from 1 to N .

Each of the calendar intervals is that unit long. For example, a calendar year is a year long.

$$\forall I_1, n, U, I_2 [Cal_int(I_1, n, U, I_2) \rightarrow duration(I_1, U) = 1] \quad (9.86)$$

There are properties relating to the labelling of clock and calendar intervals.

$$\begin{aligned} \forall I_1, n_1, n_2, U, I_2 [(Cal_int(I_1, n_1, U, I_2) \wedge \\ Has_un(n_2, U, I_2)) \rightarrow (0 < n_1 \leq n_2)] \end{aligned} \quad (9.87)$$

There is a 1^{st} small interval, and it starts the large interval.

$$\forall n, U, I_2 [Has_un(n, U, I_2) \rightarrow \exists! I_1 Cal_int(I_1, 1, U, I_2)] \quad (9.88)$$

$$\begin{aligned} \forall S, N, I_1, n, U, I_2 [(Has_un(S, n, U, I_2) \wedge \\ Cal_int(I_1, 1, U, I_2)) \rightarrow Int_starts(I_1, I_2)] \end{aligned} \quad (9.89)$$

There is an n^{th} small interval, and it finishes the large interval.

$$\forall n, U, I_2 [Has_un(n, U, I_2) \rightarrow \exists! I_1 Cal_int(I_1, n, U, I_2)] \quad (9.90)$$

$$\begin{aligned} \forall n, U, I_2 [Has_un(n, U, I_2) \wedge \\ Cal_int(I_1, n, U, I_2) \rightarrow Int_finishes(I_1, I_2)] \end{aligned} \quad (9.91)$$

All but the last small interval have a small interval that succeeds and is met by it; all but the first small interval have a small interval that precedes and meets it.

$$\begin{aligned}
& \forall N, n, U, I_1, I_2 \\
& [(Cal_int(I_1, n, U, I_2) \wedge \\
& \quad Has_un(n, U, I_2) \wedge \\
& \quad (n < N)) \rightarrow \exists! I_2 [Cal_int(I_2, n + 1, U, I_2) \wedge \\
& \quad \quad Int_meets(y_1, I_2)]] \quad (9.92)
\end{aligned}$$

$$\begin{aligned}
& \forall N, n, U, I_2, I_2 \\
& [(Cal_int(I_2, n, U, I_2) \wedge \\
& \quad Has_un(n, U, I_2) \wedge \\
& \quad (1 < n)) \rightarrow \exists! y_1 [Cal_int(y_1, n - 1, U, I_2) \wedge \\
& \quad \quad Int_meets(y_1, I_2)]] \quad (9.93)
\end{aligned}$$

Another way to talk about durations of either clock or calendar intervals, corresponds to the introduction of predicates and functions that are specific for each temporal unit.

The following predicates are defined:

$$\forall I_1, n, I_2 [Sec_int(I_1, n, I_2) \leftrightarrow Clk_int(I_1, n, *Second*, I_2)] \quad (9.94)$$

$$\forall I_1, n, I_2 [Min_int(I_1, n, I_2) \leftrightarrow Clk_int(I_1, n, *Minute*, I_2)] \quad (9.95)$$

$$\forall I_1, n, I_2 [Hr_int(I_1, n, I_2) \leftrightarrow Clk_int(I_1, n, *Hour*, I_2)] \quad (9.96)$$

$$\forall I_1, n, I_2 [Day_int(I_1, n, I_2) \leftrightarrow Cal_int(I_1, n, *Day*, I_2)] \quad (9.97)$$

$$\forall I_1, n, I_2 [Month_int(I_1, n, I_2) \leftrightarrow Cal_int(I_1, n, *Month*, I_2)] \quad (9.98)$$

$$\forall I_1, n, I_2 [Yr_int(I_1, n, I_2) \leftrightarrow Cal_int(I_1, n, *Year*, I_2)] \quad (9.99)$$

The following functions are defined:

$$\forall I_1, n, I_2 [Sec_int(I_1, n, I_2) \leftrightarrow sec_fn(n, I_2) = I_1] \quad (9.100)$$

$$\forall I_1, n, I_2 [Min_int(I_1, n, I_2) \leftrightarrow minut_fn(n, I_2) = I_1] \quad (9.101)$$

$$\forall I_1, n, I_2 [Hr_int(I_1, n, I_2) \leftrightarrow hr_fn(n, I_2) = I_1] \quad (9.102)$$

$$\forall I_1, n, I_2 [Day_int(I_1, n, I_2) \leftrightarrow da_fn(n, I_2) = I_1] \quad (9.103)$$

$$\forall I_1, n, I_2 [Month_int(I_1, n, I_2) \leftrightarrow mon_fn(n, I_2) = I_1] \quad (9.104)$$

$$\forall I_1, n, I_2 [Yr_int(I_1, n, I_2) \leftrightarrow yr_fn(n, I_2) = I_1] \quad (9.105)$$

The *am/pm* designation of hours is handled by the predicate *Hr_12*:

$$\forall I_1, n, I_2 [Hr_12(I_1, n, *am*, I_2) \leftrightarrow Hr_int(I_1, n, I_2)] \quad (9.106)$$

$$\forall I_1, n, I_2 [Hr_12(I_1, n, *pm*, I_2) \leftrightarrow Hr_int(I_1, n + 12, I_2)] \quad (9.107)$$

Time standards

Hours, days, weeks, months, and years are relative to a time zone.²¹ Instead of referring to time zones, it is more convenient to work in terms of the time standard that is used in a time zone.

A *time standard* is a way of computing the time, relative to a world-wide system of computing time. For each time standard, there is a zone, or geographical region, and a time of the year in which it is used for describing local times. Where and when a time standard is used have to be axiomatized, and this involves interrelating a *time ontology* and a *geographical ontology*. These relations can be quite complex.

The main functions and predicates specify a clock or calendar unit interval to be the n^{th} such unit in a larger interval. The time zone need not be specified in this predication if it is already built into the nature of the larger interval. That means that the time zone only needs to be specified in the largest interval, that is, the *Common Era*. That time zone will be inherited by all smaller intervals. Thus, the *Common Era* can be considered as a function from time standards to intervals.

$$ce : Time_standards \mapsto Intervals \quad (9.108)$$

²¹This is also true of minutes, since there are regions in the world, for example, central Australia, where the hours are not aligned with GMT hours, but are, offset half an hour. Thus, predications about all clock and calendar intervals except seconds are relative to a time zone.

Days

A day as a *calendar interval* begins at and includes midnight and goes until but does not include the next midnight. By contrast, a day as a *duration* is any interval that is 24 hours in length.

Weeks

A week is any seven consecutive days. A calendar week, by contrast, according to a commonly adopted convention, starts at midnight, Saturday night, and goes to the next midnight, Saturday night. There are 52 weeks in a year, but there are not usually 52 calendar weeks in a year.

Weeks are independent of months and years. However, we can still talk about the n^{th} week in some larger period of time, for example, the third week of the month or the fifth week of the semester. So the same modes of representation are appropriate for weeks as well.²²

$$\forall I_1, n, I_2 [Wk_int(I_1, n, I_2) \leftrightarrow wk_fn(n, I_2) = I_1] \quad (9.109)$$

$$\forall I_1, n, I_2 [Wk_int(I_1, n, I_2) \leftrightarrow Cal_int(I_1, n, *Week*, I_2)] \quad (9.110)$$

The n and I_2 arguments will often be irrelevant, when we only want to say that some period is a calendar week.

The day of the week is a calendar interval of type **Day**. The n^{th} day of the week in a week is the n^{th} day in that interval.

$$\begin{aligned} \forall I_1, n, I_2 \\ [Day_of_week(I_1, n, I_2) \leftrightarrow (Day(I_1, n, I_2) \wedge \\ \exists n_1, I_3 Wk_int(I_2, n_1, I_3))] \end{aligned} \quad (9.111)$$

The days of the week have special names in English:

$$\forall I_1, I_2 [Day_of_week(I_1, 1, I_2) \leftrightarrow Sunday(I_1, I_2)] \quad (9.112)$$

$$\forall I_1, I_2 [Day_of_week(I_1, 2, I_2) \leftrightarrow Monday(I_1, I_2)] \quad (9.113)$$

$$\forall I_1, I_2 [Day_of_week(I_1, 3, I_2) \leftrightarrow Tuesday(I_1, I_2)] \quad (9.114)$$

²²The use of the function *wk_fn* depend on extensional collapse.

$$\forall I_1, I_2 [Day_of_week(I_1, 4, I_2) \leftrightarrow Wednesday(I_1, I_2)] \quad (9.115)$$

$$\forall I_1, I_2 [Day_of_week(I_1, 5, I_2) \leftrightarrow Thursday(I_1, I_2)] \quad (9.116)$$

$$\forall I_1, I_2 [Day_of_week(I_1, 6, I_2) \leftrightarrow Friday(I_1, I_2)] \quad (9.117)$$

$$\forall I_1, I_2 [Day_of_week(I_1, 7, I_2) \leftrightarrow Saturday(I_1, I_2)] \quad (9.118)$$

Since a day of the week is also a calendar day, it can be shown that it is a day long.

$$\forall I_1, I_2, n [Day_of_week(I_1, n, I_2) \rightarrow Day(I_1)] \quad (9.119)$$

One correspondance will anchor the cycle of weeks to the rest of the calendar, for example, saying that January 1, 2002 was the Tuesday of some week x .

$$\forall z [\exists I_2 Tuesday(day_fn(1, mon_fn(1, yr_fn(2002, ce(z))))), I_2] \quad (9.120)$$

We can define week days and weekend days as follows:

$$\begin{aligned} \forall I_1, I_2 [Week_day(I_1, I_2) \leftrightarrow & (Monday(I_1, I_2) \vee \\ & Tuesday(I_1, I_2) \vee \\ & Wednesday(I_1, I_2) \vee \\ & Thursday(I_1, I_2) \vee \\ & Friday(I_1, I_2))] \end{aligned} \quad (9.121)$$

$$\begin{aligned} \forall I_1, I_2 [Week_end_day(I_1, I_2) \leftrightarrow & (Saturday(I_1, I_2) \vee \\ & Sunday(I_1, I_2))] \end{aligned} \quad (9.122)$$

Months and years

Months have special names in English:

$$\forall I_1, I_2 [Month_int(I_1, 1, I_2) \leftrightarrow January(I_1, I_2)] \quad (9.123)$$

$$\forall I_1, I_2 [Month_int(I_1, 2, I_2) \leftrightarrow February(I_1, I_2)] \quad (9.124)$$

$$\forall I_1, I_2 [Month_int(I_1, 3, I_2) \leftrightarrow March(I_1, I_2)] \quad (9.125)$$

$$\forall I_1, I_2 [Month_int(I_1, 4, I_2) \leftrightarrow April(I_1, I_2)] \quad (9.126)$$

$$\forall I_1, I_2 [Month_int(I_1, 5, I_2) \leftrightarrow May(I_1, I_2)] \quad (9.127)$$

$$\forall I_1, I_2 [Month_int(I_1, 6, I_2) \leftrightarrow June(I_1, I_2)] \quad (9.128)$$

$$\forall I_1, I_2 [Month_int(I_1, 7, I_2) \leftrightarrow July(I_1, I_2)] \quad (9.129)$$

$$\forall I_1, I_2 [Month_int(I_1, 8, I_2) \leftrightarrow August(I_1, I_2)] \quad (9.130)$$

$$\forall I_1, I_2 [Month_int(I_1, 9, I_2) \leftrightarrow September(I_1, I_2)] \quad (9.131)$$

$$\forall I_1, I_2 [Month_int(I_1, 10, I_2) \leftrightarrow October(I_1, I_2)] \quad (9.132)$$

$$\forall I_1, I_2 [Month_int(I_1, 11, I_2) \leftrightarrow November(I_1, I_2)] \quad (9.133)$$

$$\forall I_1, I_2 [Month_int(I_1, 12, I_2) \leftrightarrow December(I_1, I_2)] \quad (9.134)$$

The number of days in a month have to be specified for individual months.

$$\forall m, y [January(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.135)$$

$$\forall m, y [March(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.136)$$

$$\forall m, y [April(m, y) \rightarrow Has_un(30, *Day*, m)] \quad (9.137)$$

$$\forall m, y [May(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.138)$$

$$\forall m, y [June(m, y) \rightarrow Has_un(30, *Day*, m)] \quad (9.139)$$

$$\forall m, y [July(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.140)$$

$$\forall m, y [August(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.141)$$

$$\forall m, y [September(m, y) \rightarrow Has_un(30, *Day*, m)] \quad (9.142)$$

$$\forall m, y [October(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.143)$$

$$\forall m, y [November(m, y) \rightarrow Has_un(30, *Day*, m)] \quad (9.144)$$

$$\forall m, y [December(m, y) \rightarrow Has_un(31, *Day*, m)] \quad (9.145)$$

The definition of a leap year is as follows:²³

$$\begin{aligned} \forall z [Leap_year(y) \leftrightarrow \exists n, x [Year(y, n, ce(z)) \wedge \\ ((Divides(400, n) \vee \\ (Divides(4, n) \wedge \\ \neg Divides(100, n))))]] \end{aligned} \quad (9.146)$$

Now the number of days in February can be specified:

$$\forall m, y [February(m, y) \wedge Leap_year(y) \rightarrow Has_un(29, *Day*, m)] \quad (9.147)$$

$$\forall m, y [February(m, y) \wedge \neg Leap_year(y) \rightarrow Has_un(28, *Day*, m)] \quad (9.148)$$

²³The predicate *Divides* corresponds to integer division.

Time stamps

The standard notation for times list the year, month, day, hour, minute, and second.

We define the predicate *Time_of* with 8 arguments, a time, year, month, day, hour, minute, second, and time-zone.

The *Time_of* predicate is convenient for doing temporal arithmetic. It uses the function *secs_from_start* that computes the duration, in seconds, of the interval that spans from the start of the *Common Era* till the specified date.

$$\begin{aligned} \forall t, y, m, d, h, n, s, z \ [& \text{Time_of}(t, y, m, d, h, n, s, z) \\ & \leftrightarrow \\ & \text{Begins_or_in}(t, \text{secs_from_start}(y, m, d, h, n, s, z))] \\ \text{secs_from_start}(y, m, d, h, n, s, z) = & \\ = \text{sec_fn}(s, \text{min_fn}(n, \text{hr_fn}(h, \text{da_fn}(d, \text{mon_fn}(m, \text{yr_fn}(y, \text{ce}(z))))))) & \end{aligned}$$

For example, an instant t has the time “7:45:00 pm GMT, Sunday, June 13, 2005” if the following properties hold for t :²⁴

$$\text{Time_of}(t, 2005, 6, 13, 19, 45, 00, *GMT*) \quad (9.149)$$

$$\exists w, i \ [\text{Begins_or_in}(t, w) \wedge \text{Sunday}(w, i)] \quad (9.150)$$

The relations among time zones can be expressed in terms of *Time_of*. For example, knowing that **GMT** is one hour less than the Central European Time (**CET**), we can write:

$$\begin{aligned} \forall t, y, m, d, h, n, s \ [h < 23 \rightarrow & (\text{Time_of}(t, y, m, d, h, n, s, *GMT*) \leftrightarrow \\ & \text{Time_of}(t, y, m, d, h + 1, n, s, *CET*))] \end{aligned} \quad (9.151)$$

$$\begin{aligned} \forall t, y, m, d, h, n, s \ [h \geq 23 \rightarrow & [\text{Time_of}(t, y, m, d, h, n, s, *GMT*) \leftrightarrow \\ & \text{Time_of}(t, y, m, d + 1, h - 23, n, s, *CET*)]] \end{aligned} \quad (9.152)$$

²⁴The second line says that t is in the Sunday w of some week i .

9.4.9 Linking time and events

The time ontology links to other things in the world through four predicates *At_time*, *During*, *Holds*, and *Time_span*. We assume that another ontology provides for the description of events — either a general ontology of event structure abstractly conceived, or specific, domain-dependent ontologies.

The term *eventuality* will be used to cover events, states, processes, propositions, states of affairs, and anything else that can be located with respect to time. The possible natures of eventualities would be spelled out in the event ontologies.

The predicate *At_time* relates an eventuality to an instant, and is intended to say that the eventuality holds, occurs, or is taking place at that time.

$$\forall e, t [At_time(e, t) \rightarrow Instant(t)] \quad (9.153)$$

The predicate *During* relates an eventuality to an interval, and is intended to say that the eventuality holds, occurs, or is taking place throughout that interval.

$$\forall e, T [During(e, T) \rightarrow Interval(T)] \quad (9.154)$$

If an eventuality occurs during an interval, it occurs at every instant inside the interval and during every subinterval.

$$\forall e, T [(During(e, T) \wedge Inside(t, T)) \rightarrow At_time(e, t)] \quad (9.155)$$

$$\forall e, T, T_1 [(During(e, T) \wedge Int_during(T_1, T)) \rightarrow During(e, T_1)] \quad (9.156)$$

Whether a particular process is viewed as instantaneous or as occurring over an interval is a granularity decision that may vary according to the context of use, and is assumed to be provided by the event ontology.

Often the eventualities in the event ontology are best thought of as propositions, and the relation between these and times is most naturally called *Holds*. The predicate *Holds*(*e*, *T*) would say that *e* holds at instant *T* or during interval *T*.

The predicate *Holds* would be part of the event ontology, not part of the time ontology, although its second argument would be provided by the time ontology. The designers of the event ontology may or may not want to relate *Holds* to *At_time* and *During* by axioms such as the following:

$$\forall e, t [(Holds(e, t) \wedge Instant(t)) \leftrightarrow At_time(e, t)] \quad (9.157)$$

$$\forall e, T [(Holds(e, T) \wedge Interval(T)) \leftrightarrow During(e, T)] \quad (9.158)$$

Similarly, the event ontology may provide other ways of linking events with times, for example, by including a time parameter in predications, $P(x, t)$.

The predicate *Time_span* relates eventualities to instants or intervals. For contiguous states and processes, it tells the entire instant or interval for which the state or process obtains or takes place (the predicate *Temp_seq* is defined in page 404).

$$\forall e, T [Time_span(T, e) \rightarrow (Temporal_entity(T) \wedge Temp_seq(T))] \quad (9.159)$$

$$\forall e, T [(Time_span(T, e) \wedge Interval(T)) \rightarrow During(e, T)] \quad (9.160)$$

$$\forall e, t [(Time_span(t, e) \wedge Instant(t)) \rightarrow At_time(e, t)] \quad (9.161)$$

$$\begin{aligned} \forall e, t, T [(Time_span(T, e) \wedge \\ Interval(T) \wedge \\ \neg Inside(t, T) \wedge \\ \neg Begins(t, T) \wedge \\ \neg Ends(t, T)) \rightarrow \neg At_time(e, t)] \end{aligned} \quad (9.162)$$

$$\begin{aligned} \forall e, t, t_1 [(Time_span(t, e) \wedge \\ Instant(t) \wedge \\ (t_1 \neq t)) \rightarrow \neg At_time(e, t_1)] \end{aligned} \quad (9.163)$$

Whether the eventuality obtains at the beginning and end points of its time span is a matter for the event ontology to specify. The silence here on this issue is the reason *Time_span* is not defined in terms of necessary and sufficient conditions.

The event ontology could extend temporal functions and predicates to apply to events in the obvious way, e.g.,

$$\forall e, T, t [Ev_begins(t, e) \leftrightarrow (Time_span(T, e) \wedge Begins(t, T))] \quad (9.164)$$

This would not be part of the time ontology, but would be consistent with it.

9.5 Bibliographic notes and historical remarks

Major AI conferences have devoted several papers and panels to the Knowledge Sharing Effort. Among the articles published, the following are of particular interest: [Gesenereth and Fikes 97] that includes the full specification of KIF; [Neches *et al.* 91] that presents a very good introduction to the problem of knowledge sharing; and [Patil *et al.* 92] that corresponds to progress report on the DARPA knowledge sharing effort, describing the main results that were obtained in the development of KIF, in the knowledge representation system specification, in the knowledge query and manipulation language, and in the development of ontologies.

The bulk of the work on knowledge sharing is organized under the Knowledge-Sharing Effort Project [Neches *et al.* 91], sponsored in the United States by the Air Force Office of Scientific Research (AFOSR), the Defense Advanced Research Projects Agency (DARPA), the Corporation for National Research Initiatives (CNRI), and the National Science Foundation (NSF).

KIF is described in [Gesenereth, Fikes, *et al.* 92]. (KQML) is described in [Patil *et al.* 92].

The work on knowledge sharing borrowed the term “ontology” from philosophy. The paper by [Smith 98], argues that the notion *formal ontology* was introduced in the beginning of the 20th century by the German Philosopher Edmund Husserl (1859–1938), whose work is reprinted in [Husserl 75], as the study of the interconnections of things with objects and properties, parts and wholes, relations and collectives.

The definition of ontology, as used in this chapter, was introduced by [Gruber 93]. The types of ontologies were introduced by [Guarino 98].

The time ontology presented in Section 9.4 is heavily based on the work of Jerry Hobbs [Hobbs 02]. On the Hobbs’ paper the complete time ontology is presented, together with several options that can be taken and a mapping to the DAML ontology. In our approach, we decided, both to leave out part of the description of the ontology and to adopt some of the facultative options.

Interval calculus has been studied by [Allen 83], [Allen 84], [Allen and Kautz 85], and [Allen and Hayes 89].

The overall goal of the work being done under the heading *Semantic web* is the creation and exploitation of rich semantic structures for machine-supported access to and automated processing of distributed, explicit and

implicit knowledge. This purpose drives development of new methods, invention of tools, and investigations of the Semantic Web applications of the future.

The DARPA Agent Markup Language (DAML) Program officially began in August 2000. The goal of the DAML effort is to develop a language and tools to facilitate the concept of the Semantic Web.

References

- [Alagar and Periyasemy 98] Alagar V.S. and Periyasemy K., *Specifications of Software Systems*, Heidelberg, Germany: Springer-Verlag, 1998.
- [Alchourrón, Gärdenfors and Makinson 85] Alchourrón C.E., Gärdenfors P., and Makinson D., “On the Logic of Theory Change: partial meet contraction and revision functions”, *Journal of Symbolic Logic* 50, No. 2, pp. 510–530, 1985.
- [Alchourrón and Makinson 82] Alchourrón C.E. and Makinson D., “On the logic of theory change: contraction functions and their associated revision functions”, *Theoria* 48, pp. 14–37, 1982.
- [Allen 83] Allen J.F., “Maintaining Knowledge about Temporal Intervals”, *Communications of the ACM* 26, No. 11, pp. 832–843, 1983.
- [Allen 84] Allen J.F., “Towards a General Theory of Action and Time”, *Artificial Intelligence* 23, No. 2, pp. 123–154, 1984.
- [Allen 85] Allen J.F., “A Common-sense Theory of Time”, *Proc. IJCAI-85*, pp. 528–531, 1985.
- [Allen, Fikes and Sandewall 91] Allen J., Fikes R. and Sandewall E. (eds.), *Principles of Knowledge Representation and Reasoning: Proceedings of the Second International Conference*, San Mateo, CA: Morgan Kaufmann Publishers Inc., 1991.
- [Allen and Hayes 89] Allen J.F. and Hayes P.J., “Moments and points in an interval-based temporal logic”, *Computational Intelligence* 5, pp. 225–238, 1989.
- [Allen and Kautz 85] Allen J.F. and Kautz H.A., “A Model of Naive Temporal Reasoning”, in *Formal Theories of the Commonsense World*, Hobbs and Moore (eds.), Norwood N.J.: Ablex Publishing Corp., pp. 251–268, 1985.
- [Allen and Kautz 87] Allen J.F. and Kautz H.A., “Logicism is alive and well”, *Computational Intelligence* 3, pp. 161–162, 1987.

- [**Anderson and Belnap 62a**] Anderson A.R. and Belnap N.D., “Tautological Entailments”, *Philosophical Studies* 13, pp. 9–24, 1962.
- [**Anderson and Belnap 62b**] Anderson A.R. and Belnap N.D., “The Pure Calculus of Entailment”, *Journal of Symbolic Logic* 27, pp. 19–52, 1962.
- [**Anderson and Belnap 75**] Anderson A.R. and Belnap N.D., *Entailment, The Logic of Relevance and Necessity*, Vol. 1, Princeton, N.J.: Princeton University Press, 1975.
- [**Anderson, Belnap, and Dunn 92**] Anderson A.R., Belnap N.D., and Dunn J.M., *Entailment, The Logic of Relevance and Necessity*, Vol. 2, Princeton, N.J.: Princeton University Press, 1992.
- [**Anderson and Bower 73**] Anderson J. and Bower G., *Human Associative Memory*, Washington, DC: Winston and Sons, 1973.
- [**Audi 88**] Audi R., *Belief, Justification, and Knowledge*, Belmont, CA: Wadsworth Publishing Co., 1988.
- [**Baader et al. 03**] Baader F., Calvanese D., McGuinness D., Nardi D., and Patel-Schneider P. (eds.), *The Description Logic Handbook: Theory, Implementation and Applications*, Cambridge, UK: Cambridge University Press, 2003.
- [**Baader and Nutt 03**] Baader F. and Nutt W., “Basic Description Logics”, in Baader, Calvanese, McGuinness, Nardi, and Patel-Schneider (eds.), *The Description Logic Handbook: Theory, Implementation and Applications*, Cambridge, UK: Cambridge University Press, pp. 43–95, 2003.
- [**Barcan 46**] Barcan R., “A Functional Calculus of First Order Based on Strict Implication”, *Journal of Symbolic Logic* 11, pp. 1–16, 1946.
- [**Barr and Feigenbaum 81**] Barr A. and Feigenbaum E., *The Handbook of Artificial Intelligence*, Vol. 1, Los Altos, CA: William Kaufmann, Inc., 1981.
- [**Bartlett 32**] Bartlett F., *Remembering*, Cambridge, UK: Cambridge University Press, 1932.
- [**Bechhofer 00**] Bechhofer S., “The DIG Description Logic Interface: DIG / 1.0”, Technical Report, Manchester, UK: University of Manchester, 2000. Available as: <http://potato.cs.man.ac.uk/dig/interface1.0.pdf>.
- [**Ben-Ari 03**] Ben-Ari B., *Mathematical Logic for Computer Science*, 2nd Edition, London, UK: Springer-Verlag, 2003.
- [**Besnard 89**] Besnard P., *An Introduction to Default Logic*, Heidelberg, Germany: Springer-Verlag, 1989.
- [**Birnbaum 91**] Birnbaum L., “Rigor Mortis: A Response to Nilsson’s Logic and Artificial Intelligence”, *Artificial Intelligence* 47, No. 1–3, pp. 57–77, 1991.
- [**Bobrow 80**] Bobrow D.G. (ed.), “Special Issue on Non-Monotonic Reasoning”, *Artificial Intelligence* 13, No. 1–2, 1980.

- [**Bobrow and Collins 75**] Bobrow D.G. and Collins A. (eds.), *Representation and Understanding: Studies in Cognitive Science*, New York: Academic Press, 1975.
- [**Bobrow and Winograd 77**] Bobrow D.G. and Winograd T., “An Overview of KRL, a Knowledge Representation Language”, *Cognitive Science* 1, pp. 3-46, 1977. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann, pp. 263–285, 1985.
- [**Boole 54**] Boole G., *An Investigation of the Laws of Thought*, New York, N.Y.: Dover, 1854.
- [**Borgida and Brachman 03**] Borgida A. and Brachman R.J., “Conceptual Modeling with Description Logics”, in Baader, Calvanese, McGuinness, Nardi, and Patel-Schneider (eds.), *The Description Logic Handbook: Theory, Implementation and Applications*, Cambridge, UK: Cambridge University Press, pp. 359–381, 2003.
- [**Borgida, Brachman, McGuinness, and Resnick 89**] Borgida A., Brachman R. J., McGuinness D.L., and Resnick L.A., “CLASSIC: A Structural Data Model for Objects”, *Proc. of the 1989 ACM SIGMOD International Conference on the Management of Data*, 1989.
- [**Boyer and Moore 72**] Boyer R.S. and Moore J.S., “The Sharing of Structures in Theorem-Proving Programs”, in *Machine Intelligence 7*, Meltzer and Michie (eds.), Edinburgh, UK: Edinburgh University Press, pp. 101–116, 1972.
- [**Brachman 78**] Brachman R.J., “A Structural Paradigm for Representing Knowledge”, BBN Report 3605, Cambridge, MA: Bolt Beranek & Newman, 1978.
- [**Brachman 79**] Brachman R.J., “On the Epistemological Status of Semantic Networks”, in *Associative Networks: Representation and Use of Knowledge by Computers*, Findler (ed.), New York: Academic Press, pp. 3–50, 1979. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann, pp. 191–215, 1985.
- [**Brachman 90**] Brachman R.J., “The Future of Knowledge Representation”, *Proc. AAAI-90*, Boston, MA: AAAI Press, pp. 1082–1092, 1990.
- [**Brachman, Fikes, and Levesque 83a**] Brachman R.J., Fikes R., and Levesque H., “KRYPTON: Integrating Terminology and Assertion”, *Proc. AAAI-83*, Los Altos, CA: Morgan Kaufmann, pp. 31–35, 1983.
- [**Brachman, Fikes, and Levesque 83b**] Brachman R.J., Fikes R.E., and Levesque H.J., “KRYPTON: A Functional Approach to Knowledge Representation”, *IEEE Computer* 16, No. 10, pp. 67–73, 1983. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann, pp. 412–429, 1985.

- [**Brachman and Levesque 85**] Brachman R.J and Levesque H.J. (eds.), *Readings in Knowledge Representation*, Los Altos, CA: Morgan Kaufmann Publishers Inc., 1985.
- [**Brachman, Levesque, and Reiter 89**] Brachman R.J., Levesque H.J., and Reiter R. (eds.), *Principles of Knowledge Representation and Reasoning: Proceedings of the First International Conference*, San Mateo, CA: Morgan Kaufmann Publishers Inc., 1989.
- [**Brachman et al. 91**] Brachman R.J., McGuinness D.L., Patel-Schneider P., Resnick L., and Borgida A., "Living with CLASSIC: When and How to Use a KL-ONE-Like Language", in *Principles of Semantic Networks – Explorations in the Representation of Knowledge*, Sowa (ed.), San Mateo, CA: Morgan Kaufmann Publishers Inc., pp. 401–456, 1991.
- [**Brachman and Schmolze 85**] Brachman R.J. and Schmolze J.G., "An Overview of the KL-ONE Knowledge Representation System", *Cognitive Science* 9, No. 2, pp. 171–216, 1985.
- [**Brachman and Smith 79**] Brachman R.J. and Smith B.C. (eds.), "Special Issue on Knowledge Representation", *SIGART Newsletter* 70, 1980.
- [**Bradley and Swartz 79**] Bradley R. and Swartz N., *Possible Worlds: An Introduction to Logic and its Philosophy*, Oxford, UK: Basil Blackwell Publisher, 1979.
- [**Brooks 86**] Brooks R.A., "A robust layered control system for a mobile robot", *IEEE Journal of Robotics and Automation* 2, pp. 14–23, 1986.
- [**Brooks 91**] Brooks R.A., "Intelligence without representation", *Artificial Intelligence* 47, No. 1–3, pp. 139–159, 1991.
- [**Carpenter and Thomason 90**] Carpenter B. and Thomason R., "Inheritance Theory and Path-Based Reasoning: An Introduction", in *Knowledge Representation and Defeasible Reasoning*, Kyburg et al. (eds.), Dordrecht, The Netherlands: Kluwer Academic Publishers, pp. 309–343, 1990.
- [**Cerccone and McCalla 87a**] Cerccone N. and McCalla G. (eds), *The Knowledge Frontier: Essays in the Representation of Knowledge*, New York: Springer-Verlag, 1987.
- [**Cerccone and McCalla 87b**] Cerccone N. and McCalla G., "What is Knowledge Representation", in *The Knowledge Frontier: Essays in the Representation of Knowledge*, Cerccone and McCalla (eds.), New York: Springer-Verlag, pp. 1–43, 1987.
- [**Chapman 87**] Chapman D., "Planning for Conjunctive Goals", Technical Report 802, Cambridge, MA: Department of Computer Science, MIT, 1987.
- [**Charniak, Riesbeck, McDermott, and Meehan 87**] Charniak E., Riesbeck C., McDermott D.V., and Meehan J.R., *Artificial Intelligence Programming*, Hillsdale, N.J.: Lawrence Erlbaum Associates, 2nd Edition, 1987.

- [**Cheeseman 85**] Cheeseman P., “In Defense of Probability”, *Proc. IJCAI-85*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 1002–1009, 1985.
- [**Chellas 80**] Chellas B.F., *Modal Logic: An Introduction*, Cambridge, UK: Cambridge University Press, 1980.
- [**Church 36**] Church A., “A Note on the Entscheidungsproblem”, *Journal of Symbolic Logic* 1, pp. 40–41, 1936.
- [**Church 56**] Church A., *Introduction to Mathematical Logic*, Vol. 1, Princeton, N.J.: Princeton University Press, 1956.
- [**Cohn, Schubert, and Shapiro 98**] Cohn A.G., Schubert L., and Shapiro S.C. (eds.), *Proc. of the Sixth International Conference on Principles of Knowledge Representation and Reasoning, KR’98*, San Francisco, CA: Morgan Kaufmann Publishers Inc., 1998.
- [**Cohn, Giunchiglia, and Selman 00**] Cohn A.G., Giunchiglia F., and Selman B. (eds.), *Proc. of the Seventh International Conference on Principles of Knowledge Representation and Reasoning, KR’00*, San Francisco, CA: Morgan Kaufmann Publishers Inc., 2000.
- [**Collins and Quillian 70**] Collins A.M. and Quillian M.R., “Facilitating Retrieval from Semantic Memory: The Effect of Repeating Part of an Inference”, in *Acta Psychologica 33 Attention and Performance III*, Sanders (ed.), Amsterdam, The Netherlands: North-Holland Publishing Co, pp. 304–314, 1970.
- [**Collins and Quillian 72**] Collins A.M. and Quillian M.R., “How to Make a Language User”, in *Organization of Memory*, Tulving and Donaldson (eds.), New York: Academic Press, pp. 309–351, 1972.
- [**Corcoran 72**] Corcoran J., “Conceptual Structure of Classical Logic”, *Philosophy and Phenomenological Research* 33, pp. 25–47, 1972.
- [**Corcoran 83**] Corcoran J. (ed.), *Logic, Semantics, Metamathematics*, Indianapolis, IN: Hackett Publishing Co., 2nd Edition, 1983.
- [**Cravo 94**] Cravo M.R., *Personal Communication*, 1994.
- [**Cravo and Martins 90**] Cravo M.R. and Martins J.P., “Path-Based Inference Revisited”, *Current Trends in SNePS – Semantic Network Processing System: Proc. of the First Annual Workshop*, Kumar (ed.), Lecture Notes in Artificial Intelligence 437, Heidelberg, Germany: Springer-Verlag, pp. 14–26, 1990.
- [**Cravo and Martins 91**] Cravo M.R. and Martins J.P., “Being Aware of Assumptions”, *Proc. of the Seventh Austrian Conference on Artificial Intelligence*, Kaindl (ed), Informatik-Fachberichte 287, Heidelberg, Germany: Springer-Verlag, pp. 137–146, 1991.
- [**Cravo and Martins 93**] Cravo M.R. and Martins J.P., “SNePSwD: A Newcomer to the SNePS Family”, *Journal of Experimental and Theoretical Artificial Intelligence* 5, pp. 135–148, 1993.

- [**Davies and Morgenstern 04**] Davies E. and Morgenstern L. (eds.), *Artificial Intelligence 153* – Special Issue on Logical formalization of Commonsense Reasoning, 2004
- [**de Kleer 84**] de Kleer J., “Choices without Backtracking”, *Proc. AAAI-84*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 79–85, 1984.
- [**de Kleer 86**] de Kleer J., “An Assumption-based TMS”, *Artificial Intelligence* 28, No. 2, pp. 127–162, 1986. Also in *Readings in Nonmonotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 280–297, 1987.
- [**de Kleer, Doyle, Steele, and Sussman 77**] de Kleer J., Doyle J., Steele G., and Sussman G., “AMORD: Explicit Control of Reasoning”, *Proc. Symposium on Artificial Intelligence and Programming Languages, SIGART Newsletter 64*, pp. 116–125, 1977. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers, Inc., pp. 345–355, 1985.
- [**de Kleer and Williams 87**] de Kleer J. and Williams B.C., “Diagnosing Multiple Faults”, *Artificial Intelligence* 32, No. 1, pp. 97–130, 1987.
- [**Delgrande and Mylopoulos 87**] Delgrande J.P. and Mylopoulos J., “Knowledge Representation: Features of Knowledge”, in *Fundamentals of Artificial Intelligence: An Advanced Course*, Bibel and Jorrand (eds.), Heidelberg, Germany: Springer-Verlag, pp. 3–36, 1987.
- [**Dennett 81**] Dennett D.C., *Brainstorms: Philosophical Essays on Mind and Psychology*, Cambridge, MA: The MIT Press, 1981.
- [**Dhar and Ranganathan 90**] Dhar V. and Ranganathan N., “Integer Programming vs. Expert Systems: An Experimental Comparison”, *Communications of the ACM* 33, No. 3, pp. 323–336, 1990.
- [**Doyle 77**] Doyle J., “Truth Maintenance Systems For Problem-Solving”, *Proc. IJCAI-77*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 247–249, 1977.
- [**Doyle 79**] Doyle J., “A Truth Maintenance System”, *Artificial Intelligence* 12, No. 3, pp. 231–272, 1979. Also in *Readings in Artificial Intelligence*, Webber and Nilsson (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 496–516, 1981. Also in *Readings in Nonmonotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 259–279, 1987.
- [**Doyle 83**] Doyle J., “The Ins and Outs of Reason Maintenance”, *Proc. IJCAI-83*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 349–351, 1983
- [**Doyle and London 80**] Doyle J. and London P. (eds.), “A Selected Description-Indexed Bibliography to the Literature on Belief Revision”, *SIGART Newsletter* 71, pp. 7– 23, 1980.

- [Doyle, Sandewall, and Torasso 94] Doyle J., Sandewall E., and Torasso P. (eds), *Principles of Knowledge Representation and Reasoning: Proceedings of the Fourth International Conference*, San Francisco, CA: Morgan Kaufmann Publishers Inc., 1994.
- [Dressler 88] Dressler O., “An Extended Basic ATMS”, *Proc. of The Second International Workshop on Non-Monotonic Reasoning*, Reinfrank, de Kleer, Ginsberg and Sandewall (eds.), Lecture Notes in Computer Science 346, Heidelberg, Germany: Springer-Verlag, pp. 143–163, 1988.
- [Drummond 87] Drummond M.E., “A Representation of Action and Belief for Automatic Planning Systems”, *Proc. 1986 Workshop on Reasoning about Actions and Plans*, Georgeff and Lansky (eds.) Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 189–211, 1987.
- [Dubois, Welty, and Williams 04] Dubois D., Welty C., and Williams M-A. (eds.) *Principles of Knowledge Representation and Reasoning: Proceedings of the Ninth International Conference*, menlo Park, CA: AAAI Press, 2004.
- [Dunn and Restall 02] Dunn J.M. and Restall G., “Relevance Logic”, in Gabbay and Guenther (eds), *Handbook of Philosophical Logic*, Vol. 6, Dordrecht, The Netherlands: Kluwer Academic Publishers, pp. 1–128, 2002.
- [Eklund 91] Eklund P., *An Epistemic Approach to Interactive Design in Multiple Inheritance Hierarquies*, Linköping Studies in Science and Technology. Dissertation 252. Linköping, Sweden: Department of Computer and Information Science, Linköping University, 1991.
- [Ernst and Newell 69] Ernst G.W. and Newell A., *GPS: A Case Study in Generality and Problem-solving*, New York: Academic Press, 1969.
- [Etherington 87a] Etherington D.W., “Formalizing Nonmonotonic Reasoning Systems”, *Artificial Intelligence* 31, No. 1, pp. 41–85, 1987.
- [Etherington 87b] Etherington D.W., “More on Inheritance Hierarquies with Exceptions: Default Theories and Inferential Distance”, *Proc. AAAI-87*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 352–357, 1987.
- [Etherington 87c] Etherington D.W., “A Semantics for Default Logic”, *Proc. IJCAI-87*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 495–498, 1987.
- [Etherington 88] Etherington D.W., *Reasoning with Incomplete Information*, London, UK: Pitman Publishing, 1988.
- [Etherington and Reiter 83] Etherington D.W. and Reiter R., “On Inheritance Hierarchies With Exceptions”, *Proc. AAAI-83*, Los Altos, CA: Morgan Kaufmann, pp. 104–108, 1983.
- [Fahlmann 79] Fahlmann S., *NETL: A System for Representing and Using Real World Knowledge*, Cambridge, MA: The MIT Press, 1979.

- [Fensel, Giunchiglia, McGuinness, and Williams 02] Fensel D., Giunchiglia F., McGuinness D., and Williams M-A. (eds.), *Proc. of the Eighth International Conference on Principles of Knowledge Representation and Reasoning, KR'02*, San Francisco, CA: Morgan Kaufmann Publishers Inc., 2002.
- [Fikes 75] Fikes R.E., "Deductive Retrieval Mechanisms for State Description Models", *Proc. IJCAI-75*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 99–106, 1975.
- [Fikes 96] Fikes R.E., "Ontologies: What Are They, and Where's The Research?", *Proc. of the Fifth International Conference on Principles of Knowledge Representation and Reasoning, KR'96*, San Francisco, CA: Morgan Kaufmann Publishers Inc., pp. 652–655, 1996.
- [Fikes and Kehler 85] Fikes R.E. and Kehler T., "The Role of Frame-Based Representation in Reasoning", *Communications of the ACM* 28, No. 9, pp. 904–920, 1985.
- [Fikes and Nilsson 71] Fikes R. and Nilsson N., "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving", *Artificial Intelligence* 2, No. 3–4, pp. 189–208, 1971.
- [Fillmore 68] Fillmore C., "The Case for Case", in *Universals in Linguistic Theory*, Bach and Harms (eds.), Chicago, IL: Holt Rinehart and Winston, pp. 1–88, 1968.
- [Filman 88] Filman R.E., "Reasoning with Worlds and Truth Maintenance in a Knowledge-Based Programming Environment", *Communications of the ACM* 31, pp. 382–401, 1988.
- [Fitch 52] Fitch F., *Symbolic Logic: An Introduction*, New York: The Ronald Press Company, 1952.
- [Forbus and de Kleer 88] Forbus K.D. and de Kleer J., "Focusing the ATMS", *Proc. AAAI-88*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 193–198, 1988.
- [Gärdenfors 88] Gärdenfors P., *Knowledge in Flux: Modeling the Dynamics of Epistemic States*, Cambridge, MA: The MIT Press, 1988.
- [Gärdenfors and Makison 88] Gärdenfors P. and Makison D., "Revisions of Knowledge Systems using Epistemic Entrenchment", *Proc. of the Second Conference on Theoretical Aspects of Reasoning about Knowledge*, Vardi (ed.), Los Altos, CA: Morgan Kaufmann, pp. 83–95, 1988.
- [Garson 03a] Garson, J., "Modal Logic", *The Stanford Encyclopedia of Philosophy*, Edward N. Zalta (ed.), <http://plato.stanford.edu/archives/win2003/entries/logic-modal/>, 2003.
- [Garson 03b] Garson, J., "Temporal Logic", *The Stanford Encyclopedia of Philosophy*, Edward N. Zalta (ed.), <http://plato.stanford.edu/archives/win2003/entries/logic-temporal/>, 2003.

- [**Genesereth, Fikes, et. al. 92**] Genesereth M.R., Fikes R.E., et. al., “Knowledge Interface Format, Version 3.0 Reference Manual”, Technical Report, Logic-92-01, Department of Computer Science, Stanford University, 1992.
- [**Genesereth and Fikes 97**] Genesereth M.R. and Fikes R.E., “Knowledge Interface Format Specification”, <http://logix.stanford.edu/kif/kif.html>, 1997.
- [**Genesereth and Nilsson 87**] Genesereth M.R. and Nilsson N.J., *Logical Foundations of Artificial Intelligence*, Los Altos, CA: Morgan Kaufmann Publishers Inc., 1987.
- [**Gentzen 69**] Gentzen G., *Collected Papers of Gerhard Gentzen*, Amsterdam, The Netherlands: North-Holland, 1969.
- [**Ginsberg 86**] Ginsberg M.L., “Multiple-Valued Logics”, *Proc. AAAI-86*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 243–247, 1986. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 251–255, 1987.
- [**Ginsberg 87**] Ginsberg M.L., *Readings in Nonmonotonic Reasoning*, Los Altos, CA: Morgan Kaufmann Publishers Inc., 1987.
- [**Gödel 30**] Gödel K., *Über die Vollständigkeit des Logikkalküls*, Ph.D. Thesis, Vienna, Austria: University of Vienna, 1930.
- [**Gödel 31**] Gödel, K. “Über Formal Unentscheidbare Sätze der Principia Mathematica und Verwandter Systeme, I”, *Monatshefte für Math. u. Physik* 38, pp. 173–198, 1931.
- [**Goodwin 82**] Goodwin J., *An Improved Algorithm for Nonmonotonic Dependency Network Update*, Technical Report LITH-MAT-R-82-23, Linköping, Sweden: Department of Computer and Information Science, Linköping University, 1982.
- [**Gruber 92**] Gruber T.R., “Ontolingua: A Mechanism to Support Portable Ontologies”, Technical Report KSL 91-66, Knowledge Systems Laboratory, Stanford University, 1992.
- [**Gruber 93**] Gruber T.R., “A translation approach to portable ontologies” *Knowledge Acquisition* 5, No. 2, pp. 199–220, 1993.
- [**Guarino 98**] Guarino N., “Formal Ontology and Information Systems”, in Guarino (ed.), *Formal Ontology in Information Systems*, pp. 3–15, Amsterdam, The Netherlands: IOS Press, 1998.
- [**Guarino, Carrara, and Giaretta 94**] Guarino N., Carrara M. and Giaretta P., “Formalizing Ontological Commitment”, *Proc. AAAI-94*, pp. 560–567, Menlo Park, CA: AAAI Press, 1994.
- [**Haack 78**] Haack S., *Philosophy of Logics*, Cambridge, UK: Cambridge University Press, 1978.

- [Haarslev and Möller 01] Haarslev V. and Möller R., “RACER System Description”, *Proc. International Joint Conference on Automated Reasoning*, pp. 701–705, Heidelberg, Germany: Springer-Verlag, 2001.
- [Haarslev and Möller 03] Haarslev V. and Möller R., “RACER User Guide and Reference Manual”, Version 1.7.7., Technical Report, Montreal, Canada: Computer Science Department, Concordia University, 2003.
- [Halpern and Moses 84] Halpern J.Y. and Moses Y., “Towards a Theory of Knowledge and Ignorance: Preliminary Report”, *Proc. of the AAAI Workshop on Non-Monotonic Reasoning*, Menlo Park, CA: American Association for Artificial Intelligence, pp. 125–143, 1984.
- [Hanks and McDermott 86] Hanks S. and McDermott D., “Default Reasoning, Nonmonotonic Logics, and the Frame Problem”, *Proc. AAAI-86*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 328–333, 1986. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 390–395, 1987.
- [Harman 86] Harman G., *Change in View: Principles of Reasoning*, Cambridge, MA: The MIT Press, 1986.
- [Harper 76] Harper W.L., “Ramsey test conditionals and iterated belief change”, in *Foundations of Probability Theory, Statistical Inference, and Statistical Theories of Science, Vol. 1*, Harper and Hooker (eds.), pp. 117–135, Dordrecht, The Netherlands: Reidel, 1976.
- [Hayes 75] Hayes P.J., “A Representation for Robot Plans”, *Proc. IJCAI-75*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 181–188, 1975.
- [Hayes 77] Hayes P.J., “In Defense of Logic”, *Proc. IJCAI-77*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 559–565, 1977.
- [Hayes 79] Hayes P.J., “The Logic of Frames”, in *Frame Conceptions and Text Understanding*, Metzing (ed.), Berlin, Germany: Walter de Gruyter and Co., pp. 46–61, 1979. Also in *Readings in Artificial Intelligence*, Webber and Nilsson (eds.), Palo Alto, CA: Tioga, pp. 451–458, 1979. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 287–295, 1985.
- [Hawkins and Allen 91] Hawkins J.M. and Allen R., *The Oxford Encyclopedic English Dictionary*, Oxford, UK: Clarendon Press, 1991.
- [Hendrix 79] Hendrix G.G., “Encoding Knowledge in Partitioned Networks”, in *Associative Networks: Representation and Use of Knowledge by Computers*, Findler (ed.), New York: Academic Press, pp. 51–92, 1979.
- [van Heijenoort 67] van Heijenoort J. (ed.), *From Frege to Gödel*, Harvard, MA: Harvard University Press, 1967.

- [**Herbrand 71**] Herbrand J., “Recherches sur la théorie de la démonstration”, in *Logical Writings*, Goldfarb (ed.), Cambridge, MA: Harvard University Press, 1971.
- [**Hewitt 69**] Hewitt C.E., “A Language for Proving Theorems in Robots”, *Proc. IJCAI-69*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 295–301, 1969.
- [**Hewitt, Bishop, and Steiger 73**] Hewitt C.E., Bishop P., and Steiger R., “A Universal Modular ACTOR Formalism for Artificial intelligence”, *Proc. IJCAI-73*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 235–245, 1973.
- [**Hobbs 02**] Hobbs J.R., *A DAML Ontology of Time*, www.cs.rochester.edu/~ferguson/daml/daml-time-nov2002.txt, 2002.
- [**Horty 91**] Horty J.F., “A Credulous Theory of Mixed Inheritance”, in *Inheritance Hierarchies in Knowledge Representation and Programming Languages*, Lenzerini, Nardi and Simi (eds.), New York: John Wiley and Sons Ltd, pp. 13–28, 1991.
- [**Horty and Thomason 88**] Horty J.F. and Thomason R., “Mixing Strict and Defeasible Inheritance”, *Proc. AAAI-88*, San Mateo, CA: Morgan Kaufmann, pp. 427–432, 1988.
- [**Horty, Thomason, and Touretzky 90**] Horty J.F., Thomason R., and Touretzky D.S., “A Skeptical Theory of Inheritance in Nonmonotonic Semantic Networks”, *Artificial Intelligence* 42, No. 2–3, pp. 311–348, 1990.
- [**Hughes and Cresswell 68**] Hughes G.E. and Cresswell M.J., *An Introduction to Modal Logic*, London, UK: Methuen and Co., 1968.
- [**Hursel 70**] Hursel E., *Logical Investigations*, London, UK: Routledge, 1970.
- [**Huth and Ryan 04**] Huth M. and Ryan M., *Logic in Computer Science: Modelling and Reasoning about Systems*, Cambridge, UK: Cambridge University Press, 2004.
- [**Intellicorp 85**] Intellicorp, “KEE Software Development System: User’s Manual”, Document Number 2.1-UZ-1, Intellicorp, 1985.
- [**Israel 80**] Israel D., “What’s Wrong With Non-monotonic Logic?”, *Proc. AAAI-80*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 99–101, 1980. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 53–55, 1987.
- [**Israel 87**] Israel D., “Some Remarks on the Place of Logic in Knowledge Representation”, in *The Knowledge Frontier: Essays in the Representation of Knowledge*, Cercone and McCalla (eds.), New York: Springer-Verlag, pp. 80–91, 1987.

- [Israel 93] Israel D., “The role(s) of logic in artificial intelligence”, in Gabbay, Hogger, and Robinson (eds.), *Handbook of Logic in Artificial Intelligence and Logic Programming*, Vol. 1, Oxford, UK: Clarendon Press, pp. 1–29, 1993.
- [Jalote 97] Jalote P., *An Integrated Approach to Software Engineering*, Heidelberg, Germany: Springer-Verlag, 1997.
- [Janlert 87] Janlert L.-E., “Modeling Change: The Frame Problem”, in *The Robot’s Dilemma: The Frame Problem in Artificial Intelligence*, Pylyshyn (ed.), Norwood N.J.: Ablex Publishing Corporation, pp. 1–40, 1987.
- [Jaskowski 34] Jaskowski S., “On the Rules of Supposition in Formal Logic”, *Studia Logica* 1, 1934.
- [Jeffreys 42] Jeffreys H., “Does a Contradiction Entail Every Proposition?”, *Mind* 51, 1942.
- [Keene 89] Keene S.E., *Object-Oriented Programming in COMMON LISP: A Programmer’s Guide to CLOS*, Reading, MA: Addison-Wesley Publishing Co., 1989.
- [Kirsh 91] Kirsh D., “Foundations of AI: The Big Issues”, *Artificial Intelligence* 47, pp. 3–30, 1991.
- [Kleene 52] Kleene S.C., *Introduction to Metamathematics*, New York: American Elsevier Publishing Co, Inc., 1952.
- [Kline 72] Kline M., *Mathematical Thought: From Ancient to Modern Times*, Oxford, UK: Oxford University Press, 1972.
- [Kneale and Kneale 88] Kneale W. and Kneale M., *The Development of Logic*, Oxford, UK: Clarendon Press, 1988.
- [Konolige 86] Konolige K., *A Deduction Model of Belief*, London, UK: Pitman Publishing, 1986.
- [Kowalski 75] Kowalski R., “A Proof Procedure Using Connection Graphs”, *Journal of the ACM* 22, pp. 573–595, 1975.
- [Kramosil 75] Kramosil I., “A Note on Deduction Rules with Negative Premises”, *Proc. IJCAI-75*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 53–56, 1975.
- [Kripke 63] Kripke S.A., “Semantical Analysis of Modal Logic I, Normal Propositional Calculi”, *Zeitschrift für Mathematische Logik und Grundlagen der Mathematik* 9, pp. 67–96, 1963.
- [Léa Sombé 90] Léa Sombé, *Reasoning under Incomplete Information in Artificial Intelligence*, New York: John Wiley and Sons, Inc., 1990.
- [Lehmann 92a] Lehmann F. (ed.), *Semantic Networks in Artificial Intelligence*, N.Y.: Pergamon Press, 1992.

- [**Lehmann 92b**] Lehmann F., “Semantic Networks”, in Lehmann F. (ed.), *Semantic Networks in Artificial Intelligence*, pp. 1–50, N.Y.: Pergamon Press, 1992.
- [**Lehrer 90**] Lehrer K., *Theory of Knowledge*, London, UK: Routledge, 1990.
- [**Lemmon 78**] Lemmon E.J., *Beginning Logic*, Indianapolis, IN: Hackett Publishing Co., 1978.
- [**Lenat and Guha 90**] Lenat D.B. and Guha R.V., *Building Large Knowledge-Based Systems: Representation and Inference in the Cyc Project*, Reading, MA: Addison-Wesley Publishing Co., 1990.
- [**Lenzerini, Nardi, and Simi 91**] Lenzerini M., Nardi D., and Simi M., *Inheritance Hierarchies in Knowledge Representation and Programming Languages*, New York: John Wiley and Sons, 1991.
- [**Levi 77**] Levi I., “Subjunctions, dispositions, and chances”, *Synthese* 34, pp. 423–455, 1977.
- [**Levesque 84**] Levesque H.J., “Foundations of a Functional Approach to Knowledge Representation”, *Artificial Intelligence* 23, No. 2, pp. 155–212, 1984.
- [**Lewis 18**] Lewis C.I., *A Survey of Symbolic Logic*, Berkeley, CA: University of California Press, 1918.
- [**London 78**] London P., “Dependency Networks as Representation for Modelling in General Problem Solvers”, Ph.D. Dissertation, Technical Report 698, College Park, MD: Department of Computer Science, University of Maryland, 1978.
- [**Lukaszewicz 90**] Lukaszewicz W., *Non-Monotonic Reasoning: Formalization of Commonsense Reasoning*, Chichester, UK: Ellis Horwood Limited, 1990.
- [**MacGregor 91**] MacGregor R., “The Evolving Technology of Classification-Based Knowledge Representation Systems”, in *Principles of Semantic Networks: Explorations in the Representation of Knowledge*, Sowa (ed.), San Mateo: CA, Morgan Kaufmann, pp. 385–400, 1991.
- [**Machtey and Young 78**] Machtey M. and Young P., *An Introduction to the General Theory of Algorithms*, New York: Elsevier North Holland Inc., 1978.
- [**Maida 92**] Maida A.S., “Frames”, in *Encyclopedia of Artificial Intelligence*, 2nd Edition, Shapiro (ed.), New York: John Wiley and Sons, pp. 493–507, 1992.
- [**Maida and Shapiro 85**] Maida A.S. and Shapiro S.C., “Intensional Concepts in Propositional Semantic Networks”, *Cognitive Science* 6, No. 4, pp. 291–330, 1982. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann, pp. 169–189, 1985.
- [**Martins 83**] Martins J.P., “Reasoning in Multiple Belief Spaces”, Ph.D. Dissertation, Technical Report 203, Buffalo, N.Y.: Department of Computer Science, State University of New York at Buffalo, 1983.

- [**Martins 87**] Martins J.P., “Belief Revision”, in *Encyclopedia of Artificial Intelligence*, Shapiro (ed.), New York, N.Y.: John Wiley and Sons, pp. 58–62, 1987.
- [**Martins 90**] Martins J.P., “The Truth, The Whole Truth, and Nothing But The Truth: An Indexed Bibliography to the Literature on Truth Maintenance Systems”, *AI Magazine* 11, No. 5, pp. 7–25, 1990.
- [**Martins 91**] Martins J.P., “Computational Issues in Belief Revision”, in *The Logic of Theory Change*, Fuhrmann and Morreau (eds.), Lecture Notes in Artificial Intelligence 465, Heidelberg, Germany: Springer-Verlag, pp. 51–71, 1991.
- [**Martins 92**] Martins J.P., “A Structure for Epistemic States”, in *New Directions for Intelligent Tutoring Systems*, Costa (ed.), NATO ASI Series F, Heidelberg, Germany: Springer-Verlag, pp. 198–212, 1992.
- [**Martins and Shapiro 83**] Martins J.P. and Shapiro S.C., “Reasoning in Multiple Belief Spaces”, *Proc. IJCAI-83*, Los Altos, CA: William Kaufmann, pp. 370–373, 1983.
- [**Martins and Shapiro 88**] Martins J.P. and Shapiro S.C., “A Model for Belief Revision”, *Artificial Intelligence* 35, No. 1, pp. 25–79, 1988.
- [**Matos and Martins 89**] Matos P.A. and Martins J.P., “SNePSLOG – A Logical Interface to SNePS”, Technical Report 89/03, Lisbon, Portugal: Instituto Superior Técnico, 1989.
- [**McAllester 78**] McAllester D., “A three-valued truth maintenance system”, Technical Report Memo 473, Cambridge, MA: AI-Lab, Massachusetts Institute of Technology, 1978.
- [**McCarthy 68**] McCarthy J., “Programs with Common Sense”, in *Semantic Information Processing*, Minsky (ed.), Cambridge, MA: The MIT Press, pp. 403–410, 1968. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 299–307, 1985.
- [**McCarthy 80**] McCarthy J., “Circumscription – A Form of Non-Monotonic Reasoning”, *Artificial Intelligence* 13, No. 1–2, pp. 27–39, 1980. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 145–152, 1987.
- [**McCarthy 84**] McCarthy J., “Applications of Circumscription to Formalizing Common-Sense Knowledge”, *Proc. of the AAAI Workshop on Non-monotonic Reasoning*, Menlo Park, CA: American Association for Artificial Intelligence, pp. 295–324, 1984. Also in *Artificial Intelligence* 28, No. 1, pp. 89–116, 1986. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 153–166, 1987.

- [**McCarthy 88**] McCarthy J., “Mathematical Logic in Artificial Intelligence”, in *The Artificial Intelligence Debate—False Starts, Real Foundations*, Grabaud (ed.), Cambridge, MA: The MIT Press, pp. 297–311, 1988. Also in *Formalizing Commonsense: Papers by John McCarthy*, Lifschitz (ed.), Norwood, N.J.: Ablex Publishing Co., pp. 237–249, 1990.
- [**McCarthy and Hayes 69**] McCarthy J. and Hayes P., “Some Philosophical Problems from the Standpoint of Artificial Intelligence”, in *Machine Intelligence 4*, Meltzer and Michie (eds.), Edinburgh, UK: Edinburgh University Press, pp. 463–502, 1969.
- [**McCawley 81**] McCawley J.D., *Everything that Linguists have Always Wanted to Know about Logic (but were ashamed to ask)*, Chicago, IL: The University of Chicago Press, 1981.
- [**McDermott and Forgy 78**] McDermott J. and Forgy C., “Production Systems Conflict resolution Strategies”, in *Pattern-directed Inference Systems*, Waterman and Hayes-Roth (eds), New York, NY: Academic Press Inc., pp. 177–199, 1978.
- [**McKay and Shapiro 80**] McKay D.P. and Shapiro S.C., “MULTI - A LISP Based Multiprocessing System”, *Proc. 1980 LISP Conference*, pp. 29–37, 1980.
- [**McKay and Shapiro 81**] McKay D.P. and Shapiro S.C., “Using Active Connection Graphs for Reasoning with Recursive Rules”, *Proc. IJCAI-81*, Los Altos, CA: Morgan Kaufman Publishers Inc., pp. 368–374, 1981.
- [**Mendelson 87**] Mendelson E., *Introduction to Mathematical Logic*, Belmont, CA: Wadsworth Publishing Co., 1987.
- [**Minsky 75**] Minsky M., “A Framework for Representing Knowledge”, in *The Psychology of Computer Vision*, Winston (ed.), New York: McGraw-Hill Book Co., pp. 211–277, 1975. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 245–262, 1985.
- [**Moore 82**] Moore R.C., “The Role of Logic in Knowledge Representation and Commonsense Reasoning”, *Proc. AAAI-82*, pp. 428–433, 1982. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 335–341, 1985.
- [**Moore 83**] Moore R.C., “Semantical Considerations on Non-monotonic Logic”, *Proc. IJCAI-83*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 272–279, 1983.
- [**Moore 84a**] Moore R.C., “Possible-World Semantics for Autoepistemic Logic”, *Proc. of the AAAI Workshop on Non-monotonic Reasoning*, Menlo Park, CA: American Association for Artificial Intelligence, pp. 344–354, 1984. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 137–142, 1987.

- [**Moore 84b**] Moore R.C., “The Role of Logic in Artificial Intelligence”, Technical Note 335, Stanford, CA: SRI International, Artificial Intelligence Center, 1984.
- [**Moore 85**] Moore R.C., “Semantical Considerations on Non-monotonic Logic”, *Artificial Intelligence* 25, No. 1, pp. 75–94, 1985. Also in *Readings in Non-monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 127–136, 1987.
- [**Moore 88**] Moore R.C., “Autoepistemic Logic”, in *Non-Standard Logics for Automated Reasoning*, Smets et. al. (eds.), New York: Academic Press, pp. 105–127, 1988.
- [**Morris 87a**] Morris P.H., “Curing Anomalous Extensions”, *Proc. AAAI-87*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 437–442, 1987.
- [**Morris 87b**] Morris P.H., “A Truth Maintenance Based Approach to the Frame Problem”, *Proc. of the 1987 Workshop on the Frame Problem in Artificial Intelligence*, Brown (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 297–307, 1987.
- [**Nado and Fikes 87**] Nado R. and Fikes R., “Semantically Sound Inheritance for a Formally Defined Frame Language with Defaults”, *Proc. AAAI-87*, Los Altos, CA: Morgan Kaufmann, pp. 443–449, 1987.
- [**Nebel 90**] Nebel B., *Reasoning and Revision in Hybrid Representation Systems*, Lecture Notes in Artificial Intelligence 422, Heidelberg, Germany: Springer-Verlag, 1990.
- [**Nebel, Rich, and Swartout 92**] Nebel B., Rich C., and Swartout W. (eds.), *Principles of Knowledge Representation and Reasoning: Proceedings of the Third International Conference*, San Mateo, CA: Morgan Kaufmann Publishers Inc., 1992.
- [**Neches et al. 91**] Neches R., Fikes R., Finin T., Gruber T., Patil R., Senator T., and Swartout W., “Enabling Technology for Knowledge Sharing”, *AI Magazine* 12, No. 2, pp. 36–55, 1991.
- [**Newell 81**] Newell A., “The Knowledge Level”, *AI Magazine* 2, No. 2, pp. 1–20, 1981. Also in *Artificial Intelligence* 18, No. 1, pp. 87–127, 1982. Also in Rosenbloom, Laird, and Newell (eds.), *The SOAR Papers*, Vol. 1, Cambridge, MA: The MIT Press, pp. 136–176, 1993.
- [**Newell 82**] Newell A., “Physical Symbol Systems”, *Cognitive Science* 4, No. 2, pp. 135–183, 1982.
- [**Newell and Simon 76**] Newell A. and Simon H.A., “Computer Science as Empirical Inquiry: Symbols and Search”, *Communications of the ACM* 19, No. 3, pp. 113–126, 1976. Also in *The Philosophy of Artificial Intelligence*, Boden (ed.), Oxford, UK: Oxford University Press, pp. 105–132, 1990.

- [**Nilsson 71**] Nilsson N., *Problem Solving Methods in Artificial Intelligence*, New York: McGraw-Hill Book Co., 1971.
- [**Nilsson 91**] Nilsson N., "Logic and Artificial Intelligence", *Artificial Intelligence* 47, No. 1–3, pp. 31–56, 1991.
- [**Nutter 83**] Nutter J.T., "What Else is Wrong with Non-Monotonic Logics? Representational and Informational Shortcomings", *Proc. of the Fifth Annual Conference of the Cognitive Science Society*, 1983.
- [**Patel-Schneider and Swartout 93**] Patel-Schneider P.F. and Swartout B., "Description Logic Knowledge Representation System Specification from the KRSS Group of the ARPA Knowledge Sharing Effort", 1993. <http://www-db.research.bell-labs.com/user/pfps/papers/krss-spec.ps>.
- [**Paterson and Wegman 78**] Paterson M.S. and Wegman M.N., "Linear Unification", *Journal of Computer and System Sciences* 16, No. 2, pp. 158–167, 1978.
- [**Patil et al. 92**] Patil R.S., Fikes R.E., Patel-Schneider P.F., McKay D., Finin T., Gruber T., and Neschies R., "The DARPA Knowledge Sharig Effort: Progress Report", *Proc. of the Third International Conference on Principles of Knowledge Representation and Reasoning, KR'92*, San Mateo, CA: Morgan Kaufmann Publishers Inc., pp. 777–788, 1992.
- [**Perlis 87**] Perlis D., "On the Consistency of Commonsense Reasoning", *Computational Intelligence* 2, pp. 180–190, 1987.
- [**Petrie 86**] Petrie C.J., "New Algorithms for Dependency-Directed Backtracking", Technical Report AI TR86-33, Austin, Texas: Artificial Intelligence Laboratory, The University of Texas at Austin, 1986.
- [**Petrie 87**] Petrie C.J., "Revised dependency-Directed Backtracking for Default Reasoning", *Proc Sixth National Conference on Artificial Intelligence*, pp. 167–172, Los Altos, CA: Morgan Kaufmann Publishers Inc., 1987.
- [**Pinto-Ferreira 91**] Pinto-Ferreira C., *Personal Communication*, 1991.
- [**Pinto-Ferreira and Martins 90**] Pinto-Ferreira C. and Martins J.P., "A Formal System for Reasoning about Change", *Proc. ECAI-90*, Aiello (ed.), London, UK: Pitman Publishing, pp. 503–508, 1990.
- [**Poigné 92**] Poigné A., "Basic Cathegory Theory", in Abramsky, Gabby, and Maibaum (eds.), *Handbook of Logic in Computer Science*, Vol. 1, Oxford, UK: Claredon Press, pp. 413–640, 1992.
- [**Post 43**] Post E., "Formal Reductions of the General Combinatorial Problem", *American Journal of Mathematics* 65, pp. 197–268, 1943.
- [**Priest, Routley, and Norman 89**] Priest G., Routley R., and Norman, J. (eds.) *Paraconsistent Logic: Essays on the Inconsistent*, Munich, Germany: Philosophia Verlag, 1989.

- [**Priest 02**] Priest G. “Paraconsistent Logic”, *Handbook of Philosophical Logic*, 2nd Edition, Vol. 6, Gabbay and Guentner (eds.), pp. 287–393, Dordrecht, Netherlands: Kluwer Academic Publishers, 2002.
- [**Quillian 68**] Quillian M.R., “Semantic Memory” in *Semantic Information Processing*, Minsky (ed.), Cambridge, MA: The MIT Press, pp. 227–270, 1968.
- [**Quine 86**] Quine W.V., *Philosophy of Logic*, Cambridge, MA: Harvard University Press, 2nd Edition, 1986.
- [**Ramsay 88**] Ramsay A., *Formal Methods in Artificial Intelligence*, Cambridge, UK: Cambridge University Press, 1988.
- [**Rapaport 92a**] Rapaport W.J., “Logic”, in *Encyclopedia of Artificial Intelligence*, Shapiro (ed.), 2nd Edition, New York: John Wiley and Sons, pp. 851–853, 1992.
- [**Rapaport 92b**] Rapaport W.J., “Predicate Logic”, in *Encyclopedia of Artificial Intelligence*, Shapiro (ed.), 2nd Edition, New York: John Wiley and Sons, pp. 866–873, 1992.
- [**Raphael 68**] Raphael B., “SIR: Semantic Information Retrieval”, in *Semantic Information Processing*, Minsky (ed.), Cambridge, MA: The MIT Press, pp. 33–145, 1968.
- [**Rautenberg 87**] Rautenberg W. (ed.), *Ω -Bibliography of Mathematical Logic*, Volume II: Non-Classical Logics, Berlin, Germany: Springer-Verlag, 1987.
- [**Read 88**] Read S., *Relevant Logic: A Philosophical Examination of Inference*, Oxford, UK: Basil Blackwell Ltd, 1988.
- [**Reinfrank 89**] Reinfrank M., *Fundamentals and Logical Foundations of Truth Maintenance*, Linköping Studies in Science and Technology. Dissertation 221. Linköping, Sweden: Department of Computer and Information Science, Linköping University, 1989.
- [**Reiter 78a**] Reiter R., “On Reasoning by Default”, *Proc. Theoretical Issues in Natural Language Processing 2*, pp. 210–218, 1978. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 401–410, 1985.
- [**Reiter 78b**] Reiter R., “On Closed World Data Bases”, in *Logic and Data Bases*, Gallaire and Minker (eds.), New York: Plenum Press, pp. 55–76, 1978. Also in *Readings in Artificial Intelligence*, Webber and Nilsson (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 119–140, 1981.
- [**Reiter 80**] Reiter R., “A Logic for Default Reasoning”, *Artificial Intelligence 13*, No. 1–2, pp. 81–132, 1980. Also in *Readings in Monotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 68–93, 1987.

- [**Reiter 87**] Reiter R., “Non-monotonic Reasoning”, in *Exploring Artificial Intelligence*, Shrobe (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 439–481, 1988. Also in *Annual Review of Computer Science 2*, Tramb (ed.), Palo Alto, CA: Annual Reviews Inc., pp. 947–986, 1987.
- [**Reiter and Criscuolo 81**] Reiter R. and Criscuolo G., “On Interacting Defaults”, *Proc. IJCAI-81*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 270–276, 1981. Also in *Readings in Monmonotonic Reasoning*, Ginsberg (ed.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 94–100, 1987.
- [**Roberts 73**] Roberts D.D., *The Existential Graphs of Charles S. Pierce*, The Hage, The Netherlands: Mouton, 1973.
- [**Roberts and Goldstein 77**] Roberts R.B. and Goldstein I.P., “The FRL Manual”, AI Memo 409, Artificial Intelligence Laboratory, Cambridge, MA: Massachusetts Institute of Technology, 1977.
- [**Robinson 65**] Robinson J.A., “A Machine-Oriented Logic Based on the Resolution Principle”, *Journal of the ACM* 12, No. 1, pp. 23–41, 1965.
- [**Sandewall 72**] Sandewall E., “An Approach to the Frame Problem and its Implementation”, *Machine Intelligence 7*, Meltzer and Michie (eds.), Edinburgh, Scotland: Edinburgh University Press, pp. 195–204, 1972.
- [**Schank 75**] Schank R.C., *Conceptual Information Processing*, Amsterdam, The Netherlands: North-Holland Publishing Co., 1975.
- [**Schank and Abelson 77**] Schank R.C. and Abelson R.P., *Scripts, Plans, Goals and Understanding*, Hillsdale, NJ: Lawrence Erlbaum Associates, Publishers, 1977.
- [**Schank and Riesbeck 81**] Schank R.C. and Riesbeck C.K., *Inside Computer Understanding: Five Programs plus Miniatures*, Hillsdale, NJ: Lawrence Erlbaum Associates, Publishers, 1981.
- [**Schmidt-Schauß and Smolka 91**] Schmidt-Schauß M. and Smolka G., “Attributive Concept Descriptions with Complements”, *Artificial Intelligence* 48, No. 1, pp. 1–26, 1991.
- [**Schmolze and Lipkis 83**] Schmolze J.G. and Lipkis T., “Classification in the KL-ONE Knowledge Representation System”, *Proc. IJCAI-83*, Los Altos, CA: Morgan Kaufmann, pp. 330–332, 1983.
- [**Schubert 76**] Schubert L.K., “Extending the Expressive Power of Semantic Networks”, *Artificial Intelligence* 7, No. 2, pp. 163–198, 1976.
- [**Schubert, Goebel, and Cerccone 79**] Schubert L.K., Goebel R.G., and Cerccone N.J., “The Structure and Organization of a Semantic Net for Comprehension and Inference”, in *Associative Networks: Representation and Use of Knowledge by Computers*, Findler (ed.), New York: Academic Press, pp. 121–175, 1979.

- [**Shapiro 71**] Shapiro S.C., “A Net Structure for Semantic Information Storage, Deduction, and Retrieval”, *Proc. IJCAI-71*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 512–523, 1971.
- [**Shapiro 77**] Shapiro S.C., “Representing and locating deduction rules in a semantic network”. *Proceedings of the Workshop on Pattern-Directed Inference Systems, SIGART Newsletter 63*, pp. 14–18, 1977.
- [**Shapiro 79a**] Shapiro S.C., “The SNePS Semantic Network Processing System”, in *Associative Networks: Representation and Use of Knowledge by Computers*, Findler (ed.), New York, N.Y.: Academic Press Inc., pp. 179–203, 1979.
- [**Shapiro 79b**] Shapiro S.C., “Using Non-standard Connectives and Quantifiers for Representing Deduction Rules in a Semantic Network”, presented at “Current Aspects of AI research”, a seminar held at the Electrotechnical Laboratory, Tokyo, August 27–28, 1979.
- [**Shapiro and Martins 90**] Shapiro S.C. and Martins J.P., “Recent Advances and Developments: The SNePS 2.1 Report”, in *Current Trends in SNePS – Semantic Network Processing System*, Kumar (ed.), Lecture Notes in Artificial Intelligence 437, Heidelberg, Germany: Springer-Verlag, pp. 1–13, 1990.
- [**Shapiro McKay, Martins, and Morgado 81**] Shapiro S.C., McKay D.P., Martins J.P., and Morgado E.J., “SNePSLOG, a Higher Order Logic Programming Language”, presented at *Workshop on Logic Programming for Intelligent Systems*, R.M.S. Queen Mary, Long Beach, CA, 1981.
- [**Shapiro and Rapaport 87**] Shapiro S.C. and Rapaport W.J., “SNePS Considered as a Fully Intensional Propositional Semantic Network”, in *The Knowledge Frontier: Essays in the Representation of Knowledge*, Cercone and McCalla (eds.), New York: Springer Verlag, pp. 262–315, 1987.
- [**Shapiro and Rapaport 92**] Shapiro S.C. and Rapaport W.J., “The SNePS Family”, in *Semantic Networks in Artificial Intelligence*, Lehmann (ed.), Oxford, UK: Pergamon Press Ltd., pp. 243–275, 1992.
- [**Shapiro and Rapaport 03**] Shapiro S.C. and Rapaport W.J., “SNePS: An Interactive Approach”, Buffalo, NY: Department of Computer Science and Engineering, State University of New York at Buffalo, 2003. Also available at <http://www.cse.buffalo.edu/sneps/Tutorial/>.
- [**Shapiro, et al. 93**] Shapiro S.C., Rapaport W.J., Cho S-H., Choi J., Feit E., Haller S., and Kumar D., “A Dictionary of SNePS Case Frames”, Buffalo, N.Y.: Department of Computer Science. State University of New York at Buffalo, 1993.
- [**Shapiro, et al. 95**] Shapiro S.C. and the SNePS Implementation Group, “SNePS 2.3 User’s Manual”, Department of Computer Science, State University of New York at Buffalo, 1995.

- [**Shapiro and Wagn 76**] Shapiro S. and Wand M., “The Relevance of Relevance”, Technical Report No. 46, Bloomington, IN: Computer Science Department, Indiana University, 1976.
- [**Sickel 76**] Sickel S., “A Search Technique for Clause Interconnectivity Graphs”, *IEEE Transactions on Computers*, Vol. C-25, No. 8, pp. 823–835, 1976.
- [**Simmons 73**] Simmons R.F., “Semantic Networks: Their Computation and Use for Understanding English Sentences”, in *Computer Models of Thought and Language*, Schank and Colby (eds.), San Francisco, CA: W.H. Freeman and Co., pp. 63–113, 1973.
- [**Smith 82**] Smith B.C., “Reflection and Semantics in a Procedural Language”, Technical Report MIT/LCS/TR-272, Cambridge, MA: Massachusetts Institute of Technology, 1982. Partially published in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 31–39, 1985.
- [**Smith 85**] Smith B.C., “The limits of correctness”, *ACM SIGCAS Computers and Society 14*, pp. 18–26, 1985.
- [**Smith 98**] Smith B., “Basic Concepts of Formal Ontology”, in Guarino (ed.), *Formal Ontology in Information Systems*, pp. 19–28, Amsterdam, The Netherlands: IOS Press, 1998.
- [**Smith and Kelleher 88**] Smith B. and Kelleher G., *Reason Maintenance Systems and Their Applications*, Chichester, UK: Ellis Horwood Limited, 1988.
- [**Snyder 91**] Snyder A., “Inheritance in Object-Orientated Programming”, in *Inheritance Hierarchies in Knowledge Representation and Programming Languages*, Lenzerini, Nardi and Simi (eds.), New York: John Wiley and Sons, pp. 153–172, 1991.
- [**Sommerville 96**] Sommerville I., *Software Engineering*, Reading, MA: Addison-Wesley, 1996.
- [**Sowa 84**] Sowa J.F., *Conceptual Structures: Information Processing in Mind and Machine*, Reading, MA: Addison-Wesley, 1984.
- [**Sowa 91**] Sowa J.F.(ed.), *Principles of Semantic Networks: Explorations in the Representation of Knowledge*, San Mateo, CA: Morgan Kaufmann Publishers Inc., 1991.
- [**Sowa 00**] Sowa J.F., *Knowledge Representation: Logical, Philosophical, and Computational Foundations*, Pacific Grove, CA: Brooks/Cole, 2000.
- [**Stallman and Sussman 77**] Stallman R. and Sussman G., “Forward Reasoning and Dependency-Directed Backtracking in a System for Computer-Aided Circuit Analysis”, *Artificial Intelligence 9*, No. 2, pp. 135–196, 1977.
- [**Stalnaker 84**] Stalnaker R., *Inquiry*, Cambridge, MA: The MIT Press, 1984.

- [**Stefik 79**] Stefik M., “An Examination of a Frame-structured Representation System”, *Proc. IJCAI-79*, Los Altos, CA: Morgan Kaufmann, pp. 845–852, 1979.
- [**Struss 88**] Struss P., “Extensions to ATMS-based Diagnosis”, in *Artificial Intelligence in Engineering: Diagnosis and Learning*, Gero (ed.), Amsterdam, The Netherlands: Elsevier, pp. 3–28, 1988.
- [**Summers 72**] Summers G.J., *Test Your Logic: 50 Puzzles in Deductive Reasoning*, New York, N.Y.: Dover Publications Inc., 1972.
- [**Sussman, Winograd, and Charniak 71**] Sussman G., Winograd T., and Charniak E., “Micro-Planner Reference Manual”, Technical Report 203A, Cambridge, MA: AI Lab., Massachusetts Institute of Technology, 1971.
- [**Tarski 65**] Tarski A., *Introduction to Logic and to the Methodology of Deductive Sciences*, New York: Oxford University Press, 1965.
- [**Tennant 84**] Tennant N., “Perfect Validity, Entailment and Paraconsistency”, *Studia Logica* 43, pp. 181–200, 1984.
- [**Thayse 88**] Thayse A., *From Standard Logic to Logic Programming*, Chichester, UK: John Wiley and Sons, 1988.
- [**Thomason 92**] Thomason R., “NETL and Subsequent Path-based Inheritance Theories”, in *Semantic Networks in Artificial Intelligence*, Lehmann (ed.), Oxford, UK: Pergamon Press Ltd., pp. 179–204, 1992.
- [**Thomason, Horty, and Touretsky 87**] Thomason R., Horty J., and Touretsky D., “A Calculus for Inheritance in Monotonic Semantic Nets”, *Proc. Second International Symposium on Methodologies for Intelligent Systems*, Ras and Zemankova (eds.), North-Holland, pp. 280–287, 1987.
- [**Touretsky 86**] Touretzky D.S., *The Mathematics of Inheritance Systems*, London, UK: Pitman Publishing, 1986.
- [**Touretsky 92**] Touretzky D.S., “Inheritance Hierarchy”, in *Encyclopedia of Artificial Intelligence*, 2nd Edition, Shapiro (ed.), New York: John Wiley and Sons, pp. 690–701, 1992.
- [**Touretsky, Horty, and Thomason 87**] Touretzky D.S., Horty J.F. and Thomason R.H., “A Clash of Intuitions: The Current State of Nonmonotonic Multiple Inheritance Systems”, *Proc. IJCAI-87*, Los Altos: Morgan Kaufmann, pp. 476–482, 1987.
- [**Turner 84**] Turner R., *Logics for Artificial Intelligence*, Chichester, UK: Ellis Horwood Ltd., 1984.
- [**Turner 90**] Turner R., *Truth and Modality for Knowledge Representation*, London, UK: Pitman Publishing, 1990.

- [**Waterman and Hayes-Roth 78**] Waterman D.A. and Hayes-Roth F., “An Overview of Pattern-directed Inference Systems”, in *Pattern-directed Inference Systems*, Waterman and Hayes-Roth (eds), New York, NY: Academic Press Inc., pp. 3–22, 1978.
- [**Whitehead and Russell 10**] Whitehead A.N. and Russell B., *Principia Mathematica*, Cambridge, UK: Cambridge University Press, 1910 (2nd Edition, 1987).
- [**Wilks 73**] Wilks Y., “An Artificial Intelligence Approach to Machine Translation”, in *Computer Models of Thought and Language*, Schank and Colby (eds.), San Francisco, CA: W.H. Freeman and Co., pp. 114–151, 1973.
- [**Wilks 77**] Wilks Y., “Good and Bad Arguments about Semantic Primitives”, D.A.I. Research Report N. 42, University of Essex, UK, 1977.
- [**Williams and Rott 01**] Williams M-A. and Rott H. (eds.), *Frontiers in Belief Revision*, Dordrecht, The Netherlands: Kluwer, 2001
- [**Winograd 72**] Winograd T., *Understanding Natural Language*, N.Y.: Academic Press, 1972.
- [**Winograd 75**] Winograd T., “Frame Representation and the Declarative / Procedural Controversy”, in *Representation and Understanding: Studies in Cognitive Science*, Bobrow and Collins (eds.), New York, NY: Academic Press Inc., pp. 185–210, 1975.
- [**Wolfram 89**] Wolfram S., *Philosophical Logic, An Introduction*, London, UK: Routledge, 1989.
- [**Woods 75**] Woods W.A., “What’s in a Link: Foundations for Semantic Networks”, in *Representation and Understanding, Studies in Cognitive Science*, Bobrow and Collins (eds.), New York: Academic Press, pp. 35–82, 1975. Also in *Readings in Knowledge Representation*, Brachman and Levesque (eds.), Los Altos, CA: Morgan Kaufmann, pp. 217–241, 1985.
- [**Woods 87**] Woods W.A., “Knowledge Representation: What’s Important About It?”, in *The Knowledge Frontier: Essays in the Representation of Knowledge*, Cercone and McCalla (eds.), New York: Springer-Verlag, pp. 44–79, 1987.
- [**Woods 91**] Woods W.A., “Understanding Subsumption and Taxonomy: A Framework for Progress”, in *Principles of Semantic Networks, Explorations in the Representation of Knowledge*, Sowa (ed.), San Mateo, CA: Morgan Kaufmann Publishers Inc., pp. 45–94, 1991.
- [**Woods 92**] Woods W.A. and Schmolze J.G., “The KL-ONE Family”, in *Semantic Networks in Artificial Intelligence*, Lehmann (ed.), Oxford, UK: Pergamon Press Ltd., pp. 133–177, 1992.
- [**Zadeh 85**] Zadeh L., “Syllogistic Reasoning as a Basis for Combination of Evidence in Expert Systems”, *Proc. IJCAI-85*, Los Altos, CA: Morgan Kaufmann Publishers Inc., pp. 417–419, 1985.